Deep Learning for Computer Vison (EN 525.733.8VL +8VL2)

Dr. Nasser M. Nasrabadi

Virtual Lectures, Tuesday or Thursdays, 7.20-10pm

nnasrab1@jhu.edu

Zoom class meeting link:

https://wse.zoom.us/j/97388824681

Passcode: 525619

Introduction to Deep Learning

Class Format

The course consists of lectures given by me and students with discussions, reading journal paper assignments, and homework assignments using dedicated neural network software in MatLab, PyTorch, ad Tensor Flow. Each student will carry out several mini-independent projects and 1 final project in selected research area subject to the instructor's approval but mainly in biometrics and vision.

Grading:

- Homework Assignments, 30% **(You can only submit two late HWs)**
- Mini Projects (15 minutes oral presentation): 30%
- Final Project (15 minutes oral presentation): 40%

Course Notes Will be provided

Class Schedule

Week 1: Introduction and review of classical artificial neural network architectures (MLP, CNN)

Week 2: Review of the classical convolutional neural network-based architecture (LeNet, AlexNet, VGGNet) and the functions of its layers.

Week 3: Details of Inception Architectures for GoogLeNet,

Week 4: Details of ResNet, and DenseNet achitectures

Week 5: Details of Facenet, DeepFace, HighwayNet and FractalNet, ***mini-project assignment***

Week 6: Regularization theory, sparsity and overfitting problems

Week 7: Introduction to sparse auto-encoders, Coupled CNN, and Siamese auto-encoders

Week 8: Mid-term project with oral presentation, ***Mid-term Project Due***

Week 9: Generative adversarial Networks (GAN) and its applications, ***Final project assignment***

Week 10: Conditional GAN, Cycle GAN, StyleGAn and StarGAN with their applications

Week 11: Recurrent network and LTSM networks

Week 12: Data & classifier fusion, multimodal fusion

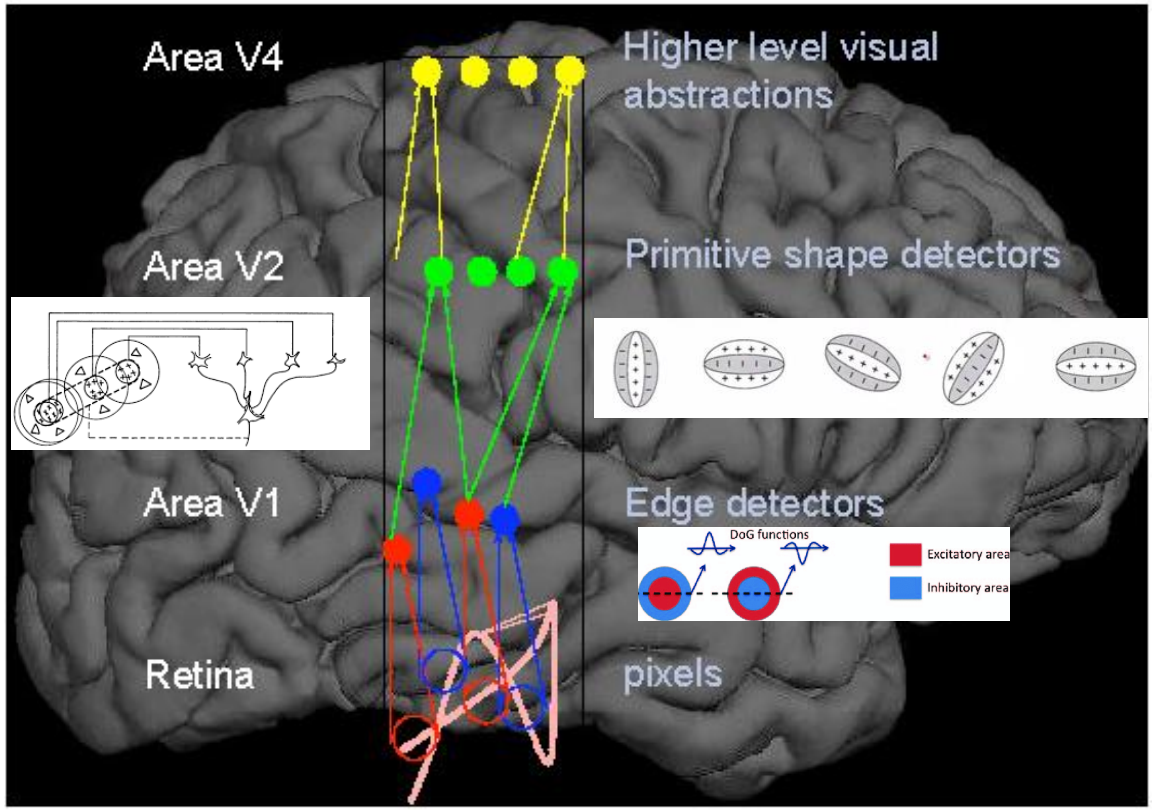
Week 13: Use of deep learning in image, video processing and biometrics (i.e., video super-resolution, face recognition, fingerprints, voice, iris)

Week 14: Applications of deep learning for data-mining and image retrieval using deep hashing

Week 15: Final project with oral presentation, ***Final project Due***

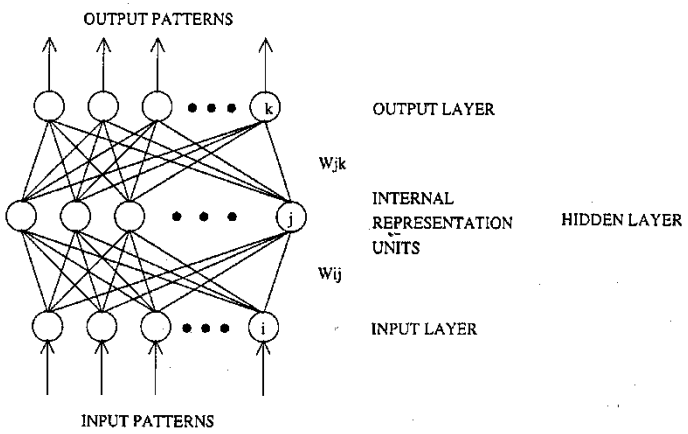
Deep Architecture in the Brian

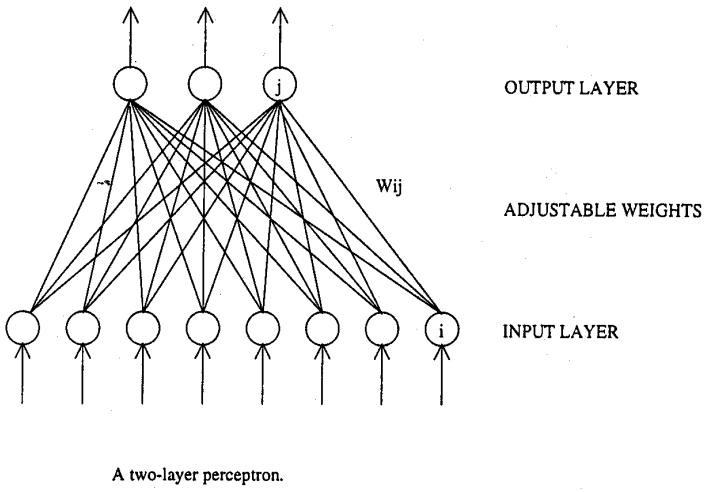
- Neural network architectures are motivated by models of our brain and nerve cells.
- Neural networks are massively parallel systems with dense arrangements of simple processors.
- Processing power of neural networks is measured mainly by the number of interconnection updates per second.
- More interconnections than number of processors.
- Neural networks are trained by presenting samples.
- Neural processors are much simpler than a CPU of a computer.



Background of Neural Networks

- Neural network architectures are motivated by models of our brain and nerve cells.
- Neural networks are massively parallel systems with dense arrangements of simple processors.
- Processing power of neural networks is measured mainly by the number of interconnection updates per second.
- More interconnections than number of processors.
- Neural networks are trained by presenting samples.
- Neural processors are much simpler than a CPU of a computer.





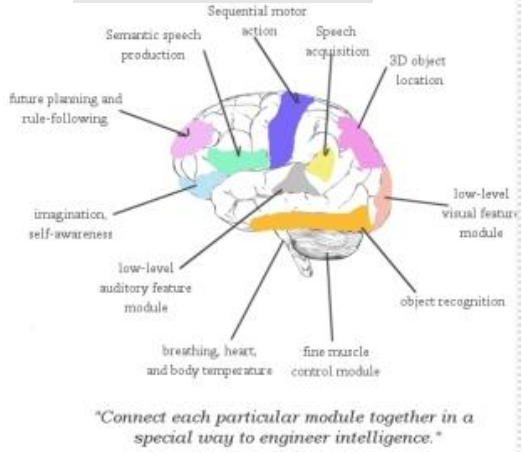
A two-layer perceptron.

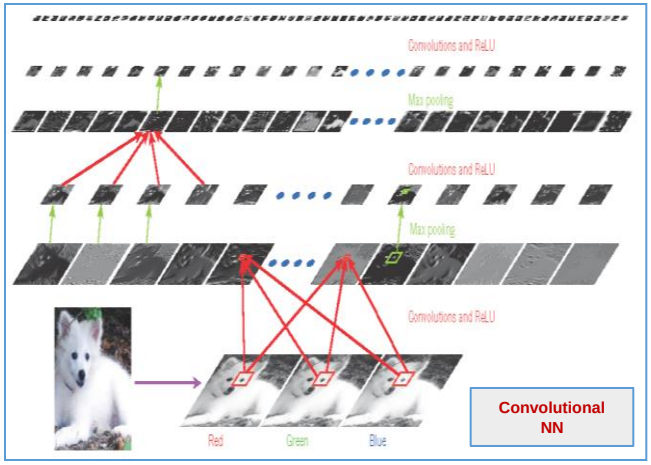
Introduction to Neural Networks (Cont.)

- Neural network architectures are different from traditional single processor computers.
- Von Neumann machines have a single CPU and computations are sequential (one instruction at a time).
- The processing power of a neural network is measured mainly by the number of interconnection updates per second.
  a. Neural networks VS parallel computers
  b. Processors in a neural network are massively interconnected
  c. More interconnections than number of processors.
  d. Neural networks are trained by presenting examples.
  e. They are not programmed.
- Neural processors are much simpler than a typical CPU processor.
- Many applications of neural networks can be considered as pattern mapping tasks, i.e., classification problem.

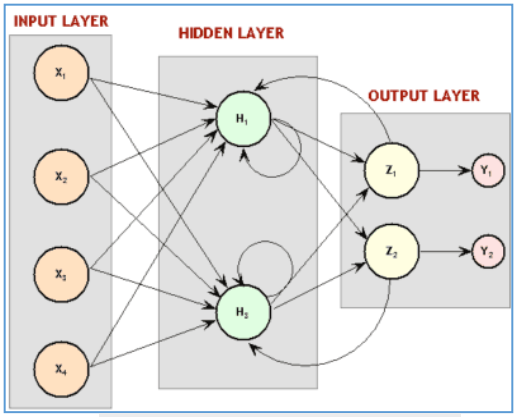
What is Deep Neural Network (deep Learning)

An Integrate system of Specialized modules

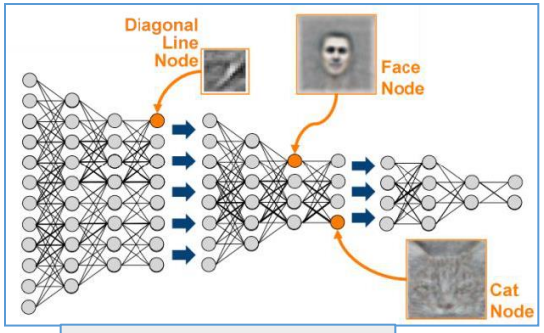




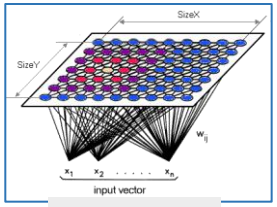
Convolutional NN



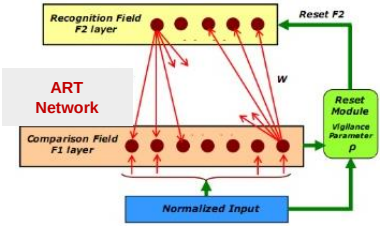
Recurrent NN

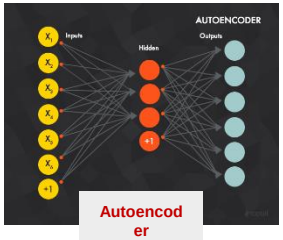


Multilayer Perceptrons (MLP 1994)

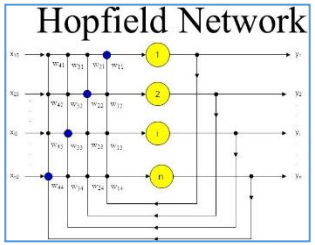


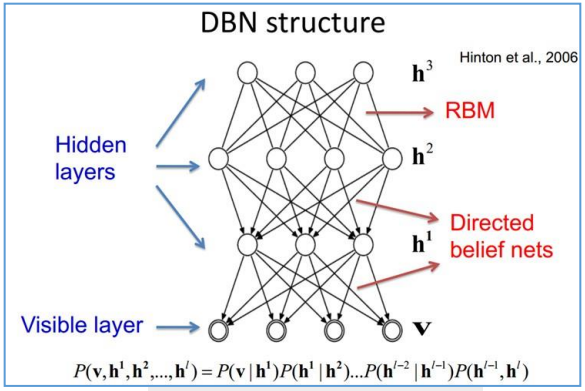
Kohonen SOFM





Autoencoder

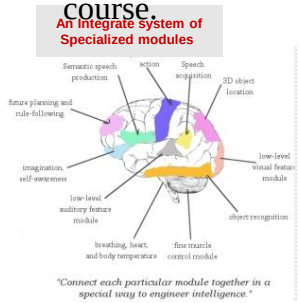


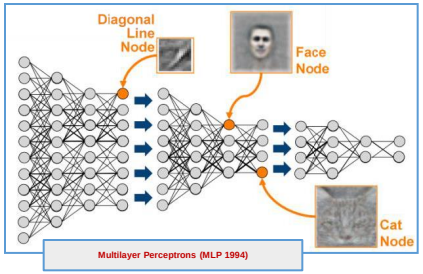


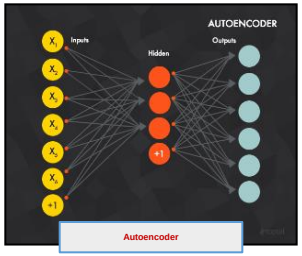
Deep Belief Network (DBN_2006)

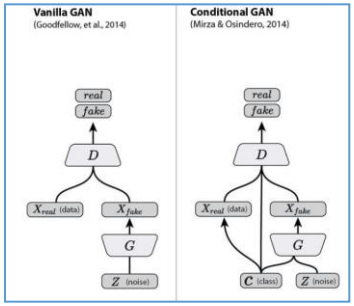
Deep Neural Network Architectures

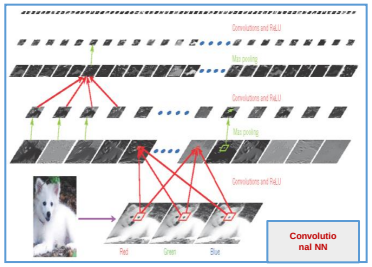
- Neural network architectures that are studied in this course.

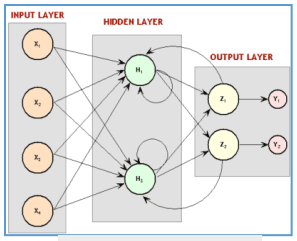


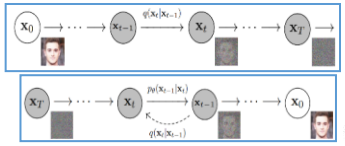


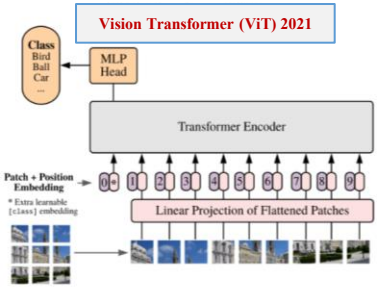


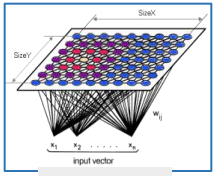


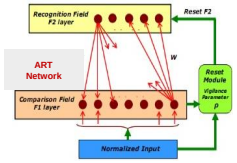


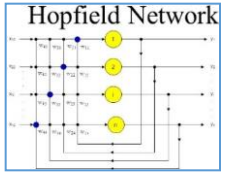


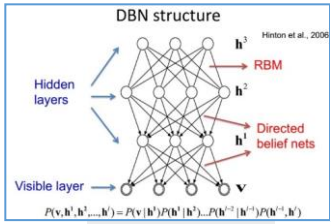


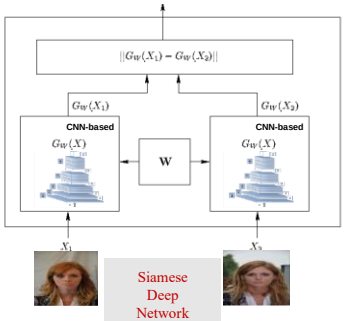












Pattern-Mapping System

- Consider a pattern-mapping system shown below; interface to the system are a variety of input and output signals.
- For each input signal (i.e., a feature vector is extracted to represent the task) the output is the result of the task (i.e., identifying the class). Pattern classification tasks, such as identification of blood cells, sonar signals, speech recognition, character recognition, prediction tasks and recalling information from partial data can be found in neural network literature.
- Optimization tasks, such as travel salesman problem and vision problems can be solved by neural networks. Many vision tasks can be converted into optimization problems (minimization tasks).
- A large number of problems in speech and vision remain to be addressed, i.e., recognition of objects at different scales, orientation, and overlapping.

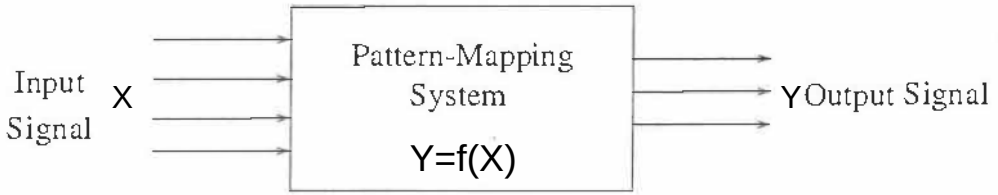
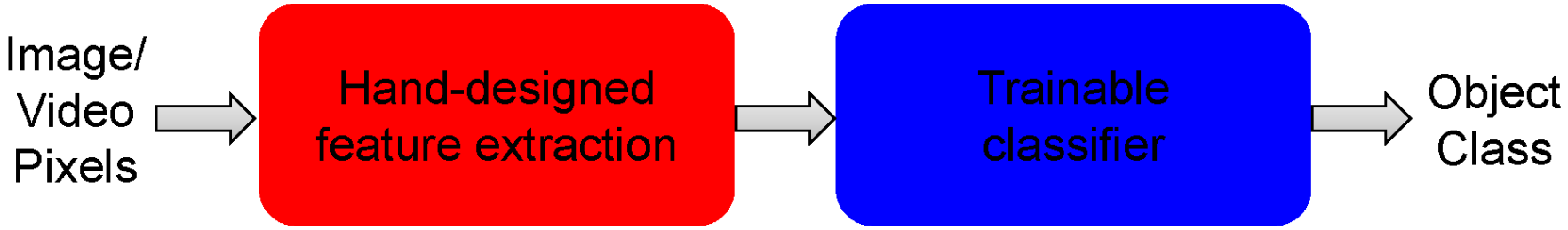


Figure Shows a simple pattern-mapping system

Shallow vs. Deep Architectures

Traditional recognition: “Shallow” architecture



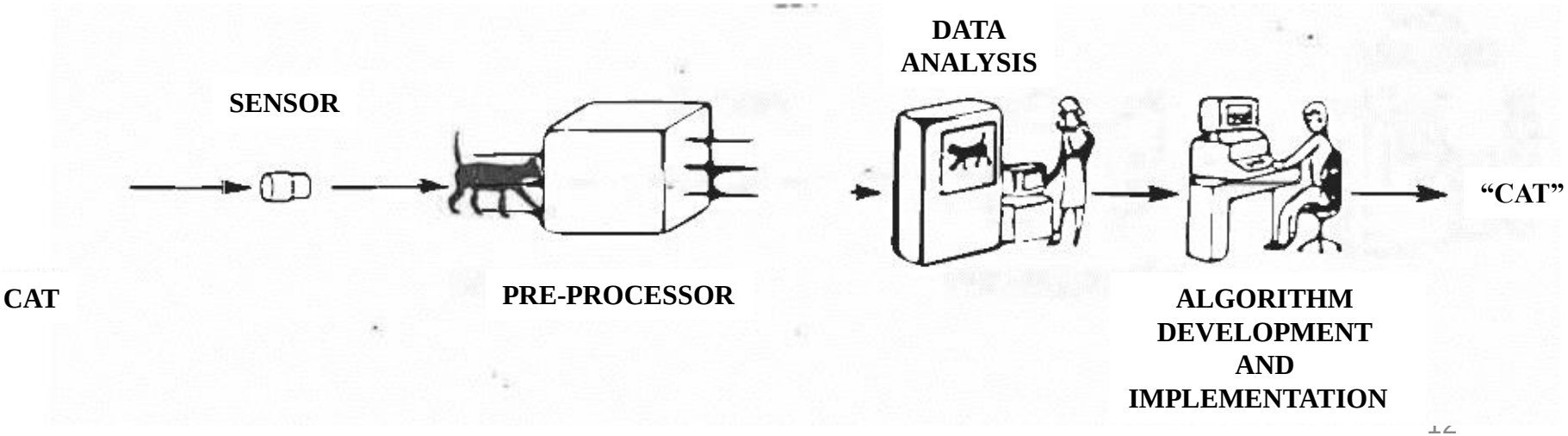
SIFT, HoG, BSIF, LBP, SURF

SVM, Logistic regression, Decision trees

Deep learning: “Deep” architecture

Traditional Computers vs Neural Networks

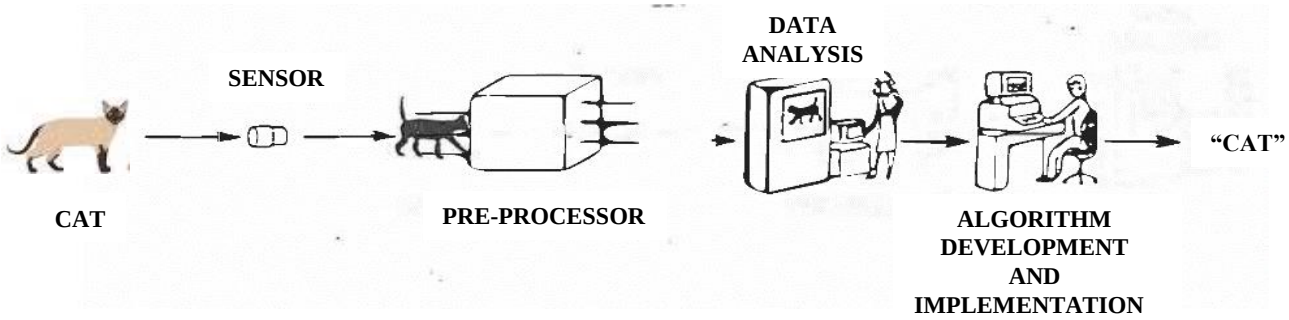
- Traditional approach to a pattern classification problem
- A program is needed to perform a task.
- An error (damage) in the program, the task cannot be accomplished.
- Information about objects to be recognized are stored in memory.



Traditional approach to object recognition

Traditional Computers vs Neural Networks

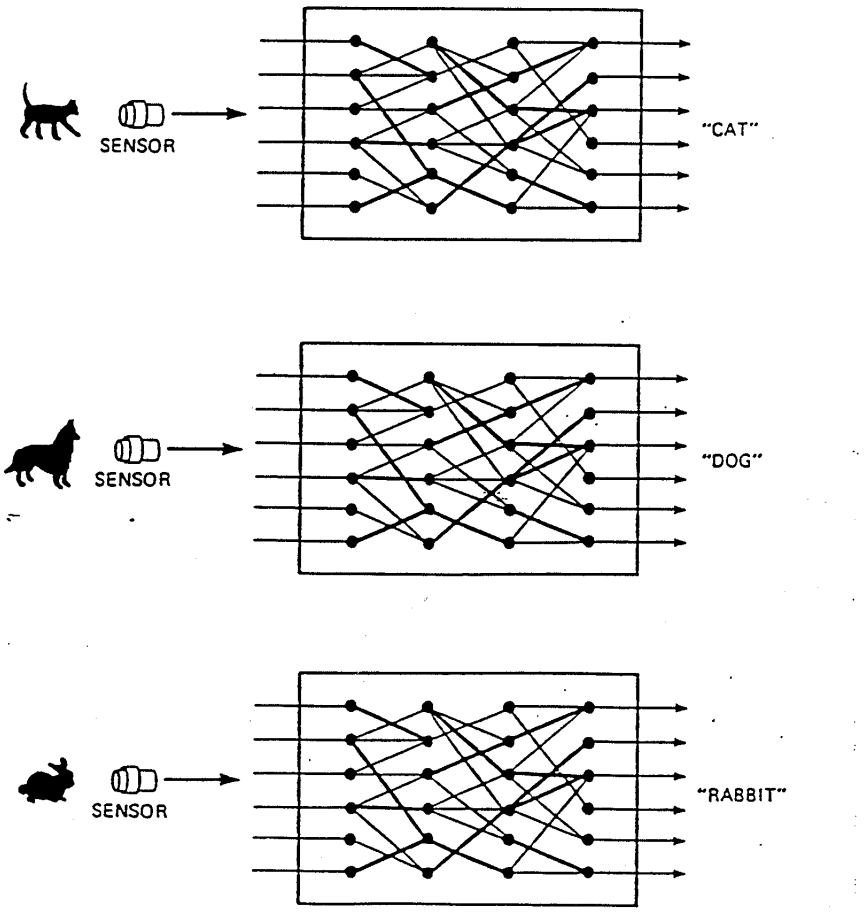
- Traditional approach to a pattern classification problem
- A program is needed to perform a task.
- An error (damage) in the program, the task cannot be accomplished.
- Information about objects to be recognized are stored in memory.



Traditional approach to object recognition

Traditional Computers VS Neural Networks

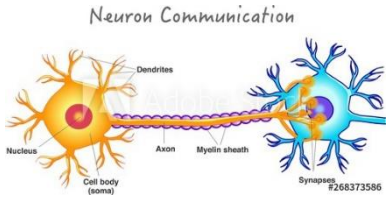
- Neural networks are good in pattern mapping tasks.
- Many signal and image processing techniques can mapped as pattern mapping tasks.
- In neural networks knowledge about objects are distributed and learned by adjusting the weights.
- Same network is used to classify different objects.

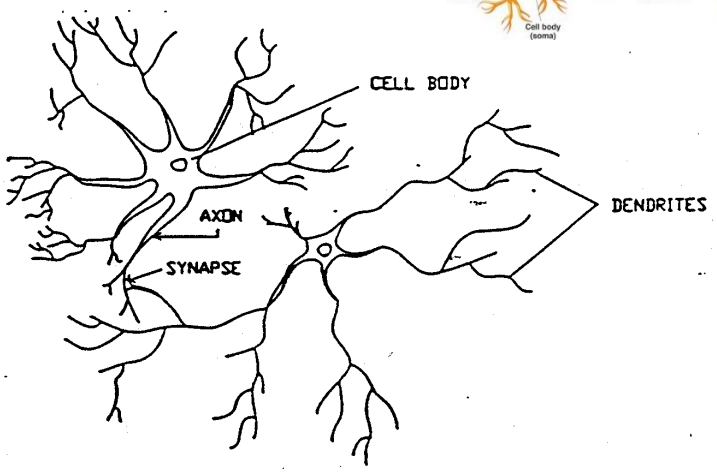


Neural network approach to a pattern classification problem

Biological Neuron

- In a human brain, there are approximately 10^{11} neurons with 10^{15} inter-connections.
- A neuron consists of three sections:
  - Cell Body: Summation-fires
  - Dendrites: Receive the signal from other cells
  - Axon: Passes the signal (pulses) to other cells
    - They could be 0.1 millimeter or up to 1 meter in length.
- Point of connections are called synapses.
- The signals received by the cell body is summed and when this sum exceeds a threshold the cell fires, sending a pulse down the axon to other neurons.

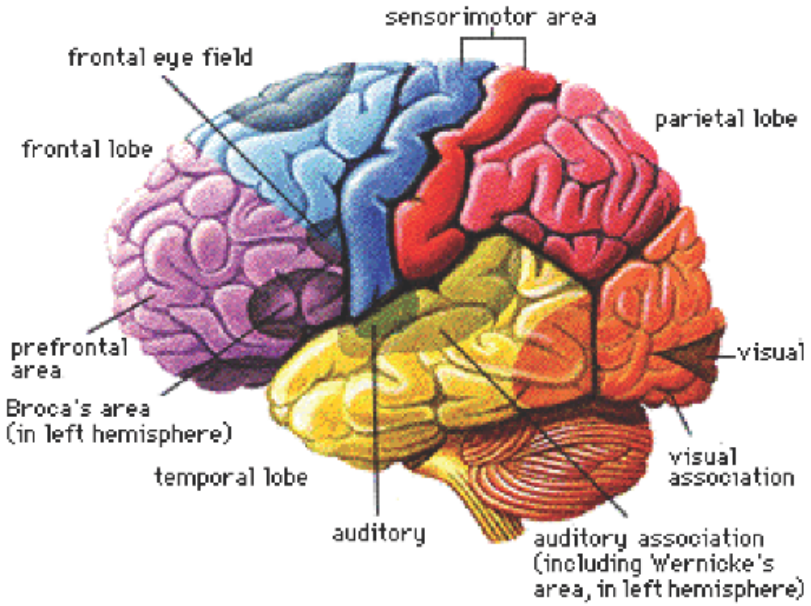




Human Brain

Neural Networks in the Brain

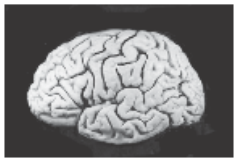
The brain is not homogeneous. At the largest anatomical scale, we distinguish **cortex**, **midbrain**, **brainstem**, and **cerebellum**. Each of these can be hierarchically subdivided into many **regions**, and **areas** within each region, either according to the anatomical structure of the neural networks within it, or according to the function performed by them.



The overall pattern of **projections** (bundles of neural connections) between areas is extremely complex, and only partially known. The best mapped (and largest) system in the human brain is the visual system, where the first 10 or 11 processing stages have been identified. We distinguish **feedforward** projections that go from earlier processing stages (near the sensory input) to later ones (near the motor output), from **feedback** connections that go in the opposite direction.

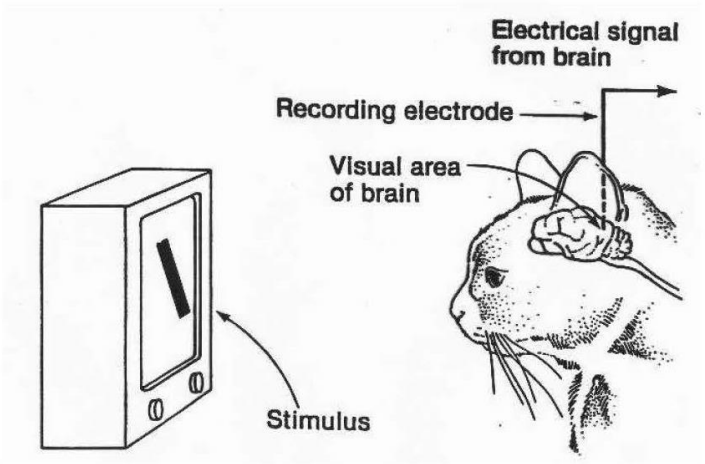
In addition to these long-range connections, neurons also link up with many thousands of their neighbours. In this way they form very dense, complex local networks:

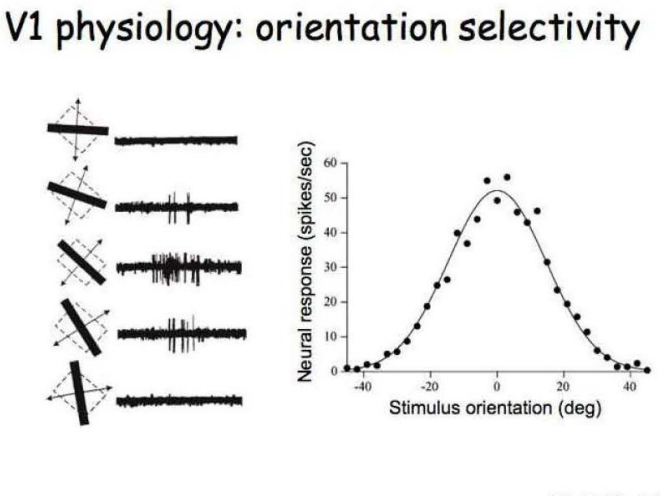
Human Brain vs Artificial Neural Network

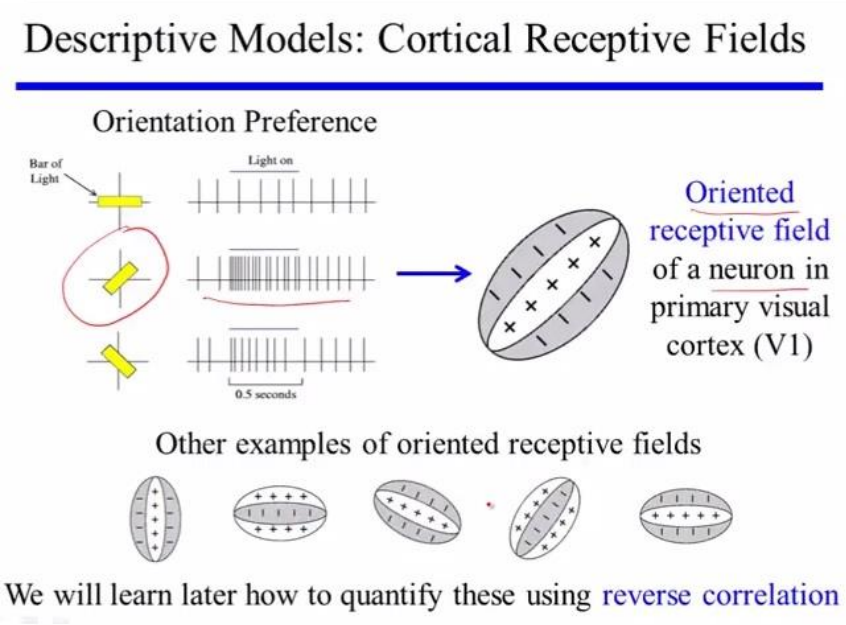
	processing elements	element size	energy use	processing speed	style of computation	fault tolerant	learns	intelligent, conscious
	10^{14} synapses	10^{-6} m	30 W	100 Hz	parallel, distributed	yes	yes	usually
	10^{8} transistors	10^{-6} m	30 W (CPU)	10^{9} Hz	serial, centralized	no	a little	not (yet)

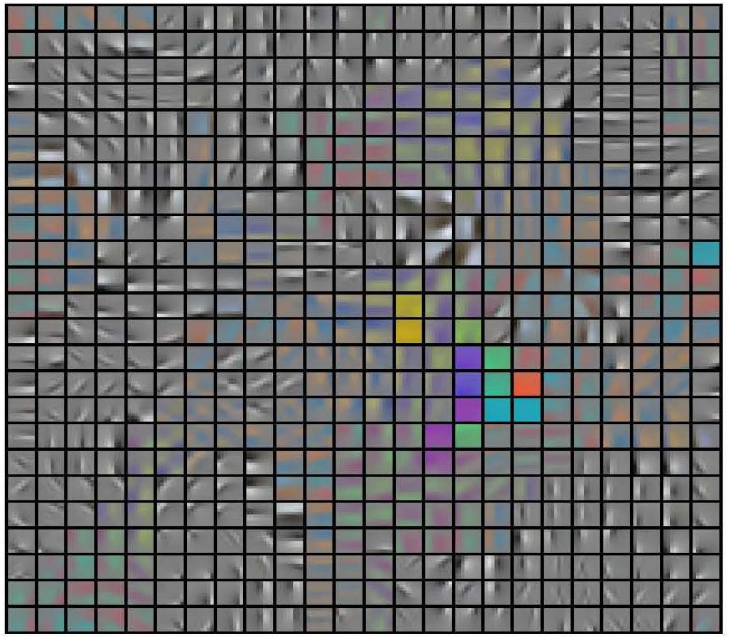
As a discipline of Artificial Intelligence, Neural Networks attempt to bring computers a little closer to the brain's capabilities by imitating certain aspects of information processing in the brain, in a highly simplified

Selectivity and Topographic Maps in VI







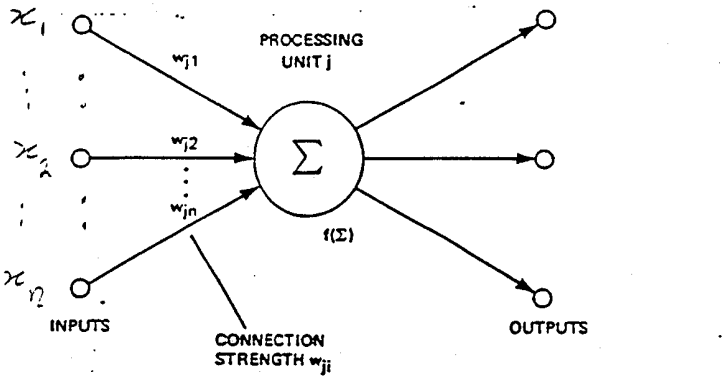


Multi-Layer Perceptrons (MLP)

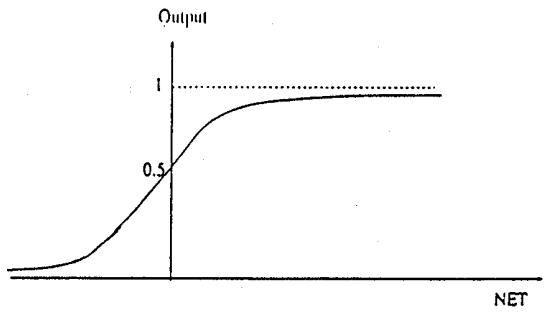
Artificial Neuron

- A typical processing unit performs a weighted sum on the inputs and uses a nonlinear function (hard or soft threshold), *f*(.), to compute its input. The output is sent to other cells.
- The **sum** is $NET_j = \mathbf{x}^{\mathrm{T}}\mathbf{w} = x_1 w_{j1} + x_2 w_{j2} + \ldots + x_n w_{jn}$
- The nonlinear function is usually chosen to be a Sigmoid function:

$$f(\mathrm{NET}_j) = \frac{1}{1 + e^{-\lambda \mathrm{NET}_j}}$$



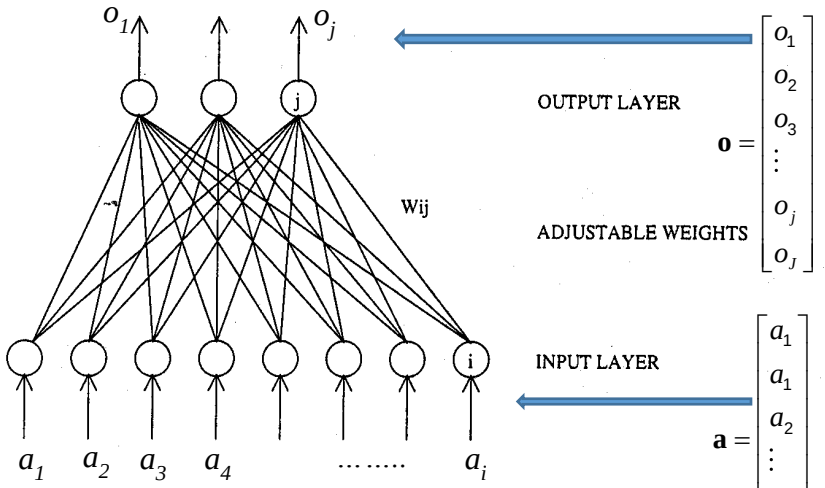
Schematic processing unit from an artificial neural network.



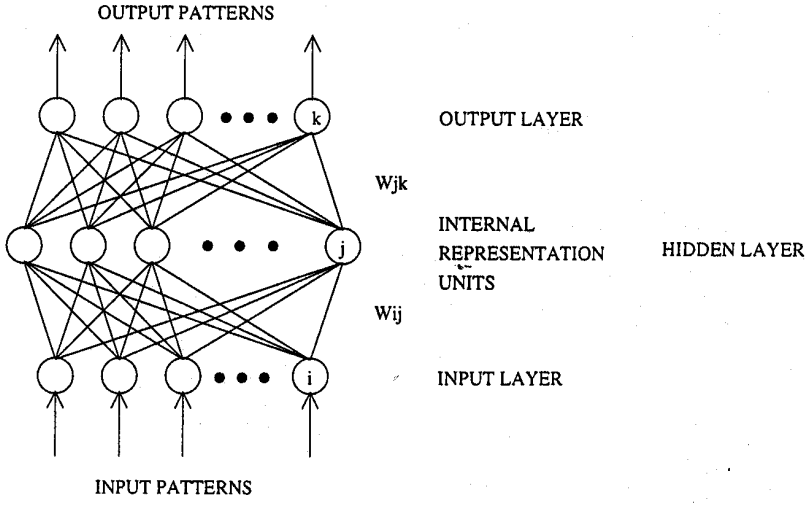
Sigmoidal Function

Examples of Feedforward Neural Networks

- Figure below shows a two layer and a three-layer networks:



A two-layer perceptron.



Historical Note

1943 McCulloch and Pitts proposed the McCulloch-Pitts neuron model

1949 Hebb published his book *The Organization of Behaviour*, in which the Hebbian learning rule was introduced

1958 Rosenblatt introduced the simple single layer networks called Perceptrons

1969 Minsky and Papert's book Perceptrons demonstrated the limitation of single layer perceptrons

1980 Grossberg introduced his Adaptive Resonance Theory (ART)

1982 Hopfield published a series of papers on Hopfield networks

1982 Kohonen developed the Self-Organizing Feature Maps

1986 Back-propagation learning algorithm for multi-layer perceptrons was re-discovered, and the whole field took off again

1990s ART-variant networks were developed

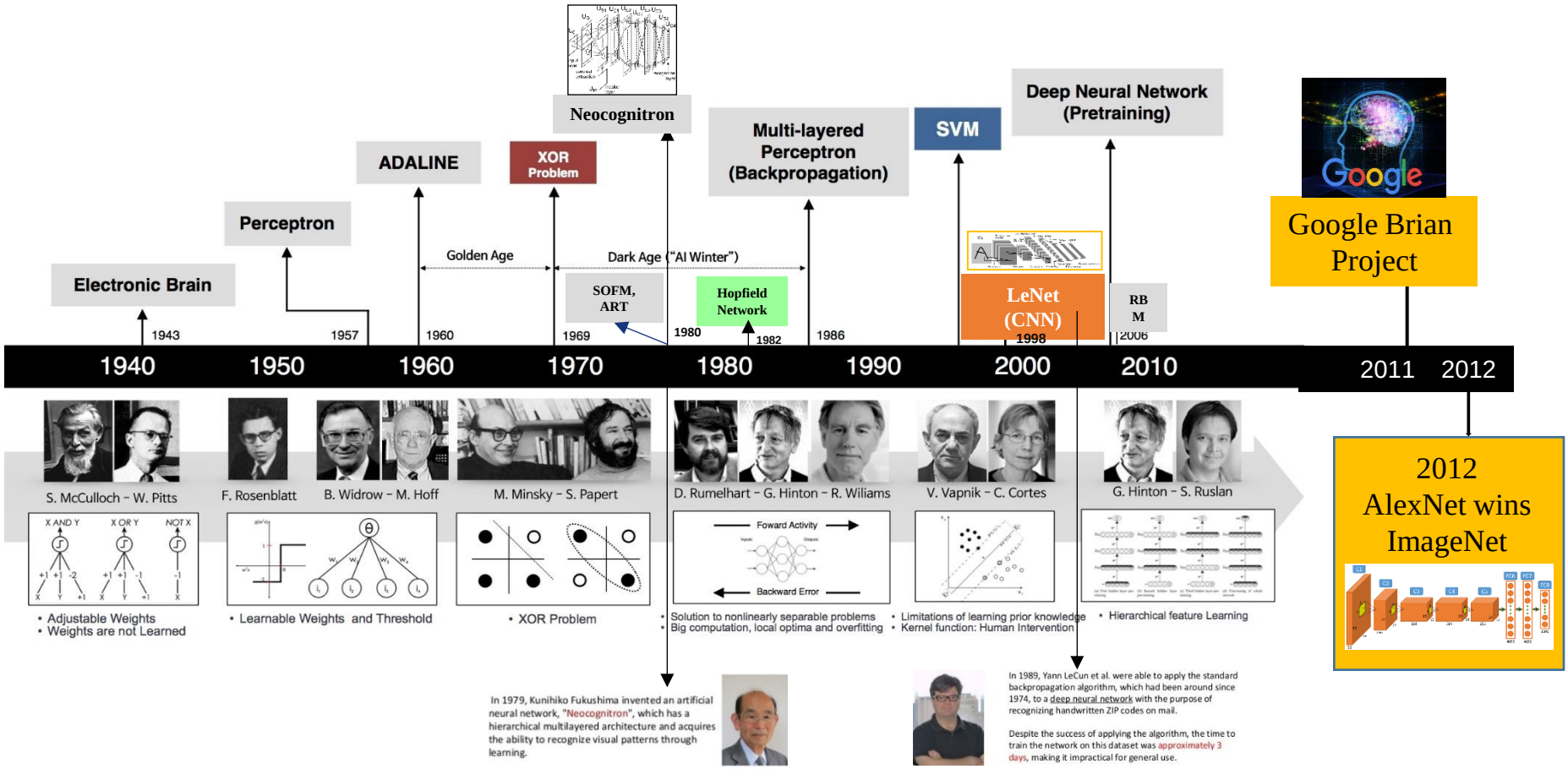
1990s Radial Basis Functions were developed

2000s Support Vector Machines were developed

Historical Note

Year	Contributer	Contribution
300 BC	Aristotle	introduced Associationism, started the history of human's attempt to understand brain.
1873	Alexander Bain	introduced Neural Groupings as the earliest models of neural network, inspired Hebbian Learning Rule.
1943	McCulloch & Pitts	introduced MCP Model, which is considered as the ancestor of Artificial Neural Model.
1949	Donald Hebb	considered as the father of neural networks, introduced Hebbian Learning Rule, which lays the foundation of modern neural network.
1958	Frank Rosenblatt	introduced the first perceptron, which highly resembles modern perceptron.
1974	Paul Werbos	introduced Backpropagation
1980	Teuvo Kohonen	introduced Self Organizing Map
	Kunihiko Fukushima	introduced Neocogitron, which inspired Convolutional Neural Network
1982	John Hopfield	introduced Hopfield Network
1985	Hilton & Sejnowski	introduced Boltzmann Machine
1986	Paul Smolensky	introduced Harmonium, which is later known as Restricted Boltzmann Machine
	Michael I. Jordan	defined and introduced Recurrent Neural Network
1990	Yann LeCun	introduced LeNet, showed the possibility of deep neural networks in practice
1997	Schuster & Paliwal	introduced Bidirectional Recurrent Neural Network
	Hochreiter & Schmidhuber	introduced LSTM, solved the problem of vanishing gradient in recurrent neural networks
2006	Geoffrey Hinton	introduced Deep Belief Networks, also introduced layer-wise pretraining technique, opened current deep learning era.
2009	Salakhutdinov & Hinton	introduced Deep Boltzmann Machines
2012	Geoffrey Hinton	introduced Dropout, an efficient way of training neural networks

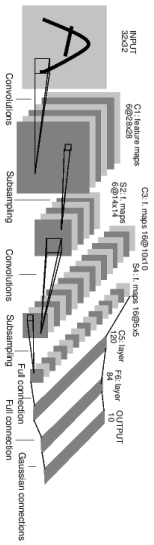
Deep Neural Network (DNN) Brief History before 2012



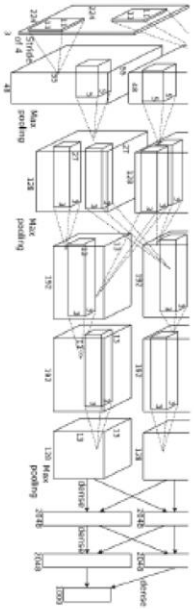
Revolution of Depth in CNN

There are several architectures in the field of Convolutional Networks

1998 LeNet
5

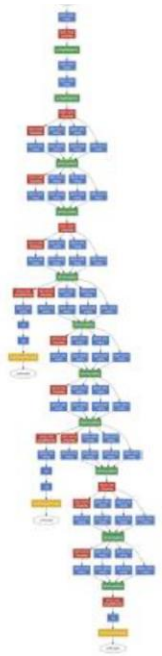


2012 AlexNet
8



2014 VGG
19

2014 GoogLeNet
22



2015 ResNET
152

Historical Note (Rosenblatt's Perceptrons)

Rosenblatt's early work on perceptrons at the Cornell Aeronautical Laboratory (1957-1959) culminated in the development and hardware construction of the Mark I Perceptron. Mark I, a visual pattern classifier, had an input (sensory) layer of 400 photosensitive units in a 20x20 grid modeling a small retina, an association layer of 512 units (stepping motors) each of which could take several excitatory and inhibitory inputs, and an output (response) layer of 8 units. The connections from the input to the association layer could be altered through plug-board wiring, but once wired they were fixed for the duration of an experiment. The connections from the association to the output layer were variable weights (motor-driven potentiometers) adjusted through the perceptron error-propagating training process. Mark I consisted of 6 racks (approximately 36 square feet) of electronic equipment and numerous experiments were conducted on this machine (can we describe some?). When Dr. Rosenblatt moved his research to Cornell University in 1959, Mark I was shipped to the Office of Naval Research (the funding agency) in Washington D.C. for demonstration purposes but was returned to Cornell rather than to the Aeronautical Lab. The Mark I Perceptron currently resides in the Smithsonian Institute.

Rosenblatt-Minsky Debates and Minsky-Papert Book:

During the late 1950s and early 1960s, much to the enjoyment of those in the audience, Rosenblatt and Minsky debated on the floors of scientific conferences the value of biologically inspired computation, Rosenblatt arguing that his neural networks could do almost anything and Minsky countering that they could do little. Minsky, wanting to decide the matter once and for all, began collaborating with Seymour Papert in the mid 1960s, and after much delay they published a book in 1969, *Perceptrons: An Introduction to Computational Geometry*, where they asserted about perceptrons (page 4), "Most of this writing ... is without scientific value..." Although Minsky was well aware that the powerful perceptrons have multiple layers and that Rosenblatt's basic feed-forward perceptrons have three layers, he defined a perceptron as a two-layer machine that can handle only linearly separable problems and, for example, cannot solve the exclusive-OR problem. Although considered a political maneuver (hatchet job) for contract funding by some knowledgeable scientists, this strong criticism of perceptrons essentially halted work on neural-based devices for a decade. [Note: Minsky and Rosenblatt knew of each other from their days at the Bronx High School of Science, graduating respectively in 1945 and 1946.]

Historical Note (Habbian Learning rule)

Hebbian theory is a theory in neuroscience that proposes an explanation for the adaptation of neurons in the brain during the learning process, describing a basic mechanism for synaptic plasticity, where an increase in synaptic efficacy arises from the presynaptic cell's repeated and persistent stimulation of the postsynaptic cell. Introduced by Donald Hebb in his 1949 book *The Organization of Behavior*, the theory is also called **Hebb's rule**, **Hebb's postulate**, and **cell assembly theory**. Hebb states it as follows:

Let us assume that the persistence or repetition of a reverberatory activity (or "trace") tends to induce lasting cellular changes that add to its stability.[…] When an axon of cell *A* is near enough to excite a cell *B* and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that *A*'s efficiency, as one of the cells firing *B*, is increased.

The theory is often summarized by Siegrid Löwel's phrase: "Cells that fire together, wire together." However, this summary should not be taken literally. Hebb emphasized that cell A needs to "take part in firing" cell B, and such causality can only occur if cell A fires just before, not at the same time as, cell B. This important aspect of causation in Hebb's work foreshadowed what is now known about spike-timing-dependent plasticity, which requires temporal precedence. The theory attempts to explain associative or **Hebbian learning**, in which simultaneous activation of cells leads to pronounced increases in synaptic strength between those cells, and provides a biological basis for errorless learning methods for education and memory rehabilitation. In the study of neural networks in cognitive function, it is often regarded as the neuronal basis of unsupervised learning.

Neural Network Applications

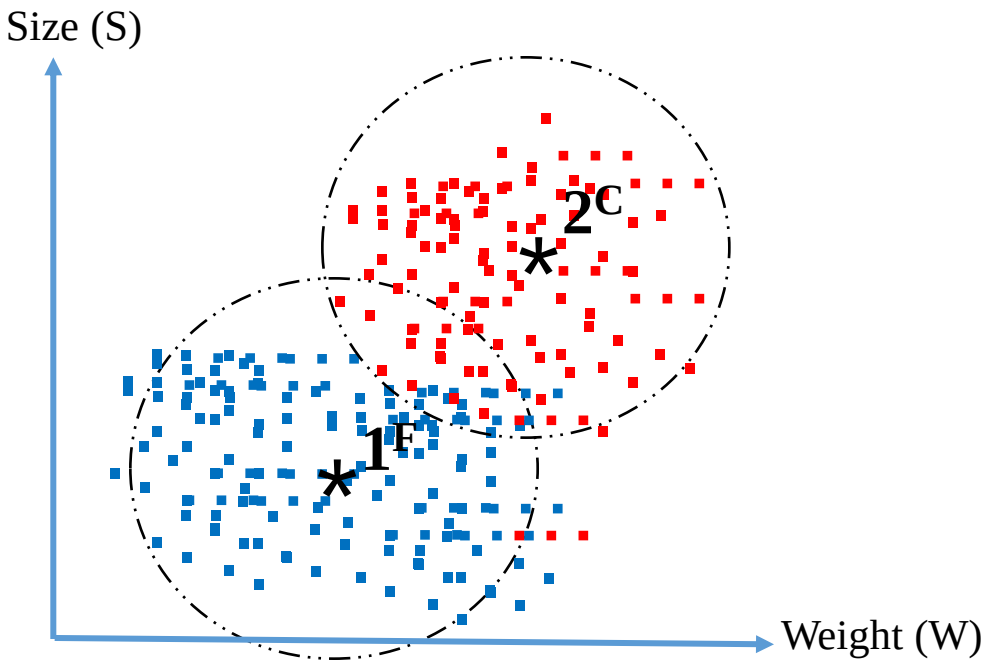
- Face recognition- Recognizing faces for biometrics applications
- Financial modelling – predicting the stock market Time
- Series prediction – climate, weather, seizures
- Computer games – intelligent agents, chess, backgammon
- Robotics – autonomous adaptable robots
- Pattern recognition – speech recognition, seismic activity, sonar signals, object classification, target detection and classification
- Data analysis – data compression, data mining
- Bioinformatics – DNA sequencing, alignment
- Biometrics – face, iris, fingerprint, signature, character recognition

Supervised and Unsupervised Learning

1. Supervised training requires the pairing of each input vector with a target vector representing the desired output. An input vector is applied, and the output of the network is calculated which is compared with the corresponding target vector. The difference (error) is feedback through the network and its weights are changed according to an algorithm that tends to minimize the error.
2. Unsupervised learning requires no target vector for the outputs. The training set consists solely of input vectors. The training algorithm modifies network weights to produce out-put vectors that are consistent. The training process extracts the statistical properties of the training set and groups similar input vectors into classes. Unsupervised training is a far more plausible model of learning in the biological system.
3. Semi-supervised Learning – a combination of supervised (labelled data) and unsupervised (unlabeled data) learning.

Pattern Recognition- Supervised and Unsupervised Classification Problems

- Consider a set of data, i.e., apples from Florida and California. Let us measure their weight W and size S. Therefore, we have only two classes to categorize any new data.

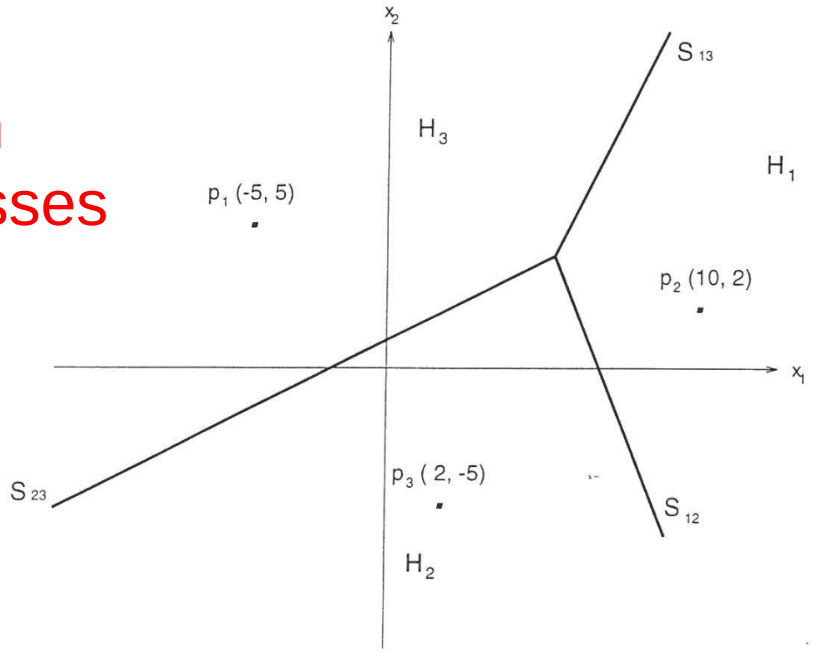


- The new data (w,s) of an apple is compared with the centroid of the cluster.
- The centroid represents the standard (pro-type) features.

Minimum – Distance Classifier

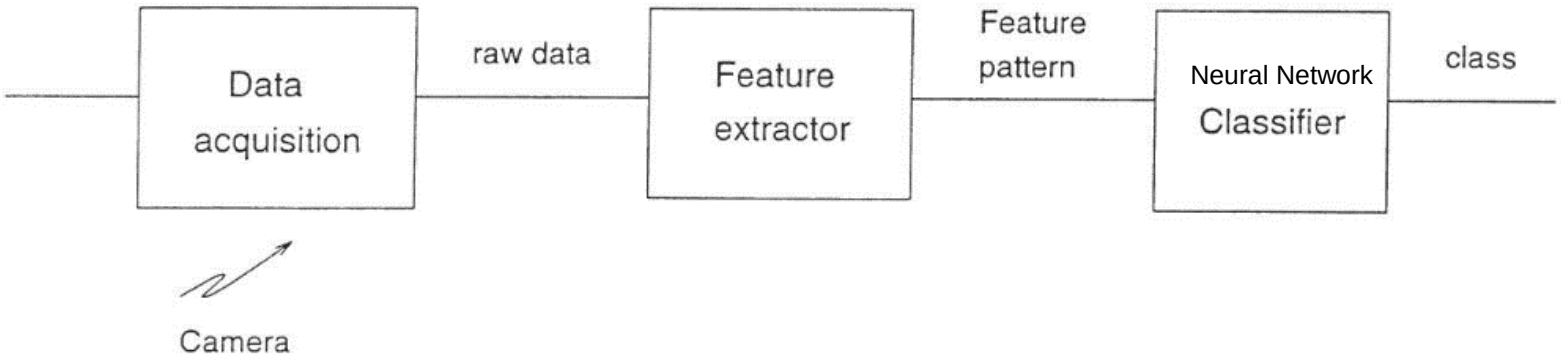
- When the decision boundaries are of linear form the classifier is called linear classifier.
- A linear classifier is also sometimes called a linear machine, also called correlation classification.
- Consider the example shown below, the feature vector is a 2- dimensional pattern and the number of classes is three.
- The centroids (center of Gravity of all points) of the clusters (classes) are the points $P_1(-5,5), P_2(10,2),$ and $P_3(2,-5)$ and S_{12}, S_{13} and S_{23} and decision boundaries.
- This n-dimensional space is called the pattern space.

In this diagram
we have 3 classes



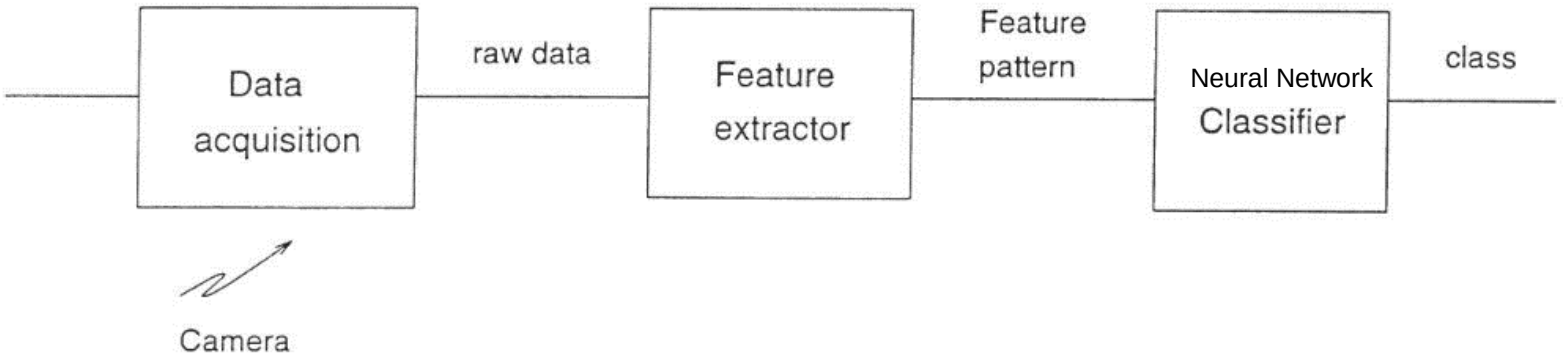
A Typical Classifier

- A useful task that a neural network can perform is pattern classification.
- A pattern feature is the quantitative description of an object.
- Patterns could be stationary or static, such as pictures, video images and speech signal.
- A typical pattern classifier is shown below.
- Data, such as pictures, speech signal, radar signal and diagnosis signals.
- Feature extractor, finds useful attributes about the input signal.



A Typical Classifier

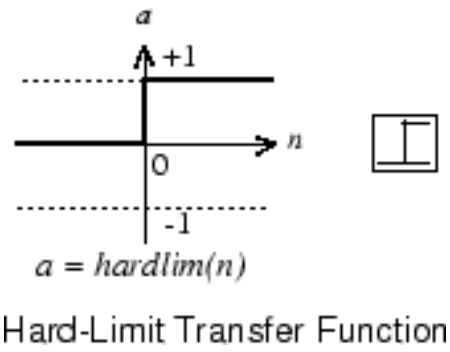
- The classifier identifies to which class the input feature belongs to.
- The feature pattern is a vector $X = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \in R^n$ of dimension *n*.
- Therefore, the input feature pattern is a point in the *n*-dimensional space R^n .



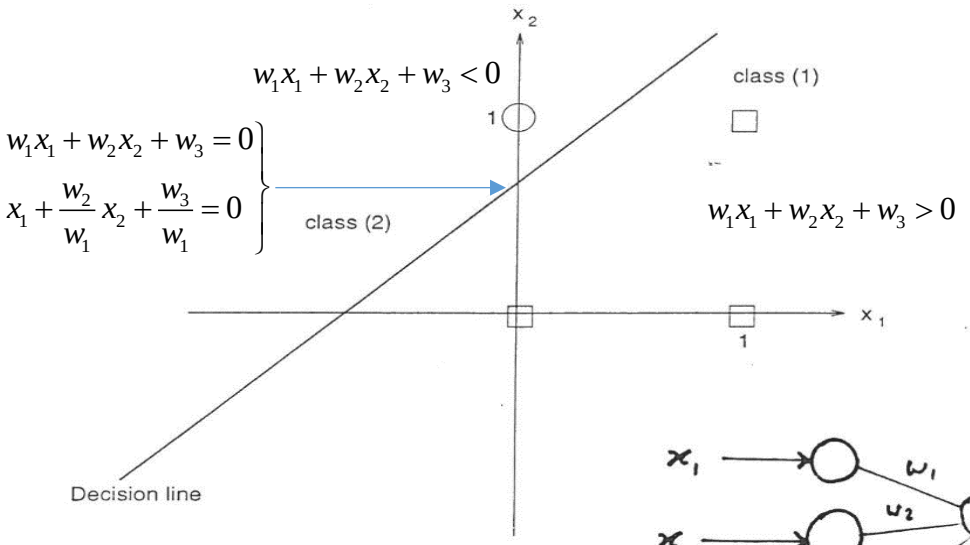
Linearly Separable AND/OR Logic Gate (Binary Classification)

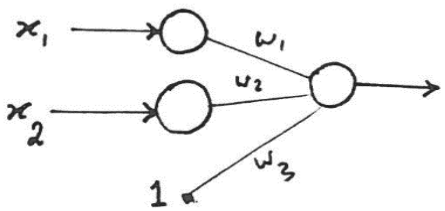
- Consider a two-dimensional input $[x_1, x_2]$ to be classified into two categories (clusters, classes).
- Let input be $[x_1, x_2] \in [0,1]$
- Here $\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \Rightarrow \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \end{bmatrix} \in$ class 1 (0)

$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \Rightarrow \begin{bmatrix} 0 \\ 1 \end{bmatrix} \in$ class 2 (1)



Hard-Limit Transfer Function

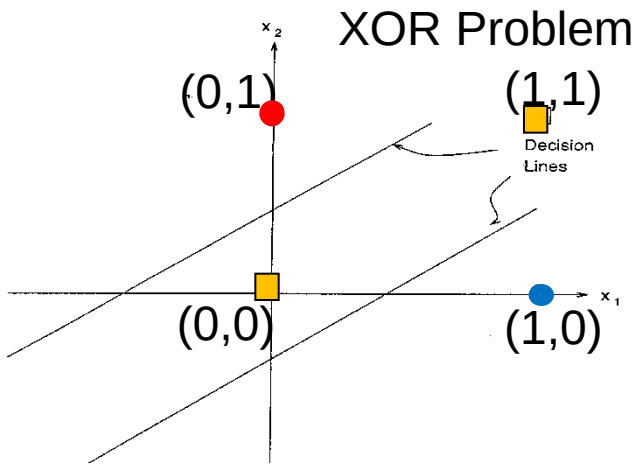




- A single line can divide the data space into two classes; therefore, it is said to be linearly separable.

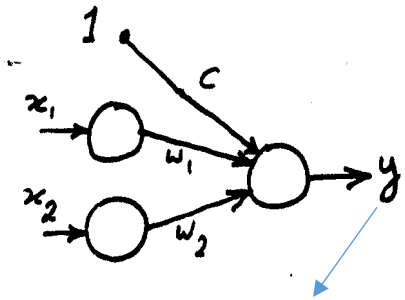
Non-separable XOR-Problem

- Consider the XOR problem which is a non-separable task.
- We need two lines to perform the classification as shown in this example. No one single line can do the classification.

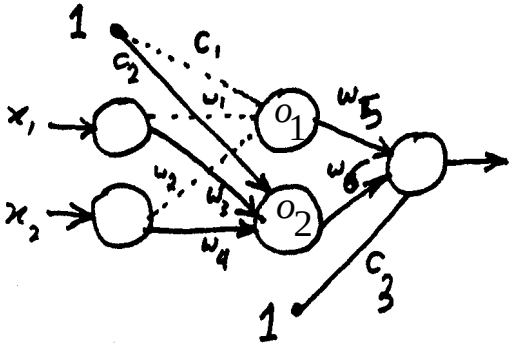


$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \Rightarrow \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix} \in \textit{class } 0$$

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \Rightarrow \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix} \in \textit{class } 1$$



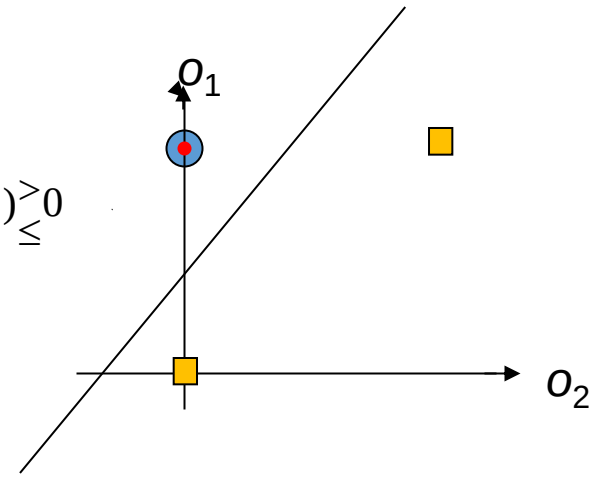
$$y = f(w_1 x_1 + w_2 x_2 + c_1) \underset{\leq}{>} 0$$



$$y = f(w_5 o_1 + w_6 o_2 + c_3) \underset{\leq}{>} 0$$

$$O_1 = f(w_1 x_1 + w_2 x_2 + c_1) \underset{\leq}{>} 0$$

$$O_2 = f(w_3 x_1 + w_4 x_2 + c_2) \underset{\leq}{>} 0$$



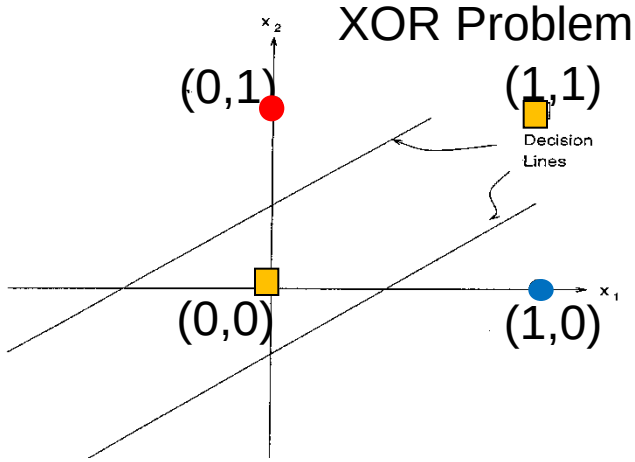
Non-separable XOR-Problem -Example

- Consider the XOR problem:

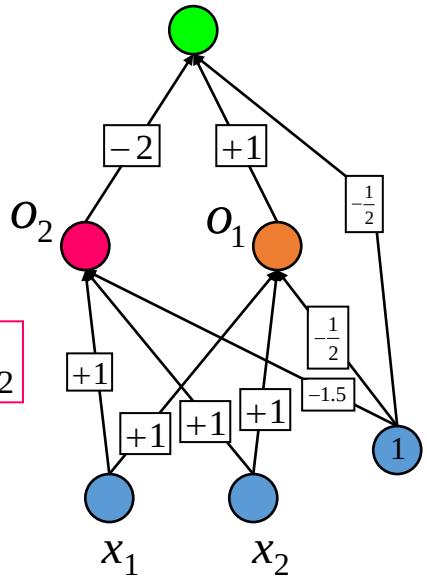
$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \Rightarrow \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix} \in class\ 0$$

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \Rightarrow \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix} \in class\ 1$$

x_1	x_2	o_1	o_2	*out*
0	0	0	0	0
1	1	1	1	0
0	1	1	0	1
1	0	1	0	1

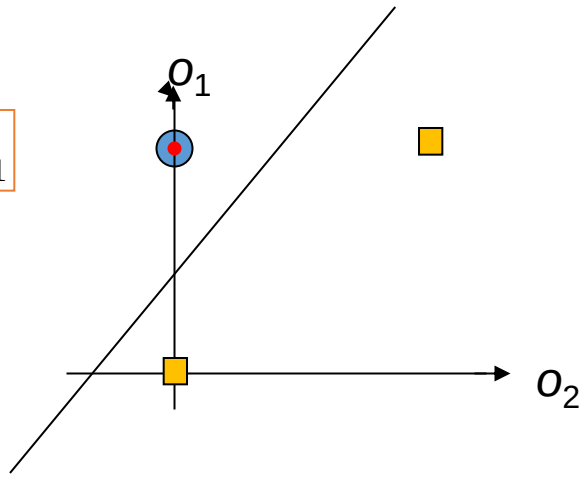


$$out = o_1 - 2o_2 - \frac{1}{2}$$

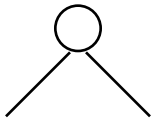
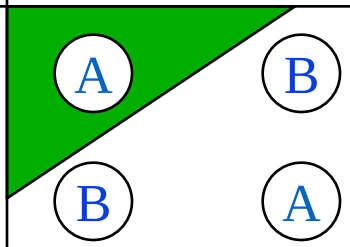
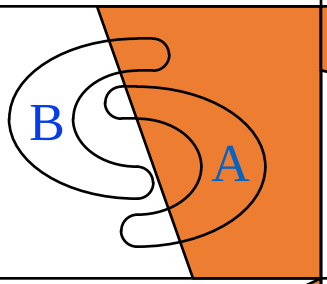
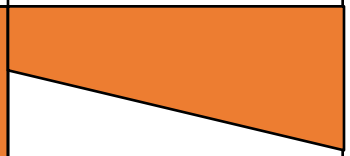
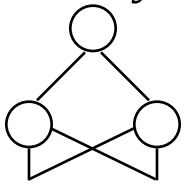
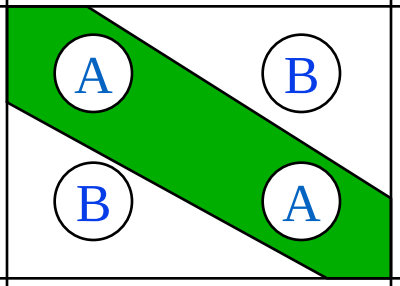
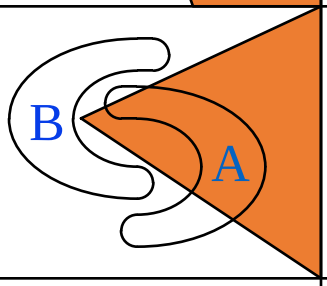
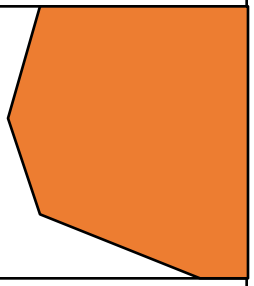
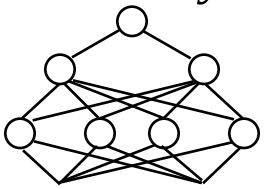
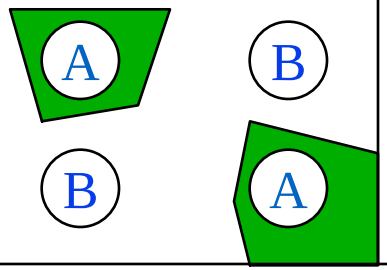
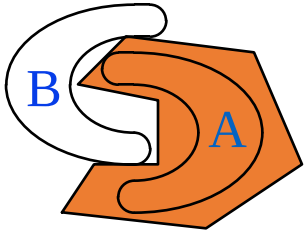
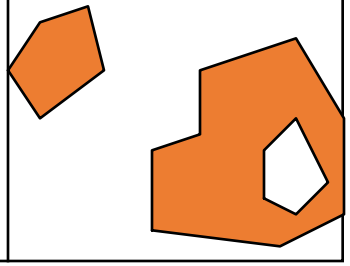


$$-1.5 + x_1 + x_2 = o_2$$

$$-0.5 + x_1 + x_2 = o_1$$

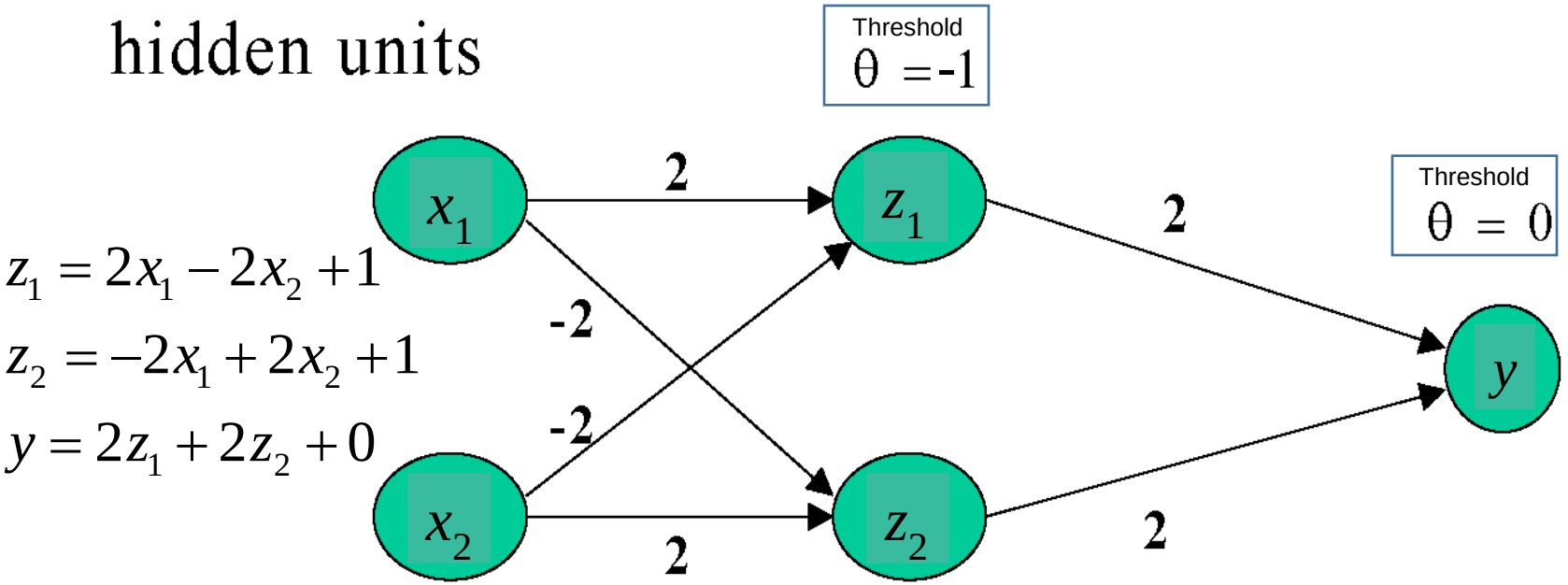


Different non-linearly separable problems

Structure	*Types of Decision Regions*	*Exclusive-OR Problem*	*Classes with Meshed regions*	*Most General Region Shapes*
Single-Layer 	*Half Plane Bounded By Hyperplane*			
Two-Layer 	*Convex Open Or Closed Regions*			
Three-Layer 	Abitrary (Complexity Limited by No. of Nodes)			

A Three Layer Perceptron for XOR

- Note **Bipolar pattern** is used for input
  - XOR can be solved by a more complex network with hidden units



(-1, -1)	(+1,+1)	+1
(-1, 1)	(-1, +1)	-1
(1, -1)	(+1, -1)	-1
(1, 1)	(+1,+1)	+1

Single-Layer Perceptron

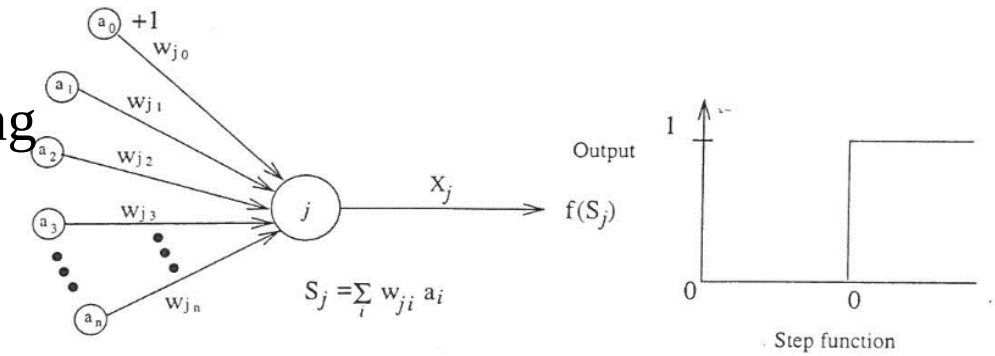
- Rosenblatt (1957) constructed a class of processing unit that transmitted signals and adapted their interconnection weights.
- Input patterns are binary valued (0 or 1) and the output of each unit is also binary valued (0 or 1).
- Each perceptron processing unit *j* performs a weighted sum of its input values (**McCulloch–Pitts (MCP) neuron model)**

$$S_j = \sum_{i=0}^{I} a_i w_{ij}$$

- The output o_j of unit *j* is given by:

$$o_j = f(S_j) = \begin{cases} 0 & \text{if} \quad S_j \le 0 \\ 1 & \text{if} \quad S_j > 0 \end{cases}$$

- where $f(S_j)$ is a non-liner function, such as hard limiting function
- The bias input a_0 is always fixed at the value of +1.



Perceptron processing unit.

OUTPUT OF $j = X_j$ { 0 IF $S_j \le 0$; 1 IF $S_j > 0$

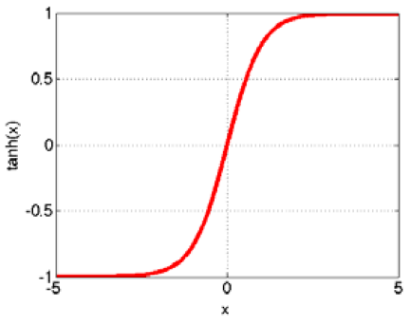
Possible Nonlinearity Functions for each Neuron

Options:

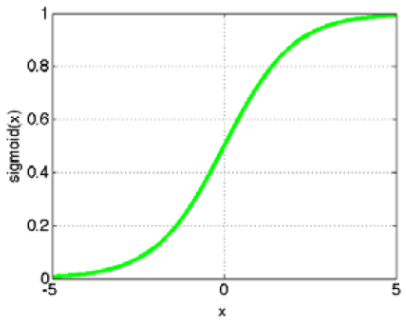
- Tanh
- Sigmoid: 1/(1+exp(-x))
- Rectified linear unit (ReLU)
  - Simplifies backpropagation
  - Makes learning faster
  - Avoids saturation issues
    → Preferred option

i.e., RELU layer will apply an elementwise activation function, such as the *Max*(**0, x**) (thresholding at zero)

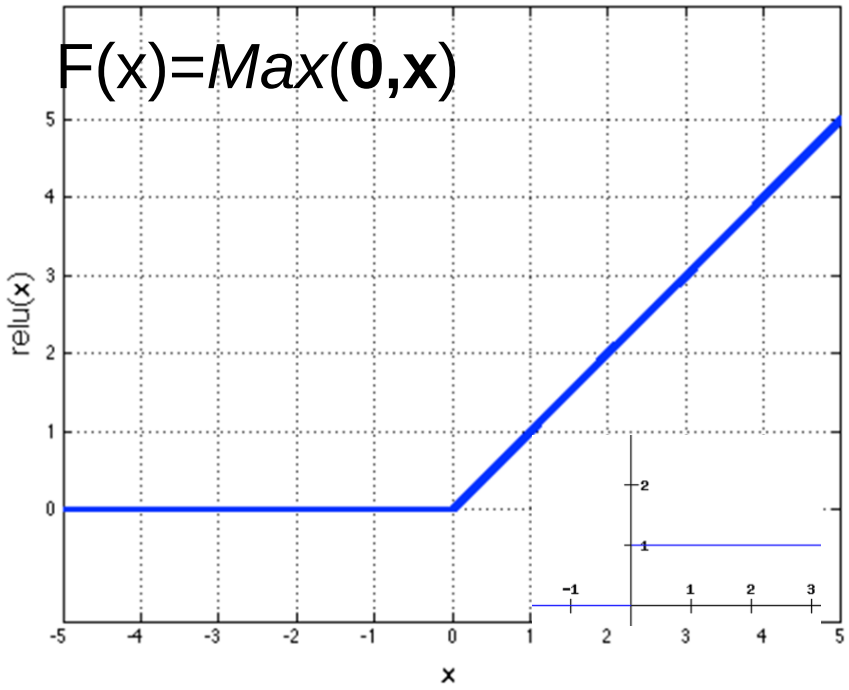
F(**x**)=Tanh(**x**)



1/(1+exp(-**x**))

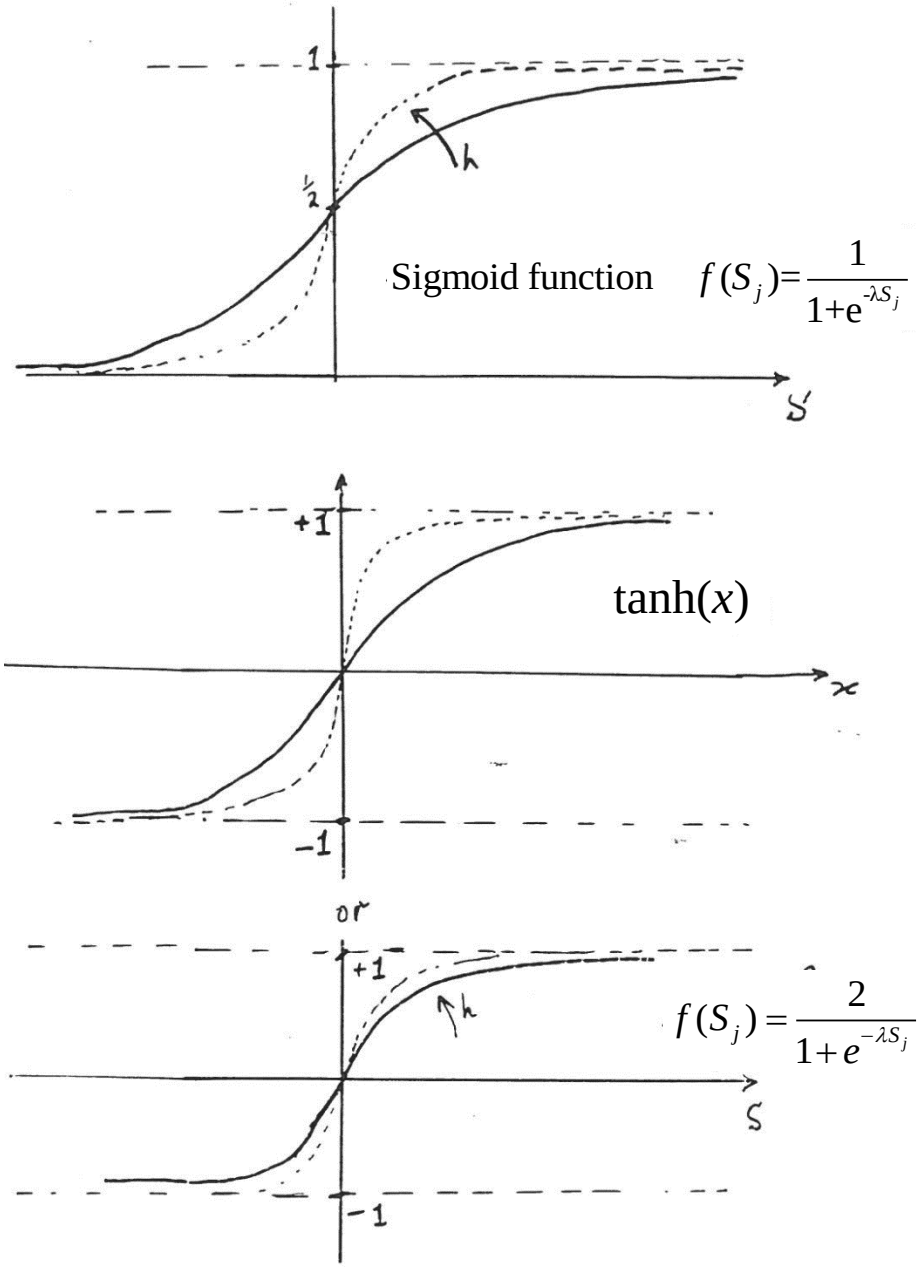


F(x)=*Max*(**0,x**)



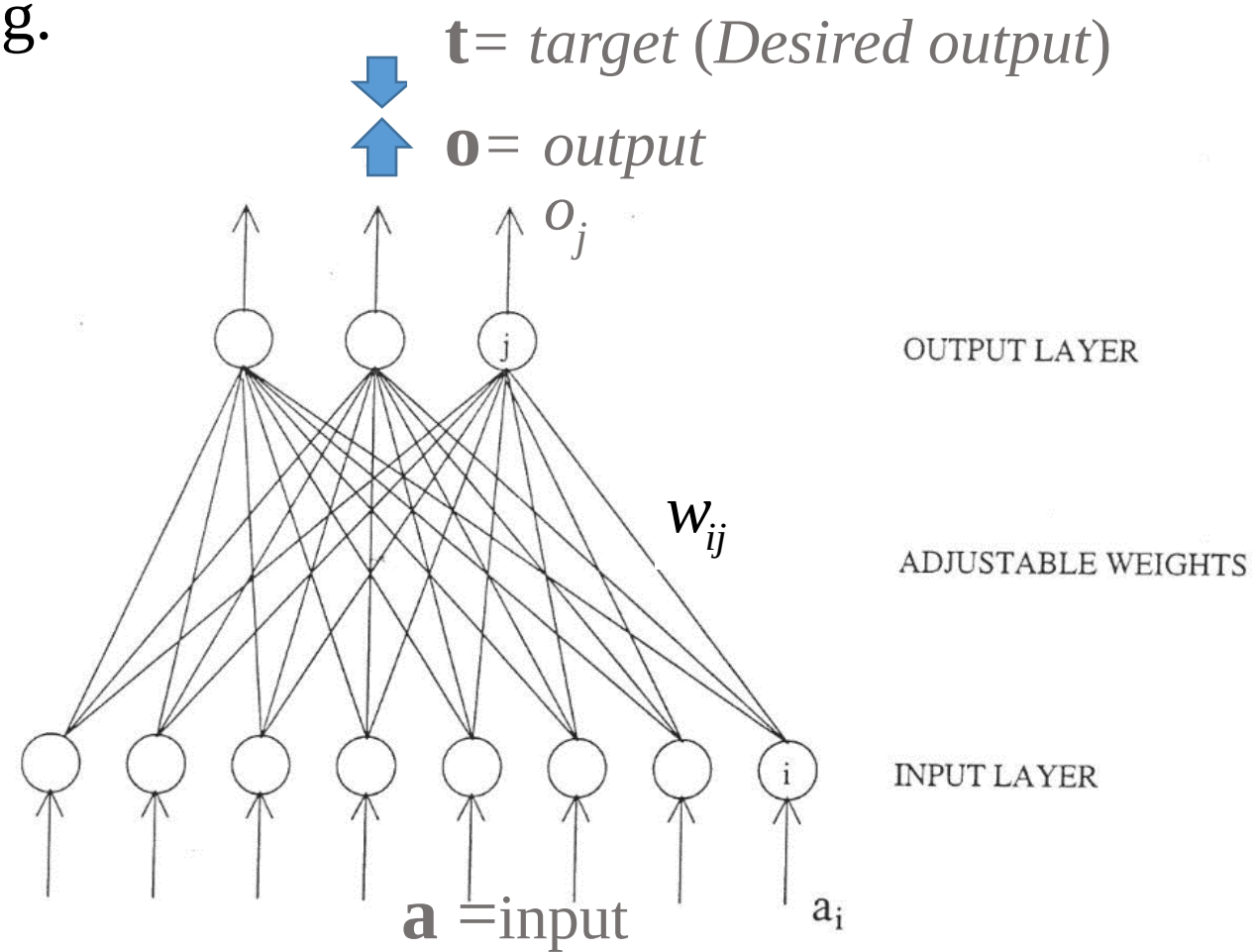
Non-Linear Logistic Function

- The bias unit acts as a constant value in the summation, it will shift the function to left or right.
- The non-linear (logistic) function could be a Sigmoid, tanh(x), or a step function.



An Example of a Single-Layer (Two-Layer) Discrete Perceptron

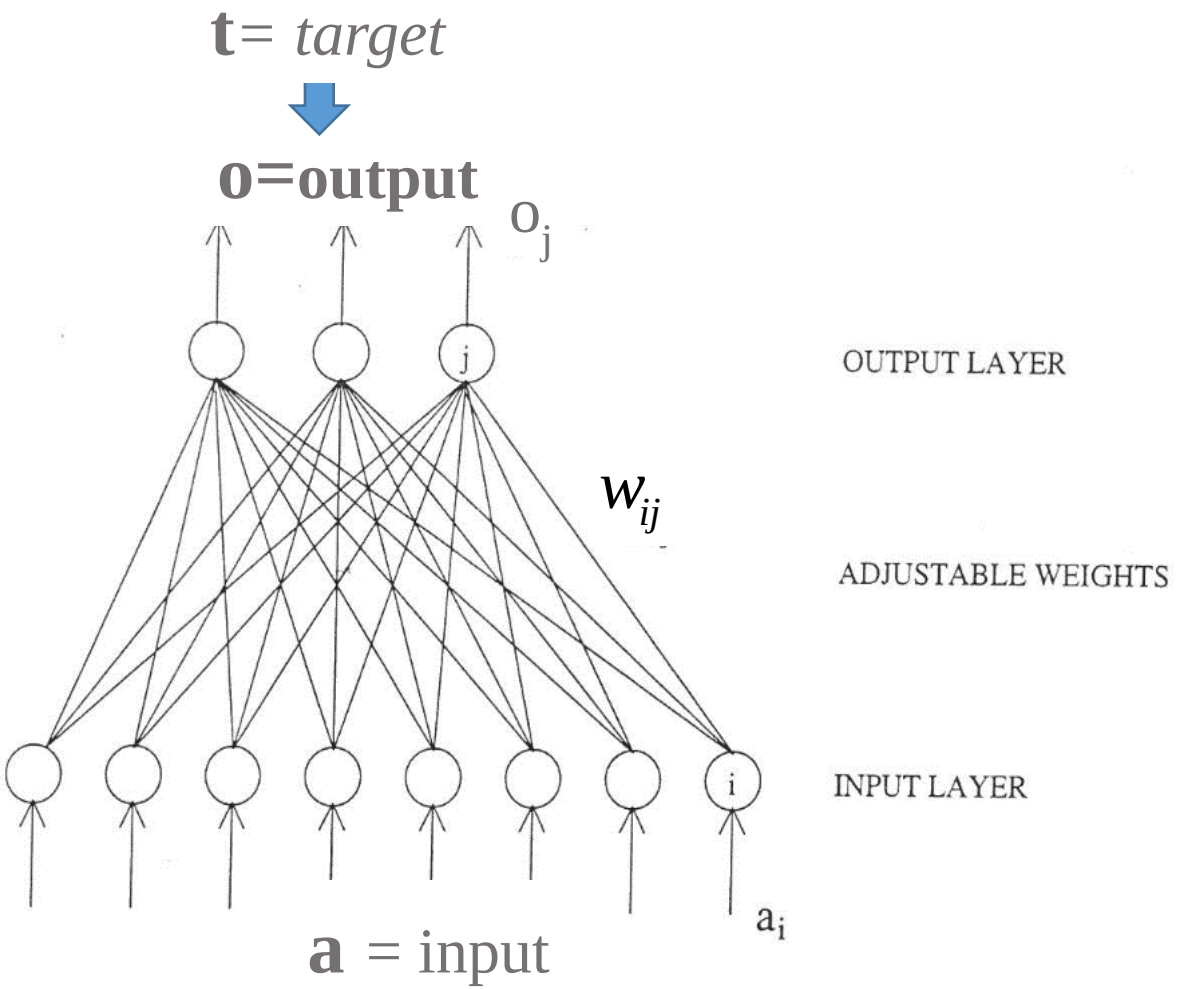
- There is only one layer of adjustable weights w_{ij} between the input layer (first layer) and the output layer (second layer).
- This network learns to classify patterns through a supervised learning.



An Example of a Single-Layer (Two-Layer) Discrete Perceptron

- The weights are obtained by training the network. They are adjusted, such that the network would produce the desired output.
- The network is first trained and then used to perform classification tasks.

- The weights w_{ij} are adjusted by a learning rule, using a set of training patterns $\mathbf{a}_p$.
- Learning is usually supervised or unsupervised.



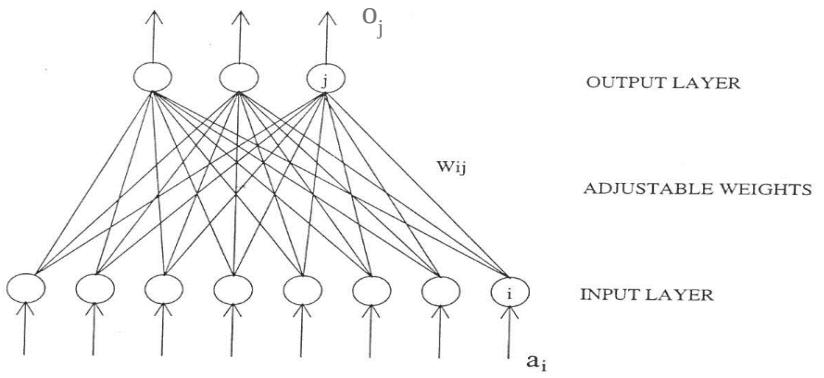
A Simple Learning Rule for a Single Layer Discrete Perceptron (Binary Input & Output)

- A constant is added or subtracted from the appropriate weights during learning:

$$w_{ij}^{t+1} = w_{ij}^{t} + \Delta w_{ij}$$

$$w_{ij}^{t+1} = w_{ij}^{t} + \eta \underbrace{(t_j - o_j)}_{\delta_j} a_i$$

$$w_{ij}^{t+1} = w_{ij}^{t} + \eta \delta_j a_i$$



where η is small constant between 0 to 1 which determines the learning rate, a_i is a binary input (0/1) at input node i, o_j is the output value (0/1) of node j, t_j is the target value (desired output of node j), and finally:

$$(t_j - o_j) = \begin{cases} 1 \text{ if } t_j = 1 \text{ and } o_j = 0 \\ 0 \; t_j = o_j \\ -1 \text{ if } t_j = 0 \text{ and } o_j = 1 \end{cases}$$

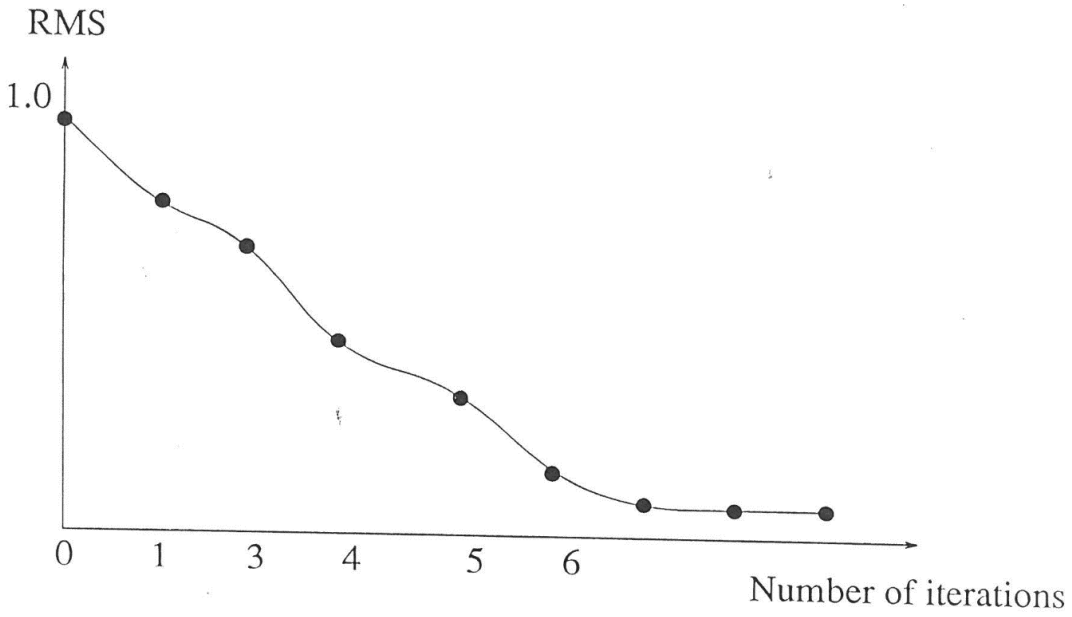
A Simple Learning Rule for a Discrete Perceptron

- As the weights are adjusted the networks performance improves.
- Performance is measured by a Root-Mean-Square (RMS) error value,

$$RMS = \sqrt{\frac{\sum_{p}^{n_p}\sum_{j}^{n_0}(t_{jp} - o_{jp})^2}{n_p n_0}}$$

where n_p = number of patterns in the training set and n_o = number of units in the output layer.

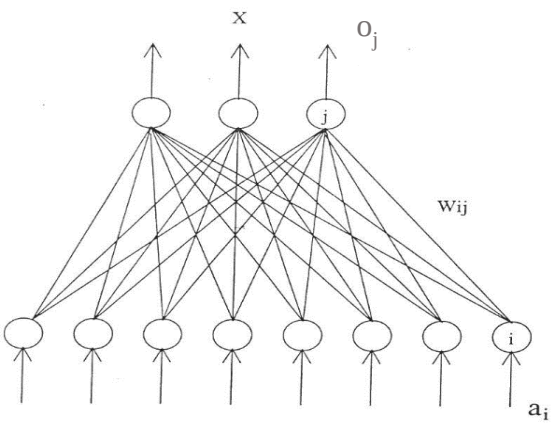
- Training process is stopped when RMS is very small.

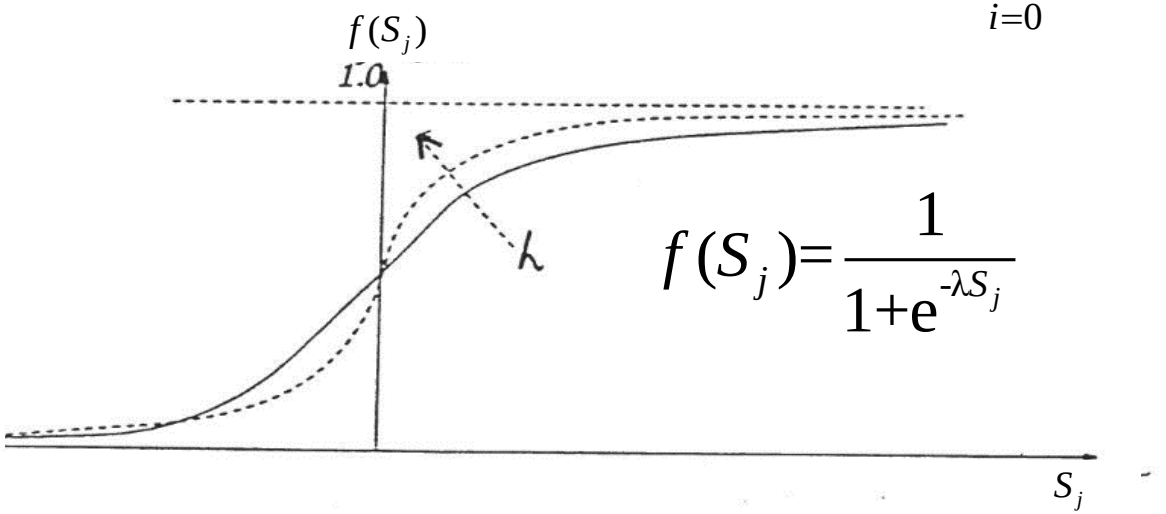


Learning for a Continuous Perceptron

- If the input and output patterns have continuous between 0 and 1 then these patterns are unipolar patterns {0,1}.
- If the input and output patterns have values between 0 and 1 then the bipolar patterns are used {-1,1}.
- The hard limiting function in the discrete perceptron is replaced with a continuous function, such as Sigmoid function.
- For the output values to be between 0,1 then the Sigmoid function is given by:

$$S_j = \sum_{i=0}^{I} w_{ij} a_i$$



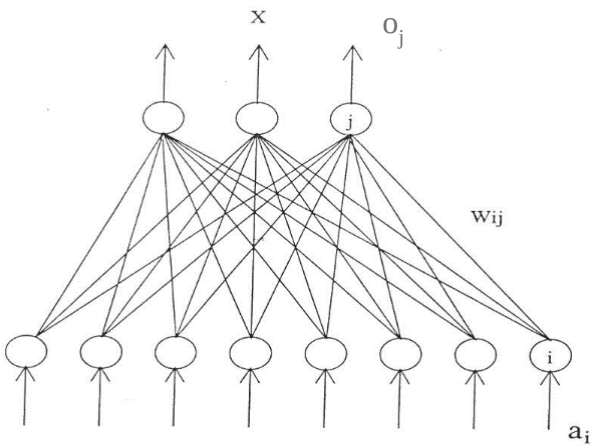


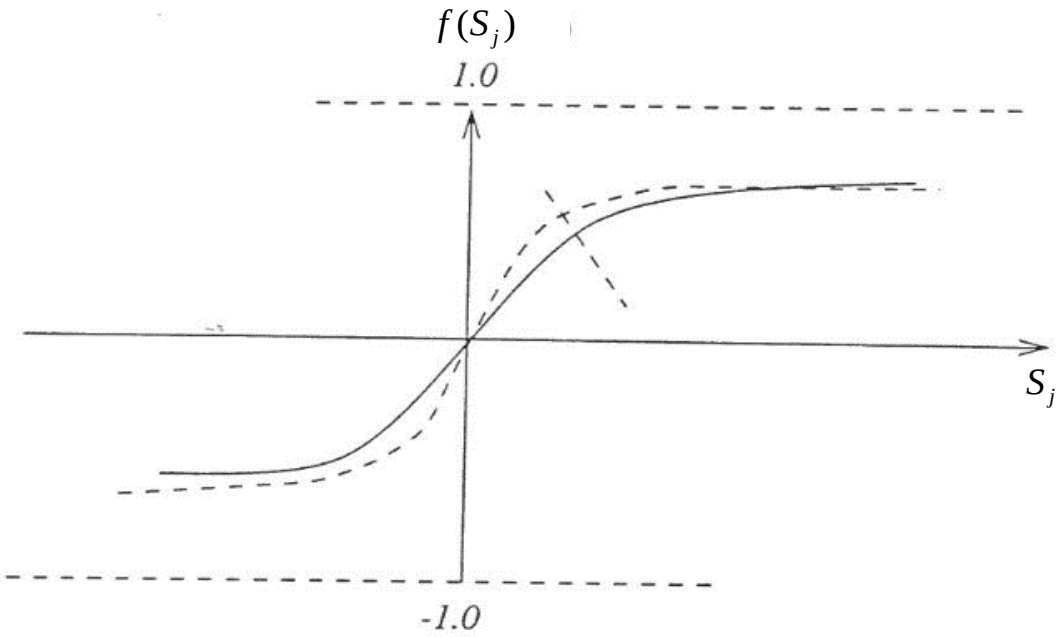
$$f(S_j) = \frac{1}{1+e^{-\lambda S_j}}$$

Learning for a Continuous Perceptron with Bipolar Patterns

- If output values are between {-1,1} then the Sigmoid function is given by

$$f(S_j) = \frac{2}{1+e^{-\lambda S_j}} - 1 \qquad S_j = \sum_{i=0}^{I} w_{ij} a_i$$

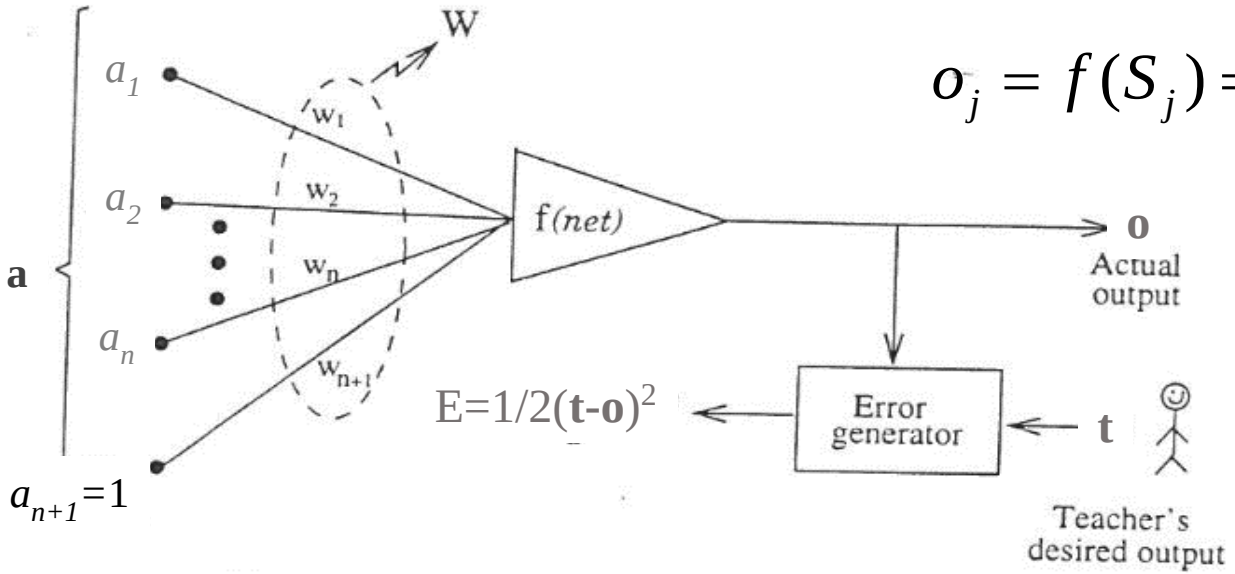




Learning for a Continuous Perceptron

- Consider a simple perceptron consisting of a single unit as shown below, let the input pattern vector be donated by **a** and the output vector **o**.

$$\mathbf{a} = \begin{bmatrix} a_1 \\ \vdots \\ a_{n+1} = 1 \end{bmatrix}$$

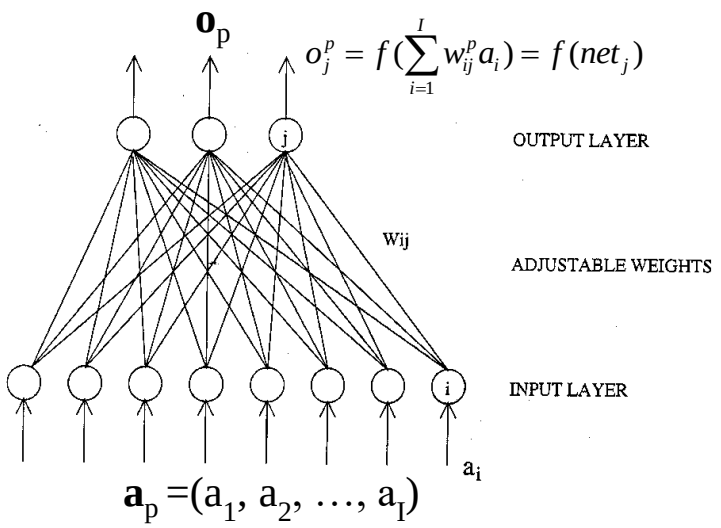


$$o_j = f(S_j) = f(\sum_{i=0}^{I} w_{ij} a_i)$$

Continuous perceptron trained using squared error $E=1/2(\mathbf{t}-\mathbf{o})^2$ Minimization.

Learning for a Continuous Perceptron

- We can adjust the weights, in such a manner that the error between the actual output $\mathbf{o}_p$ and the desired output $\mathbf{t}_p$ is minimized.



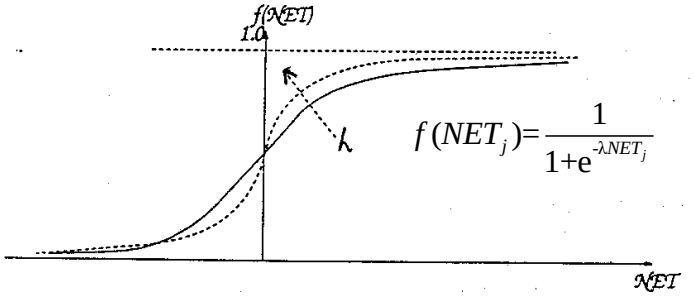
- Define the error function E_p, when presenting an input $\mathbf{a}_p$ to the network, at t^{th} training step to be

$$E_p^t = \tfrac{1}{2}(\mathbf{t}_p - \mathbf{o}_p)^2$$

$$E_p^t(\mathbf{W}) = \tfrac{1}{2}(\mathbf{t}_p - f(\mathbf{W}\mathbf{a}_p))^2$$

- This is the error due to one input presentation $\mathbf{a}_p$, we could sum overall the presentations (training set or a batch),

$$E_{total}(\mathbf{W}) = \sum_{p=1}^{N} E_p(\mathbf{W})$$



- We need to minimize $E_p(\mathbf{W})$ with respect to $\mathbf{W}$. The algorithm is the well-known least mean square (LMS) error.

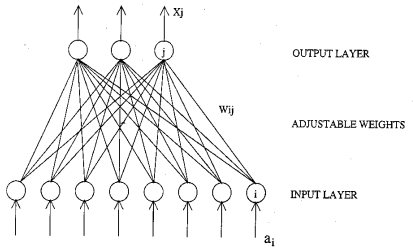
Learning for a Continuous Perceptron

- A simple weight adjustment would be to move the weights in the negative gradient direction (gradient descent algorithm) on the multi-dimensional error surface

$$\mathbf{W}^{t+1} = \mathbf{W}^t - \eta \Delta E_p(\mathbf{W}^t) = \mathbf{W}^t - \eta \frac{\partial E_p}{\partial \mathbf{W}^t} = \mathbf{W}^t - \eta \frac{\partial}{\partial \mathbf{W}} \left\{ \frac{1}{2} (\mathbf{t}_p - f(\mathbf{W}^t \mathbf{a}_p))^2 \right\}$$

- In component form:

$$w_{ij}^{t+1} = w_{ij}^t - \eta \Delta E_p(w_{ij}^t) = w_{ij}^t - \eta \frac{\partial E_p}{\partial w_{ij}^t} = w_{ij}^t - \eta \frac{\partial}{\partial w_{ij}^t} \left\{ \frac{1}{2} (t_i - \underbrace{f(\sum_{i=1}^{I} w_{ij}^t a_i)}_{o_j})^2 \right\}$$



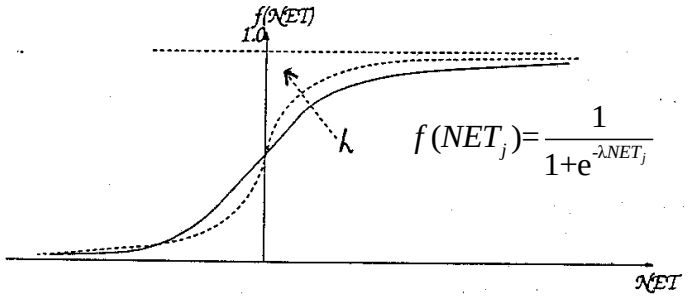
$$\Delta E_p(w_{ij}^t) = \frac{\partial E_p}{\partial w_{ij}^t} = -(t_j - o_j) f'(net_j) \frac{\partial (net_j)}{\partial w_{ij}^t}$$

and $net_j = \sum_{i=1}^{I} w_{ij}^t a_i^p$, $\therefore$ $\frac{\partial net_j}{\partial w_{ij}^t} = a_i^p$ which is the i^{th} input node for input pattern p.

- So the weight adjustment equation is given as

$$w_{ij}^{t+1} = w_{ij}^t + \eta \overbrace{(t_j^p - o_j^p) f'(net_j^p)}^{\delta_j} a_i = w_{ij}^t + \eta \delta_j a_i$$

but $f'(net_j) = f(net_j)(1 - f(net_j)) = o_j(1 - o_j)$

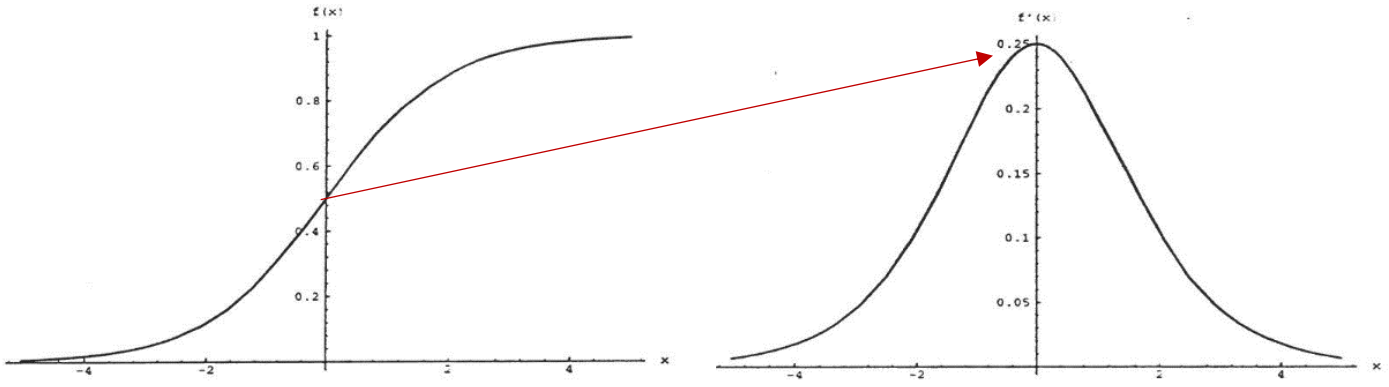


Sigmoid Function Characteristics

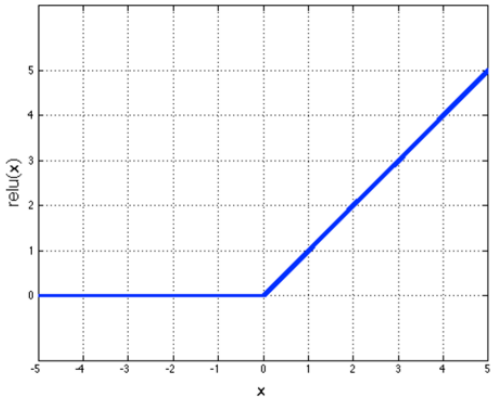
- Note the derivative, $o_{pj}(1 - o_{pj})$, reaches zero or one.
- Therefore, the amount of change in weights is most for those units that are near their mid range and, in some sense, not yet committed to being either on or off.

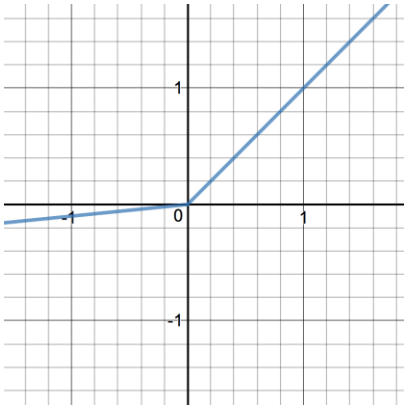
$$w_{ij}^{t+1} = w_{ij}^{t} + \eta \overbrace{(t_j^p - o_j^p) f'(net_j^p)}^{\delta_j} a_i = w_{ij}^{t} + \eta \delta_j a_i$$

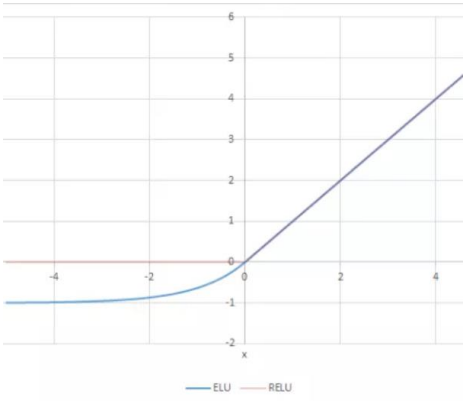
but $f'(net_j) = f(net_j)(1 - f(net_j)) = o_j(1 - o_j)$

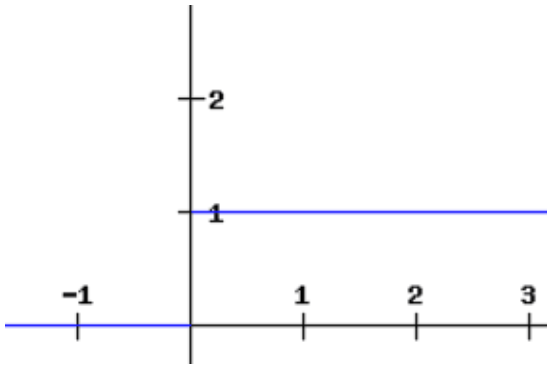


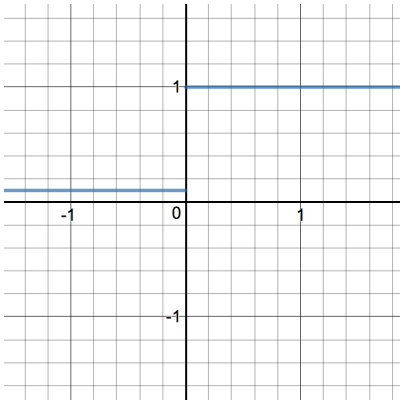
RelU & LeakyRelU & ELU Function Characteristics

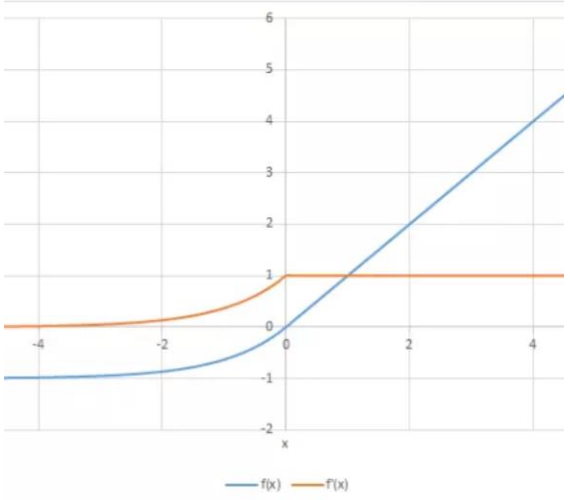








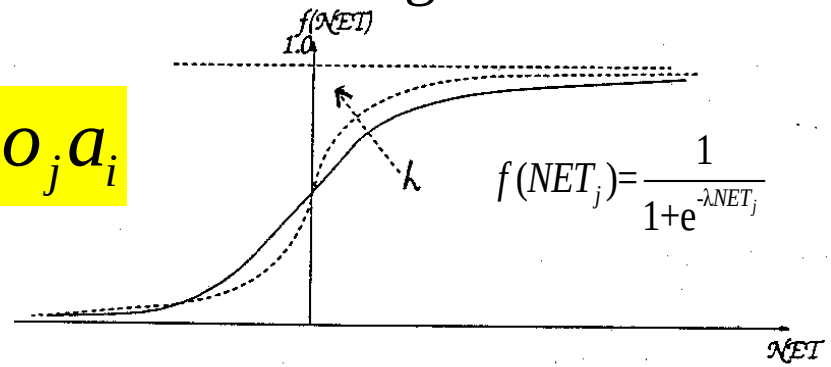




Learning for a Continuous Perceptron

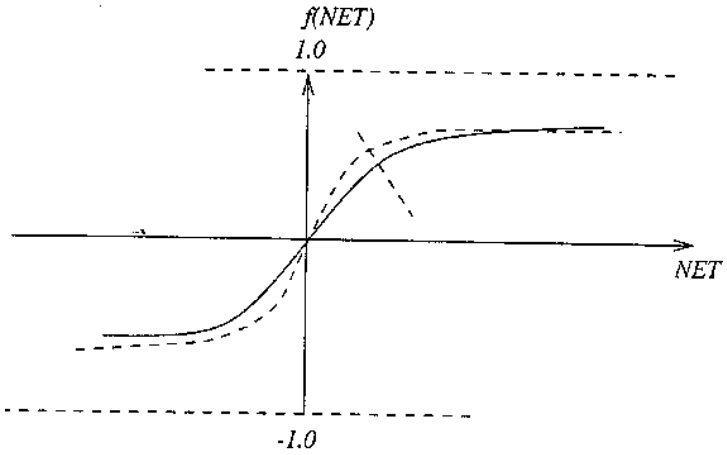
- The weight adjustment now for Sigmoid function is given as

$$w_{ij}^{t+1} = w_{ij}^{t} + \eta(t_j - o_j)(1-o_j)o_j a_i$$



- If the activation function is tanh(.)

$$f(net) = \frac{2}{1+e^{\lambda net}} - 1$$



- Then

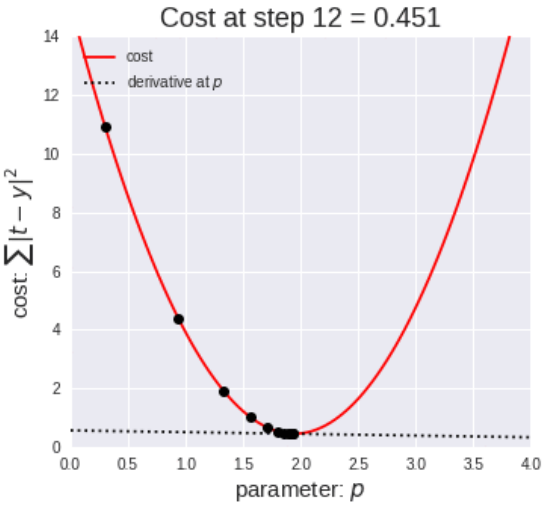
$$f'(net) = \frac{1}{2}(1 - f(net)^2)$$

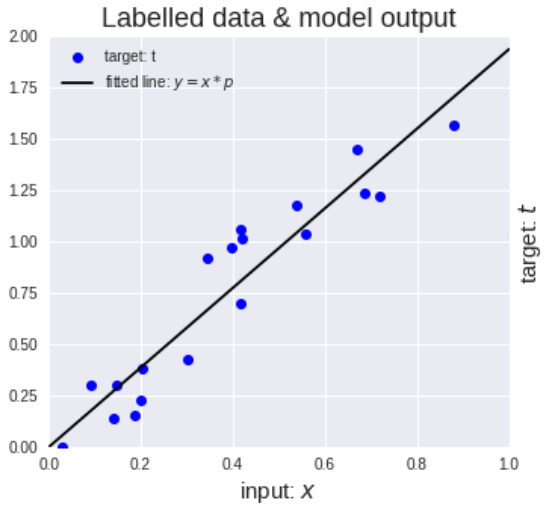
- Then the weight adjustment is now

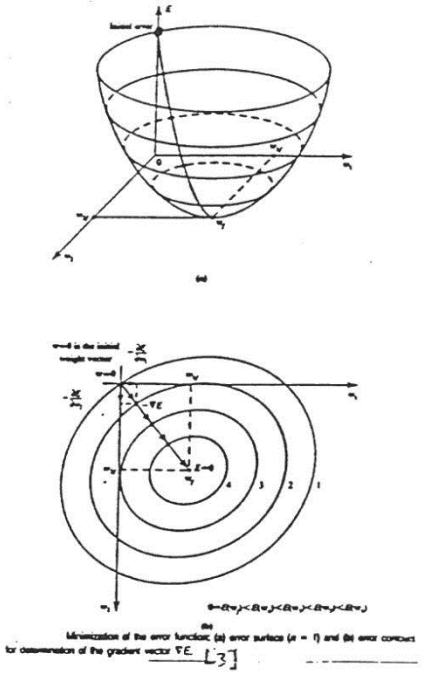
$$w_{ij}^{t+1} = w_{ij}^{t} + \tfrac{1}{2}\eta(t_j - o_j)(1-o_j^2)a_i$$

Error Surface for LMS

- Consider the error surface in terms of two weights $E(w_1, w_2)$ the surface is in a shape of a concave bowl.







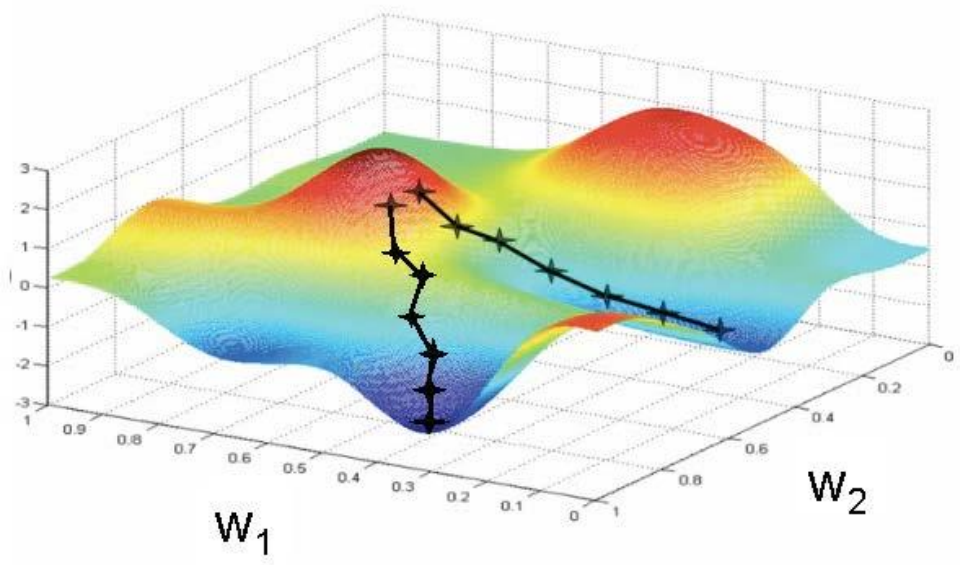
- The updating procedure moves on the error surface along the negative gradient.
- As shown in the above figure we move along the negative gradient on the error surface until we reach to a point $E(W_f)$ that is the global minimum.
- In more complicated tasks with multidimensional parameters the error surface has many local minima and the gradient descent algorithm could get trapped into one of the local minimum.

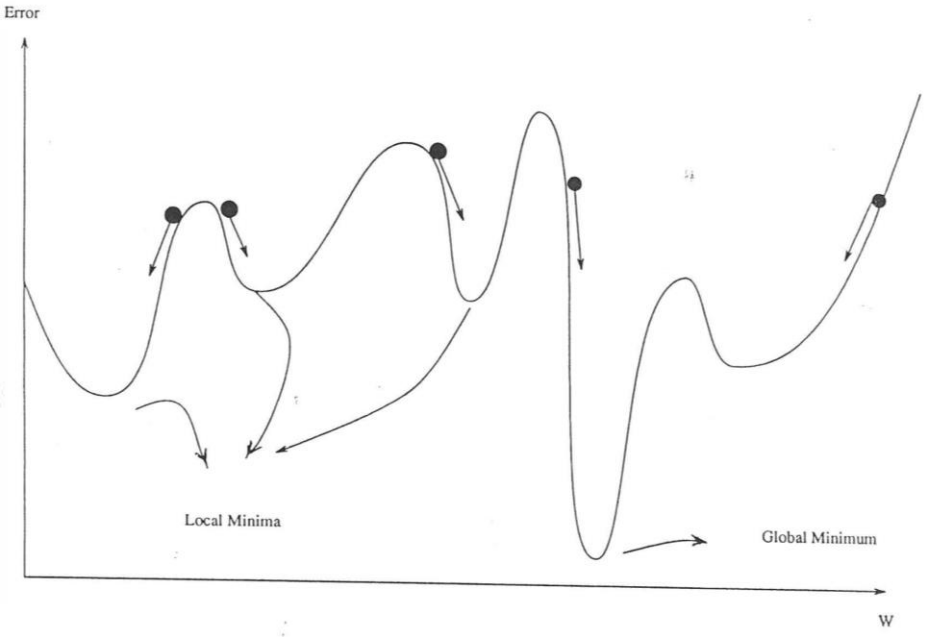
Error Surface with Local Minimums

$$E_p^t = \tfrac{1}{2}(\mathbf{t}_p - \mathbf{o}_p)^2$$

$$E_p^t(\mathbf{W}) = \tfrac{1}{2}(\mathbf{t}_p - f(\mathbf{W}\mathbf{a}_p))^2$$

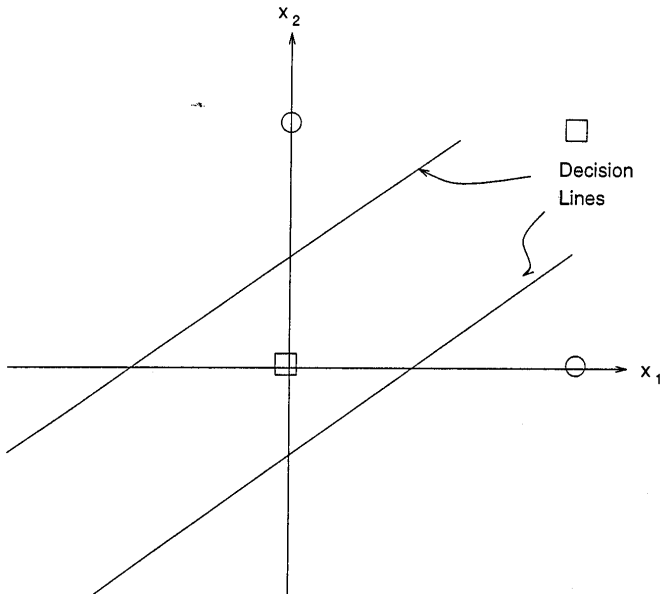
Imagine a Multi-dimensional Space

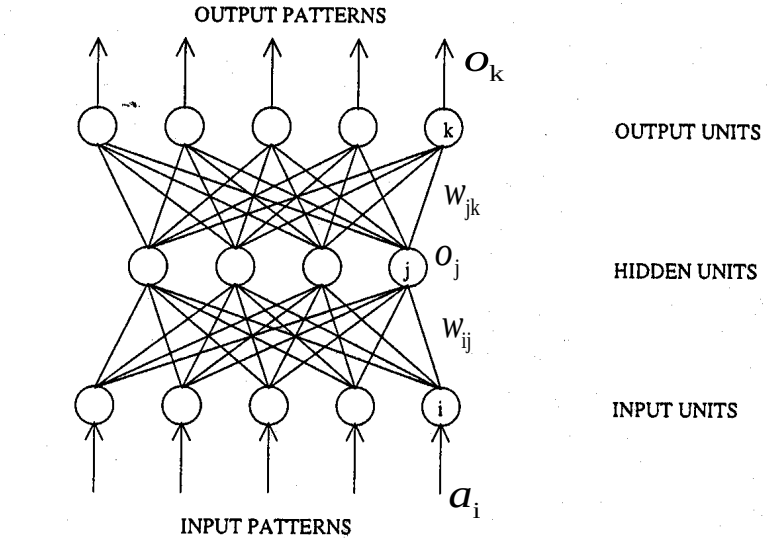




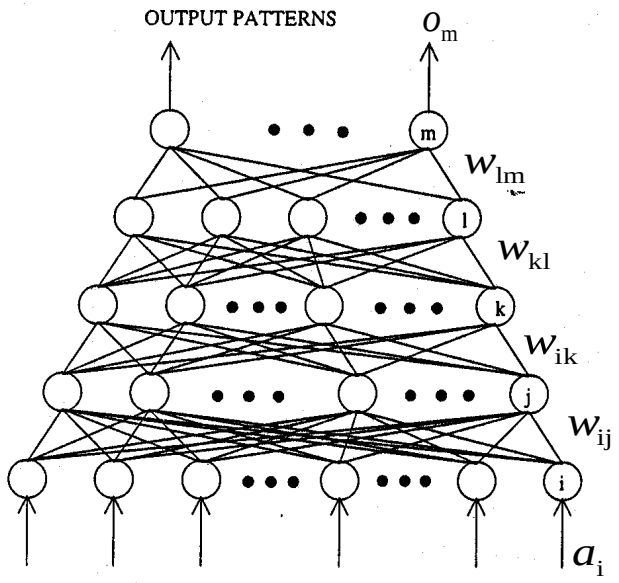
Examples of Multilayer Feedforward Perceptron

- A three layer perceptron can solve any non-linearly separable problem.
- Examples of a three-layer and a five-layer perceptron are shown:





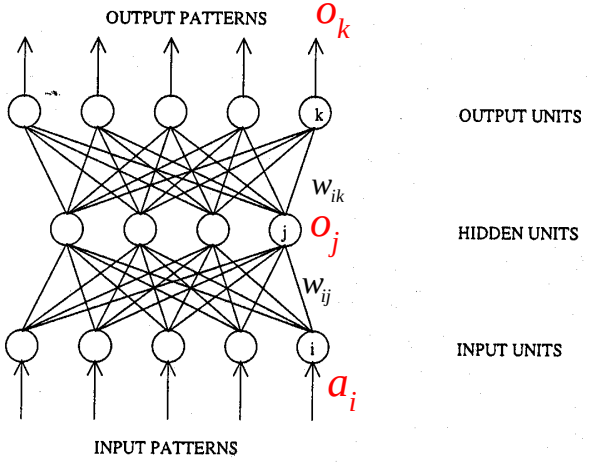
A three-layered back-propagation network, fully interconnected..



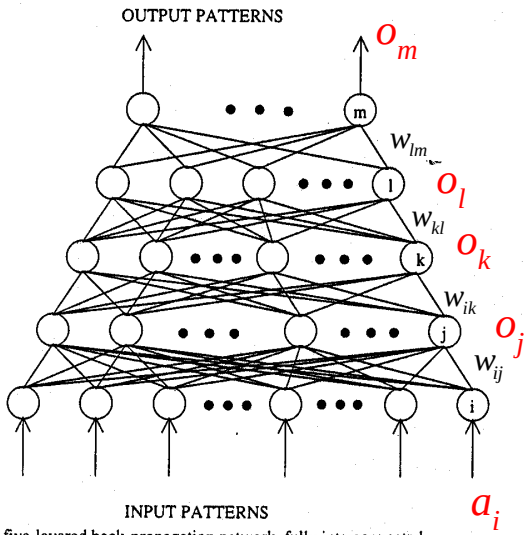
A five-layered back-propagation network, fully interconnected.

Multilayer Feedforward Perceptron

- In the case of a three-layer perceptron, there are two layers of weights w_{ij} and w_{jk} which have to trained by a learning algorithm.
- Similarly in the case of a five-layer perceptron there are four layers of weights that have to be obtained by training.
- These feedforward networks simply implement a pattern-mapping relationship.
- These networks can be used for classification tasks, for example consider the task of object recognition. The input could be a feature vector and each output node could represent a specific object (class).
- Let us assume the network has already been trained that is the weights have been optimized by a training algorithm.



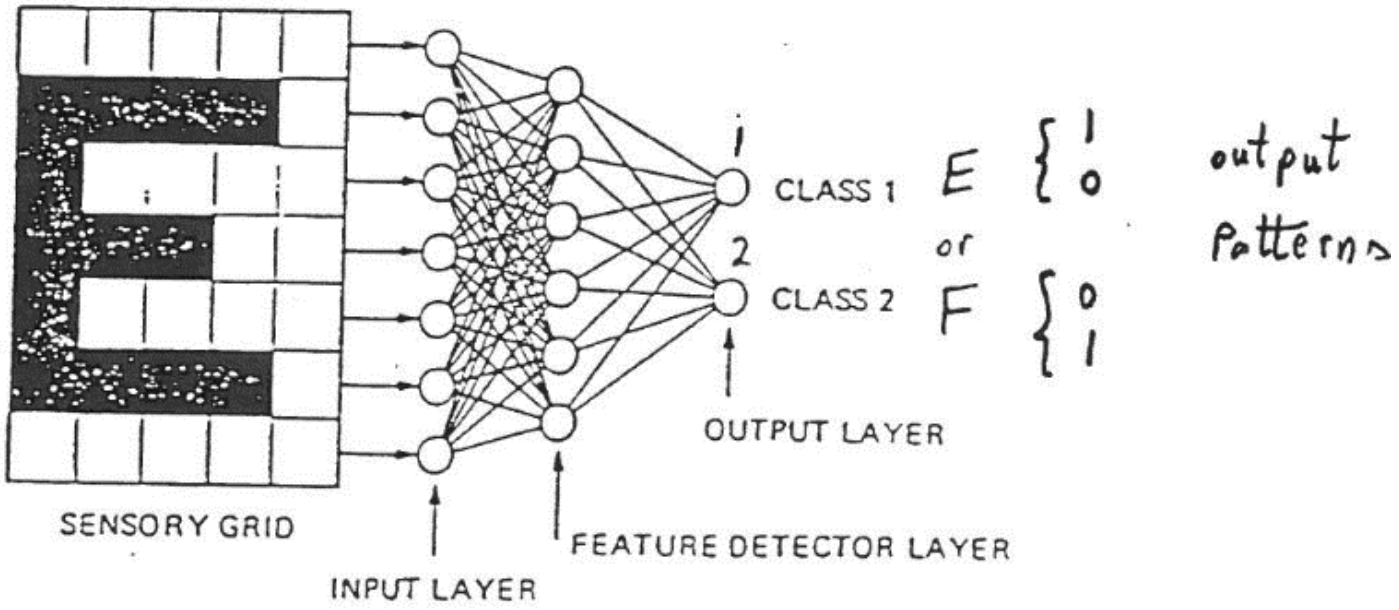
A three-layered back-propagation network, fully interconnected..



A five-layered back-propagation network, fully interconnected.

Three-Layer Perceptron Function

- Consider the network shown below, it is supposed to distinguish between the two letters E and F. When the letter E is presented to the network the first neuron is excited, corresponding to class 1, and the when the letter F is presented the second neuron should fire which corresponds to class 2.
- The most important task is the training of the weights for the multi-layer network. The major obstacle is adjusting the weights for the hidden units, since there is no target values for the hidden units.
- An important technique is the backpropagation training algorithm.
- Once the network has been trained the weights are fixed and it should be able to classify a new input data.



Three-Layer Perceptron Function

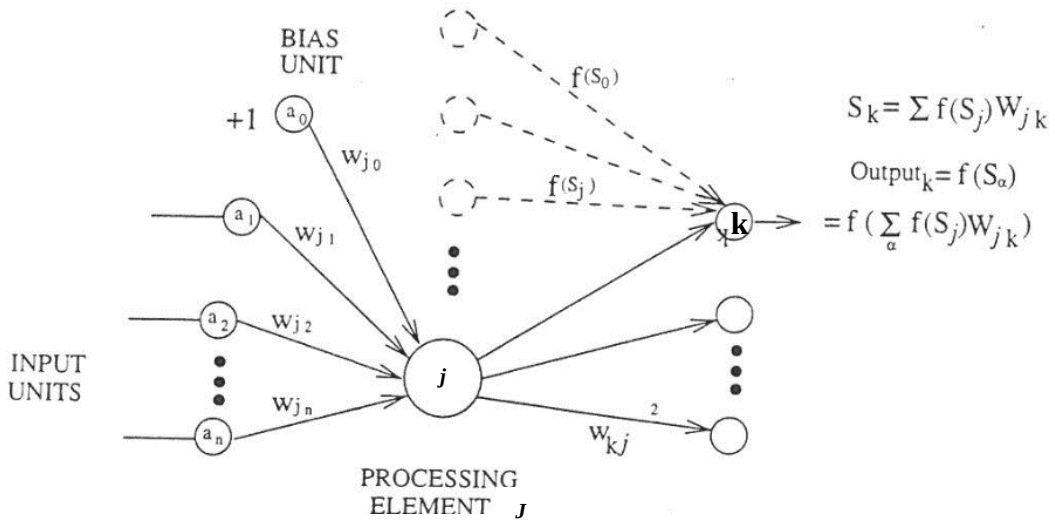
- Let us first consider how a three-layer perceptron works and then discuss the backpropagation training algorithm to find the weights.
- Consider a processing input unit of a three-layer network. When an input pattern is presented each input unit takes on the value of its entry.

$$S_j = \sum_{i=1}^{I} a_i w_{ij} \quad o_j = f(S_j) = \frac{1}{1+e^{-S_j}} = \frac{1}{1+e^{-\sum a_i w_{ij}}}$$

- Each unit j in the hidden layer receives input from the units in the layer below, its calculates a weighted sum of all its inputs

$$S_k = \sum_{j=1}^{J} f(S_j) w_{jk} \quad \text{output}_k = f(S_k) = f\left(\sum_{j=1}^{K} f(S_j) w_{jk}\right)$$

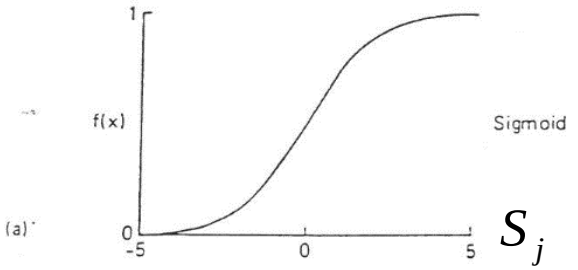
wher f(x) is a non-linear function



Three-Layer Perceptron - Non-linear Functions

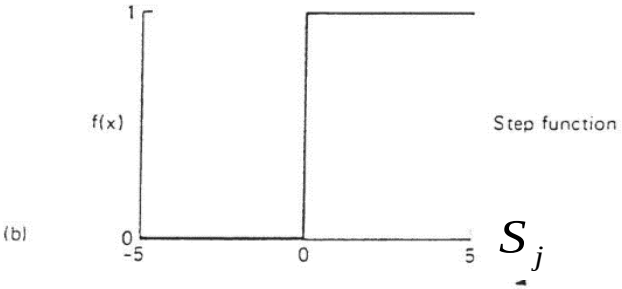
Examples:

a) Sigmoid function: $$f(S_j) = \frac{1}{1+e^{-S_j}}$$

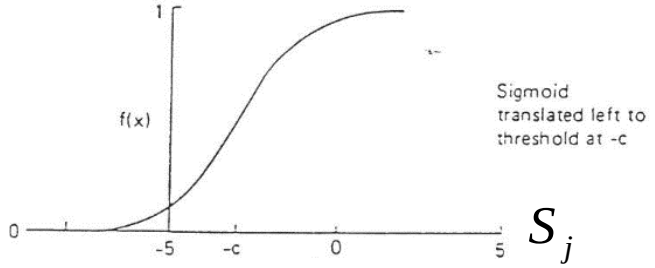


b) Step function:

$$f(S_j) = \begin{cases} 0 \ if \ S_j \le 0 \\ 1 \ if \ S_j > 0 \end{cases}$$

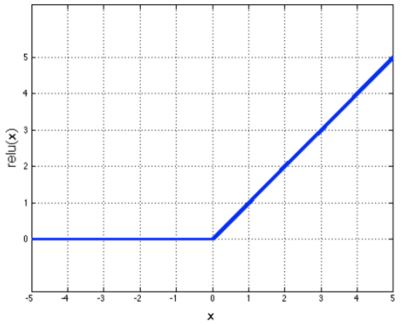


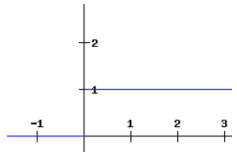
c) Shifted Sigmoid function (c) A sigmoid function moved c units to the left, the threshold at -c.



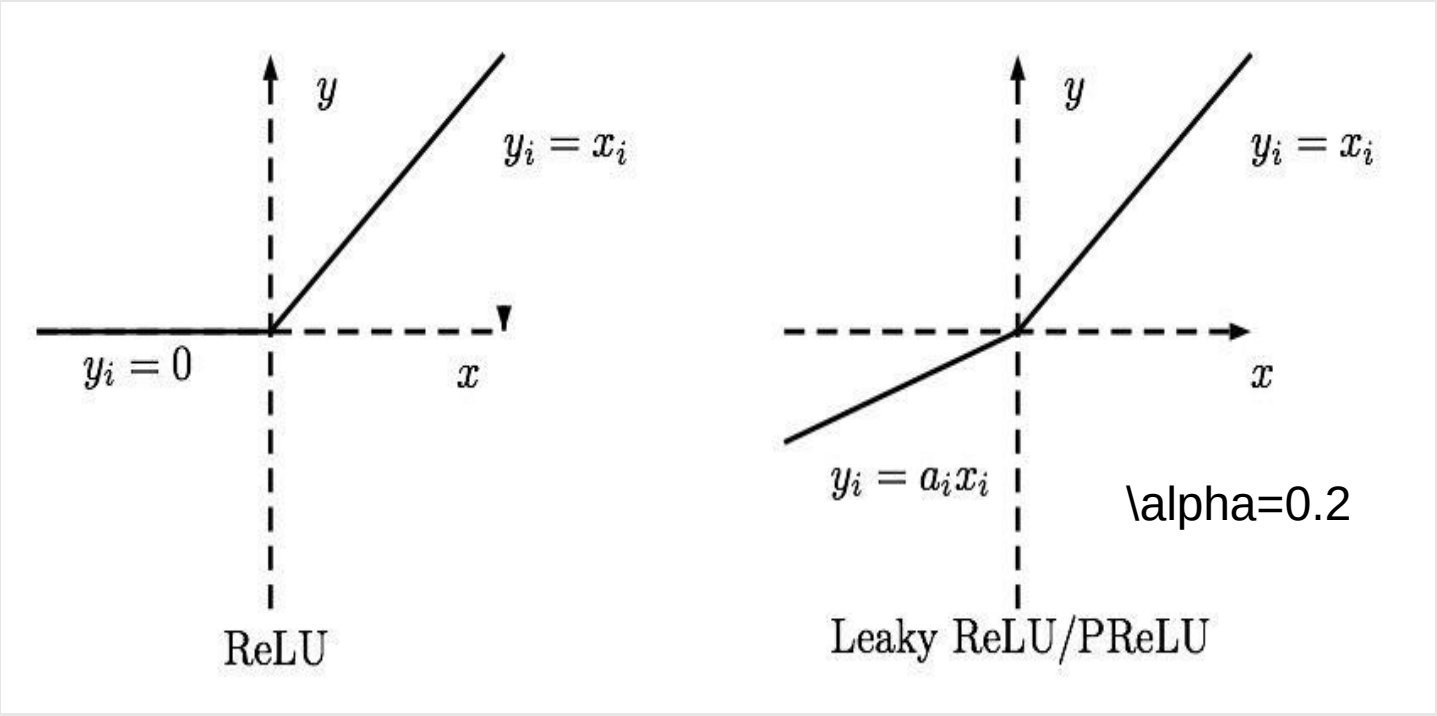
d) Rectified Linear Unit (ReLU):

F(x)=*Max*(**0**,**x**)





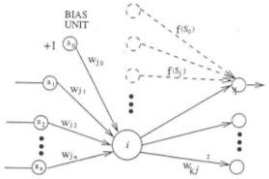
ReLu vs. LeakyReLU activation functions



Xu, B., Wang, N., Chen, T., & Li, M. (2015). Empirical Evaluation of Rectified Activations in Convolution Network. ICML Deep Learning Workshop, 06–11.

Three-Layer Perceptron Function

- The bias is set to 1 and it provides a thresholding effect on each unit that it regards. This is equivalent to translating the Sigmoid curve to the left or right. This threshold is automatically obtained when training the network, it also improves the convergence.
- Each unit in the output layer, *k*, receives input from hidden layer



$$S_k = \sum_{j=1}^{J} o_j w_{jk} = \sum_{j=1}^{J} f(S_j) w_{jk} \quad \text{then} \quad O_k = f(S_k) = f(\sum_{j=1}^{J} f(S_j) w_{jk})$$

- Substitute for $S_j = \sum_{i=1}^{I} a_i w_{ij}$, then $O_k = f(\sum_{j=1}^{J} f(\sum_{i=1}^{I} a_i w_{ij}) w_{jk})$
- Note that if in the hidden layer there was no non-linear function then the 3-layer is equivalent to a 2-layer network with weights given by

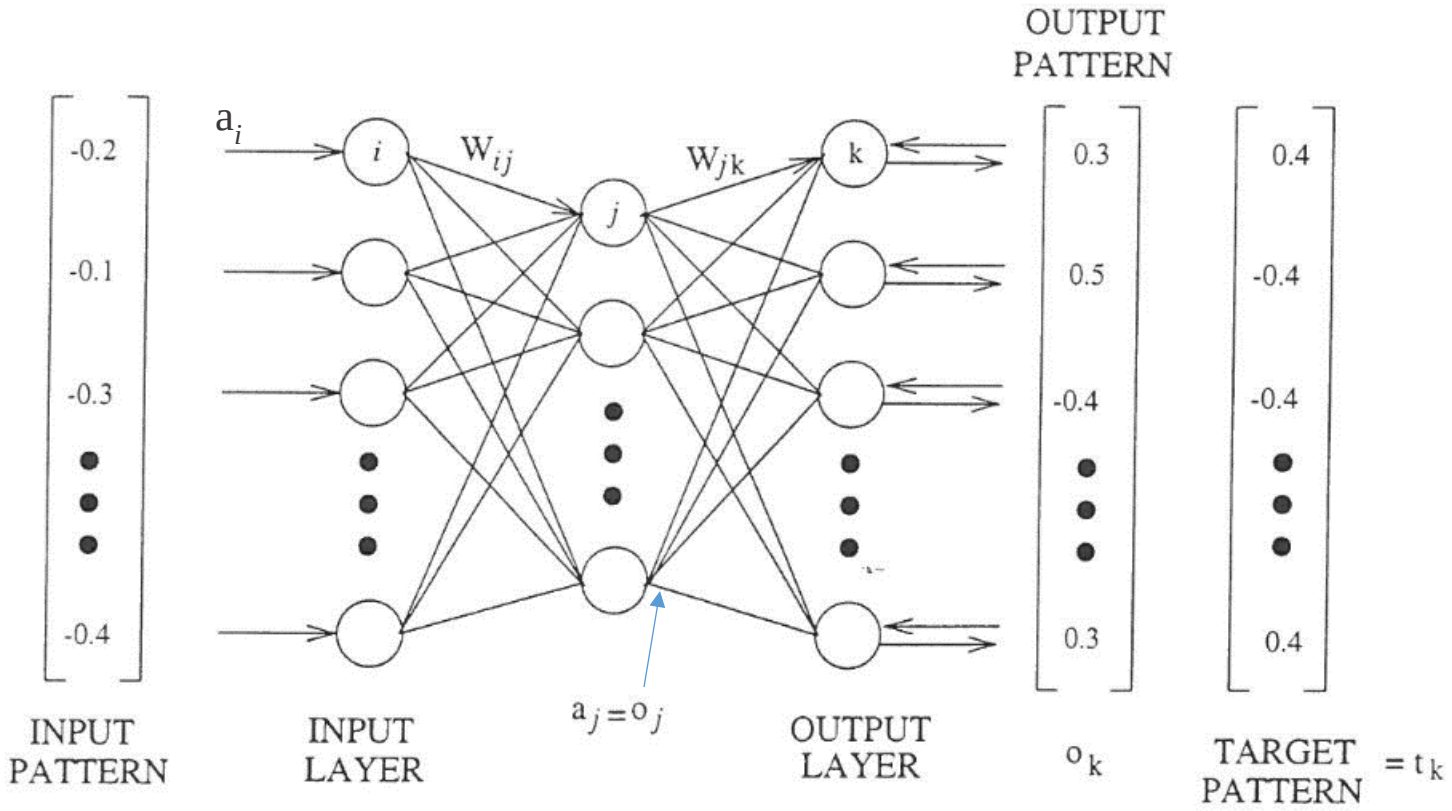
$$O_k = f(\sum_{j=1}^{J} \sum_{i=1}^{I} a_i w_{ij} w_{jk}) = f(\sum_{i=1}^{I} a_i \sum_{j=1}^{J} (w_{ij} w_{jk})) = f(\sum_{i=1}^{I} a_i w'_{ik})$$

- Where the weights can be replaced by a single set of weights

$$w'_{ik} = \sum_{j=1}^{J} (w_{ij} w_{jk})$$

Backpropagation Training Algorithm or Generalized Delta Rule

- Back-error propagation is a training algorithm for multi-layer perceptron, it is also known as the Generalized Delta Rule.
- Initial weights are assigned randomly (usually values between 0-1 are chosen).
- A set of training vector patterns and target patterns are used.



Training a Three-layer Network

- Adjusting the weights of the output layers
- Input pattern is presented to the input layer and the output pattern is calculating using the initial weights.
- For each output unit its output value is compared with its corresponding target value and an error is calculated.
- The error is given by $\delta_k = (t_k - o_k) f'(S_k)$

$$w_{ij}^{t+1} = w_{ij}^{t} + \eta \overbrace{(t_j^p - o_j^p) f'(S_j^p)}^{\delta_j} a_i$$
$$= w_{ij}^{t} + \eta \delta_j a_i$$

where

t_k = the target value for unit k

o_k = the output value for unit k

$f'(S_k)$ = the derivative of the Sigmoid function

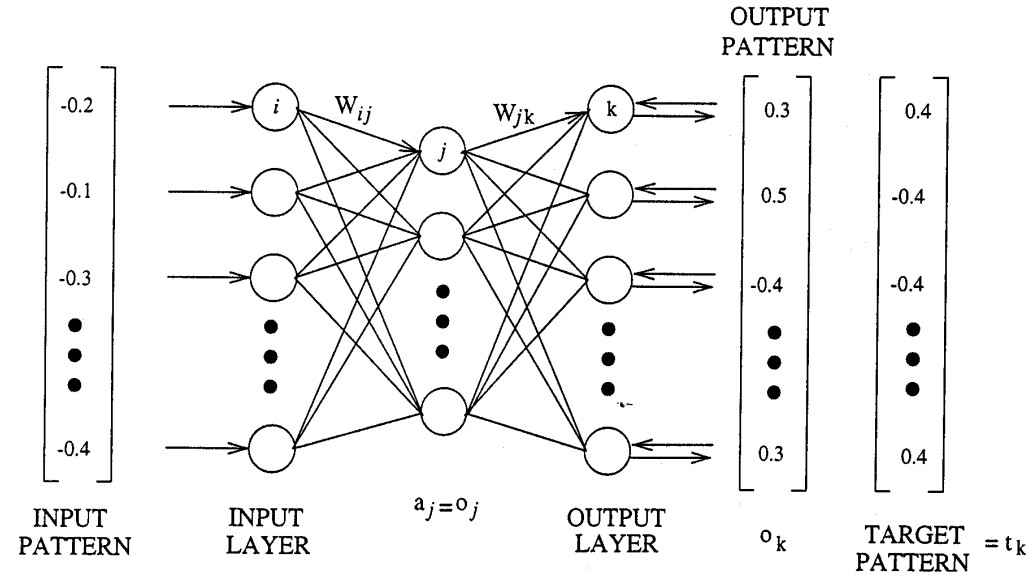
$f'(S_k) = f(S_k)(1 - f(S_k)) = o_k(1 - o_k)$

S_k = weighted sum of the inputs to k

- The quantity (t_k-o_k) reflects the amount of error.
- $f'(S_k)$ scales the error to force a stronger correction when the sum S_k is near the rapid rise in the Sigmoid function.

Adjusting the weights of the output layers

Output layer weights w_{jk}:



The weight adjustment is given by:

$\Delta w_{jk} = \eta \delta_k a_j$ but $\delta_k = (t_k - o_k) f'(S_k)$ and $a_j = o_j$

$w_{jk}(t+1) = w_{jk}(t) + \Delta w_{jk}$

$w_{jk}(t+1) = w_{jk}(t) + \eta \delta_k a_j$

$w_{jk}(t+1) = w_{jk}(t) + \eta (t_k - o_k) f'(S_k) a_j$

Weight adjustment for a single layer

$$w_{ij}^{t+1} = w_{ij}^{t} + \eta \overbrace{(t_j^p - o_j^p) f'(S_j^p)}^{\delta_j} a_i$$

$$= w_{ij}^{t} + \eta \delta_j a_i$$

η – is a learning rate between roughly 0.001 - 0.01

$\delta_k = (t_k - o_k) f'(S_k)$ – is the error value of the target unit. A larger vlaue for unit j results in a larger adjustment to its incoming weights.

$a_j = o_j = f(S_j) = f(\sum_{i=1}^{I} a_i w_{ij})$ – is the output value of the unit from hidden layer.

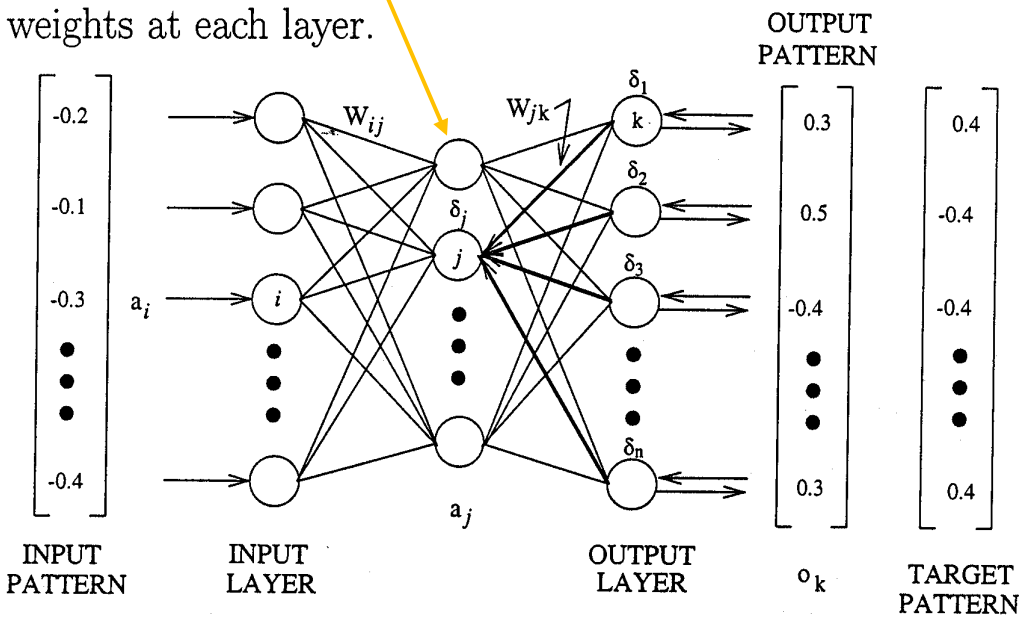
Weight Adjustment for the Hidden Layers Weights

$$w_{ij}^{t+1} = w_{ij}^{t} + \eta \overbrace{(t_j^p - o_j^p) f'(S_j^p)}^{\delta_j} a_i$$

$$\delta_k = (t_k - O_k) f'(S_k)$$

- Hidden layer weights w_{ij}:

- The hidden layers have no target vector, so the error value cannot be calculated directly.
- Backpropagation trains the hidden layers by propagating the output error back through the network layer by layer, adjusting weights at each layer.



- Consider the node j in the hidden layer, its error value is given by

$$\delta_j = \left[\sum_{k=1}^{K} \delta_k w_{jk}\right] f'(S_j)$$

- This term represents the backpropagation of the output errors back to the node j.

- The change weight is given by:

$$w_{ij}(t+1) = w_{ij}(t) + \eta \delta_j a_i = w_{ij}(t) + \eta \left\{\left[\sum_{k=1}^{K} \delta_k w_{jk}\right] f'(S_j)\right\} a_i$$

To Improve the Training Time of the Generalized Delta Rule

- The generalized Delta rule or the backpropagation algorithm can be improved in many ways, for example by modifying the weight adjustment rules.
- When the learning rate is large, it could lead to oscillation. We could decrease the learning rate with time (#iteration).
- One way to increase the learning rate without leading to oscillation is to modify the generalized delta rule to include a momentum term. This can be accomplished by

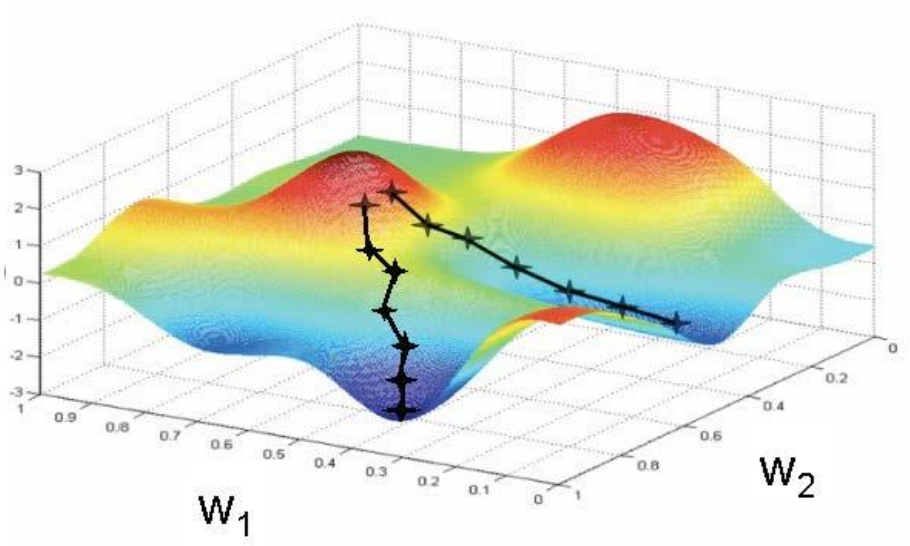
$$\Delta w_{jk}(t+1) = \eta \delta_k a_j + \overbrace{\alpha \Delta w_{jk}(t)}^{momentum\ term}$$

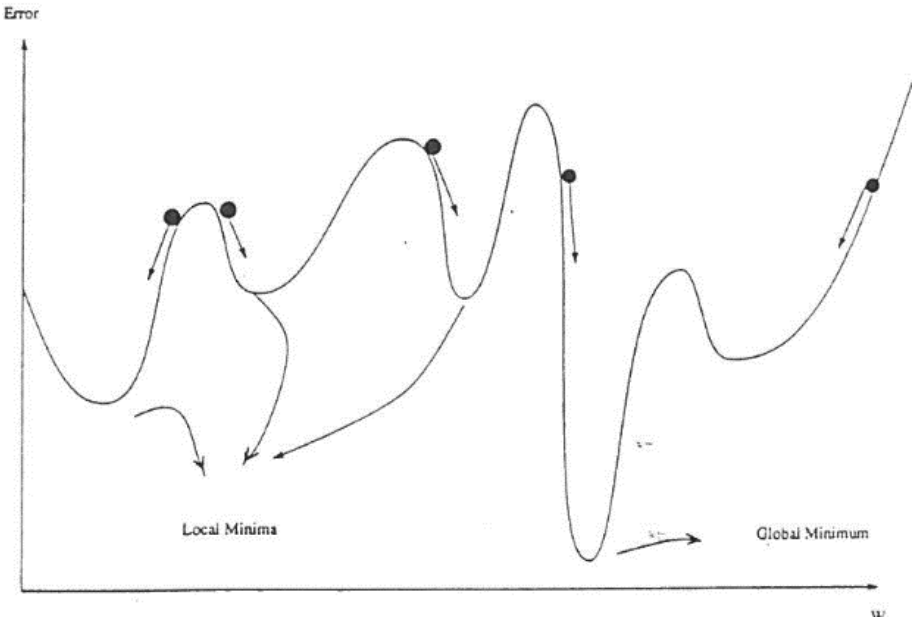
- Where α is a constant which determines the effect of past weight changes on the current direction of the movement in weight space.
- This updating procedure, effectively filters out the high-frequency variations of the error-surface in the weight space.

Strengths and Limitations of Back-Error Propagation Networks

- The number of layers, interconnections, processing units and the learning constants can be changed. Therefore, multi-layer perceptron can address a broad spectrum of applications.
- The largest drawback with back-error propagation appears to be its convergence time.
- Realistic applications may need many thousands of examples in a training set and it may take days of computing to complete training.
- Back-error propagation is susceptible to training failures.
- For example, the network can get trapped in a local minimum.

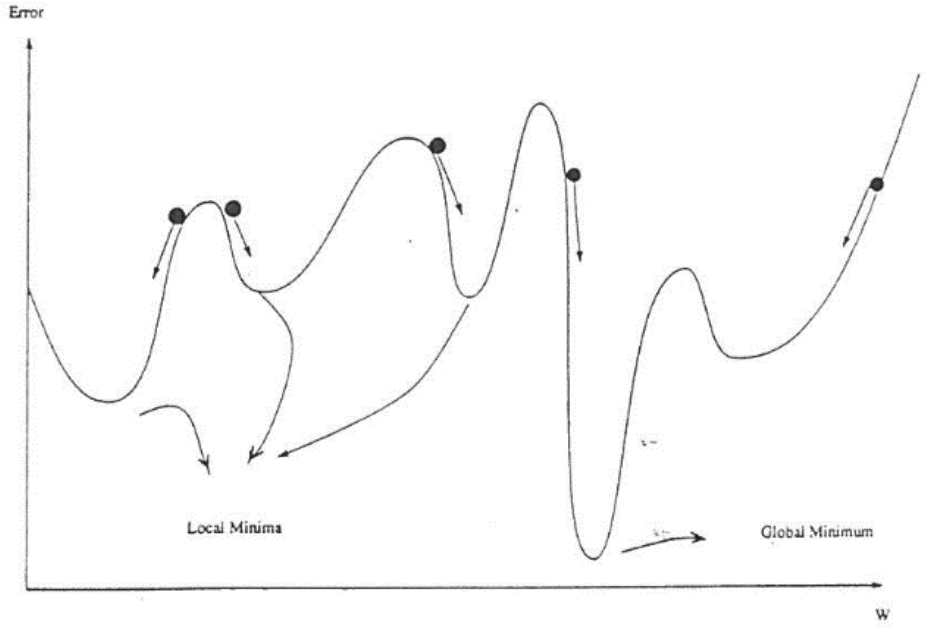
Strengths and Limitations of Back-Error Propagation Networks





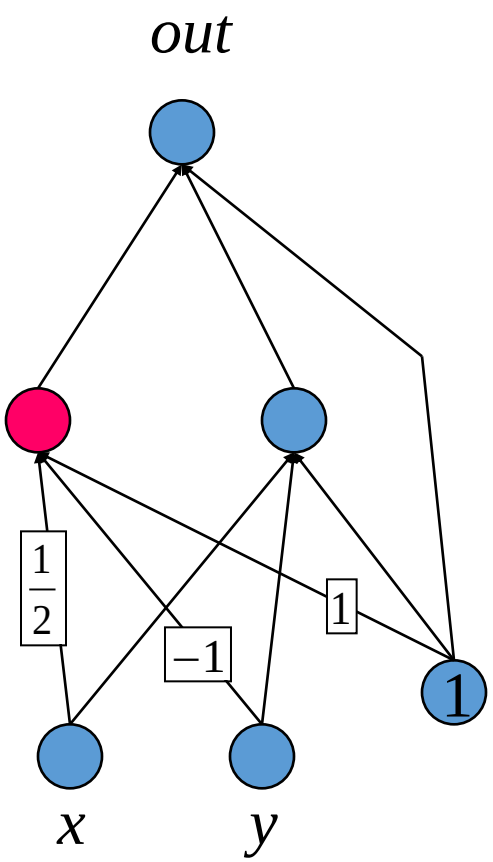
- Convergence time can be improved by lowering the learning parameter η ; gradually.
- To avoid the local minimum (jump out of it) we could add some noise to the weights.
- The multi-layer perceptron is not designed to model biological structures found at synapses and nerve interconnections.

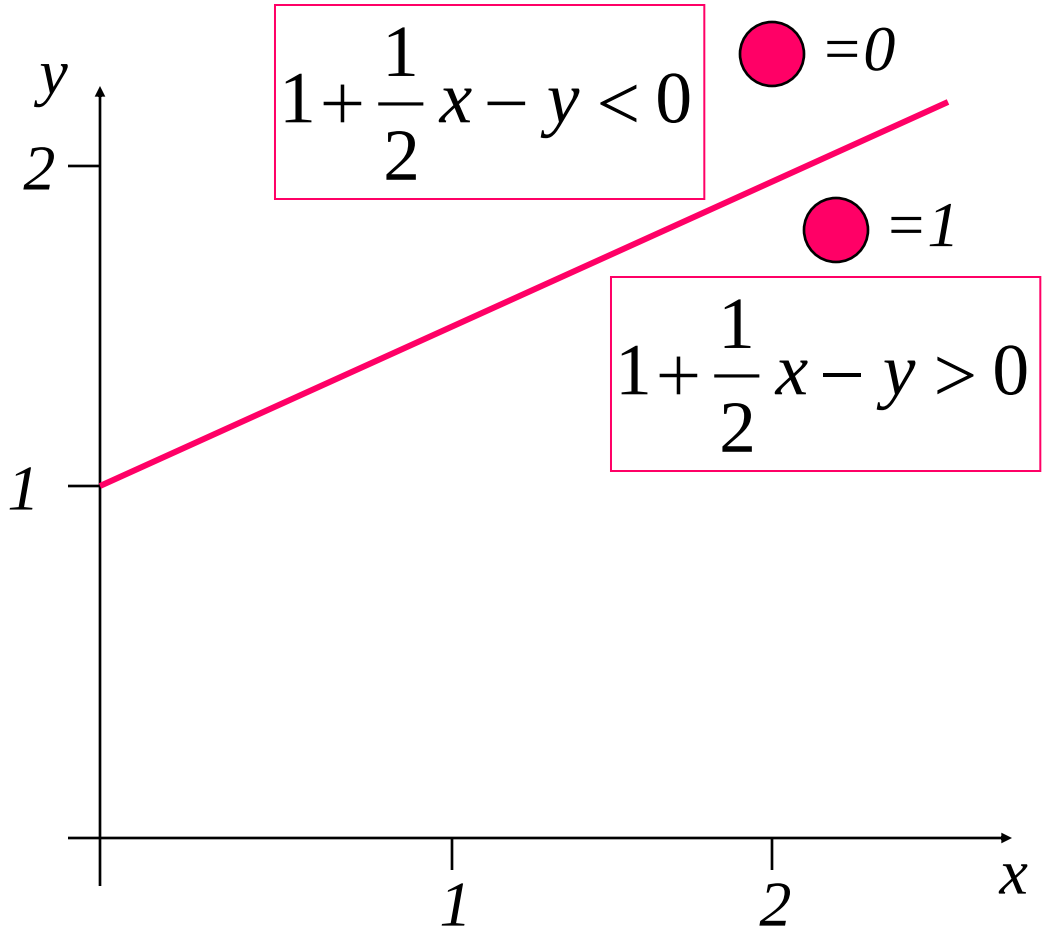
Strengths and Limitations of Back-Error Propagation Networks



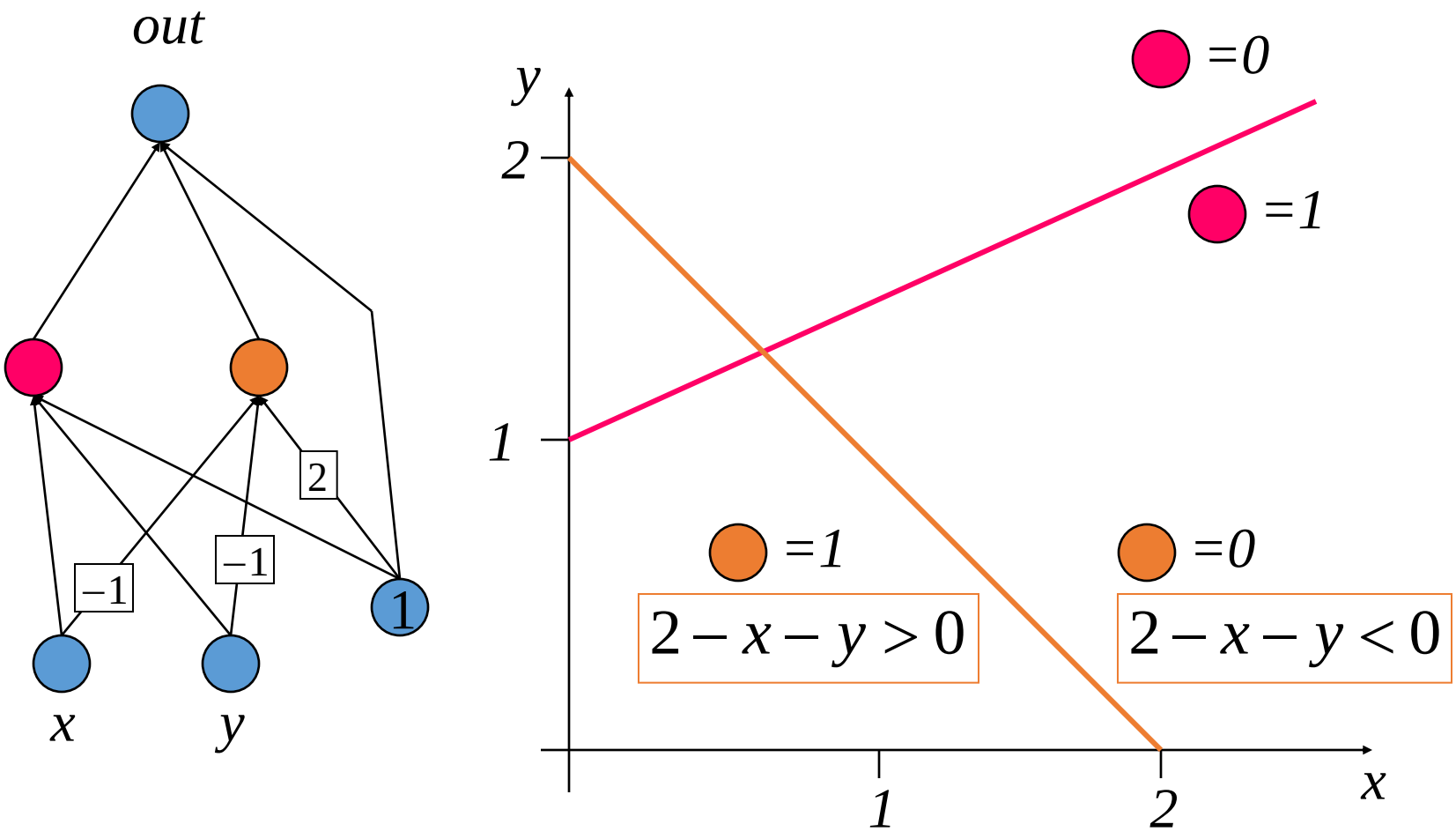
- Convergence time can be improved by lowering the learning parameter η ; gradually.
- To avoid the local minimum (jump out of it) we could add some noise to the weights.
- The multi-layer perceptron is not designed to model biological structures found at synapses and nerve interconnections.

Example: Perceptrons as Constraint Satisfaction Networks

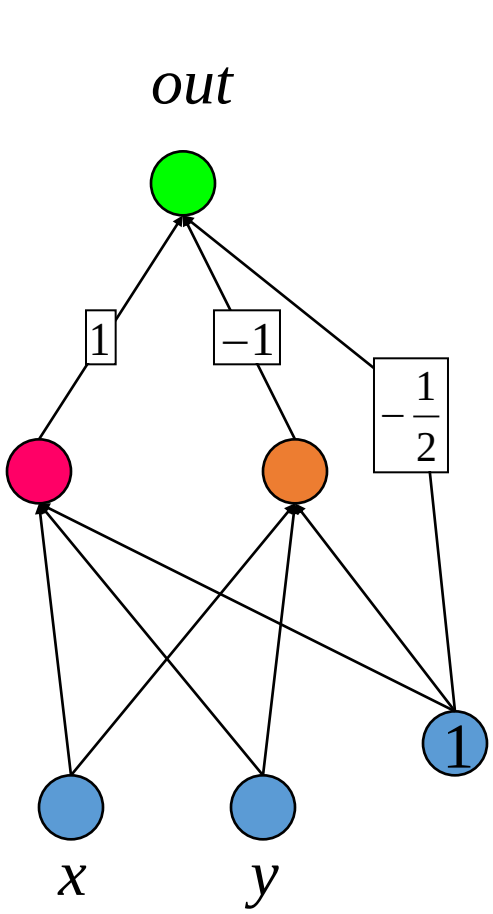


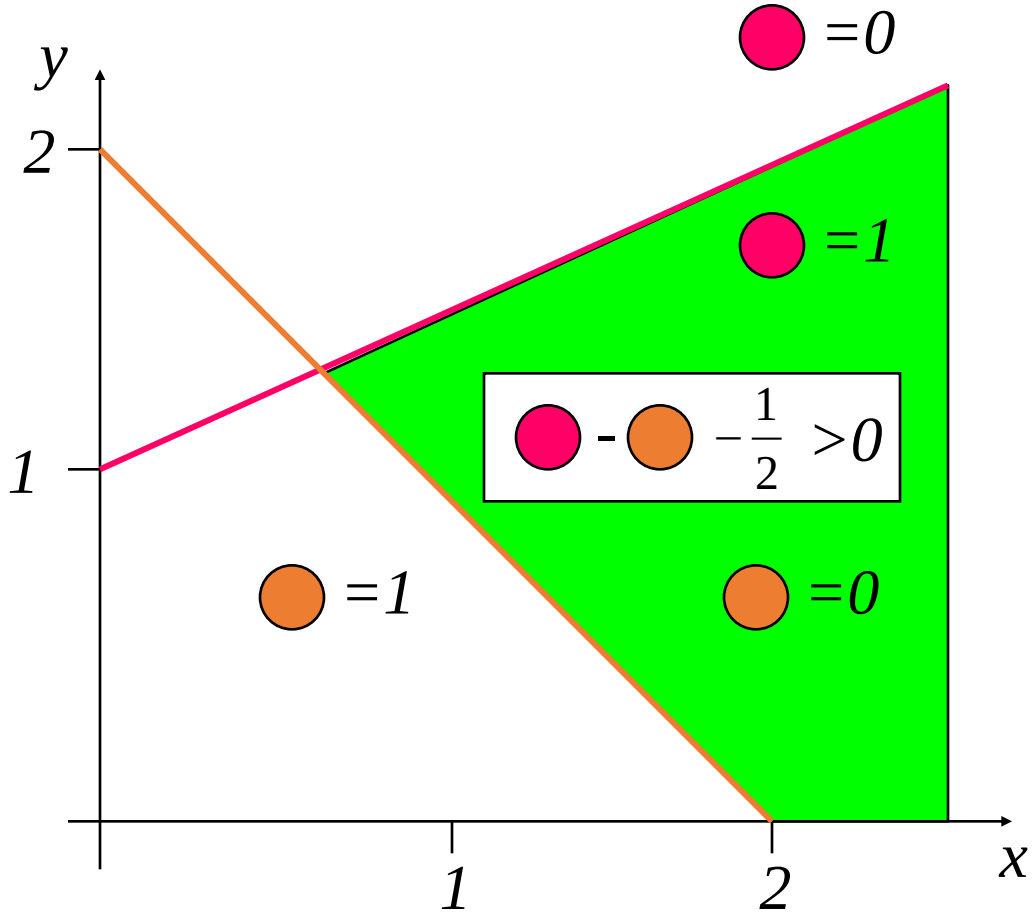


Example: Perceptrons as Constraint Satisfaction Networks

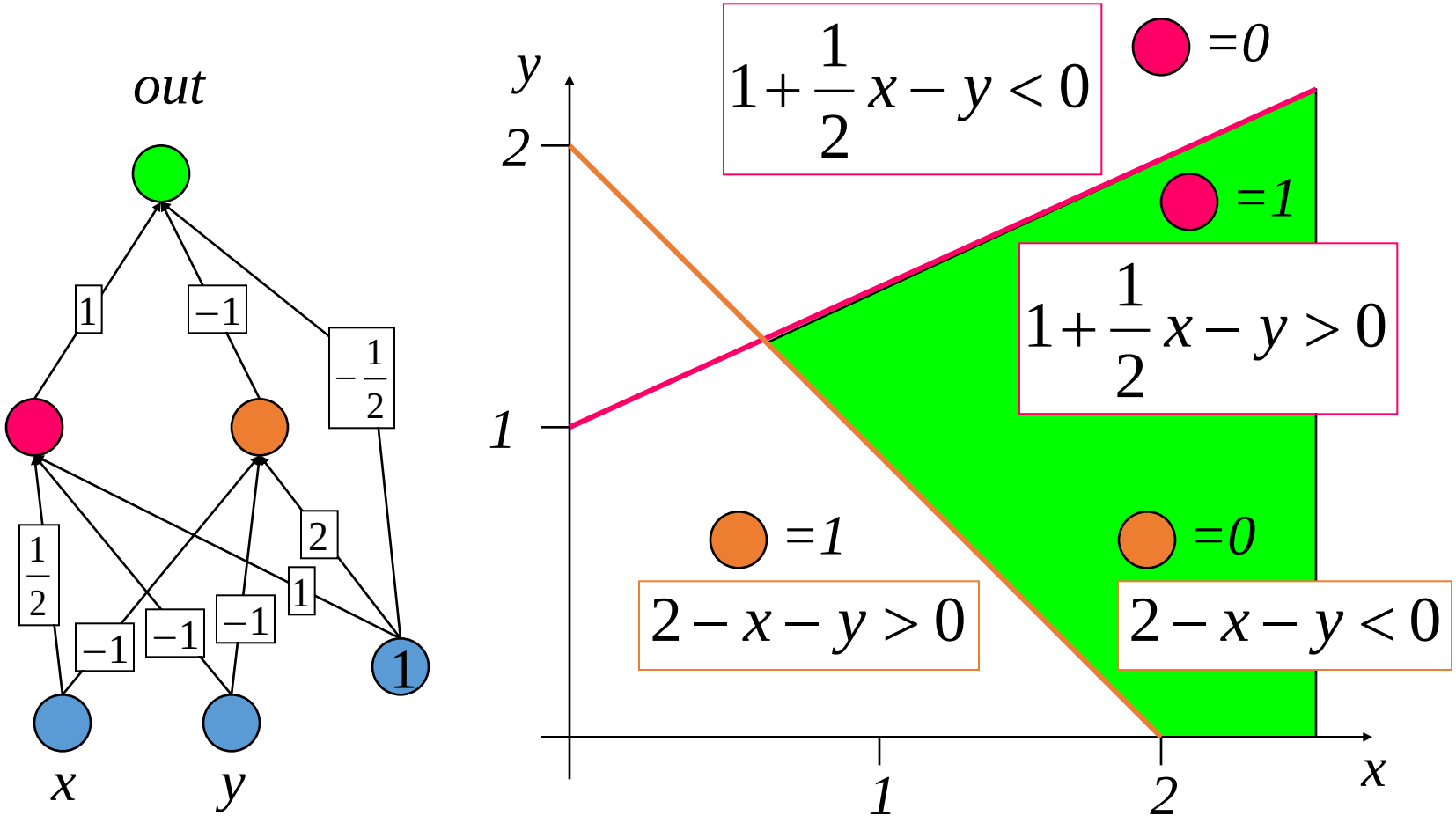


Example: Perceptrons as Constraint Satisfaction Networks





Perceptrons as Constraint Satisfaction Networks



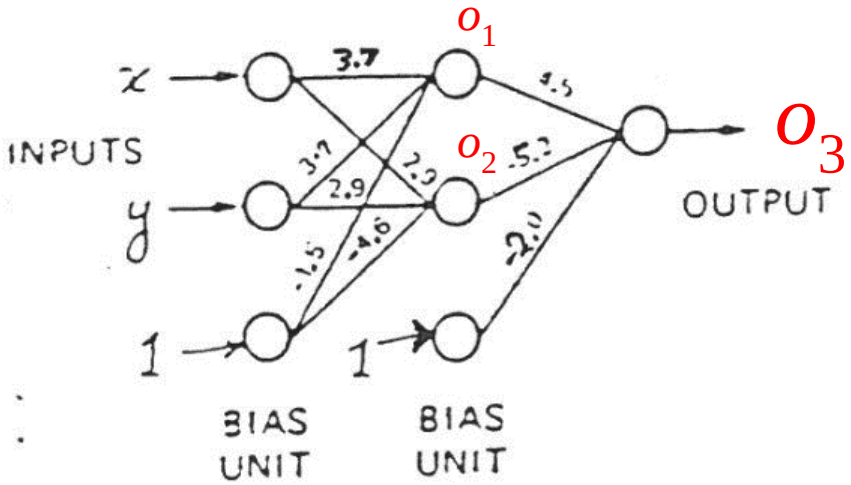
Exclusive-Or Problem with a Three-Layer Perceptron

- The XOR-problem is a classic example that a single layer network cannot solve, but a multi-layer network can solve the classification.
- Consider the network shown below that can perform the XOR function.
- Let the input be $\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$ and the output of the hidden layer $\begin{bmatrix} o_1 \\ o_2 \\ 1 \end{bmatrix}$ and the final output o_3.
- Using the given weights we can calculate the final output $o_3 = f(S_3)$ by the following expressions

$$\begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} 3.7 & 2.9 & 0 \\ 3.7 & 2.9 & 0 \\ -1.5 & -4.5 & 0 \end{bmatrix} = \begin{bmatrix} S_1 \\ S_2 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} f(S_1) & f(S_2) & 1 \end{bmatrix} \begin{bmatrix} 4.5 \\ -5.2 \\ -2.0 \end{bmatrix} = S_3$$

$o_1 = f(S_1),\ o_2 = f(S_2),\ o_3 = f(S_3)$



Exclusive-Or Problem with a Three-Layer Perceptron

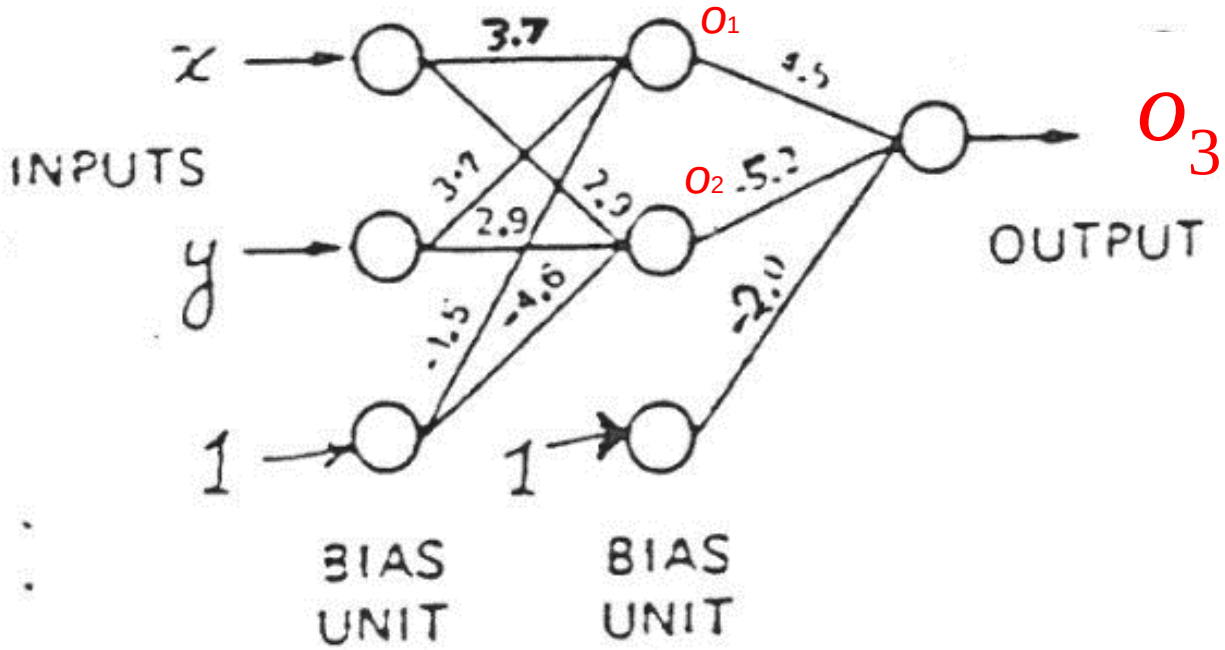
$$\begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} 3.7 & 2.9 & 0 \\ 3.7 & 2.9 & 0 \\ -1.5 & -4.6 & 0 \end{bmatrix} = \begin{bmatrix} S_1 \\ S_2 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} f(S_1) & f(S_2) & 1 \end{bmatrix} \begin{bmatrix} 4.5 \\ -5.2 \\ -2.0 \end{bmatrix} = S_3$$

$$o_1 = f(S_1),\ o_2 = f(S_2),\ o_3 = f(S_3)$$

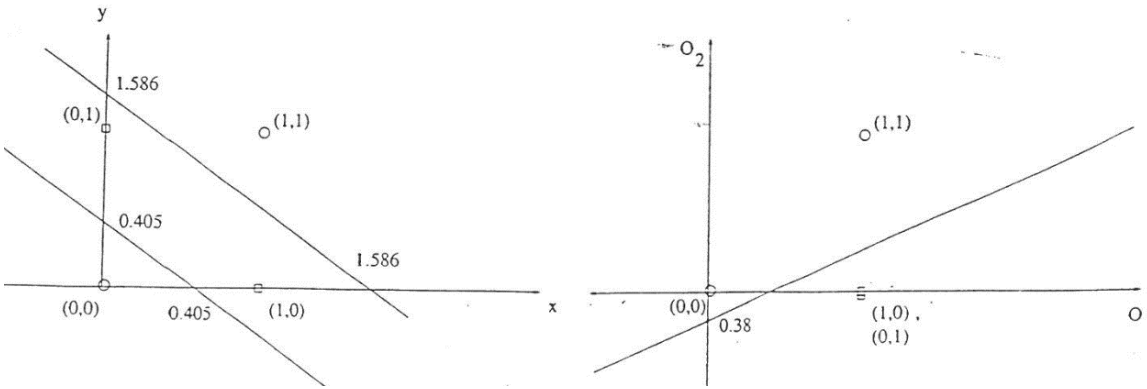
XOR-Problem

INPUTS		OUTPUTS
x	y	o_3
0	0	0
0	1	1
1	0	1
1	1	0



Exclusive-Or Problem with a Three-Layer Perceptron

- Mapping the input pattern [*x*,*y*] to o_1 and o_2 coordinates are shown below.
- The straight lines created by the weights of the first layer are $3.7x + 3.7y - 1.5 = 0$ and $2.9x + 2.9y - 4.6 = 0$
- Each hidden unit creates a straight line which partitions the *x* and *y* space into two regions as shown below.
- Straight line representing the partition in o_1 and o_2 space is given by $4.5o_1 - 5.2o_2 - 2 = 0$
- The input patterns [0,0] and [1,1] are mapped into [0,0] and [1,1] in o_1 and o_2 coordinates. The input patterns [0,1] and [1,0] are mapped into [1,0] in o_1 and o_2 coordinates.



Batch Learning and On-Line Learning Methods

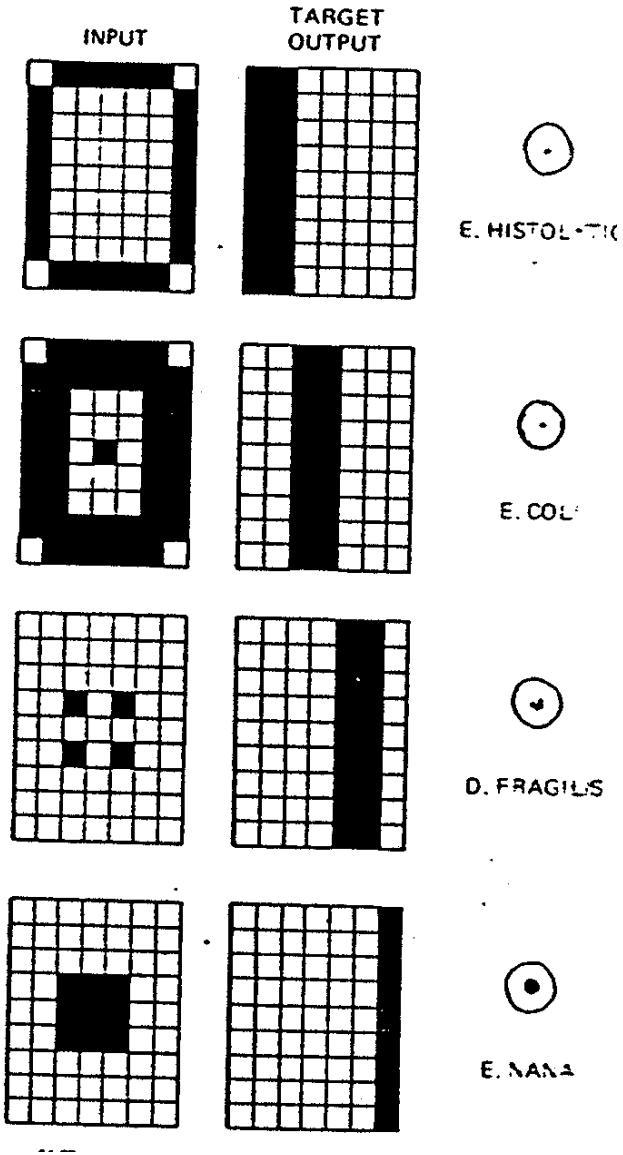
- Batch Learning Method: of supervised learning, adjustments to the weights of the multi-layer perceptron are performed after the presentation of all the P random samples from a large training samples, this constitute one ***epoch*** of training. Therefore, adjustment to the weights of the NN are done on an ***epoch-by-epoch basis***.
  1. The cost function minimized is from all the P samples.
  2. More accurate estimation of the gradient vector.
  3. Parallelization of the learning process, but storage requirement is an issue.
- On-Line Learning Method: of supervised learning, adjustments to the weights of the multi-layer perceptron are performed on an ***example-by-example basis.*** The cost function to be minimized is therefore the total instantaneous error. We can say the epoch size is one sample size.
  1. Samples are presented one at a time in random, so algorithm is referred to as stochastic gradient method.
  2. Can track small changes in the training data.
  3. Parallelization is an issue.

See PP. 127-128 of the recommended book

Applications of Multilayer Perceptrons

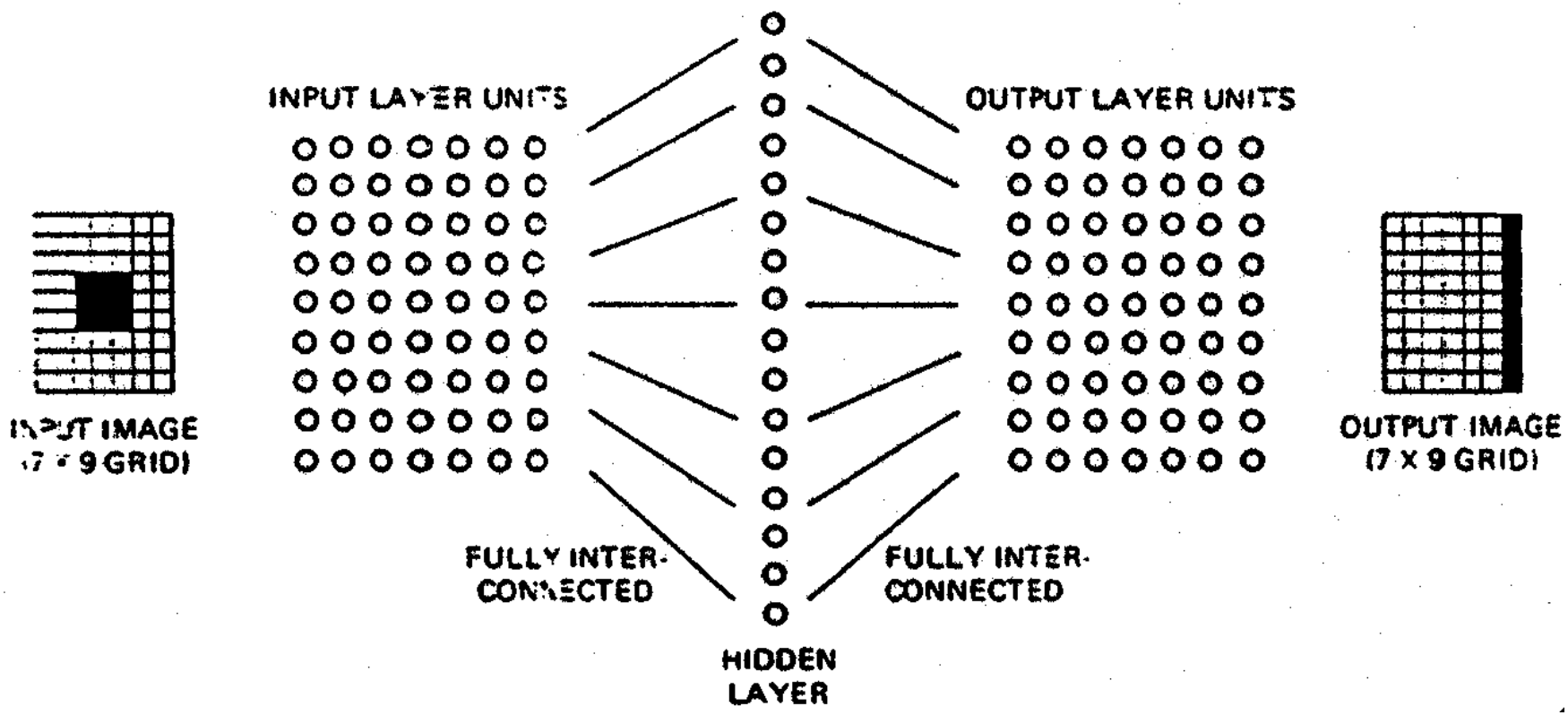
Example of Pattern Mapping:

- Four different input patterns of size 7x9 pixels.
- Four output 2-D patterns of size 7x9 pixels.
- This a four class problem
- A three layer neural network is used to perform the classification.



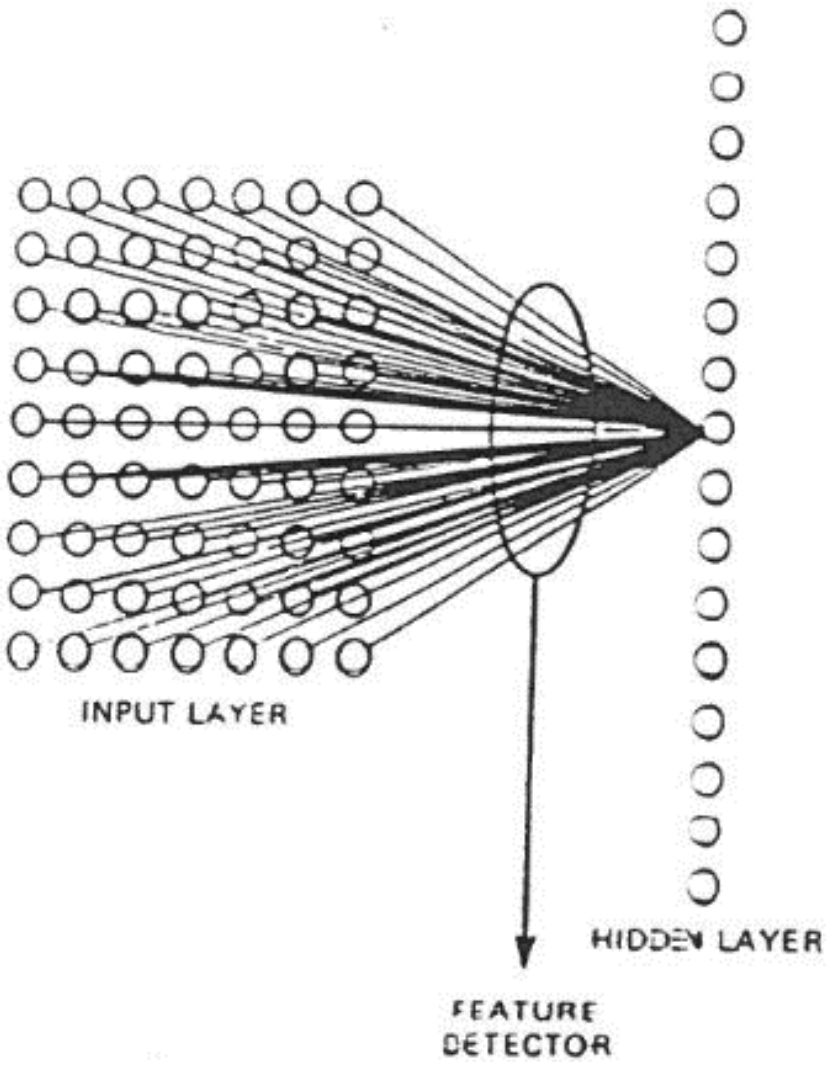
Example of Pattern Mapping

- A three layer neural network of size 2-dimensional grid (7x9)
- 16 hidden units
- An output of 7x9 grid



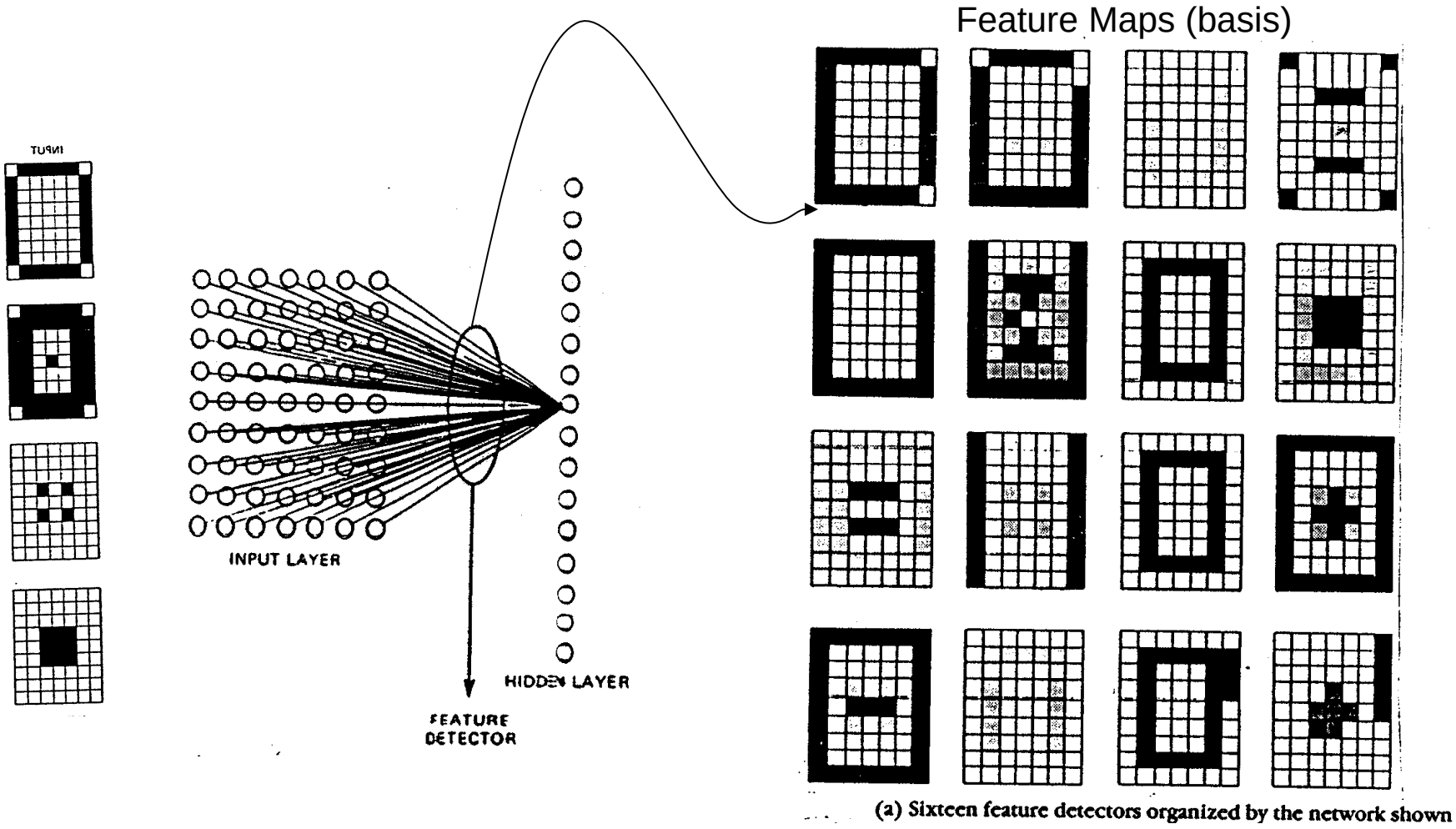
Example of Pattern Mapping

- The training set was presented to the network 3,000 times.
- The network was able to classify all the pictures in the training set, and in addition showed good performance on noisy versions.
- After the network was trained the ,weights to each neuron to the hidden layer were investigated.
- It was found these connections act as feature detectors.



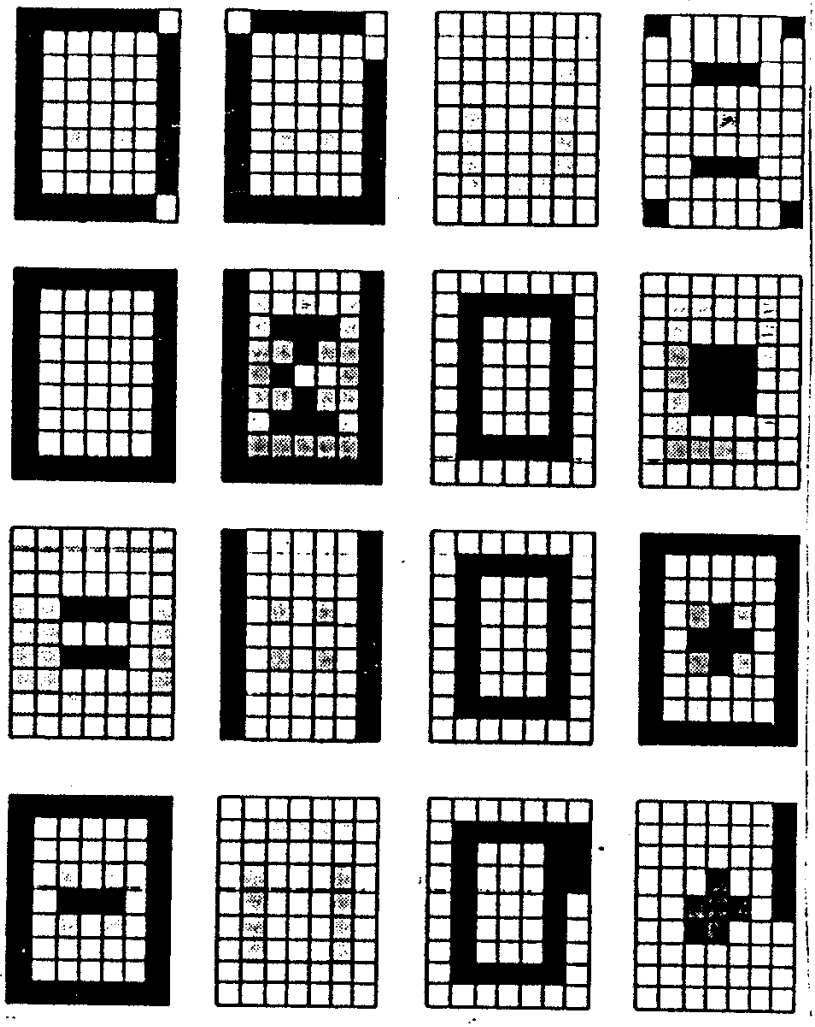
Example of Pattern Mapping

- The 16 set of weights to the hidden units are shown.
- The first layer acts like a correlation masks.

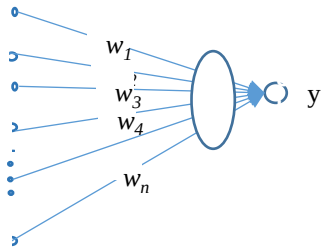


Example of Pattern Mapping

- There are 16 feature detectors, each corresponding to the weights converging to a single unit in the hidden layer.
- Values of these weights were quantized (scaled) and shown in a grid.
- Black boxes represent inhibitory connections, striped boxes represent positive (excitatory) connections, and cross-hatched boxes were strongly positive. White boxes represent weights with very low absolute values.
- Each feature detector illustrates the in coming pattern that the hidden unit respond to best.
- Therefore, an input pattern that is exactly the same as a given feature detector will activate the corresponding feature detecting unit best.
- Since the number of units in the hidden layer ,was more than enough, some of these feature detectors are redundant.



Typical Classifiers

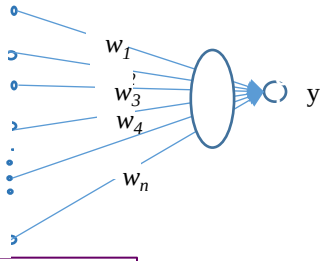


Binary Perceptron:

- Activation function $z_j = \sum_i w_{ij} x_i \qquad y_j = \begin{cases} 1 & \text{if } z_j > 0 \\ 0 & \text{otherwise} \end{cases}$
- Weight update $\Delta w_{ij} = \eta(t_j - y_j)x_i \qquad t_j \in [0,1]$

$$w_{ij}^{t+1} = w_{ij}^t + \Delta w_{ij}$$

Linear regression:



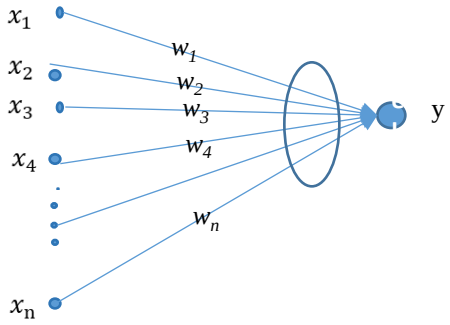
- Activation function $y_j = \sum_i w_{ij} x_i$
- Weight update $\Delta w_{ij} = \eta(t_j - y_j)x_i$

$t_j \in \{0,1\} \Rightarrow E = \frac{1}{2}\sum_j (t_j - y_j)^2$

assumes minimizing squared error loss function

$$w_{ij}^{t+1} = w_{ij}^t + \Delta w_{ij}$$

Typical Classifiers



Logistic regression (two layer continuous Perceptron):

- **Activation function** $z_j = \sum_i w_{ij} x_i \qquad y_j = \dfrac{1}{1+\exp(-z_j)}$
- **Weight update** $\Delta w_{ij} = \eta(t_j - y_j) y_j (1 - y_j) x_i \qquad t_j \in [0,1]$

$$w_{ij}^{t+1} = w_{ij}^{t} + \Delta w_{ij}$$

assumes minimizing squared error loss function: $E = \frac{1}{2}\sum_j (t_j - y_j)^2$

Softmax net (multinomial logistic regression):

- **activation function** $y_j = \dfrac{\exp(-z_j)}{\sum_k \exp(-z_k)}$
- **weight update** $\Delta w_{ij} = \eta(t_j - y_j) y_j (1 - y_j) x_i \qquad t_j \in [0,1]$

$$w_{ij}^{t+1} = w_{ij}^{t} + \Delta w_{ij}$$

assumes minimizing squared error loss function: $E = \frac{1}{2}\sum_j (t_j - y_j)^2$

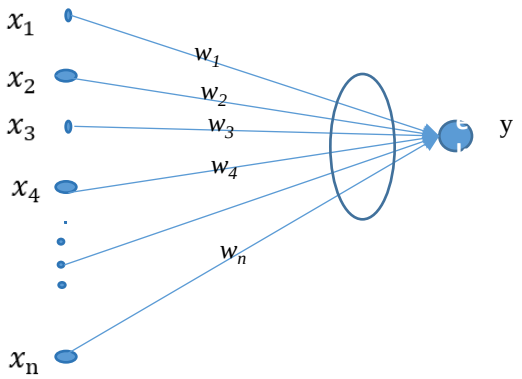
Loss Functions

- Loss functions
  1. squared error:

$$E = \frac{1}{2}\sum_j \left(t_j - y_j\right)^2 \qquad y_j = \frac{1}{1+\exp(-z_j)}, \qquad z_j = \sum_i w_{ij} x_i$$

$$\frac{\partial E_j}{\partial y_j} = (t_j - y_j)$$

$$\frac{\partial E}{\partial w_i} = \sum_j \left(\frac{\partial E_j}{\partial y_j} \times \frac{\partial y_j}{\partial z_j} \times \frac{\partial z_j}{\partial w_i}\right) = \sum_j [(t_j - y_j) y_j (1 - y_j) x_i]$$

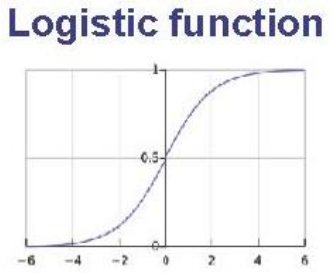


Cross-Entropy Loss Functions for Binary Logistic Regression

2. cross entropy loss:

$$\min\{E(\mathbf{w})\} = -\sum_j t_j \log y_j + (1-t_j)\log(1-y_j) \qquad t_j \in [0,1]$$

$$y_j = \frac{1}{1+\exp(-z_j)} \quad , \quad z_j = \sum_{i=1}^{n} w_i x_i$$

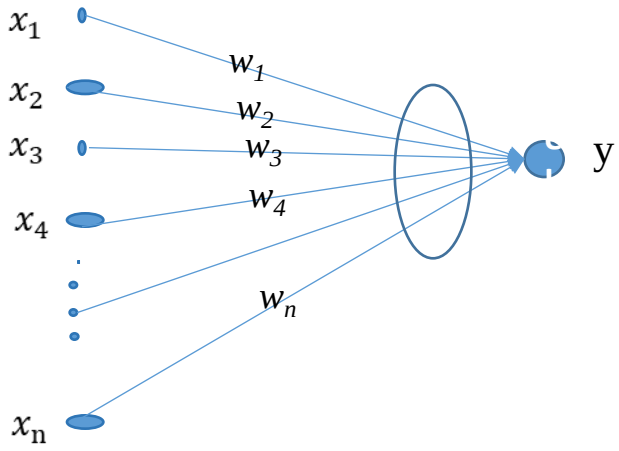


$$\frac{\partial E_j}{\partial w_i} = \frac{\partial E_j}{\partial y_j} \times \frac{\partial y_j}{\partial z_j} \times \frac{\partial z_j}{\partial w_i} \quad \text{for } i=1,\ldots n$$

$$\frac{\partial E_j}{\partial y_j} = \frac{t_j - y_j}{y_j(1-y_j)}$$

$$\frac{\partial y_j}{\partial z_j} = y_j(1-y_j)$$

$$\frac{\partial z_j}{\partial w_i} = x_i$$



$$\frac{\partial E_j}{\partial w_i} = \frac{t_j - y_j}{y_j(1-y_j)} \times y_j(1-y_j) \times x_i = (t_j - y_j)x_i \quad , \quad \frac{\partial E(\mathbf{w})}{\partial w_i} = \sum_{j=1} (t_j - y_j)x_i$$

Regularization Techniques

- Regularization forces some of the weights to take values close to zero. Criteria for model complexity (i.e., # of parameters in network)
- Weight penalty terms

- L2 weight decay

$$E = \frac{1}{2}\sum_j \left(t_j - y_j\right)^2 + \frac{\lambda}{2}\sum_{i,j} w_{ij}^2$$

$$\Delta w_{ij} = \varepsilon \delta_j x_i - \varepsilon \lambda w_{ij}$$

$$w_{ij}^{t+1} = w_{ij}^t + \Delta w_{ij}$$

- L1 weight decay

$$E = \frac{1}{2}\sum_j \left(t_j - y_j\right)^2 + \frac{\lambda}{2}\sum_{i,j} \left|w_{ij}\right|$$

$$\Delta w_{ij} = \varepsilon \delta_j x_i - \varepsilon \lambda \text{sign}(w_{ij})$$

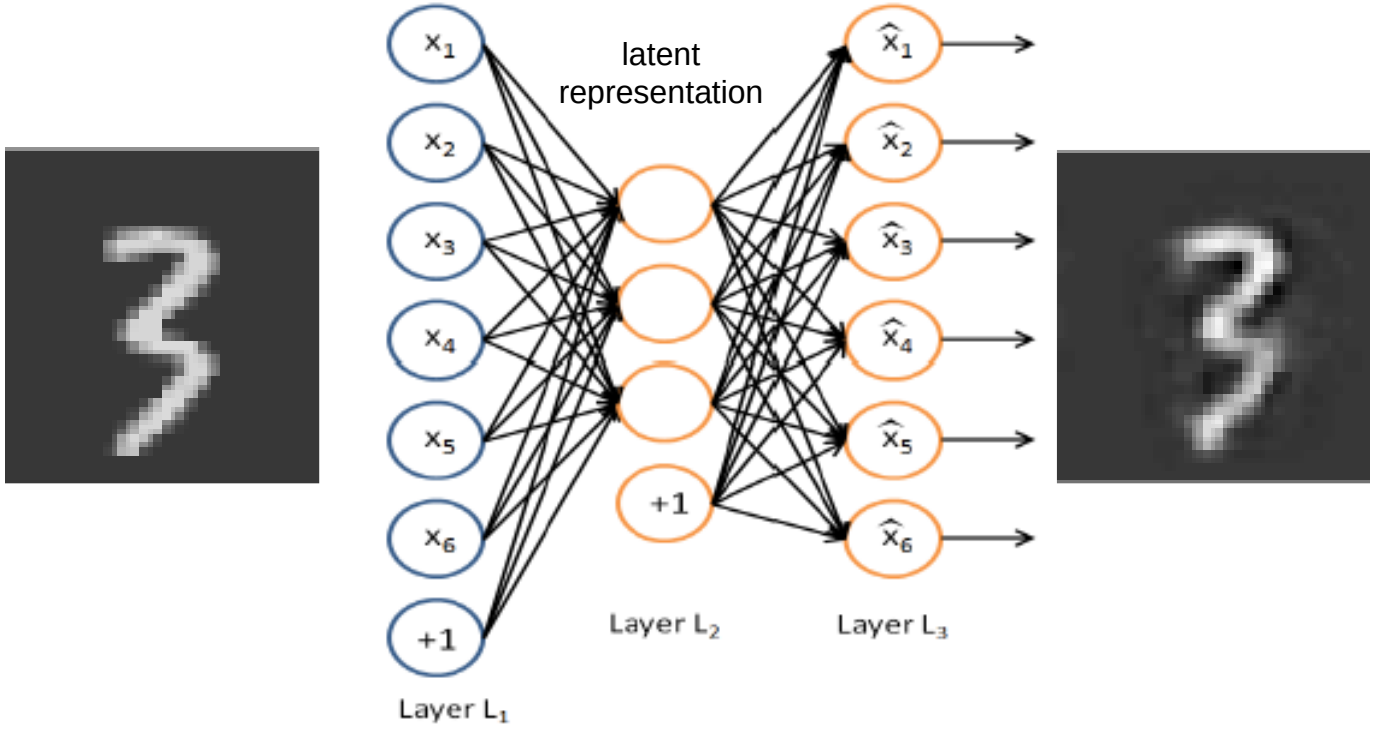
$$w_{ij}^{t+1} = w_{ij}^t + \Delta w_{ij}$$

Deep Autoencoders

See pp. 181-183 from recommended book

Autoencoders

- Autoencoders are a multi-layer neural network with a specific topology.
- The target output of the network is set to the input.
- The aim of training is to minimize the error of reconstruction.
- Often a reduced set of hidden units is used, creating an information bottleneck.



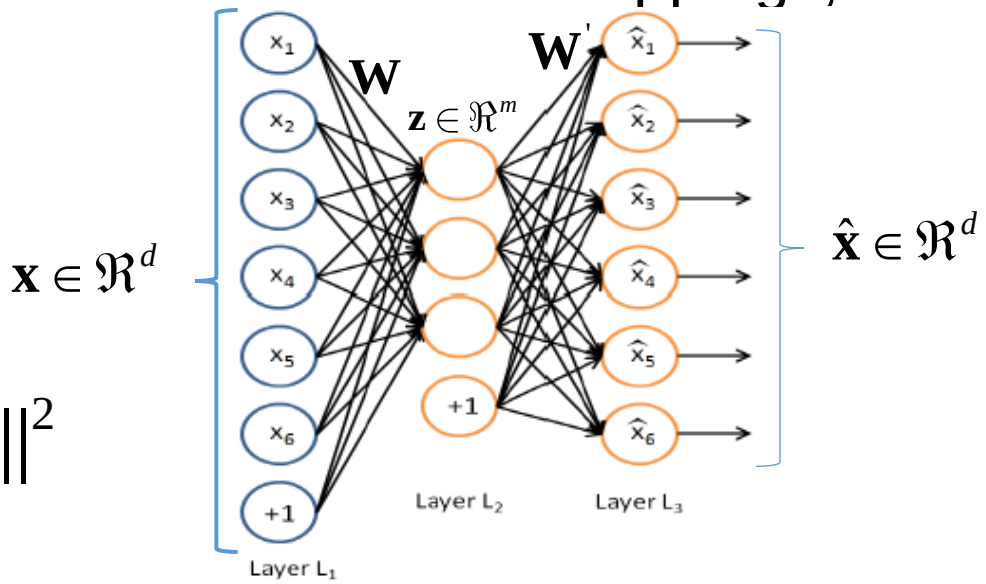
Autoencoders

- An auto-encoder always consists of two parts, the encoder and the decoder, which can be defined as mappings, such that:

$$\phi : \chi \rightarrow F$$

$$\varphi : F \rightarrow \chi$$

$$\arg\min_{\phi,\varphi} \| \mathbf{x} - (\phi \circ \varphi) \|^2$$



- In the simplest case, where there is one hidden layer, an autoencoder takes the input $\mathbf{x} \in \Re^d$ onto $\mathbf{z} \in \Re^m$:

$$\mathbf{z} = \sigma(\mathbf{W}\mathbf{x} + b)$$

$$\hat{\mathbf{x}} = \sigma(\mathbf{W}'\mathbf{z} + b')$$

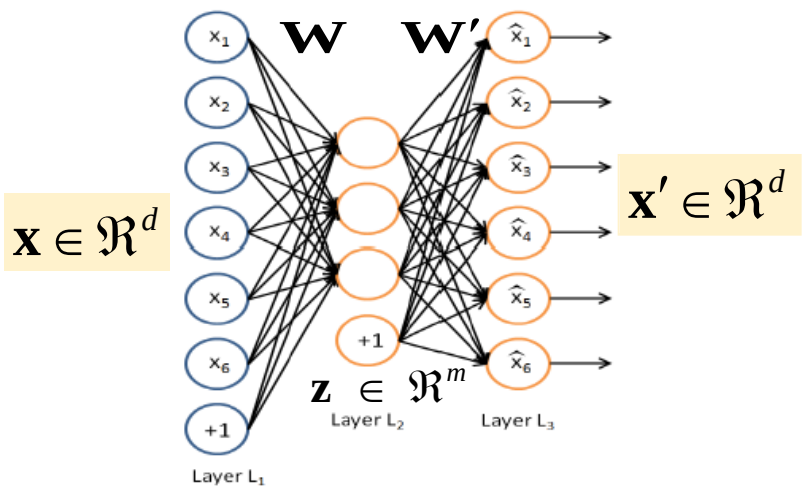
σ is the Sigmoid or Tanh or Rectified Linear Unit (RLU)

Autoencoders

- The output of hidden units $\mathbf{z}$ is usually referred to as code or latent variables (latent representation). After that, it is mapped onto the reconstruction $\mathbf{x}'$of the same input :

$$\mathbf{x}' = \sigma(\mathbf{W}'\mathbf{z} + b')$$

Note $\mathbf{W}^T = \mathbf{W}'$



- Autoencoders are also trained to minimize reconstruction errors (such as squared errors):

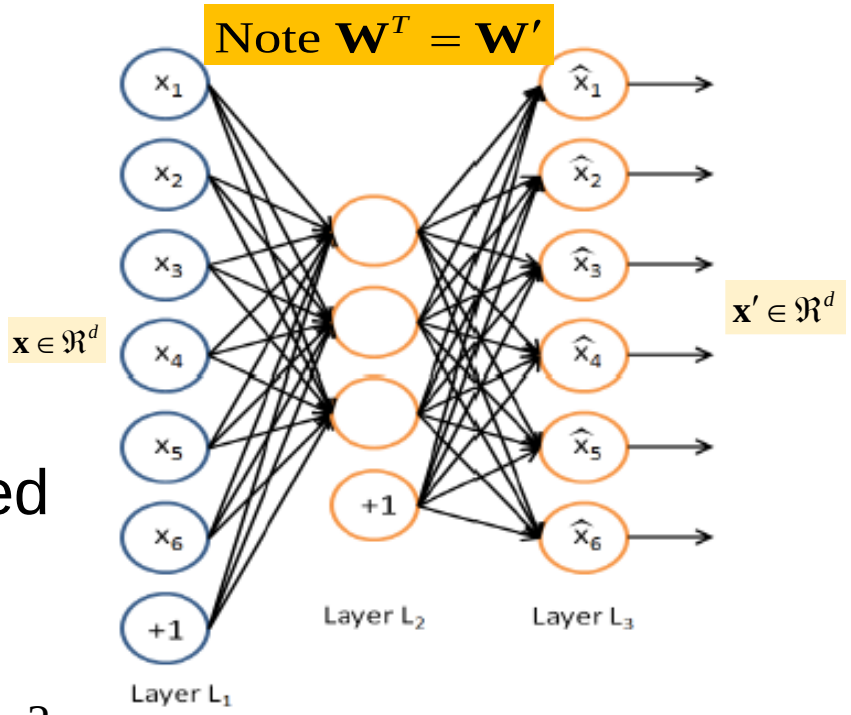
$$\ell(\mathbf{x}, \mathbf{x}') = \| \mathbf{x} - \mathbf{x}' \|^2 = \| \mathbf{x} - \sigma(\mathbf{W}'\sigma(\mathbf{W}\mathbf{x} + b) + b') \|^2$$

- If the # of hidden neurons is smaller than the input layer, $m < d$, the autoencoder can achieve dimensionality reduction.

Autoencoders

- Weights between the input and hidden layer are often tied with weights between the hidden and output layers $\mathbf{w} = \mathbf{w}'$
- Given an input vector $\mathbf{x} \in [0,1]$, hidden unit and output activations are calculated as:

$$\mathbf{z} = \sigma(\mathbf{W}\mathbf{x} + b)$$
$$\mathbf{x}' = \sigma(\mathbf{W}'\mathbf{z} + b')$$

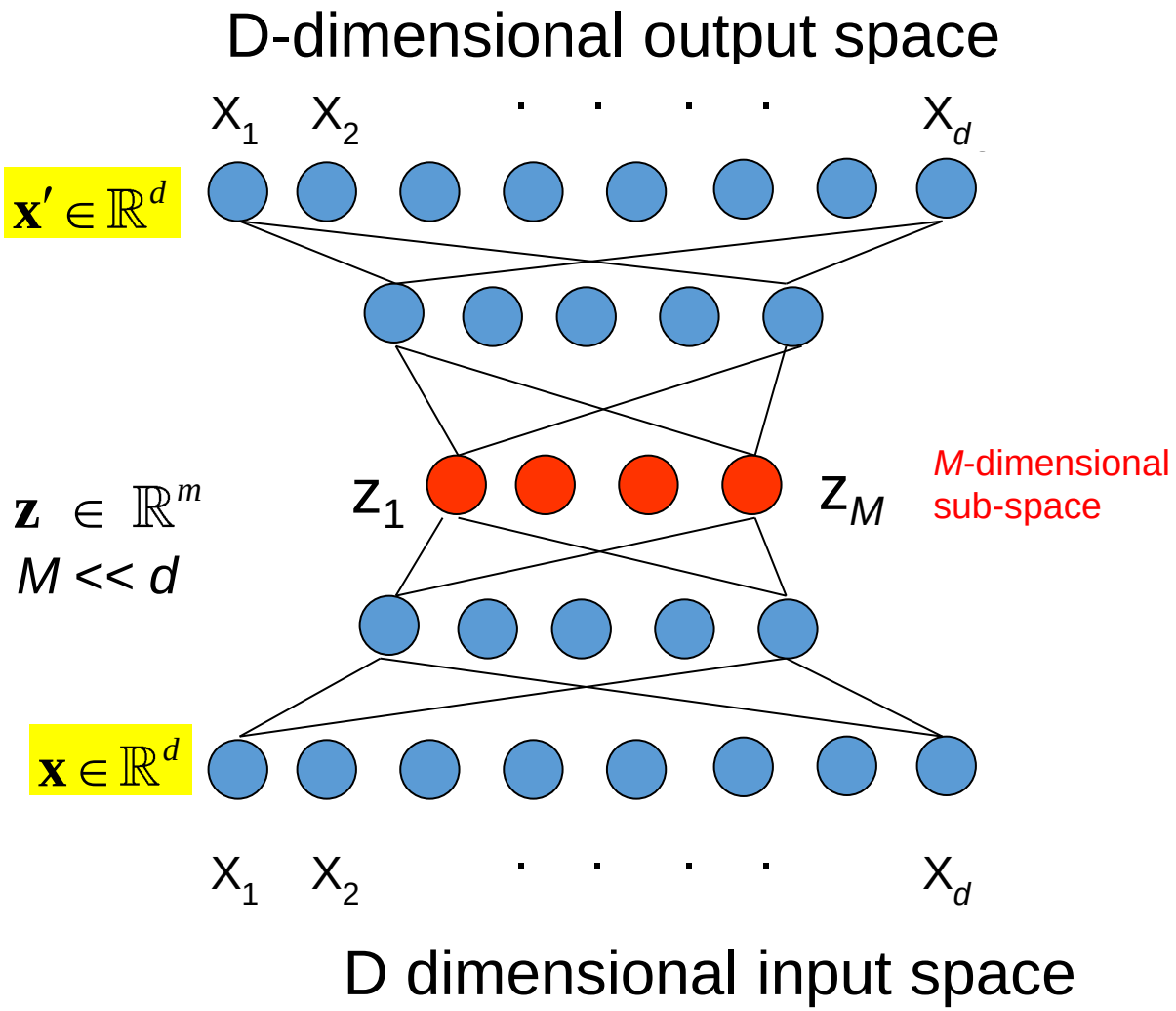


- The cost function to be minimized in order to find the weights is given by:

$$\ell(\mathbf{x}, \mathbf{x}') = \ell(\mathbf{W}, b) = \| \mathbf{x} - \mathbf{x}' \|^2$$

$$\| \mathbf{x} - \mathbf{x}' \|^2 = \| \mathbf{x} - \sigma(\mathbf{W}'\sigma(\mathbf{W}\mathbf{x} + b) + b') \|^2$$

Dimensionality Reduction

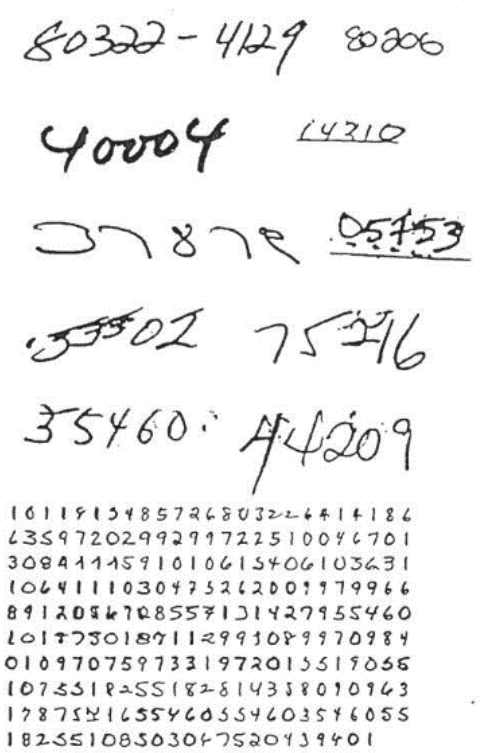


- Transformation of a *d* dimensional input space into a *M*< *d* dimensional output space
- Non-linear component analysis
- The dimensionality of the sub-space must be decided in advance.

MNIST Handwritten Dataset

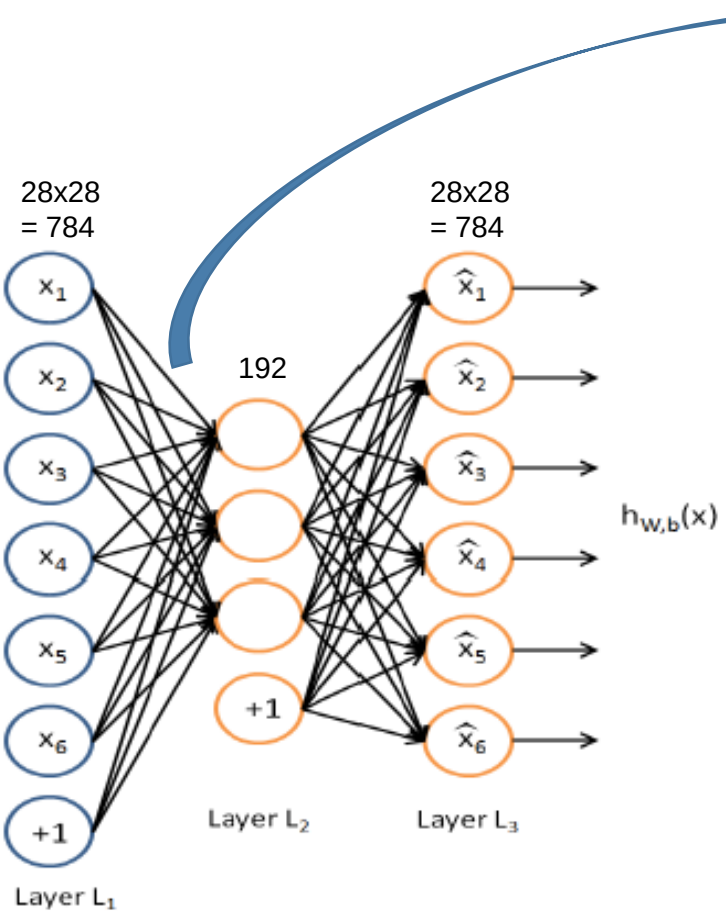
http://yann.lecun.com/exdb/mnist/

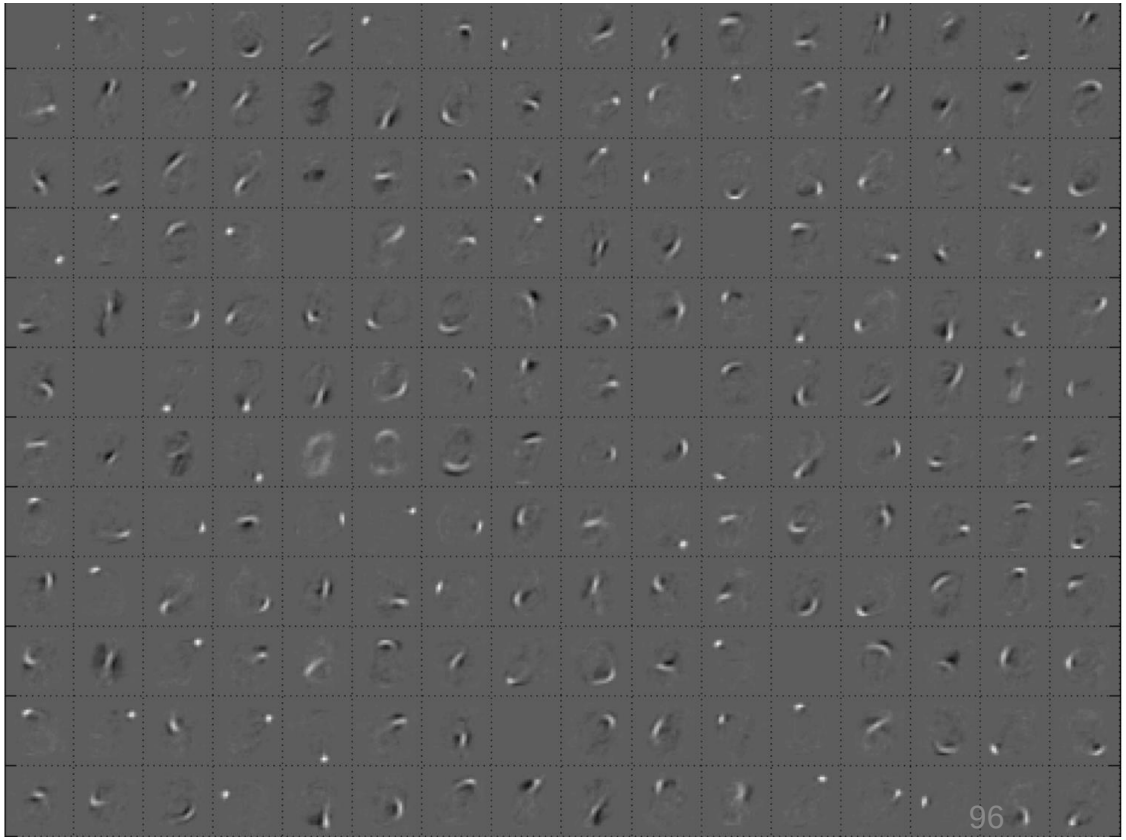
- Database of 70k handwritten digits
  - o Training set: 60k
  - o Test set: 10k
- 28 x 28 pixels
- Best performing classifiers:
  - o Linear classifier: 12% error
  - o Gaussian SVM 1.4% error
  - o ConvNets <1% error

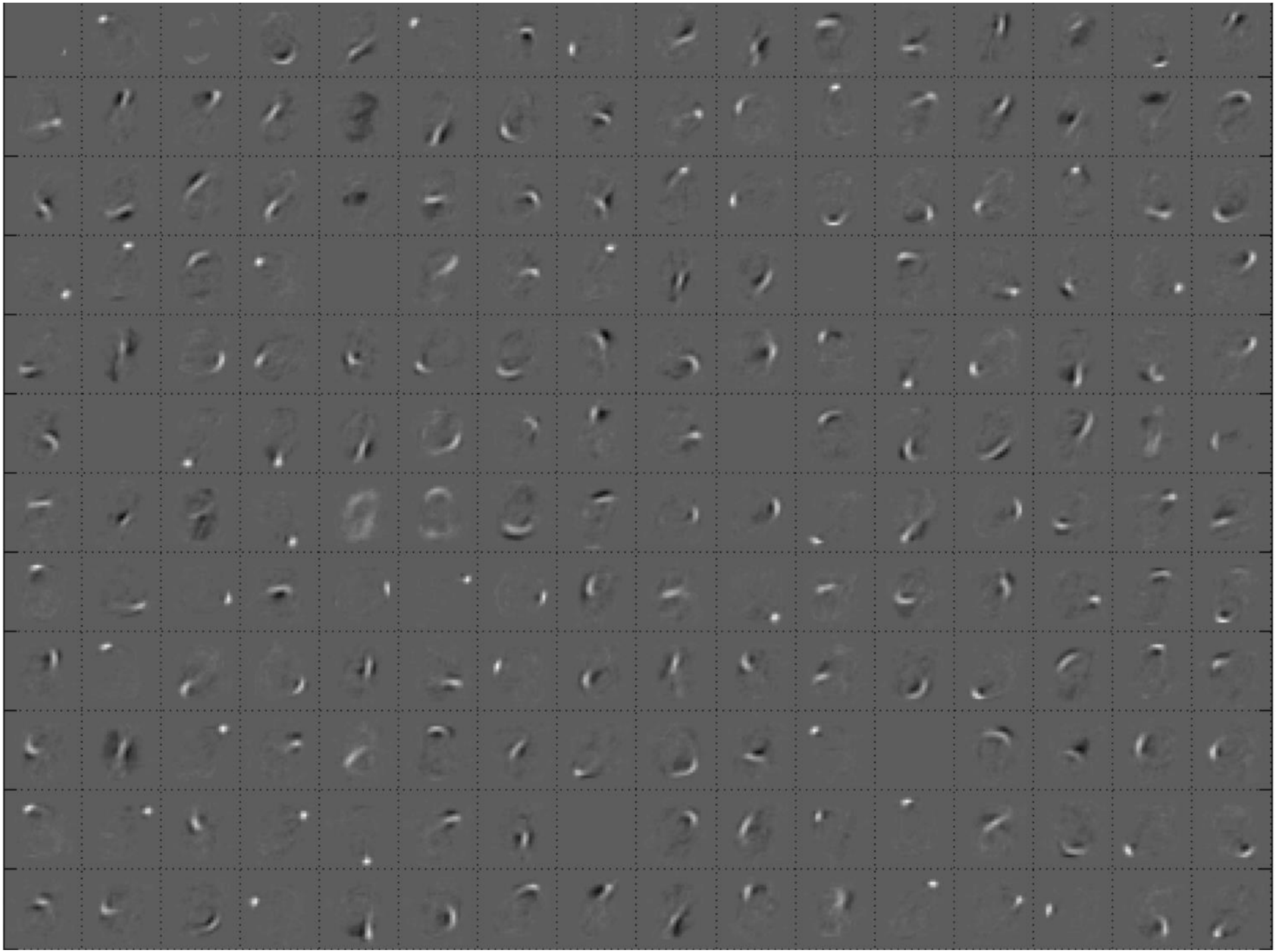


MNIST Handwritten Dataset

- Matrix of weights of size 192x(28x28), 192 hidden units producing 192 bases of 28x28 pixels

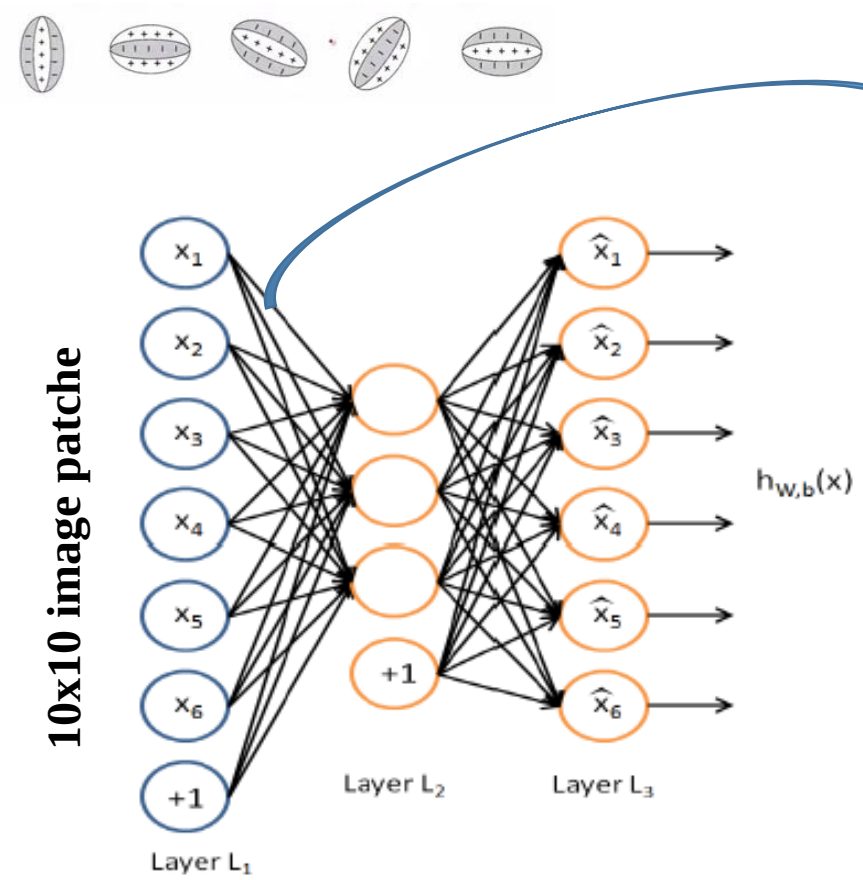


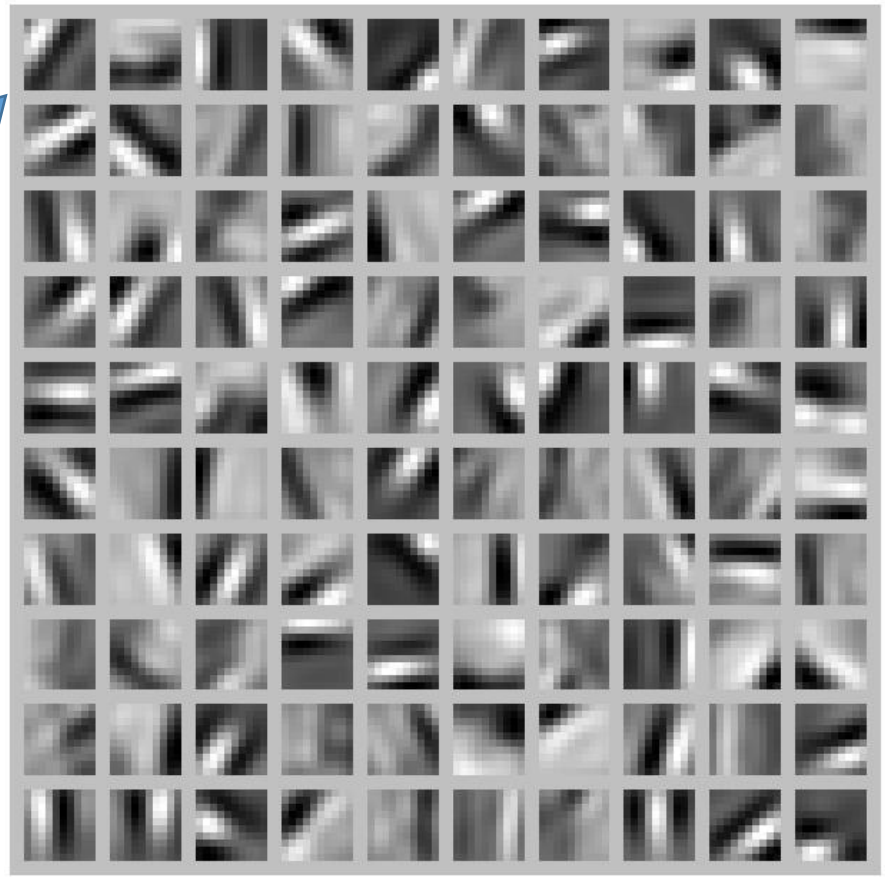




Visualization of the Weights for Input Image Patches

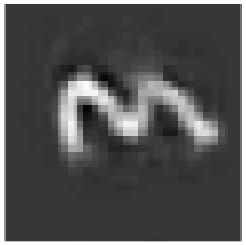
- An autoencoder trained with 100 hidden units on 10x10 image patches as inputs we get the following result (bases).
- Different hidden units have learned to detect edges at different positions and orientations in the image.

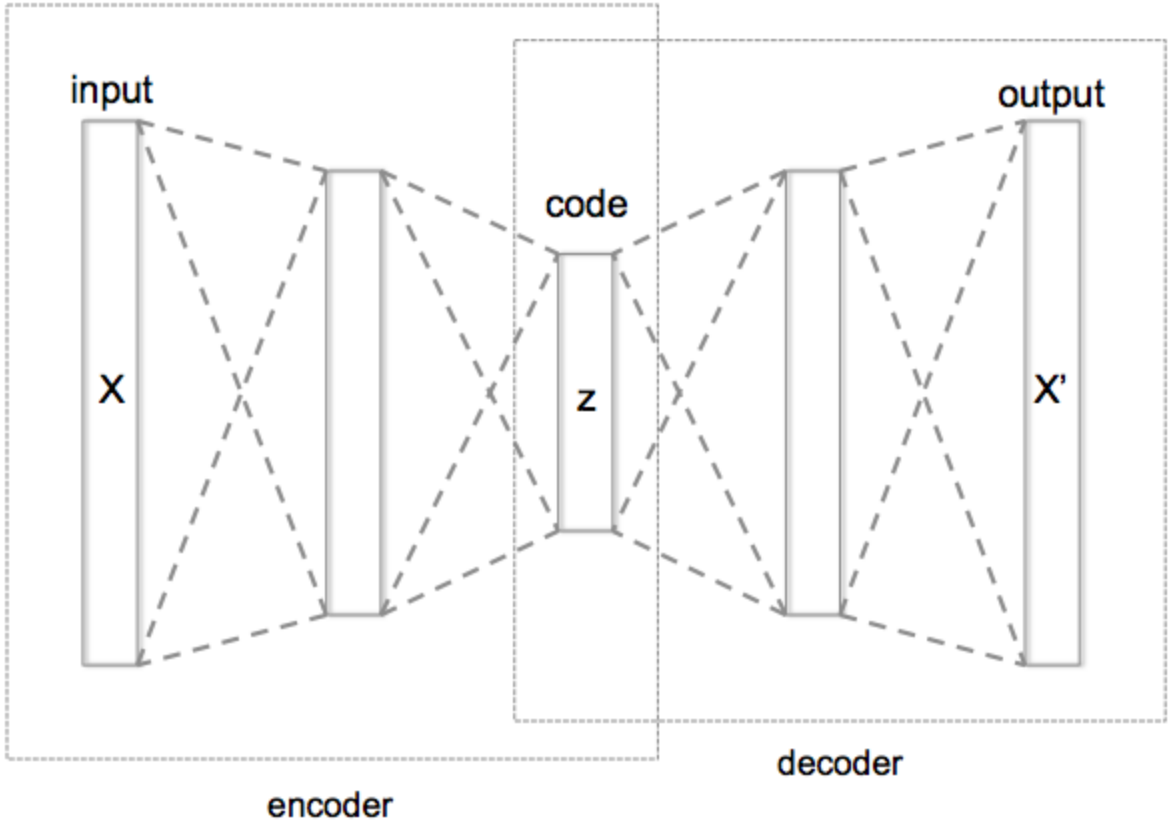


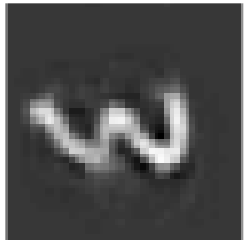


Deep Autoencoders

- We can also have multilayer Deep Autoencoders







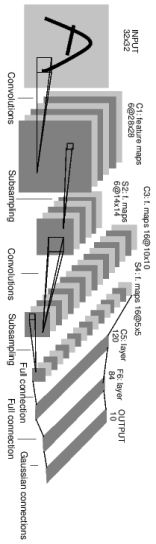
Convolutional Neural Networks (CNN)

2015 ResNET

152

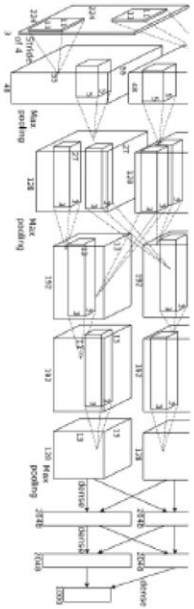
1998 LeNet

5



2012 AlexNet

8



2014 VGG

19

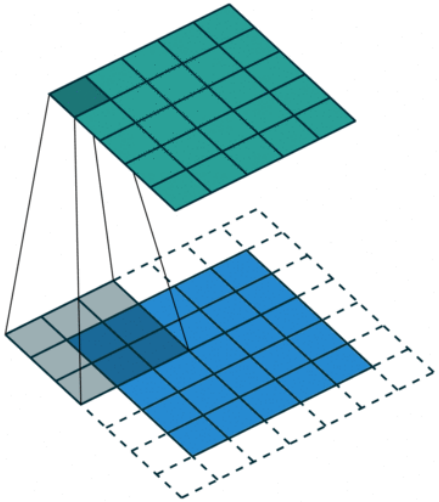
2014 GoogLeNet

22

Convolution (Regular with different stride)

$$* \begin{bmatrix} 0 & 1 & 2 \\ 2 & 2 & 0 \\ 0 & 1 & 2 \end{bmatrix} = \text{Filtered Image}$$

Filter or Kernel



3_0	3_1	2_2	1	0
0_2	0_2	1_0	3	1
3_0	1_1	2_2	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

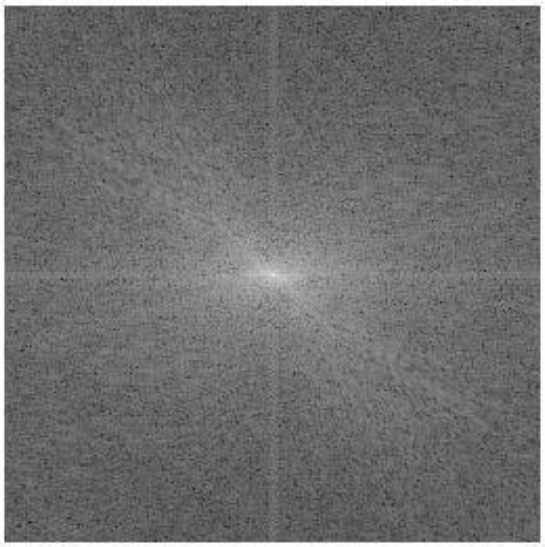
Example of Convolution in Frequency Domain

Image Lena

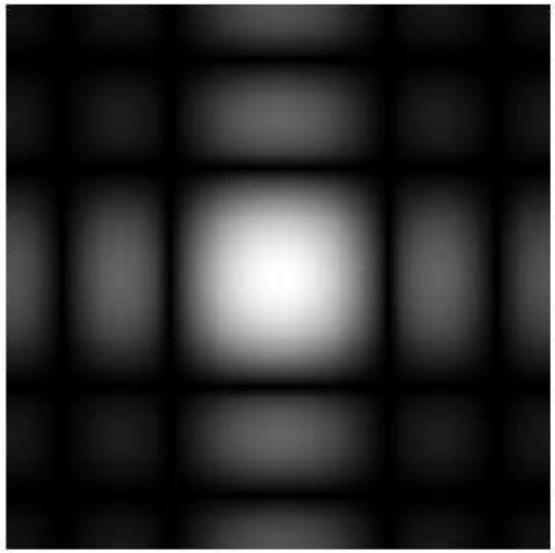
$$h = \frac{1}{25}\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Low-Pass Filter

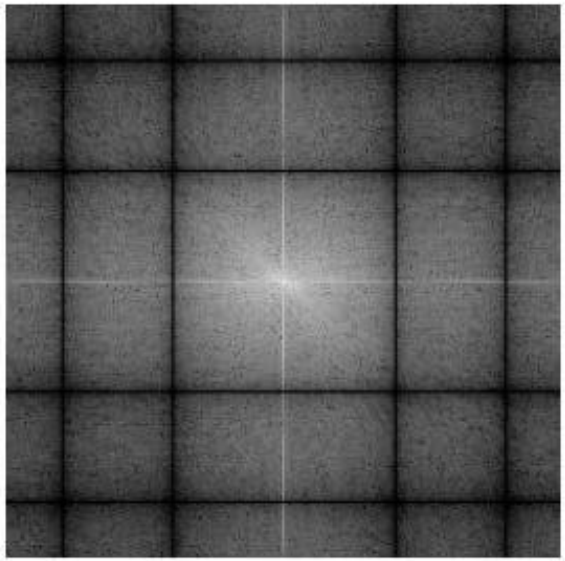
Filtered Image



Spectrum of Lena



Spectrum of Filter



Spectrum of Filtered Image

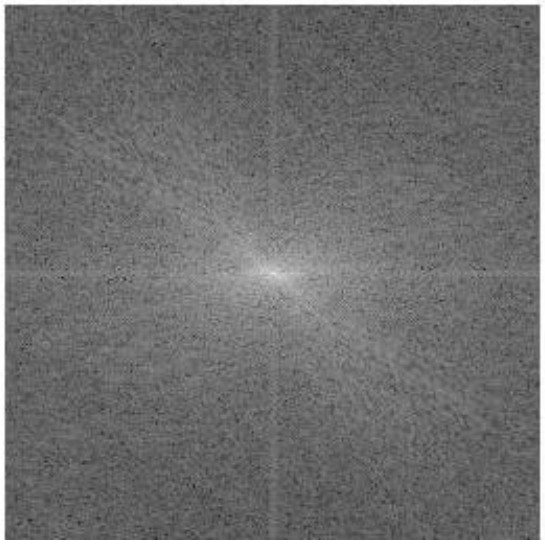
Example of Convolution in Frequency Domain

Image Lena

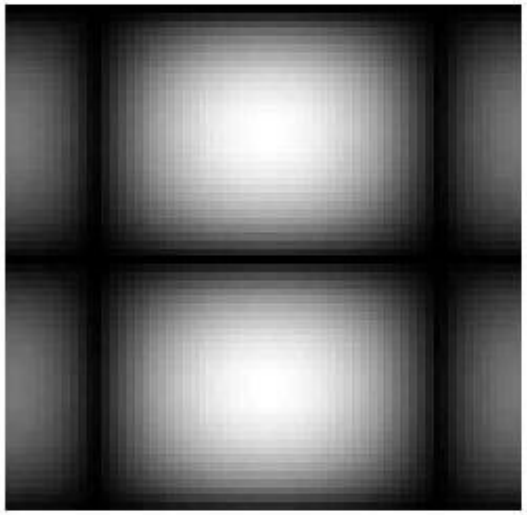
$$H_1 = \frac{1}{3}\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Filter

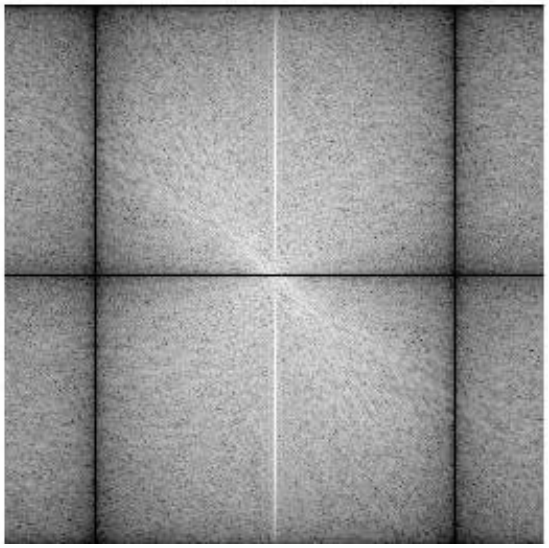
Filtered Image



Spectrum of Lena



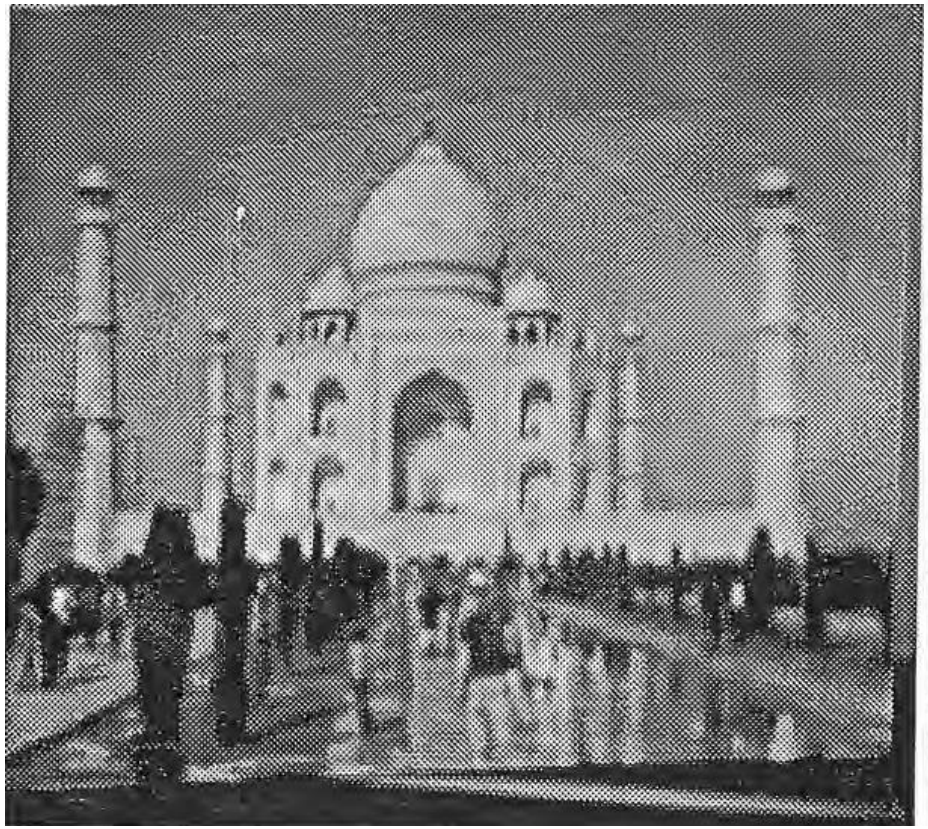
Spectrum of Filter

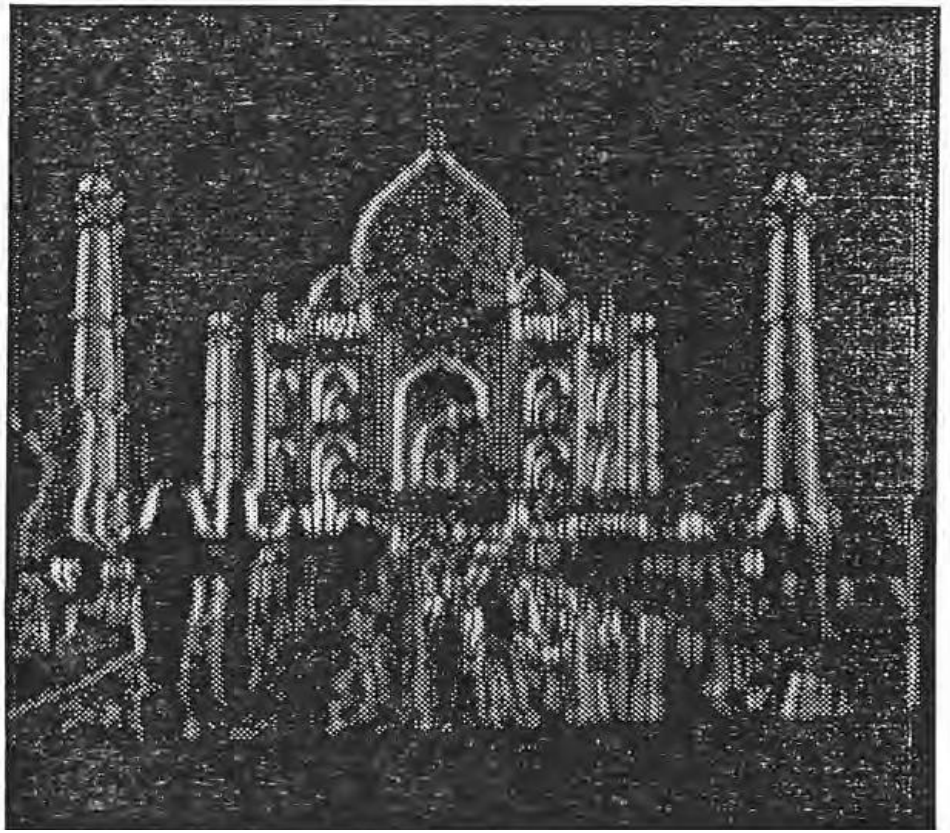


Spectrum of Filtered Image

A 2-D Filter: Vertical Edge Detector Using 90° Sobel Operator

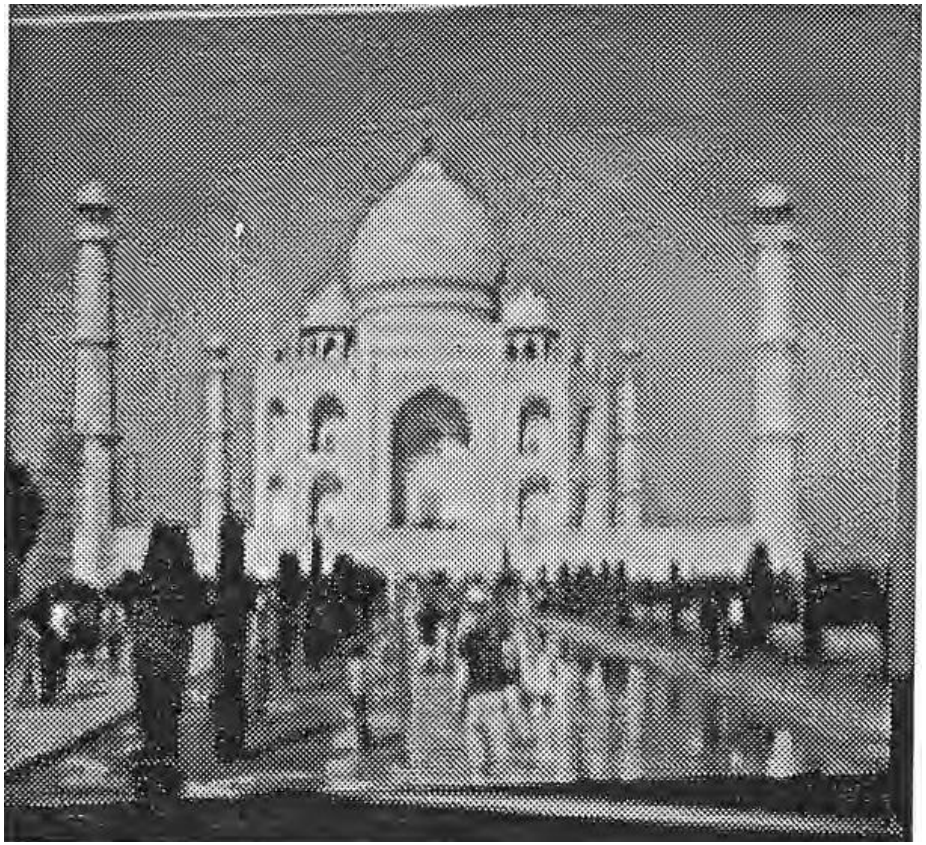
$$h(x.y)=\begin{bmatrix}-1 & 0 & 1\\ -2 & 0 & 2\\ -1 & 0 & 1\end{bmatrix}$$

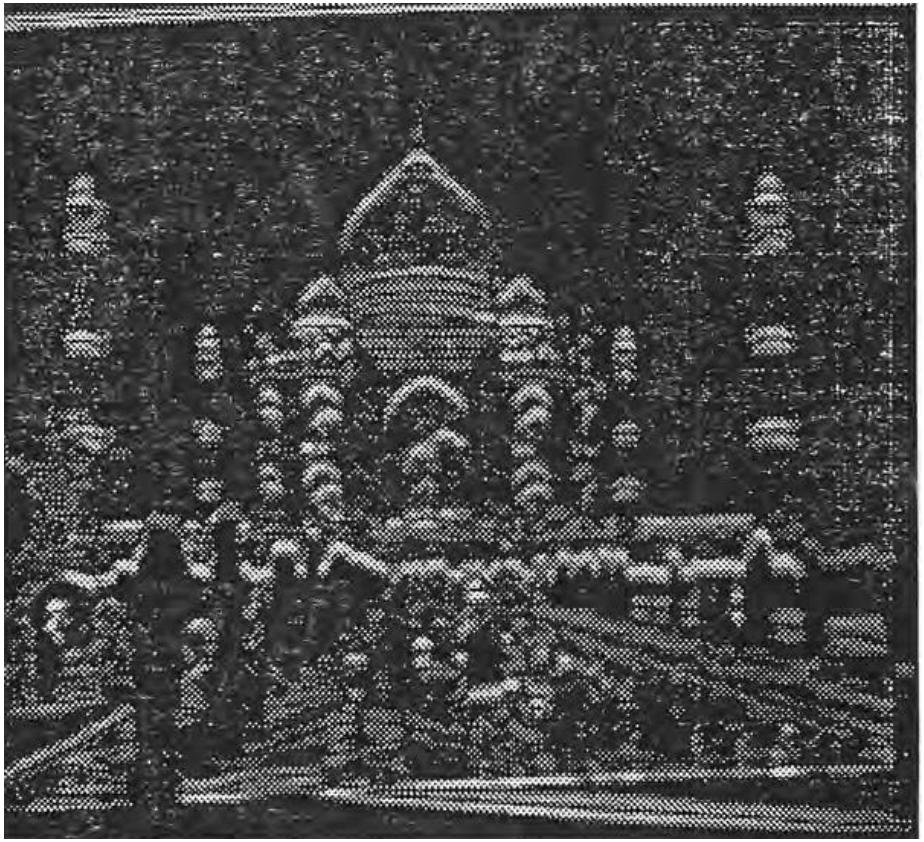




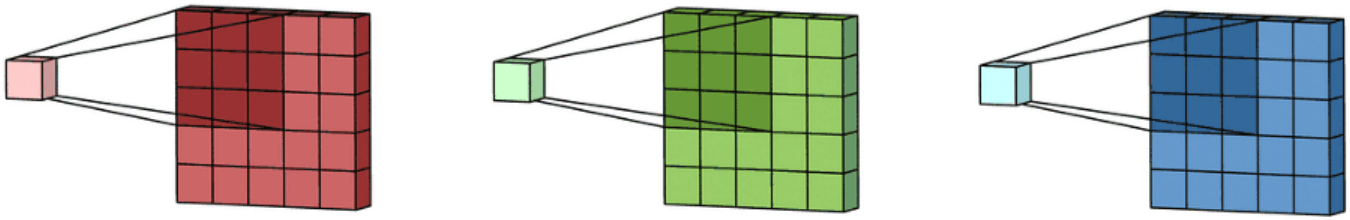
A 2D Filter: Horizontal Edge Detector Using 0° Sobel Operator

$$h(x.y)=\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$





2D Convolution Using Multiple Kernels



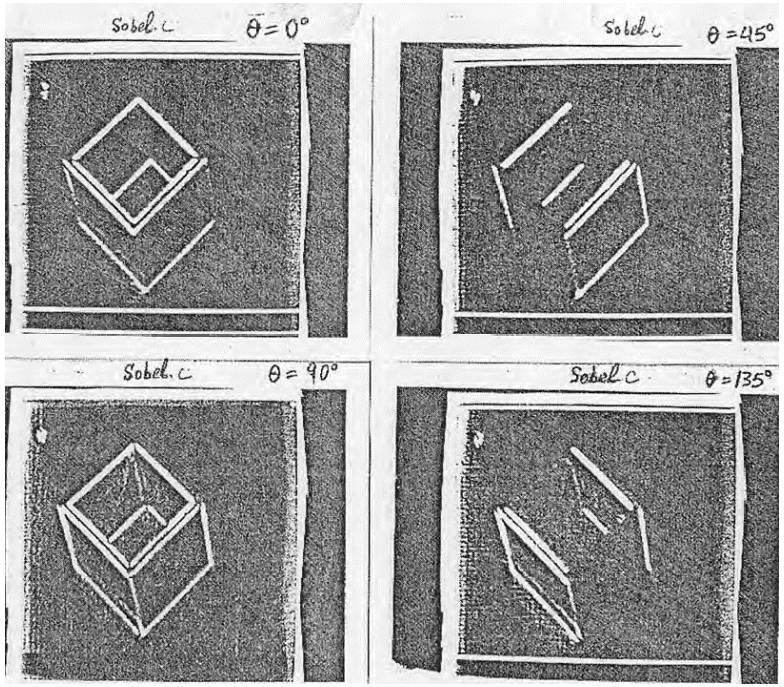
SOBEL DIRECTIONAL OPERATORS

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}, \quad \begin{bmatrix} 2 & 1 & 0 \\ 1 & 0 & -1 \\ 0 & -1 & -2 \end{bmatrix}$$

$\theta = 0°$ $\theta = 45°$

$$, \quad \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}, \quad \begin{bmatrix} 0 & -1 & -2 \\ 1 & 0 & -1 \\ 2 & 1 & 0 \end{bmatrix}$$

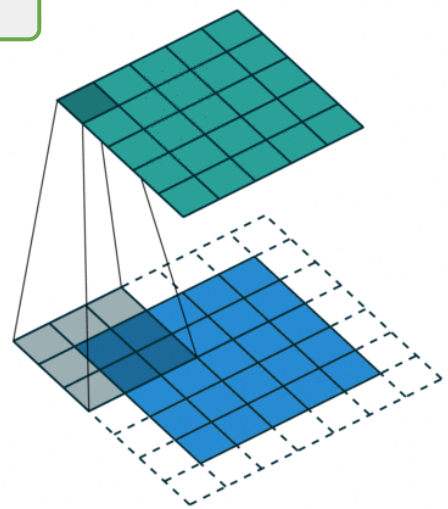
$\theta = 90°$ $\theta = 135°$



Convolution (Regular with different strides)

Regular convolutions with s=1

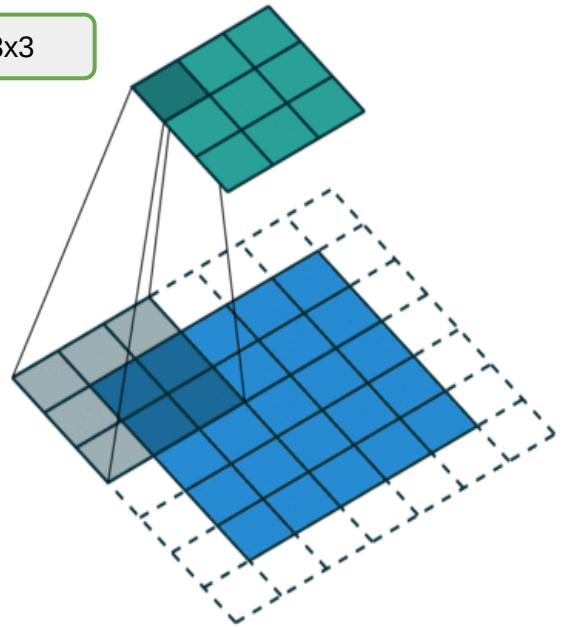
Output = 5x5



Input = 5x5
with zero-padding at border = 6x6
(stride=1)

Regular convolutions with s=2

Output = 3x3



Input = 5x5
with zero-padding at border = 6x6
(stride=2)

Stride: The stride defines the step size of the kernel when traversing the image. While its default is usually 1, we can use a stride of 2 for downsampling an image similar to MaxPooling.

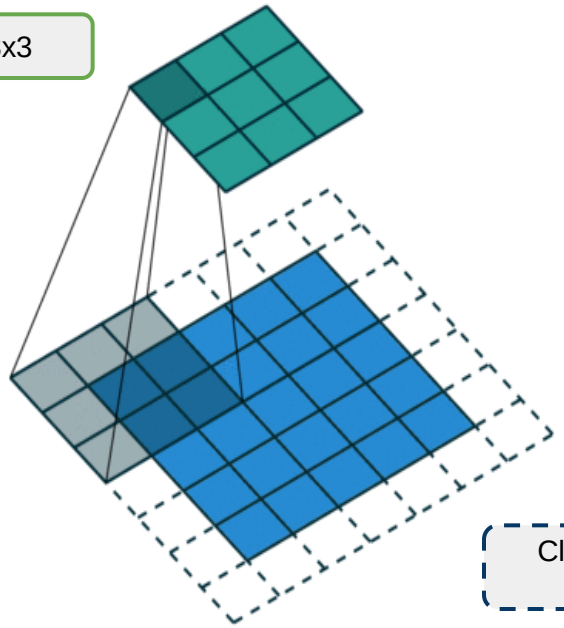
Fractional Convolution

See also:
http://datascience.stackexchange.com/questions/6107/what-are-deconvolutional-layers
https://www.reddit.com/r/MachineLearning/comments/4bbod7/which_deep_learning_frameworks_support_fractional/

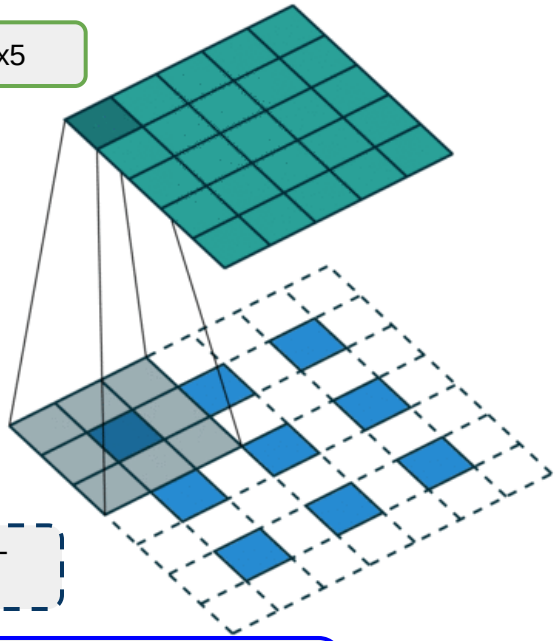
Regular convolutions

Fractionally-strided convolutions

Output = 3x3



Output = 5x5



Filter size=3x3

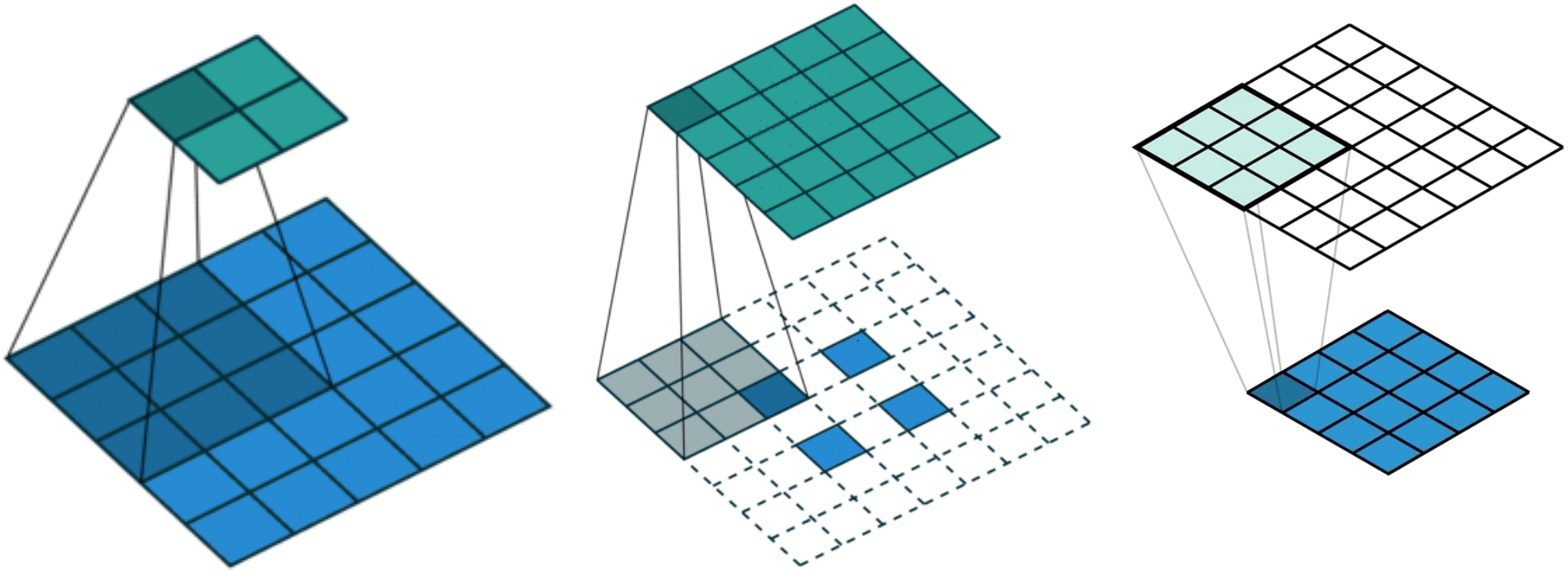
Clear dashed squares = zero-padded inputs

Input = 5x5
with zero-padding at border = 6x6
(stride=2)

Input = 3x3
Interlace zero-padding with inputs = 7x7
(stride=2)

Transposed Convolutions

- Some sources use the name deconvolution, which is inappropriate because it's not a deconvolution.
- A transposed convolutional layer carries out a regular convolution but reverts its spatial transformation.

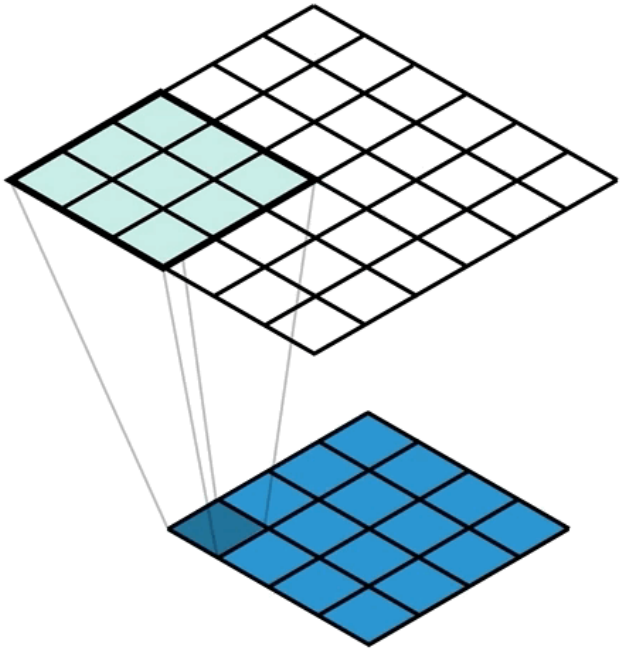


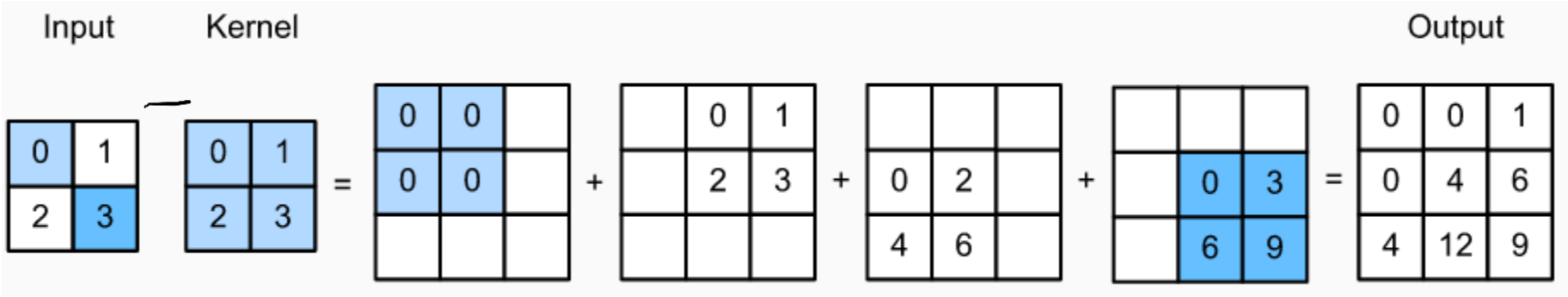
Regular convolution with s=2

Transposed convolution

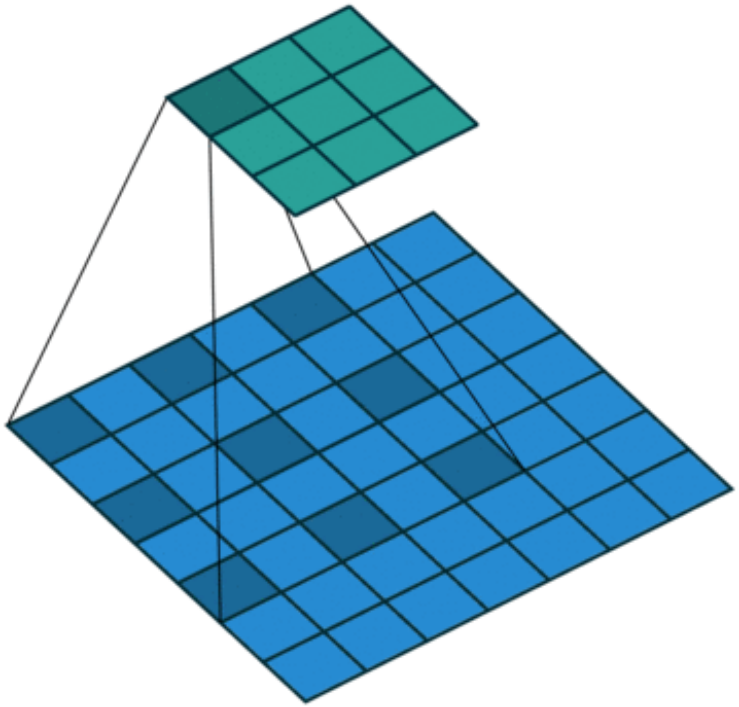
Transposed Convolutions

- We take this value and ‘distribute’ it to a neighborhood of points in the output. A kernel defines exactly how we do this, and for each output cell we multiply the input value by the corresponding weight of the kernel. We repeat this process for every value in the input and accumulate values in each output cell. Check out the Figure below for an example of this accumulation (with unit input and kernel).
- Our kernel values are hidden in the animation above, but it is important to understand that the kernel is defining the amount of the input value that’s being distributed to each of the output cells (in the neighbourhood). We can see this more clearly in the spreadsheet in Figure below, even with a unit kernel.





Dilated Convolutions



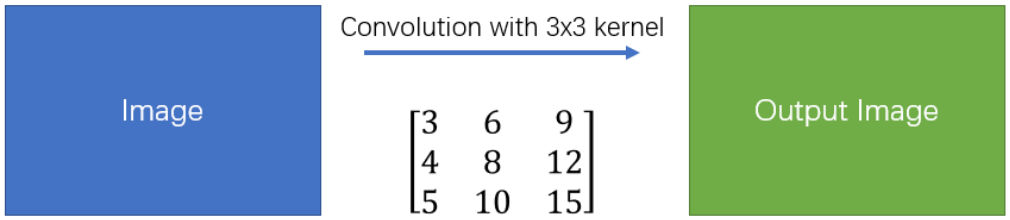
Dilated convolution

- Dilated convolutions introduce another parameter to convolutional layers called the **dilation rate**. This defines a spacing between the values in a kernel. A 3x3 kernel with a dilation rate of 2 will have the same field of view as a 5x5 kernel, while only using 9 parameters. Imagine taking a 5x5 kernel and deleting every second column and row.
- This delivers a wider field of view at the same computational cost. Dilated convolutions are particularly popular in the field of real-time segmentation. ***Use them if you need a wide field of view and cannot afford multiple convolutions or larger kernels.***

Separable Convolutions

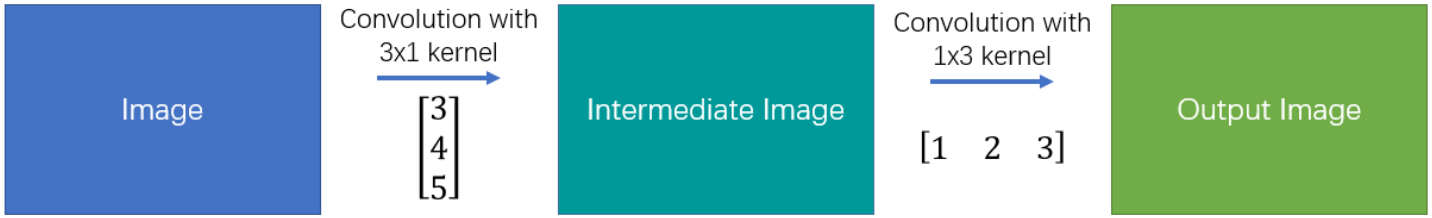
$$\begin{bmatrix} 3 & 6 & 9 \\ 4 & 8 & 12 \\ 5 & 10 & 15 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$$

Simple Convolution



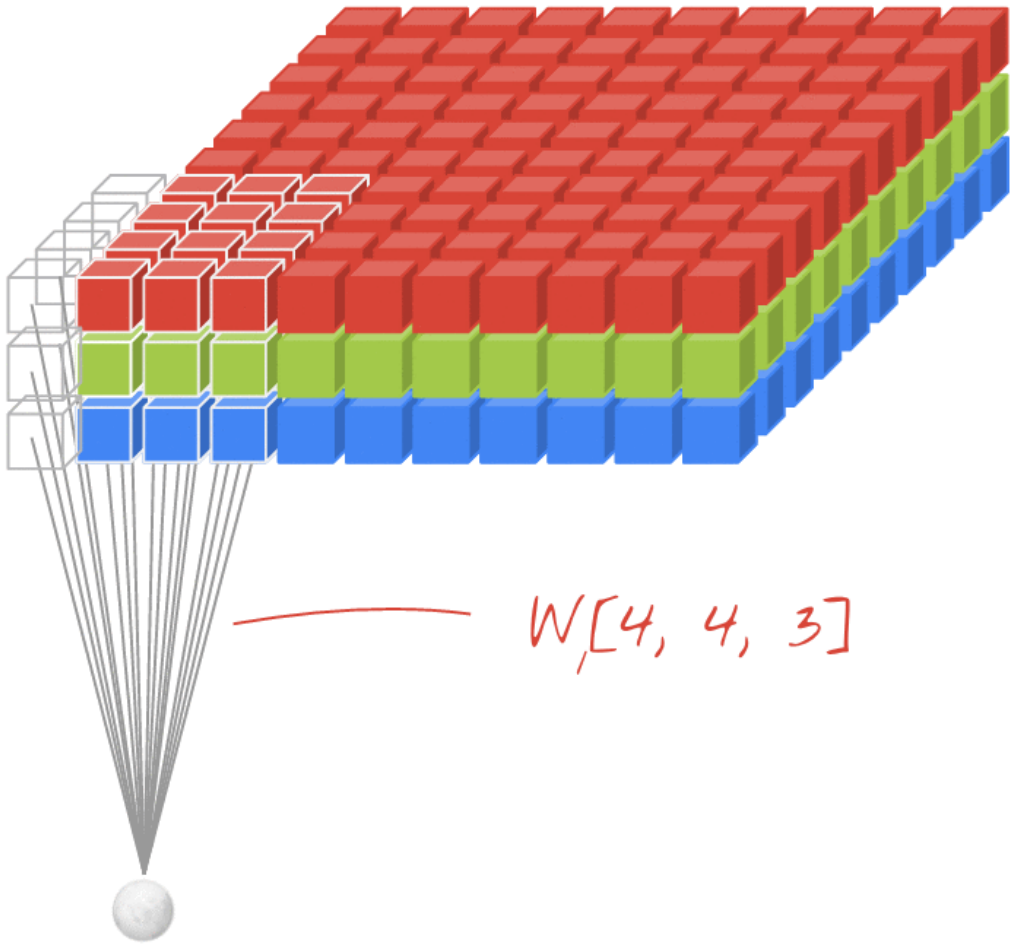
$$\begin{bmatrix} 3 & 6 & 9 \\ 4 & 8 & 12 \\ 5 & 10 & 15 \end{bmatrix}$$

Spatial Separable Convolution

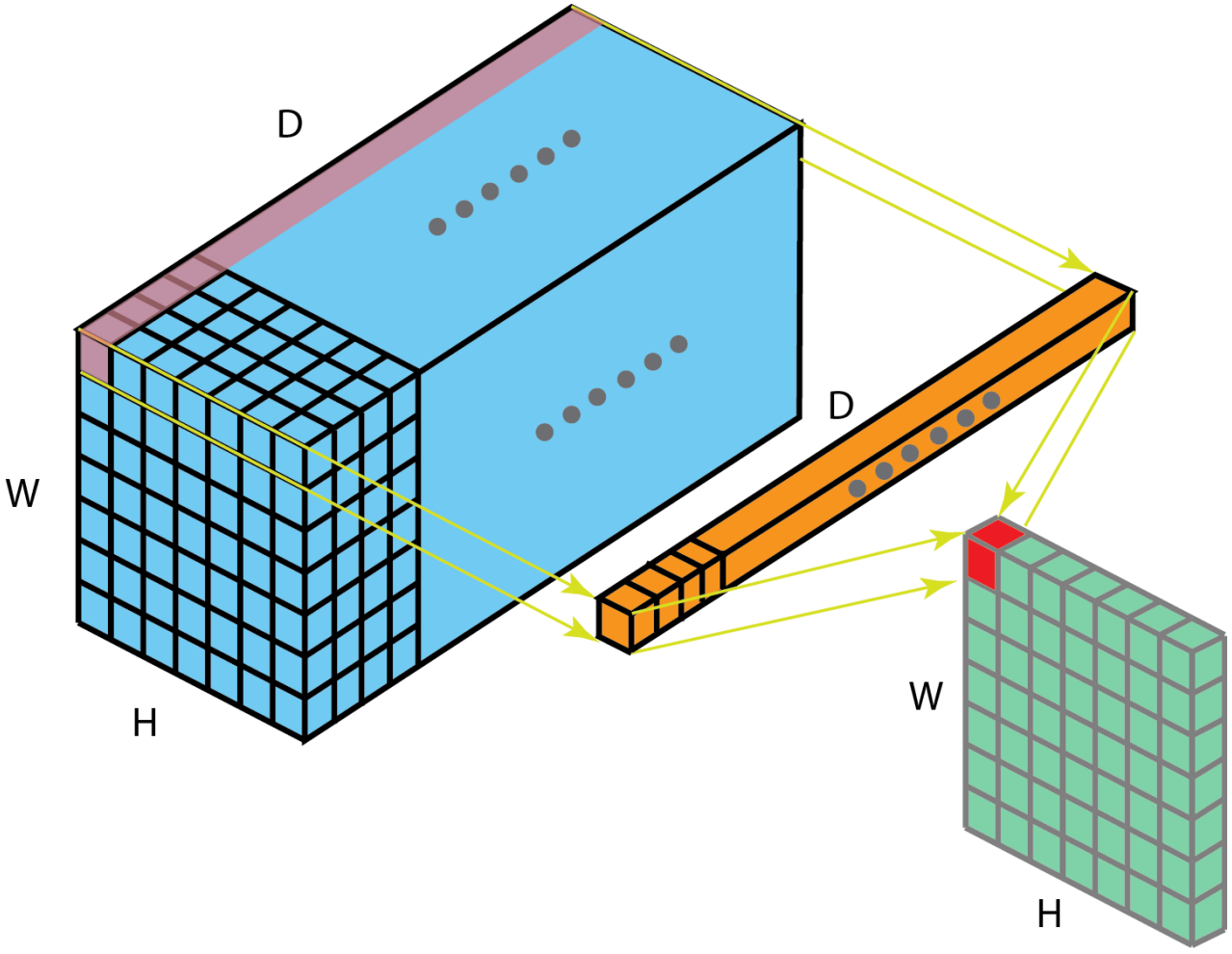


$$\begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix} \qquad \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$$

Regular Convolution with an RGB Image

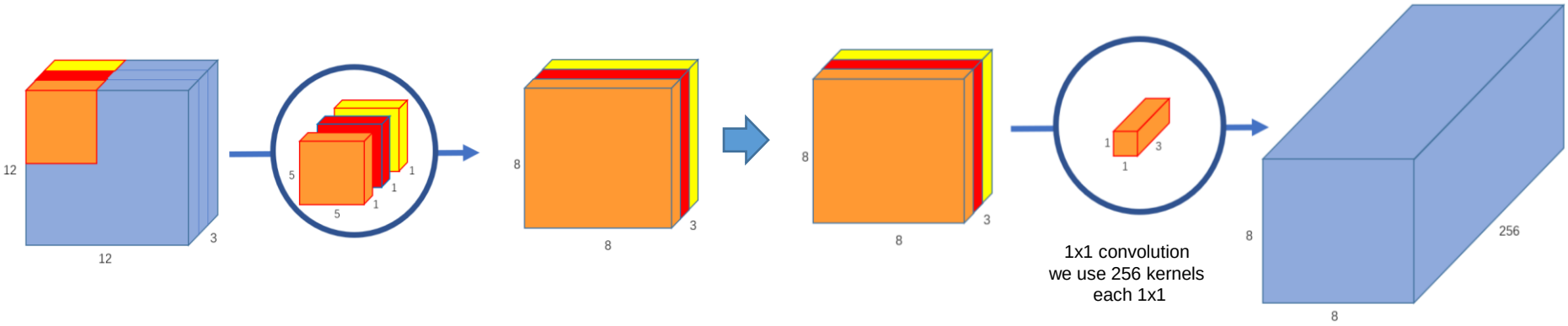


1x 1 Convolution

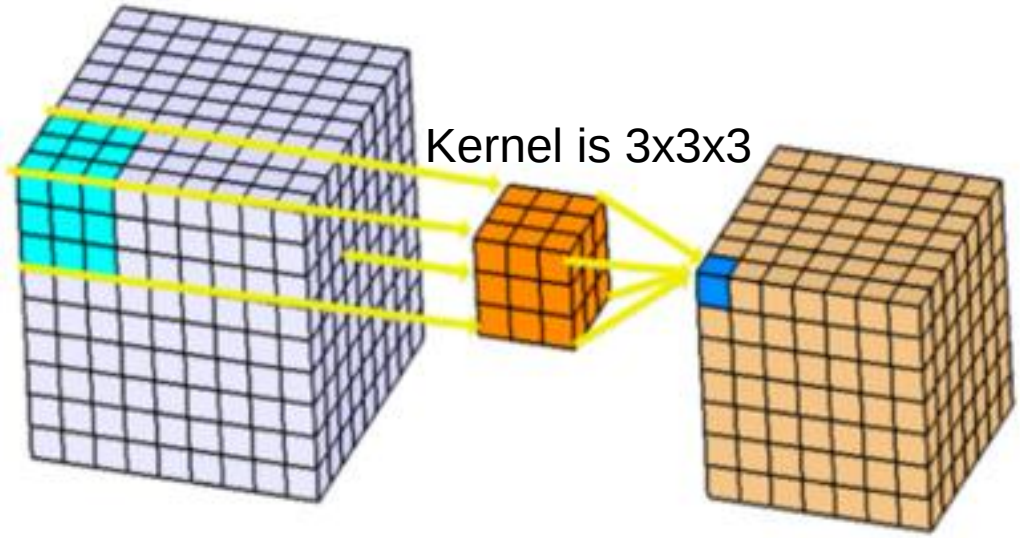


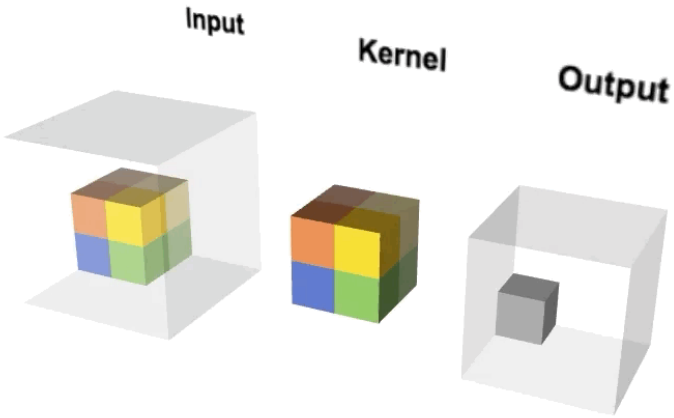
Depthwise Separable Convolutions

- Each 5x5x1 kernel iterates 1 channel of the image (note: **1 channel**, not all channels), getting the scalar products of every 25 pixel group, giving out a 8x8x1 image. Stacking these images together creates a 8x8x3 image.
- Now, we need to increase the number of channels of each image.
- The pointwise convolution is so named because it uses a 1x1 kernel, or a kernel that iterates through every single point.
- We can create 256 1x1x3 kernels that output a 8x8x1 image each to get a final image of shape 8x8x256.
- Depthwise separable convolutions are also used for mobile devices because of their efficient use of parameters.



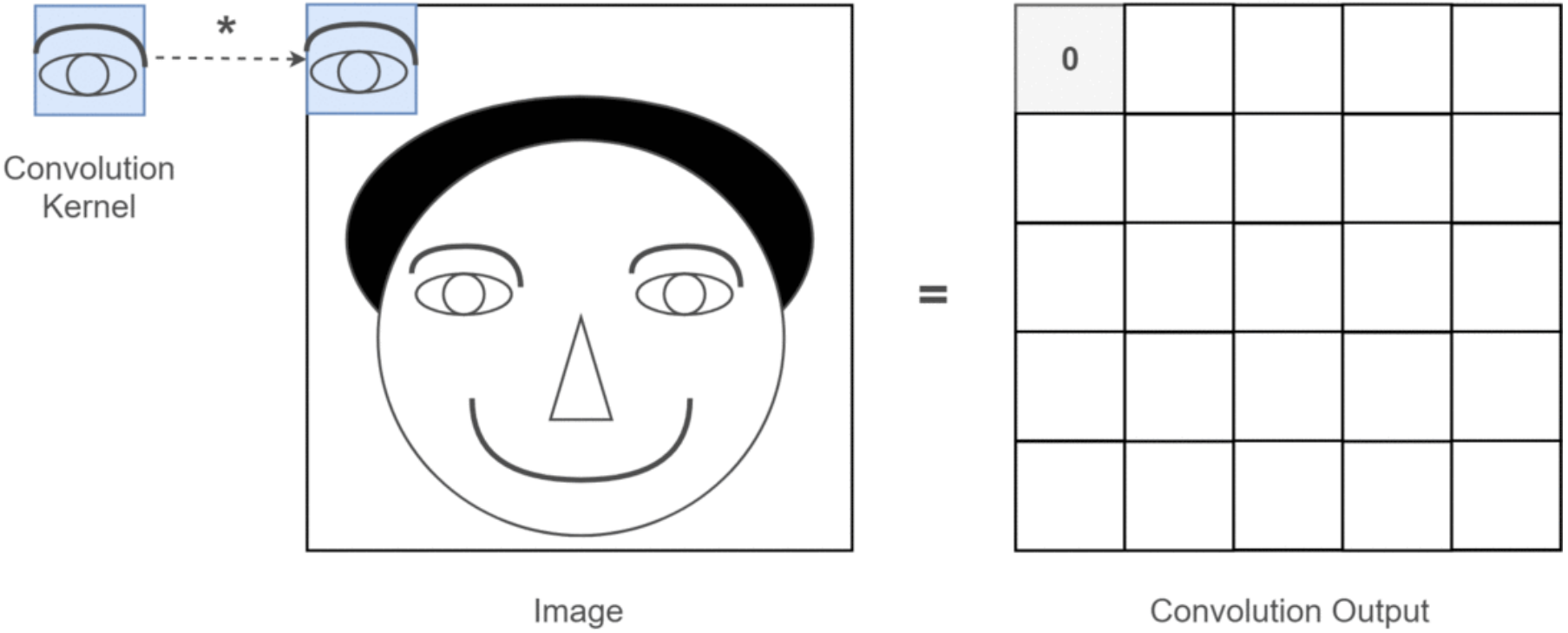
3-Dimensional Convolution





Convolution vs Correlation

- To detect eyes, perform correlation between an eye template with the face.
- This idea can be used for specific object detection (e.g., face, vehicle, targets).



NN for Handwritten Digits and Character Recognition

MNIST dataset

- Handwritten character recognition is more difficult than printed character recognition since there are various writing styles of English. Fig. 1 illustrates the variations. Discrete characters are easier to recognize than the cursive script writings where character segmentation is required.
- Most of the techniques are based on segmentation of the word into characters followed by recognition of each character.
- Each character can be recognized by segmenting it into strokes (descender and ascender) and a set of rules are used to connect these strokes to generate a symbol, as shown in the Fig. 2. This is called analysis-by-synthesis or recognition-by-generation.
- We will use MNIST dataset consisting of 9298 segmented numerals digitized from handwritten Zip-codes, as shown in Figure . The training set consists of 7291 examples and the remaining 2007 were used for testing the network.

Fig. 1

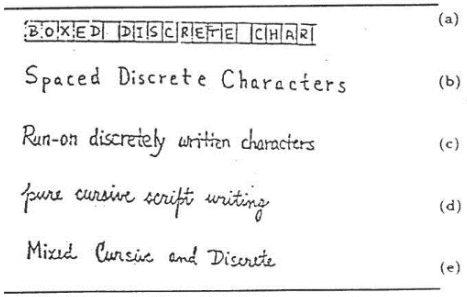
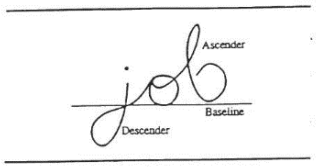
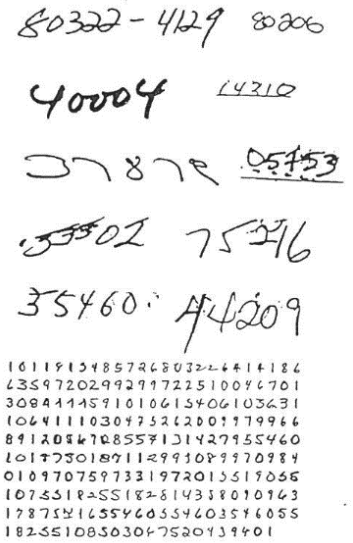


Fig. 2

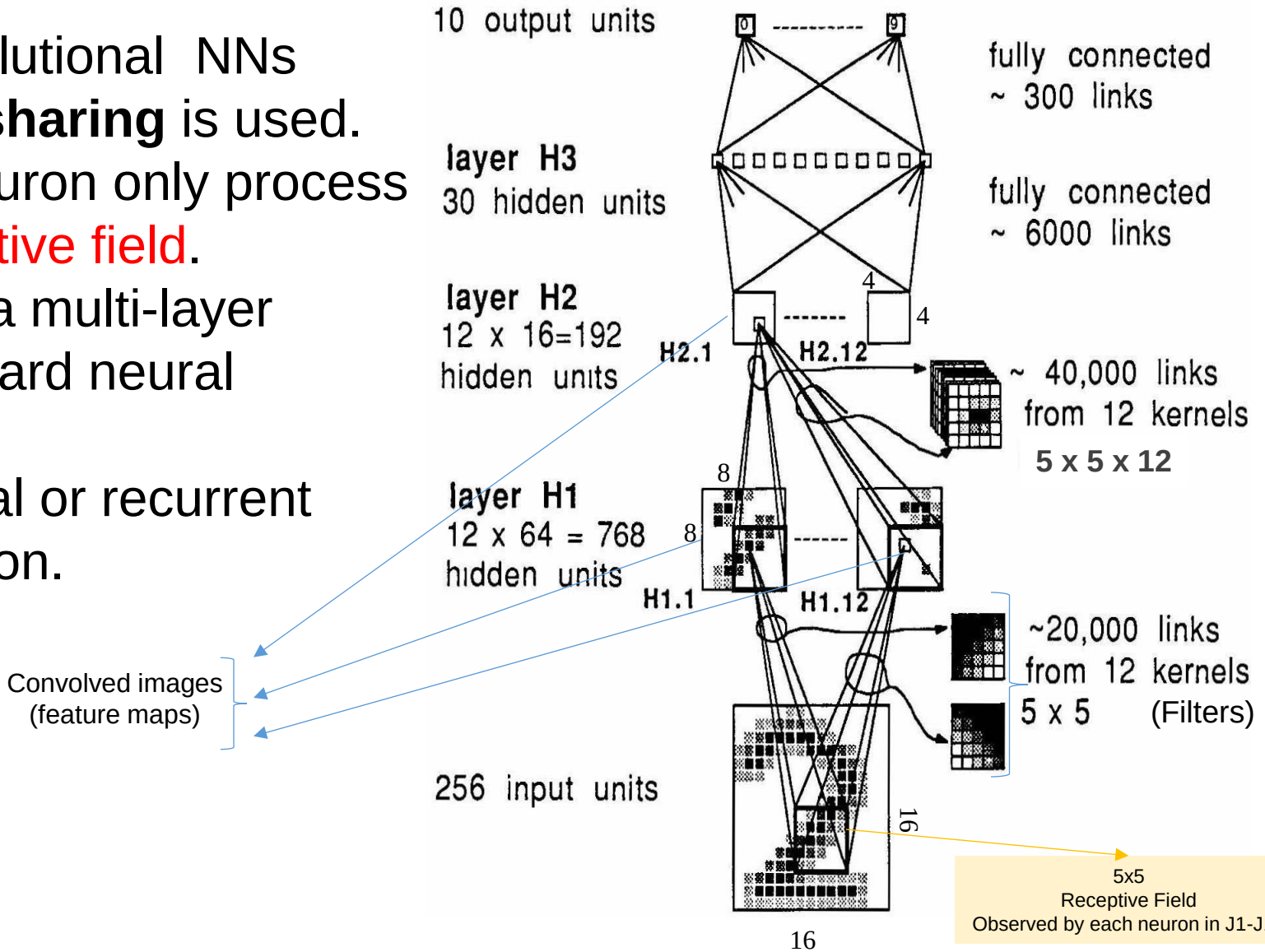




MNIST dataset

Convolutional Neural Networks (CNN)

- In convolutional NNs **weight sharing** is used.
- Each neuron only process its receptive field.
- It is still a multi-layer feedforward neural network
- No lateral or recurrent connection.



Convolutional Neural Networks (CNN)

- In convolutional NNs **weight sharing** is used.
- Each neuron only process its receptive field.
- It is still a multi-layer feedforward neural network
- No lateral or recurrent connection.

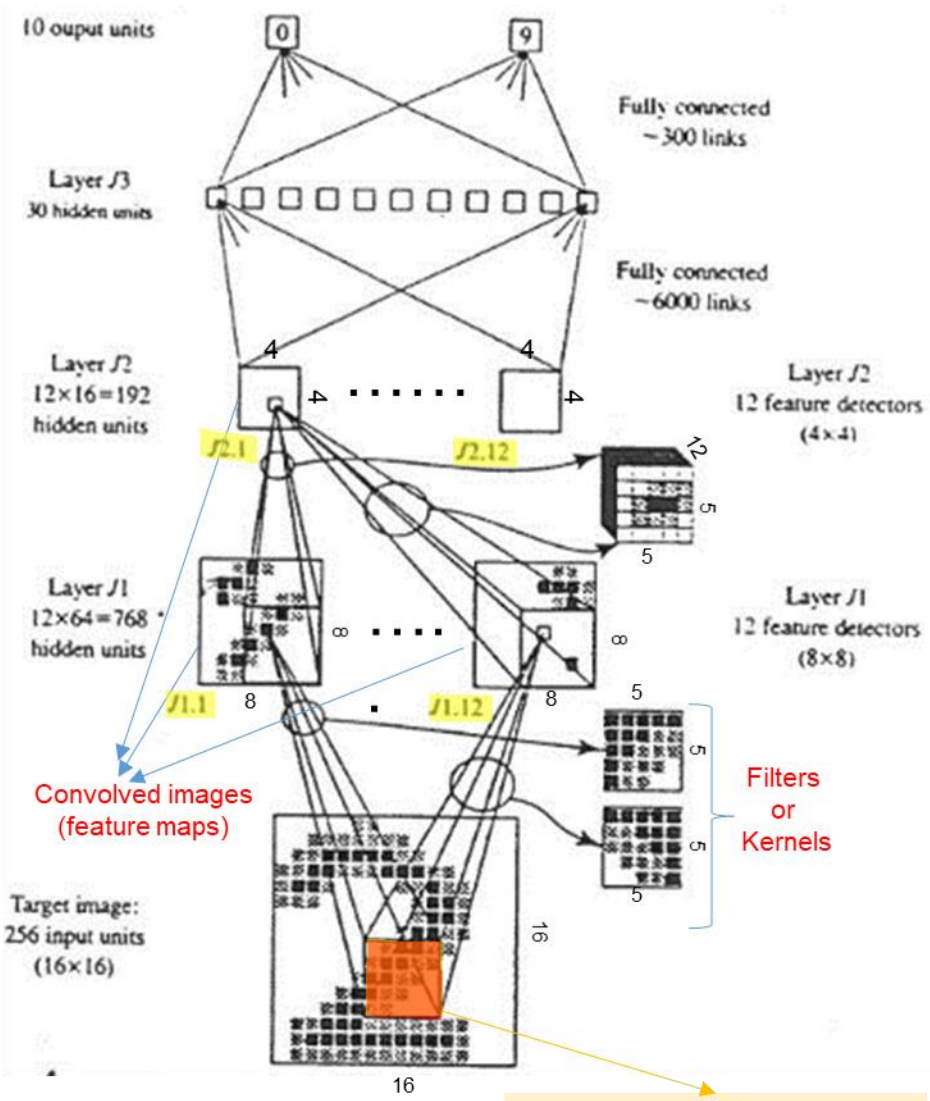


Figure 3

CNN for Handwritten Digits Recognition (MNIST Dataset)

- LeCun et al [3] have proposed a back-propagation network to recognize handwritten digits, their network was very similar to a Time Delay Neural Network (TDNN) or Convolutional Neural Network (CNN).
- The network consists of 16 x 16 normalized image, with 10 output units. This network assumes the input is the image of a single digit. This network was used to read USA Zip-codes.
- The network consists of three hidden layers, Jl, J2, and J3, as shown in Figure 3.
- Jl consists of 12 groups of 64 units per group, each arranged in an 8 x 8 square.
- Each unit in Jl has connections only from the 5 x 5 contiguous square shifts by two input pixels between neighbors in the hidden layer.
- All 64 units within a group have the same 25 weight values so they all detect the same feature, the weight for the bias input for each unit is not same.
- This weight sharing and limitation to processing of 5 x 5 receptive input fields has resulted in [(12 x 64 by (25 + 1)] weights. But only 12x(25+1) distinct weights.
- The second hidden layer J2 consists of 12 groups of 16 units (192), again taking input from 5 x 5 receptive fields.
- Layer J3 has 30 units and is fully connected to the outputs of all units in layer J2. Thus, layer J3 contains 30 x 192 weights plus 30 weights from the bias inputs (thresholds).

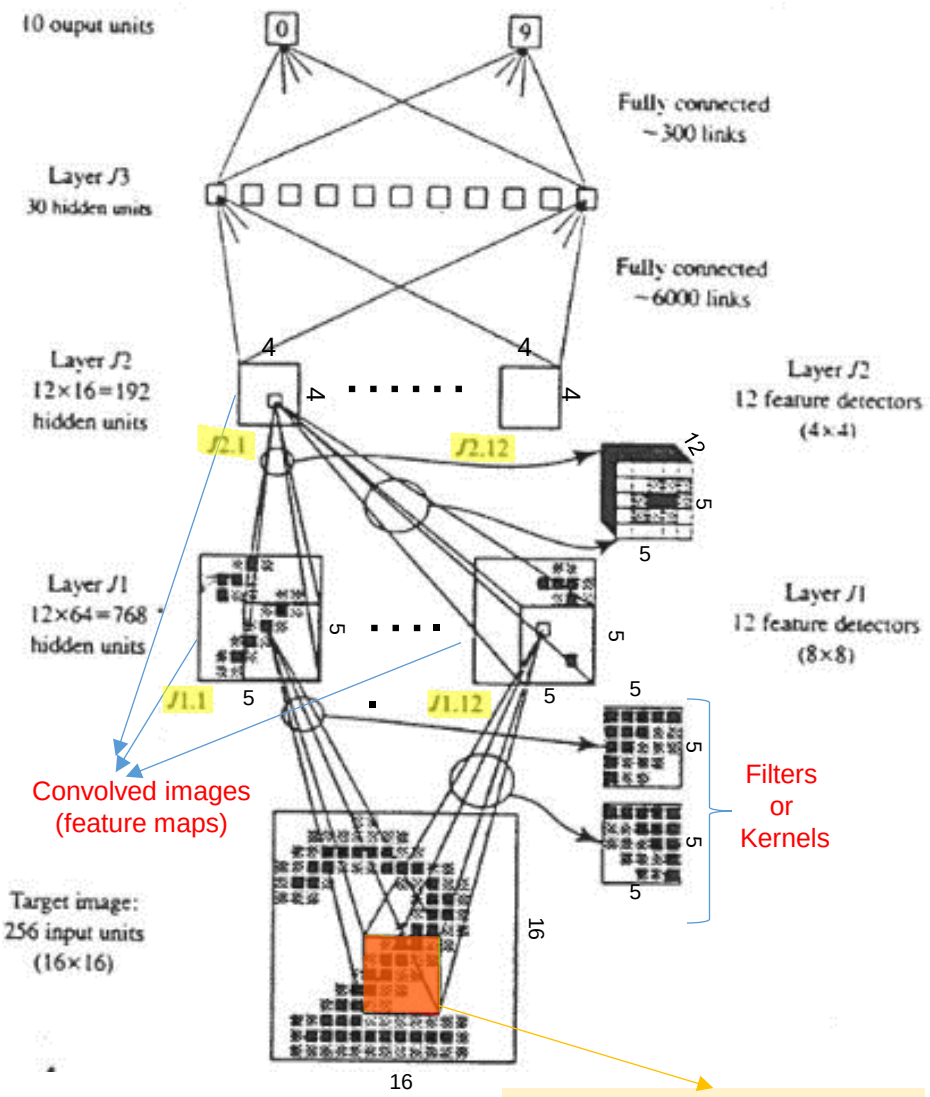
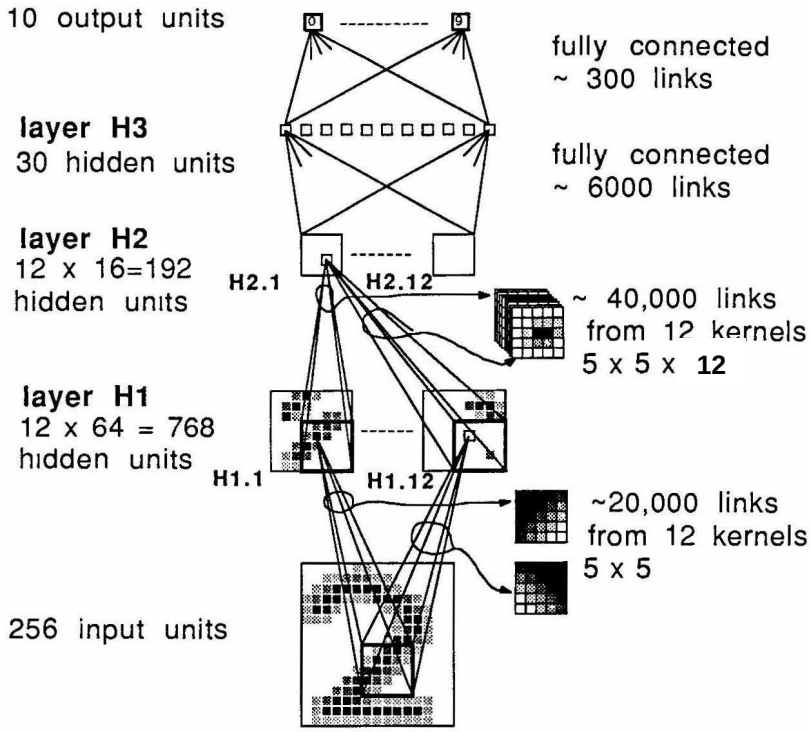
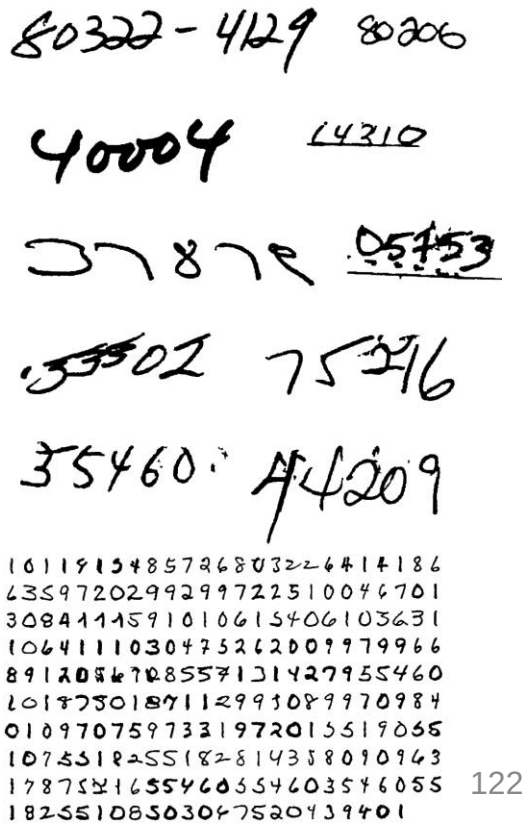


Figure 3

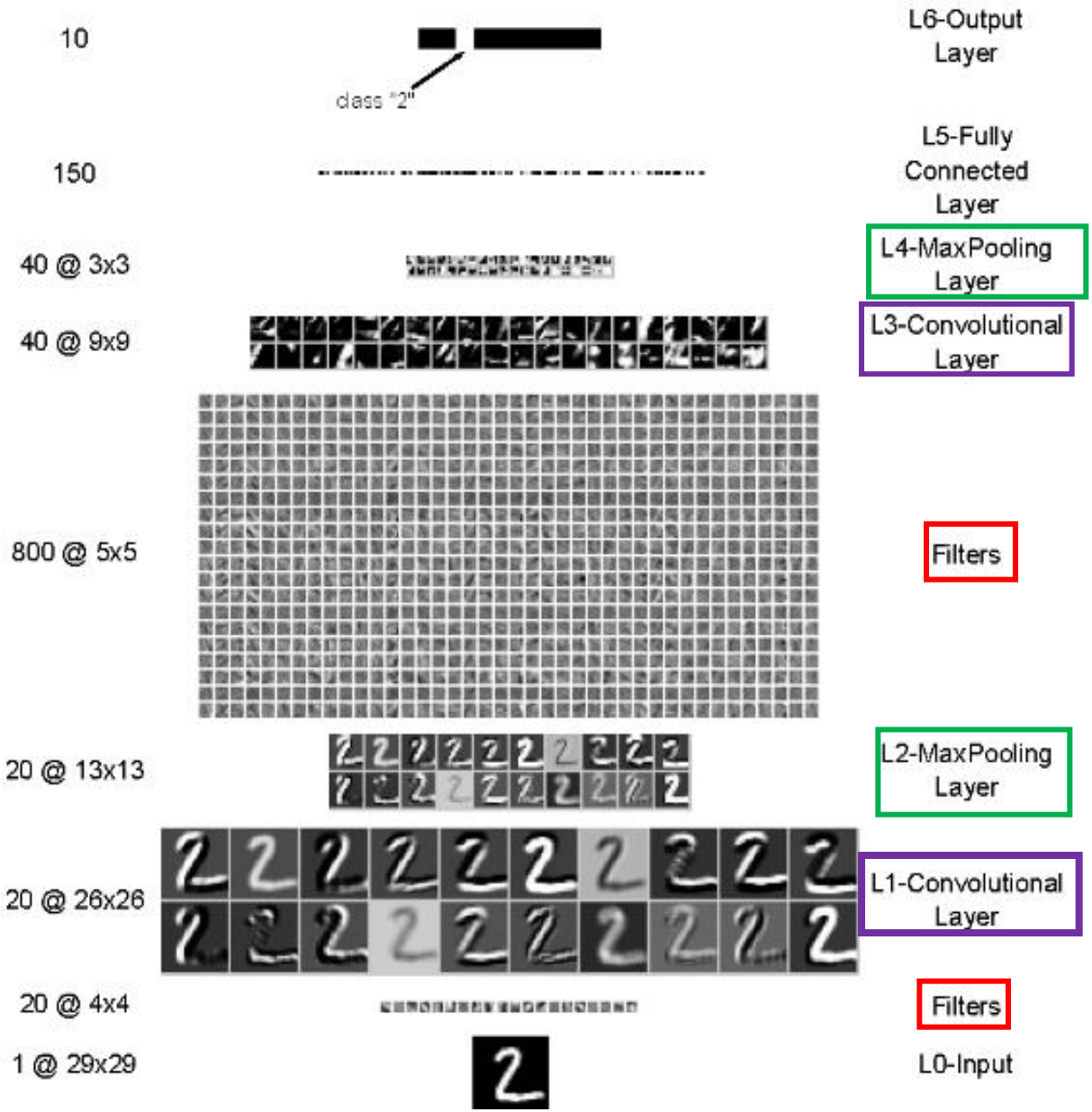
CNN for Handwritten Digits Recognition (LeCun 1989)

- The data base (MNIST) consisted of 9298 segmented numerals digitized from handwritten Zip-codes, as shown in Figure . The training set consists of 7291 examples and the remaining 2007 were used for testing the network.
- After completion of the training, only 10 digits from the entire training set had been misclassified.



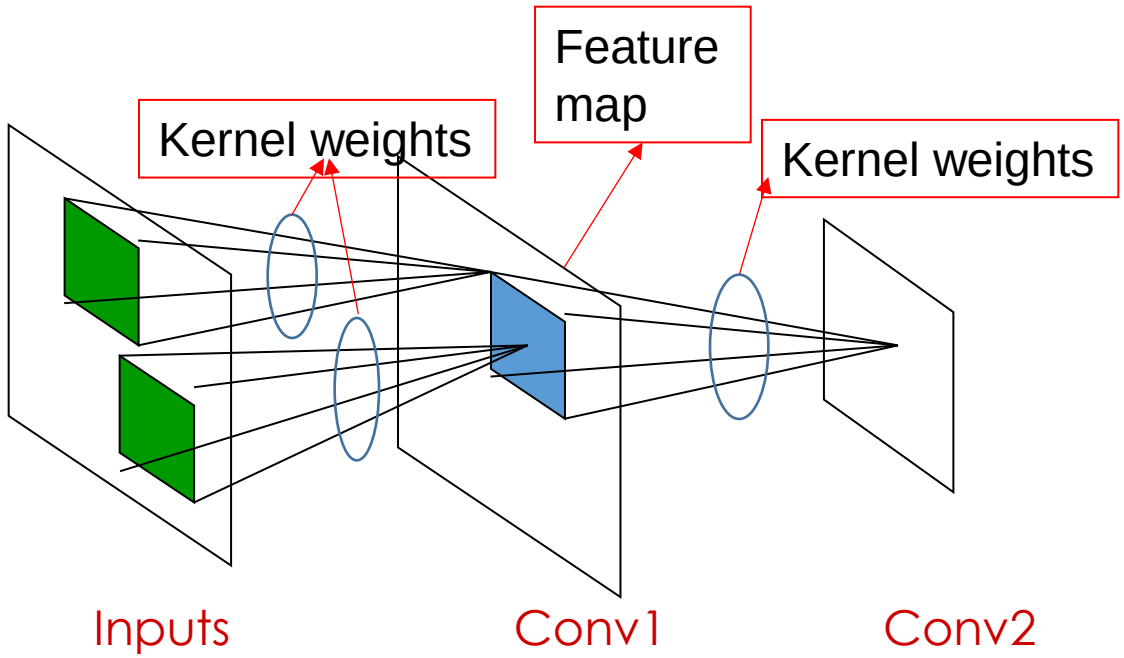


Digit Classification Using CNN (LeNet)

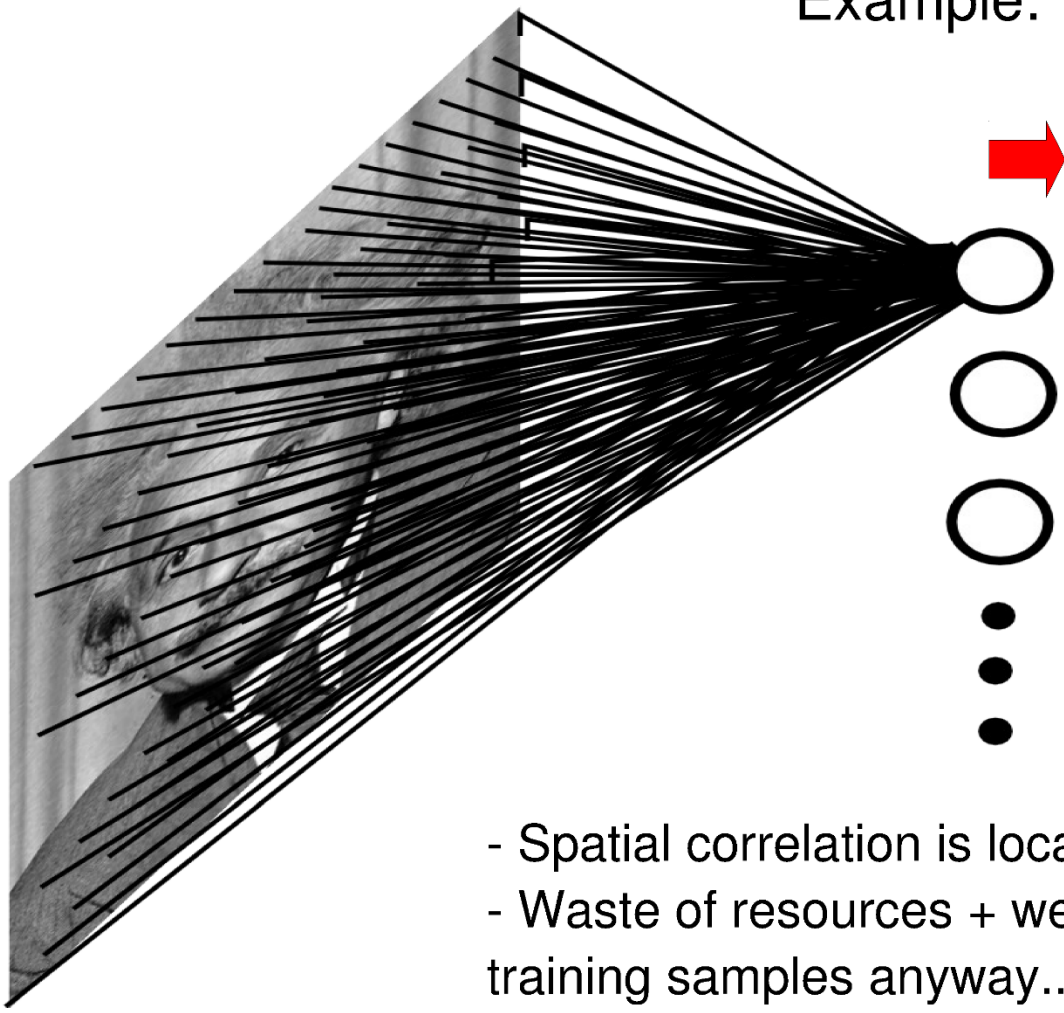


Weight Sharing in CNN

- Shared weights: all neurons in a feature map share the same kerenel weights (but not the biases).
- In this way all neurons detect the same feature at different positions in the input image.
- The number of free parameters is reduced.



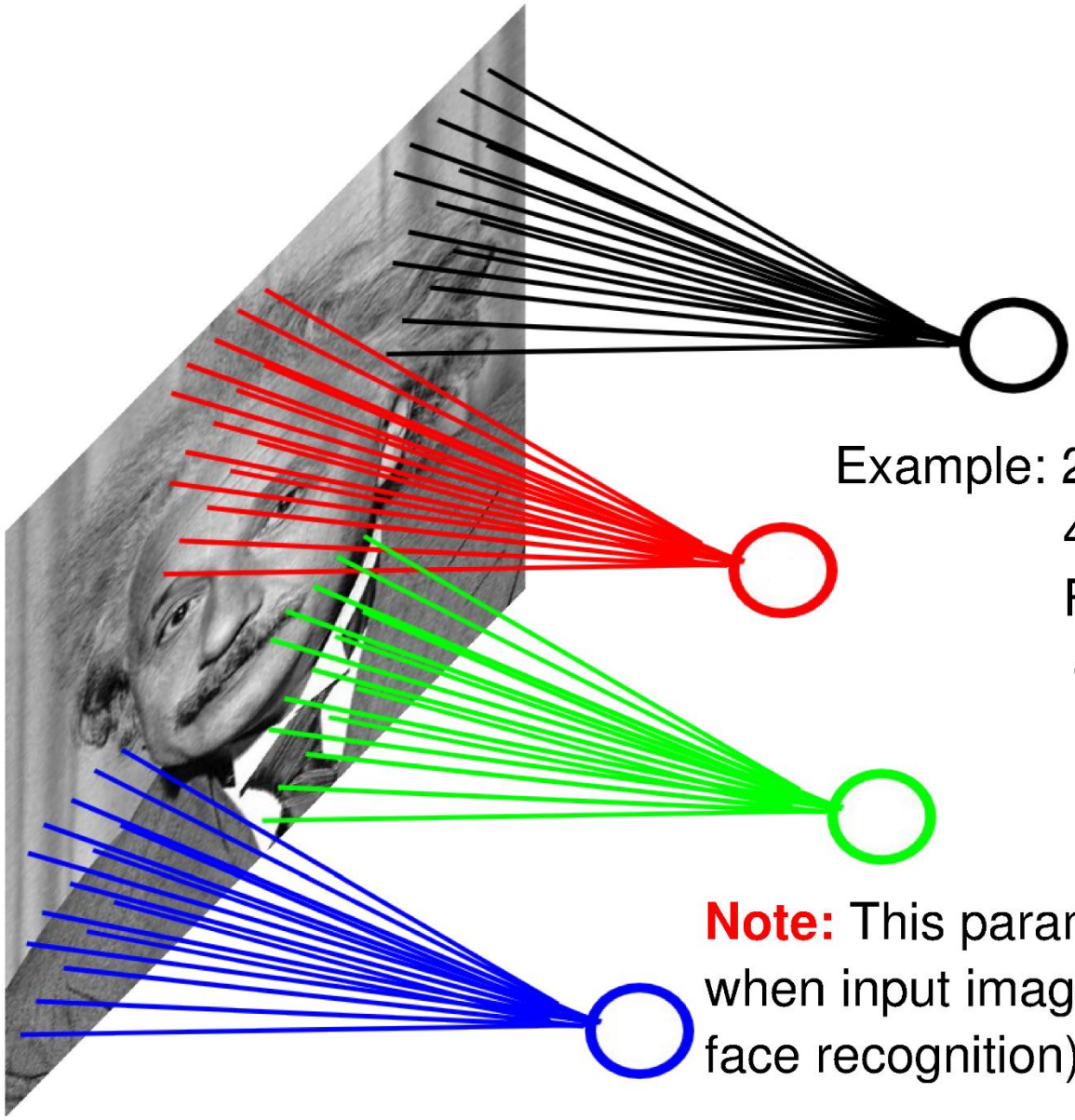
Fully Connected Neurons



Example: 200x200 image
40K hidden units
~2B parameters!!!

- Spatial correlation is local
- Waste of resources + we have not enough training samples anyway..

Locally Connected Neurons

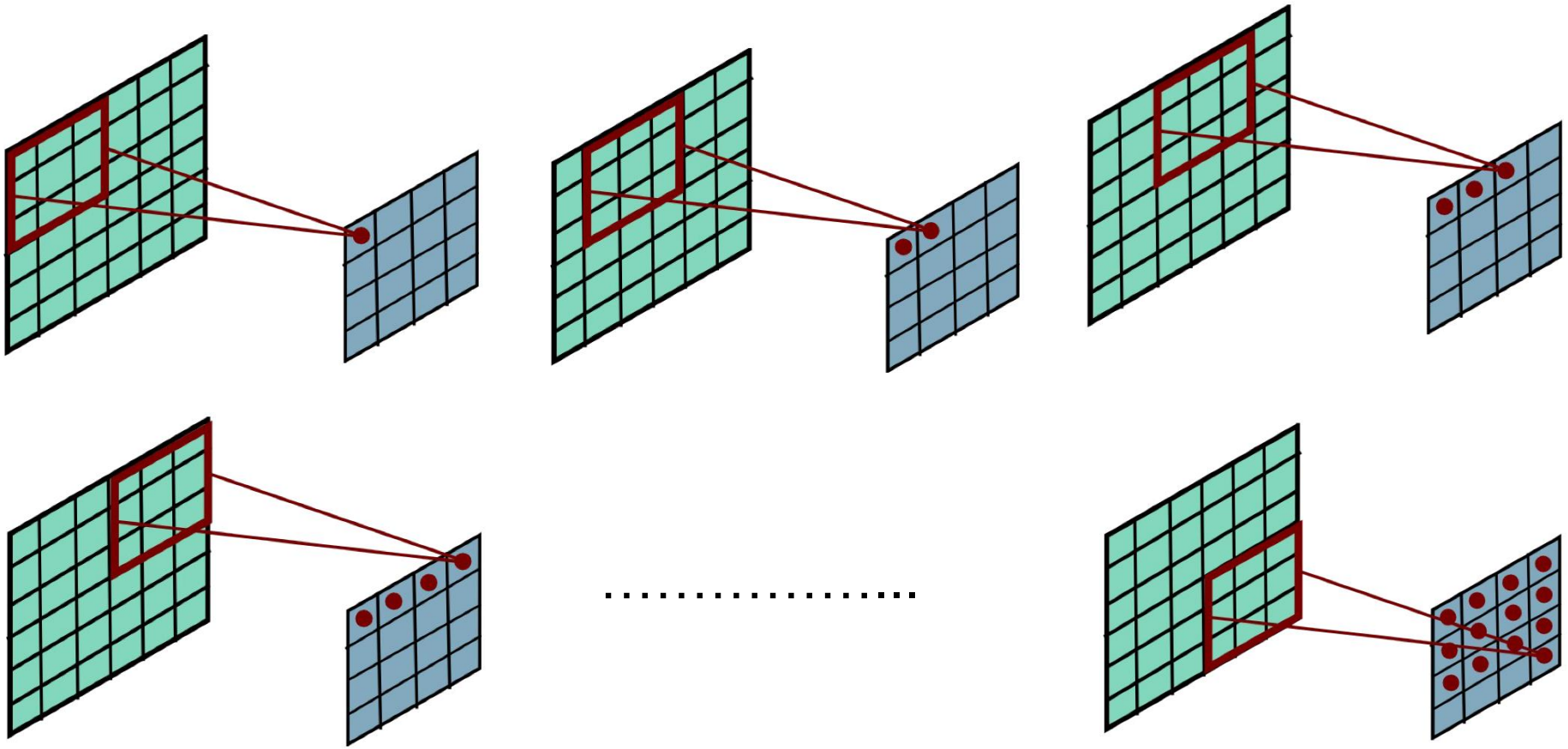


Example: 200x200 image
40K hidden units
Filter size: 10x10
4M parameters

Note: This parameterization is good when input image is registered (e.g., face recognition).

Convolutional Layer

- We shift the kernel (filter) one pixel at a time over the image and calculate.



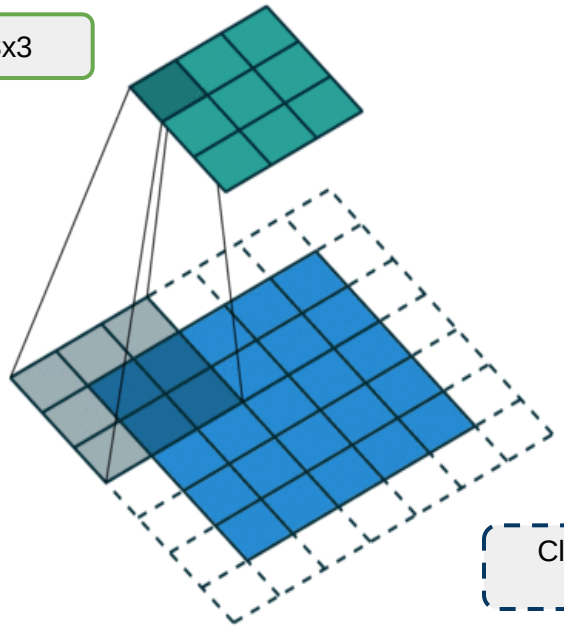
Convolution (Regular + Fractional)

See also:
http://datascience.stackexchange.com/questions/6107/what-are-deconvolutional-layers
https://www.reddit.com/r/MachineLearning/comments/4bbod7/which_deep_learning_frameworks_support_fractional/

Regular convolutions

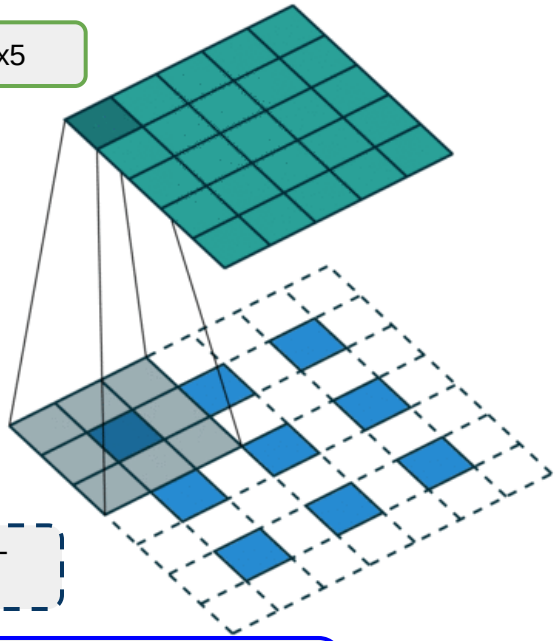
Fractionally-strided convolutions

Output = 3x3



Input = 5x5
with zero-padding at border = 6x6
(stride=2)

Output = 5x5



Filter size=3x3

Clear dashed squares = zero-padded inputs

Input = 3x3
Interlace zero-padding with inputs = 7x7
(stride=2)

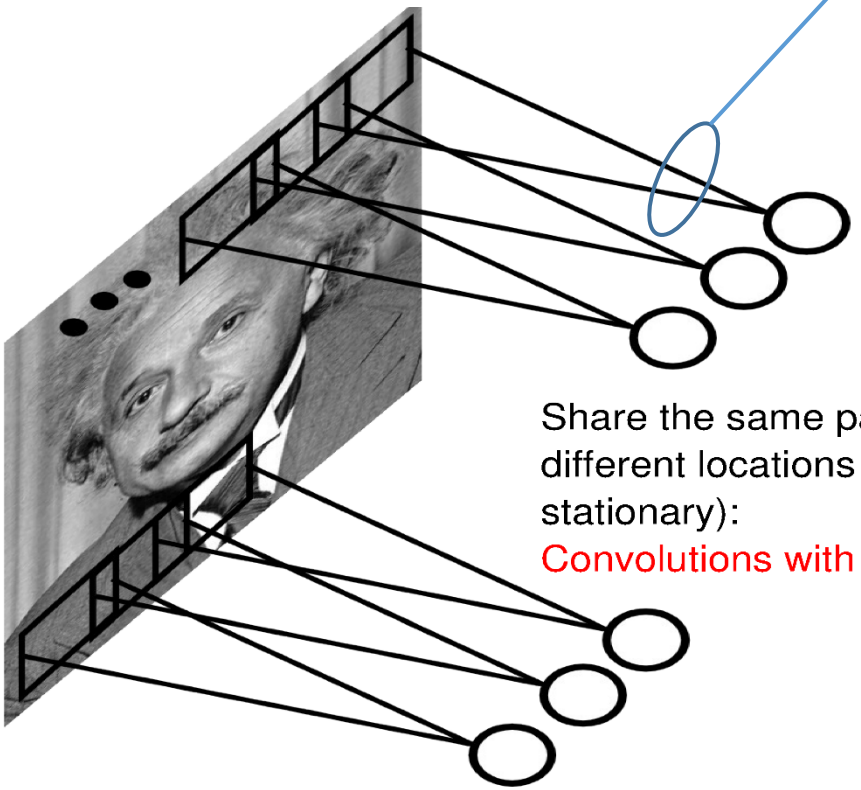
Convolutional Layer

$$h_j^n = max\left(0, \sum_{k=1}^{K} h_k^{n-1} * w_{kj}^n\right)$$

output feature map

input feature map

kernel

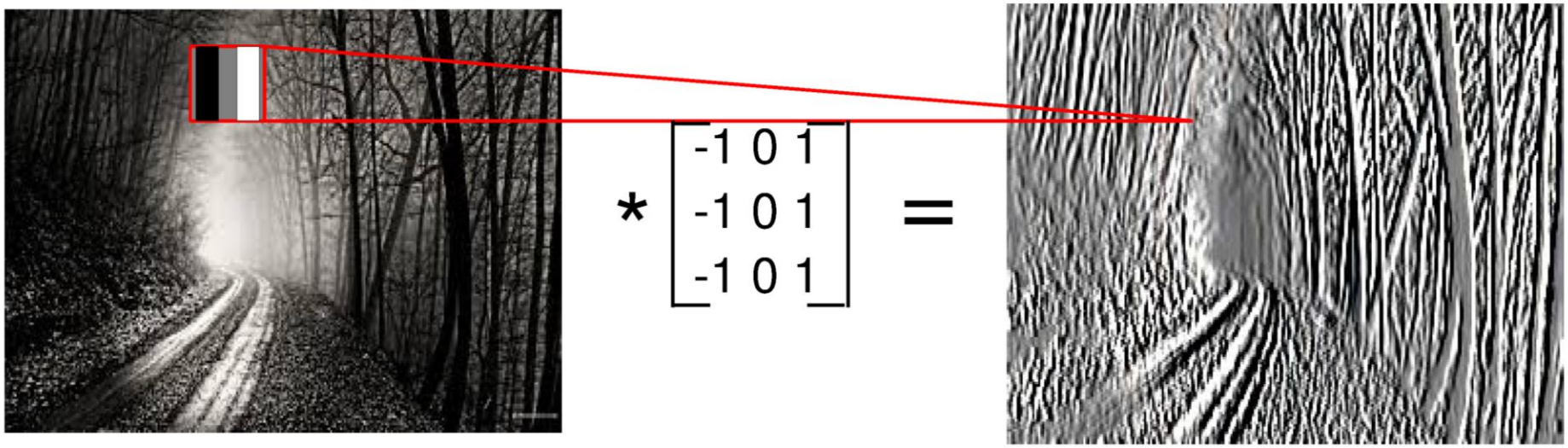


Share the same parameters across different locations (assuming input is stationary):
Convolutions with learned kernels

Similar to Convolution with 2-D Filter

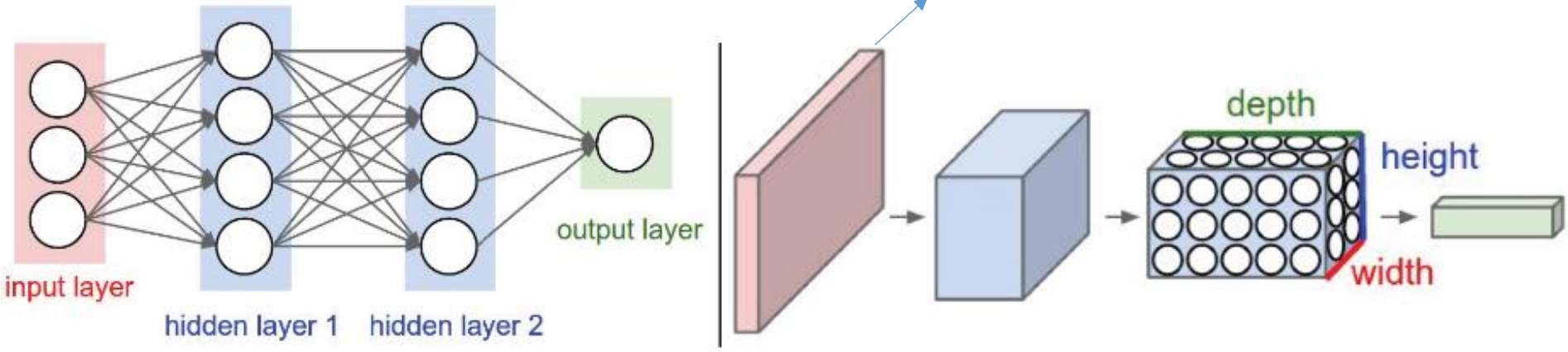
- Convolve the image with a 2-D filter extract vertical edges.
- This a low-pass vertically followed by a band-pass horizontally.
- The band-pass horizontally is a low-pass followed by high-pass.

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} -1 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{bmatrix} = \overbrace{\begin{bmatrix} -1 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 1 \end{bmatrix}}^{\text{Band-pass}} * \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 1 \end{bmatrix} * \underbrace{\begin{bmatrix} 1 \\ 1 \end{bmatrix} * \begin{bmatrix} 1 \\ 1 \end{bmatrix}}_{\text{low-pass}}$$



3-Layer Fully Connected NN vs. CNN (Convnet) Architecture

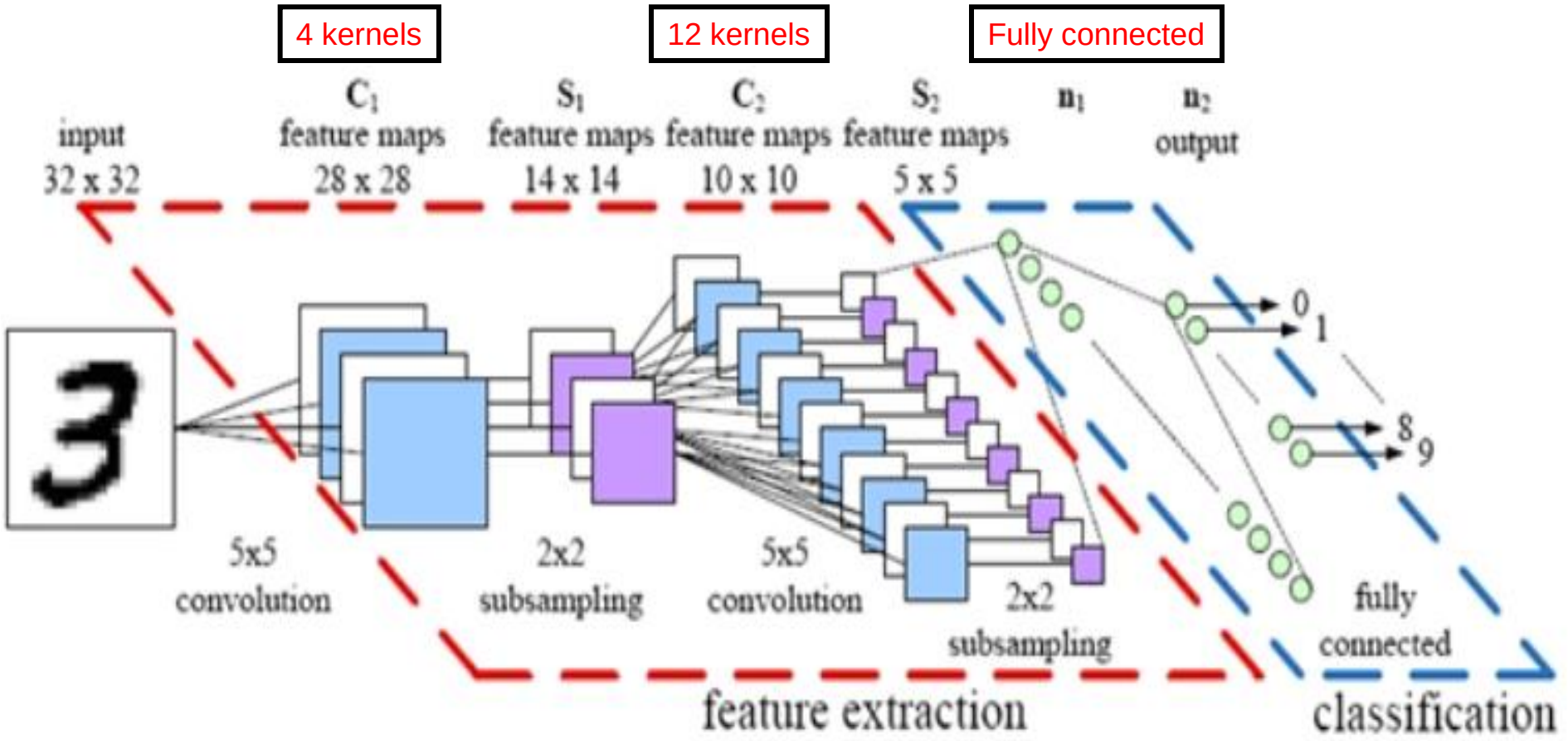
- Left: a regular 3-layer Neural Network.
- Right: a ConvNet arranges its neurons in three dimensions (width, height, depth), as visualized in one of the layers.
- Every layer of a ConvNet transforms the 3D input volume to a 3D output volume of neuron activations.
- In this example, the red input layer holds the image, so its width and height would be the dimensions of the image, and the depth would be 3 (Red, Green, Blue channels).



3-layer Perceptron

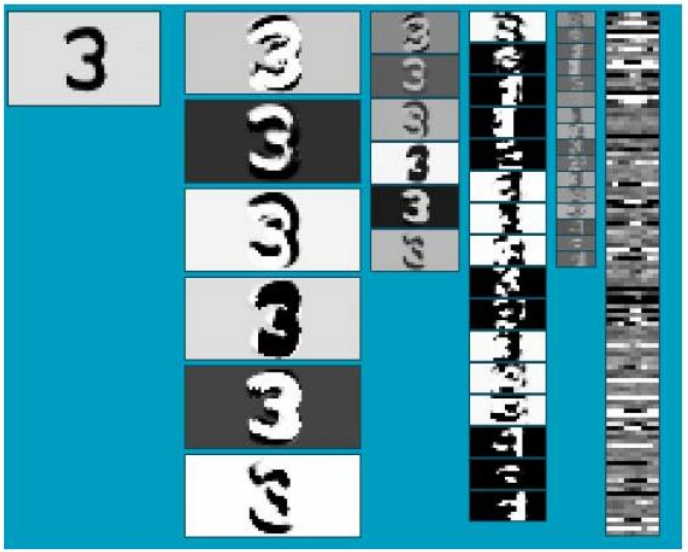
Convnet

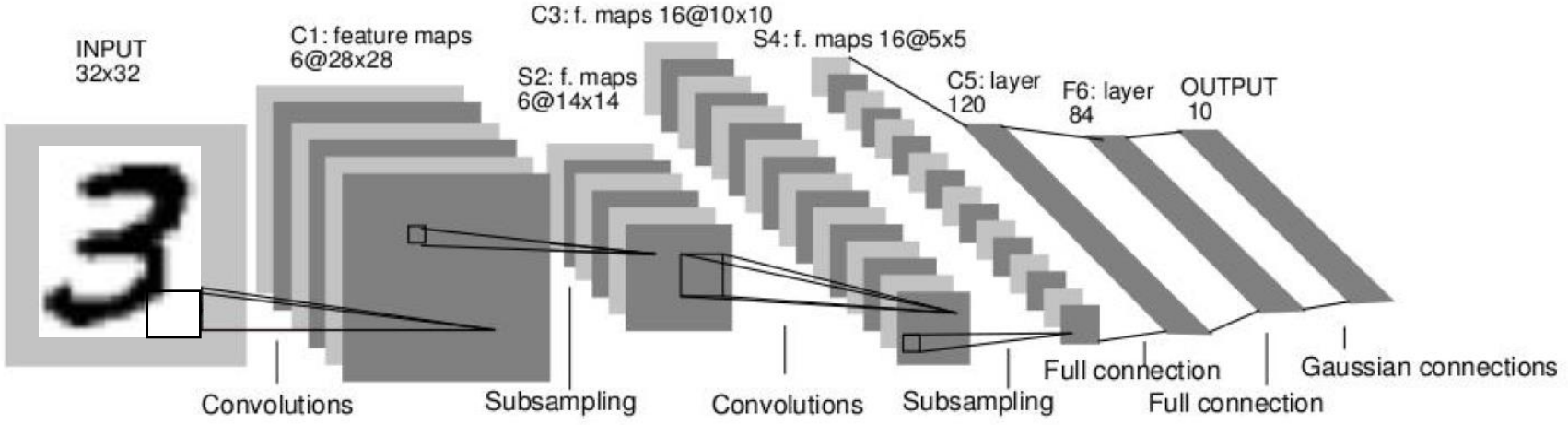
Convolutional Neural Networks (CNN)



Convolutional Neural Networks (CNN, ConvNet, LeNet)

- Neural network with specialized connectivity structure
- Stack multiple stages of feature extractors
- Higher stages compute more global, more invariant features
- Classification layer at the end

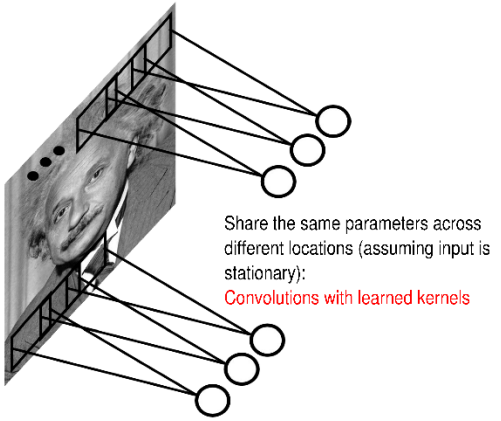


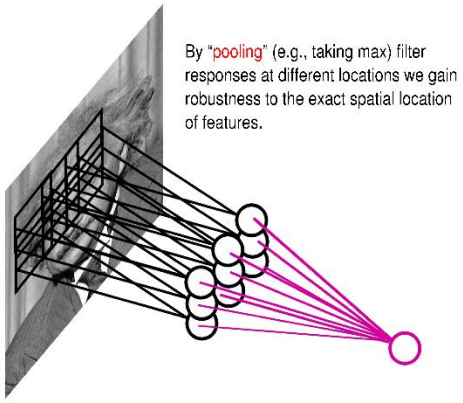


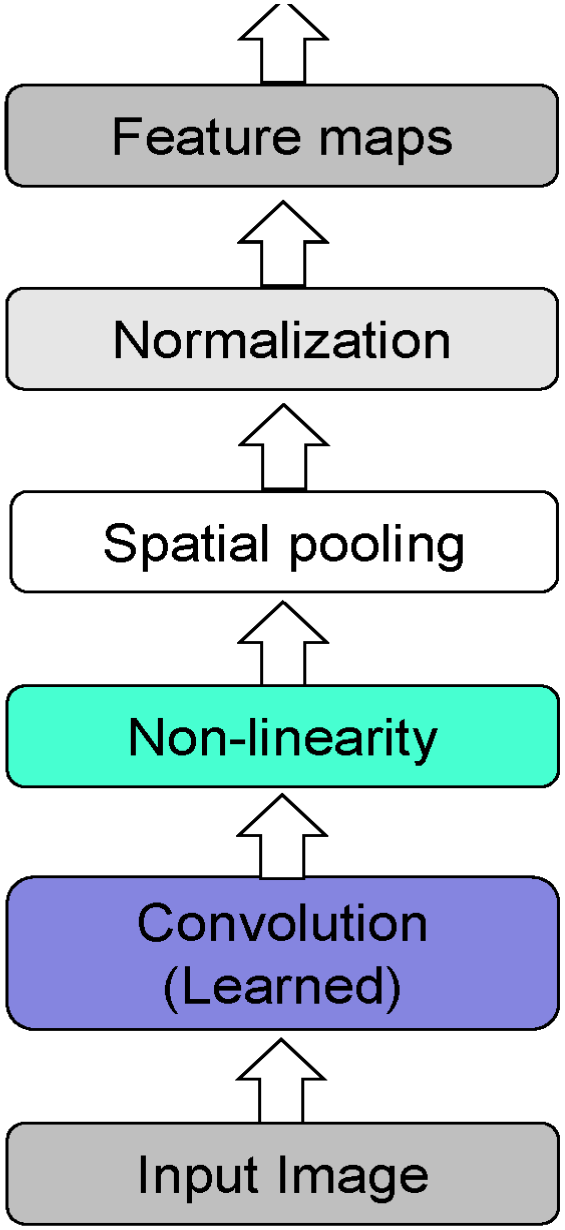
Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, Gradient-based learning applied to document recognition, Proceedings of the IEEE 86(11): 2278–2324, 1998.

Components of CNN Architecture

- Feed-forward feature extraction:
  1. Convolve input with learned filters
  2. Non-linearity
  3. Spatial pooling **(Down-sampling)**
  4. Normalization **(Helps Generalization)**
- Supervised training of convolutional filters by back-propagating classification error







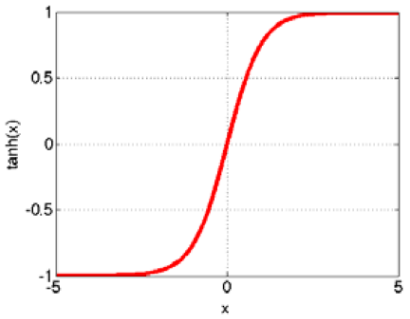
Possible Nonlinearity Functions for each Neuron

Options:

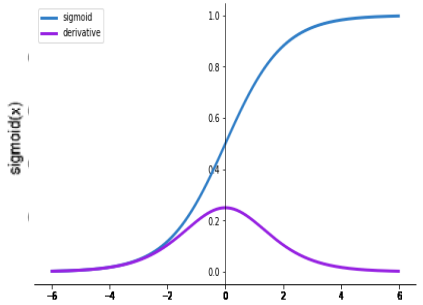
- Tanh
- Sigmoid: 1/(1+exp(-x))
- Rectified linear unit (ReLU)
  - Simplifies backpropagation
  - Makes learning faster
  - Avoids saturation issues
    → Preferred option

i.e., RELU layer will apply an element-wise activation function, such as the *Max*(**0, x**) *(*thresholding at zero)

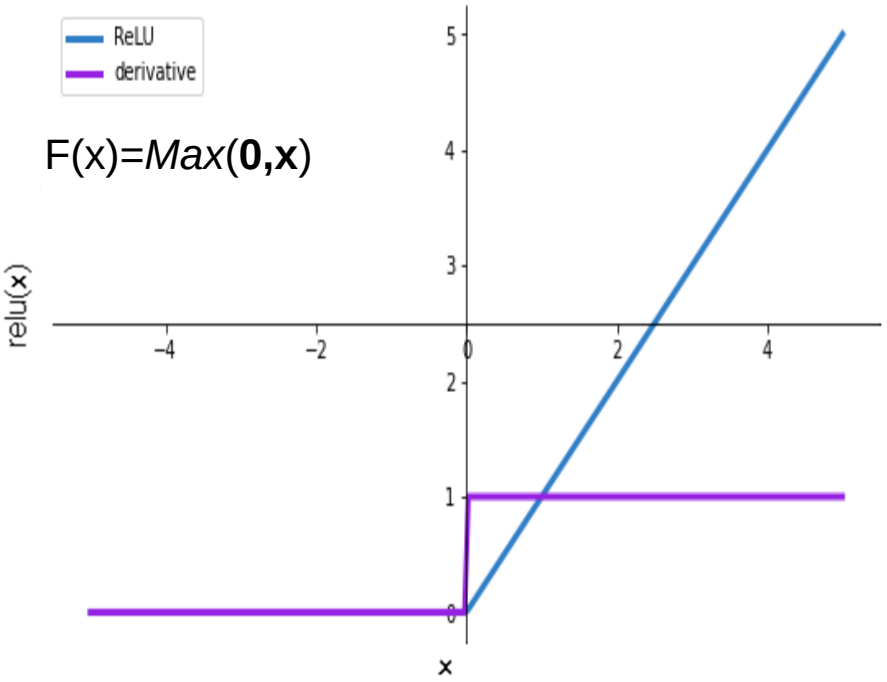
F(**x**)=Tanh(**x**)



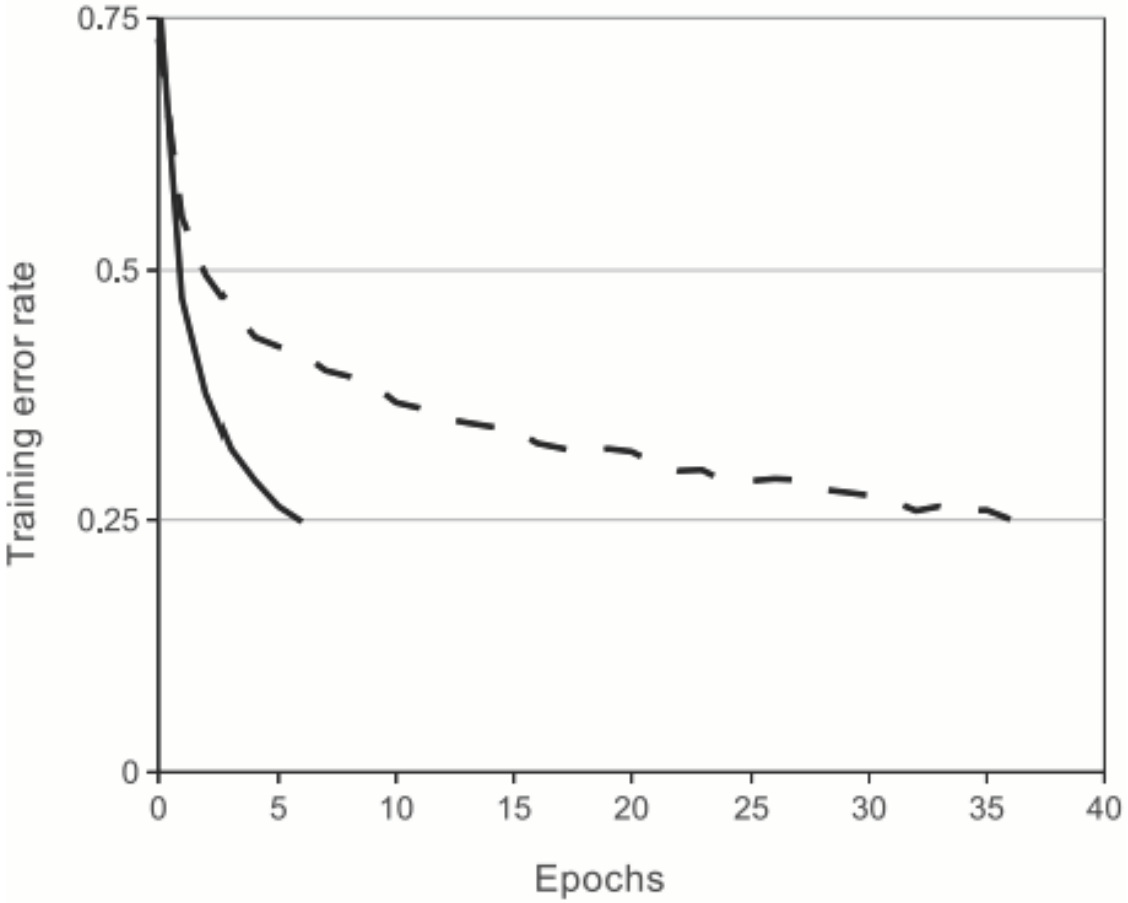
1/(1+exp(-**x**))



F(x)=*Max*(**0,x**)



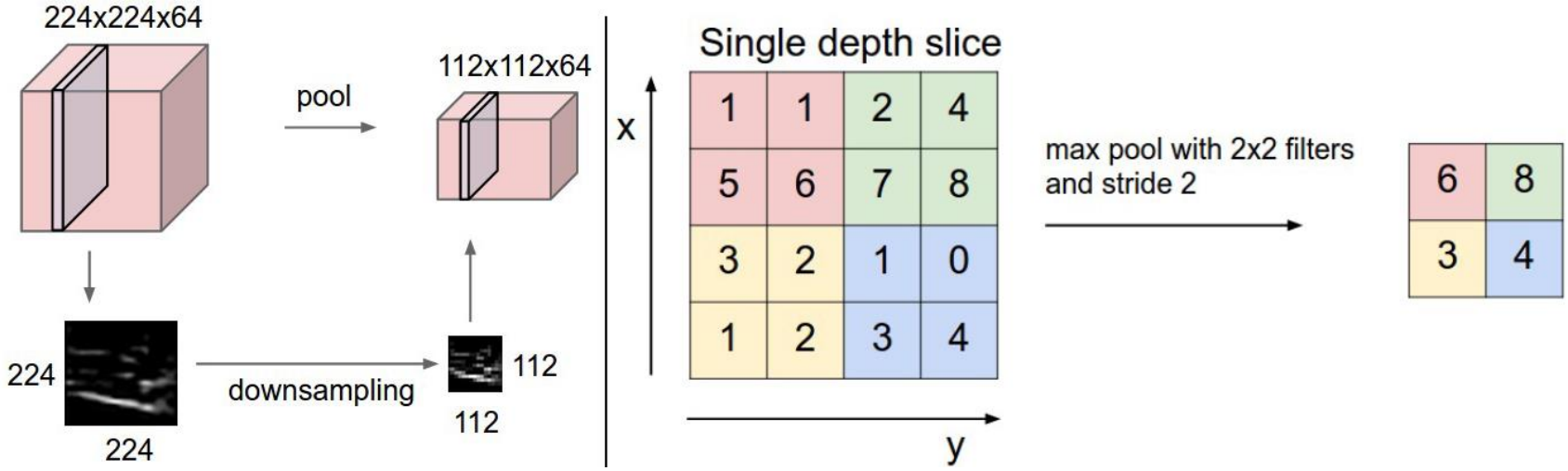
ReLUs Nonlinearity vs. Tanh Nonlinearity



A 4 layer CNN with RELUs (solid line) converges six times faster than an equivalent network with tanh neurons (dashed line) on CIFAR-10 dataset

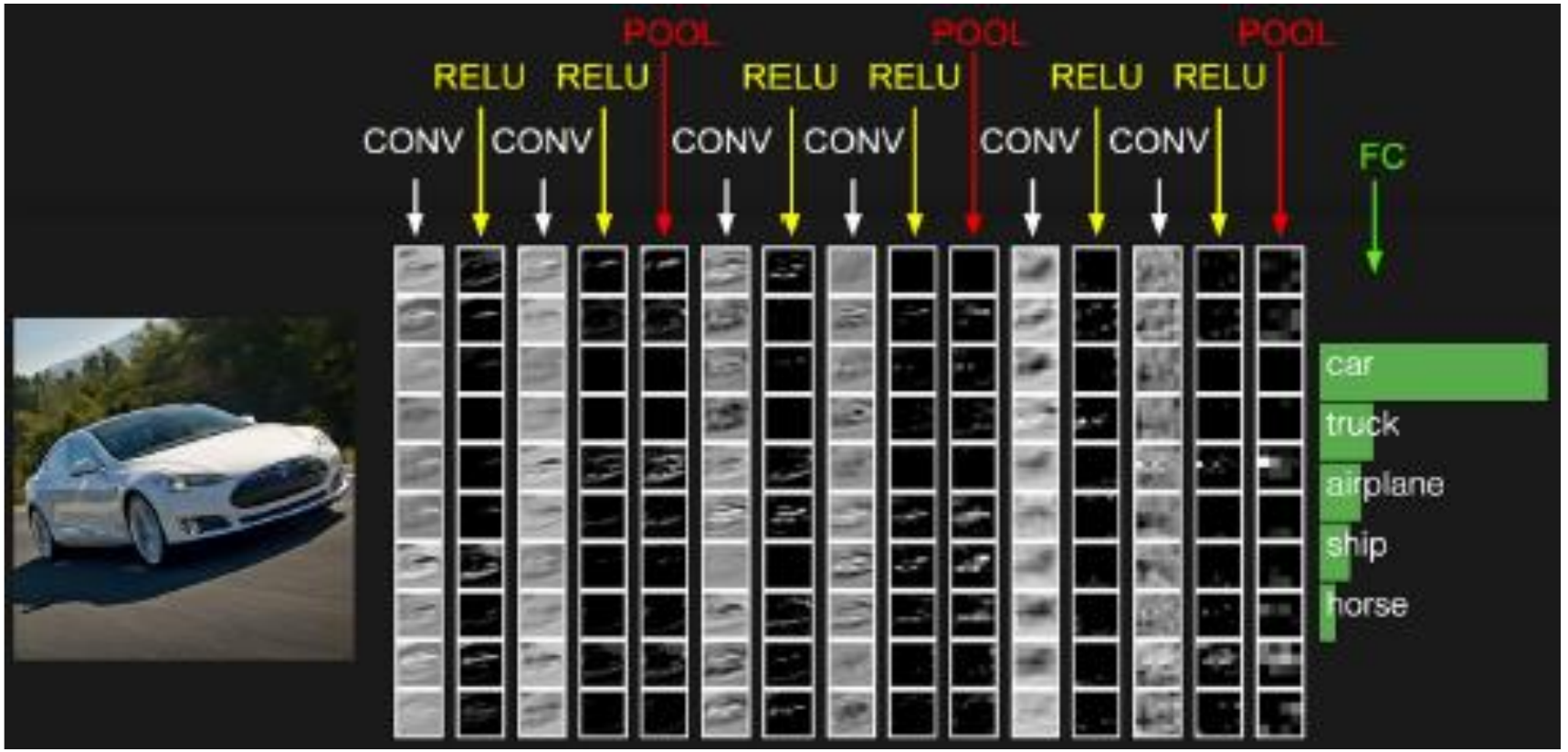
Max-Pooling Layer

- Pooling layer downsamples the volume spatially, independently in each depth slice of the input volume.
- Left: In this example, the input volume of size [224x224x64] is pooled with filter size 2, stride 2 into output volume of size [112x112x64]. Notice that the volume depth is preserved.
- Right: The most common downsampling operation is max, giving rise to max pooling, here shown with a stride of 2. That is, each max is taken over 4 numbers (little 2x2 square).
- Average pooling was often used historically but has recently fallen out of favor compared to the max pooling operation, which is said to provide invariance.



The activations of an example ConvNet architecture (VGG net)

- The initial volume stores the raw image pixels and the last volume stores the class scores.
- Each volume of activations along the processing path is shown as a column. Since it's difficult to visualize 3D volumes, we lay out each volume's slices in rows.
- The last layer volume holds the scores for each class, but here we only visualize the sorted top 5 scores, and print the labels of each one.



CNN for Handwritten Digits Recognition (MNIST Dataset)

- LeCun et al [3] have proposed a back-propagation network to recognize handwritten digits, their network was very similar to a Time Delay Neural Network (TDNN) or Convolutional Neural Network (CNN).
- The network consists of 16 x 16 normalized image, with 10 output units. This network assumes the input is the image of a single digit. This network was used to read USA Zip-codes.
- The network consisted of three hidden layers, Jl, J2, and J3, as shown in Figure 3.
- Jl consists of 12 groups of 64 units per group, each arranged in an 8 x 8 square.
- Each unit in Jl has connections only from the 5 x 5 contiguous square shifts by two input pixels between neighbors in the hidden layer.
- All 64 units within a group have the same 25 weight values so they all detect the same feature, the weight for the bias input for each unit is not same.
- This weight sharing and limitation to processing of 5 x 5 receptive input fields has resulted in [(12 x 64 by (25 + 1)] weights.
- The second hidden layer J2 consists of 12 groups of 16 units (192), again taking input from 5 x 5 receptive fields.
- Layer J3 has 30 units and is fully connected to the outputs of all units in layer J2. Thus, layer J3 contains 30 x 192 weights plus 30 weights from the bias inputs (thresholds).

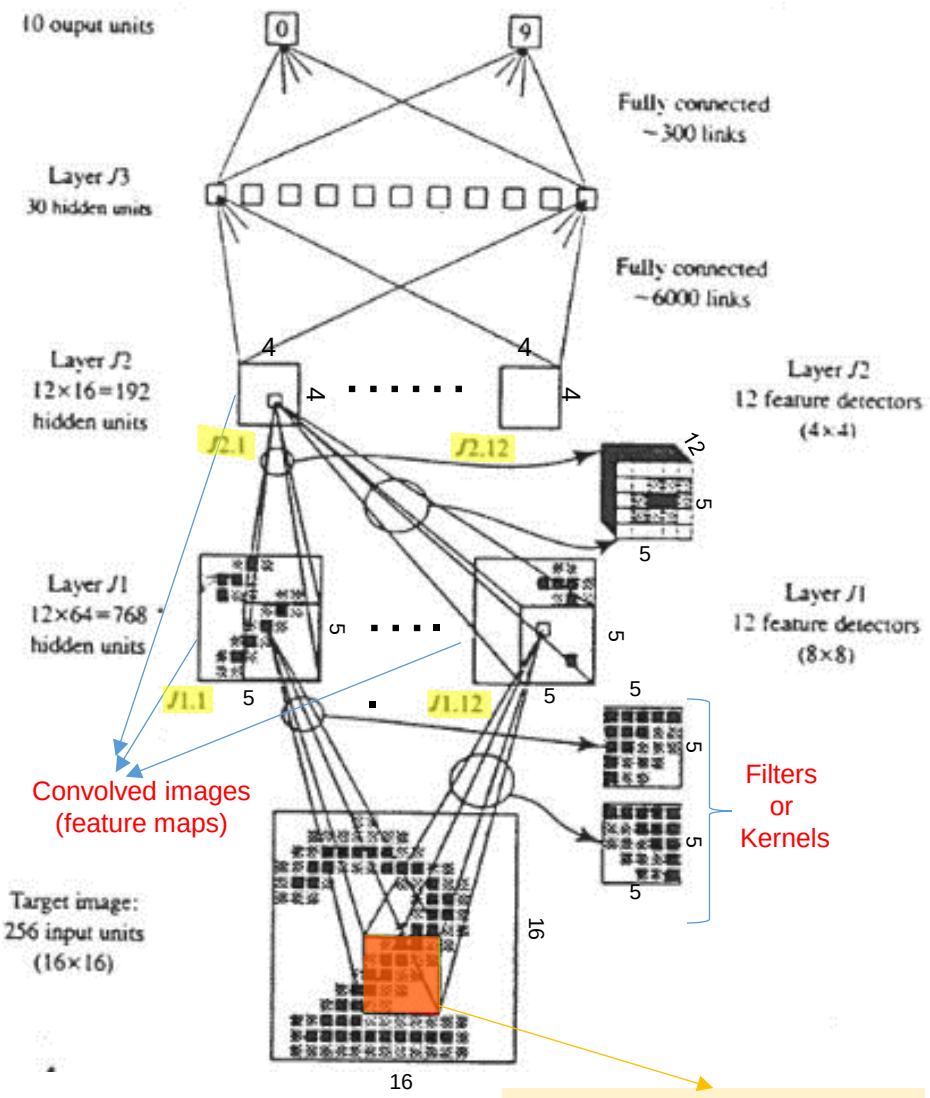
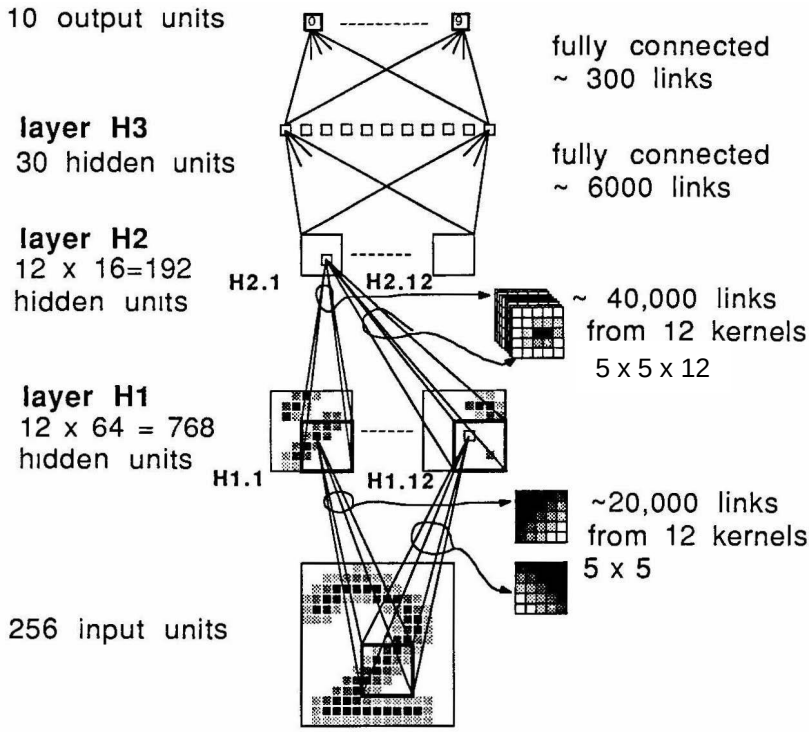
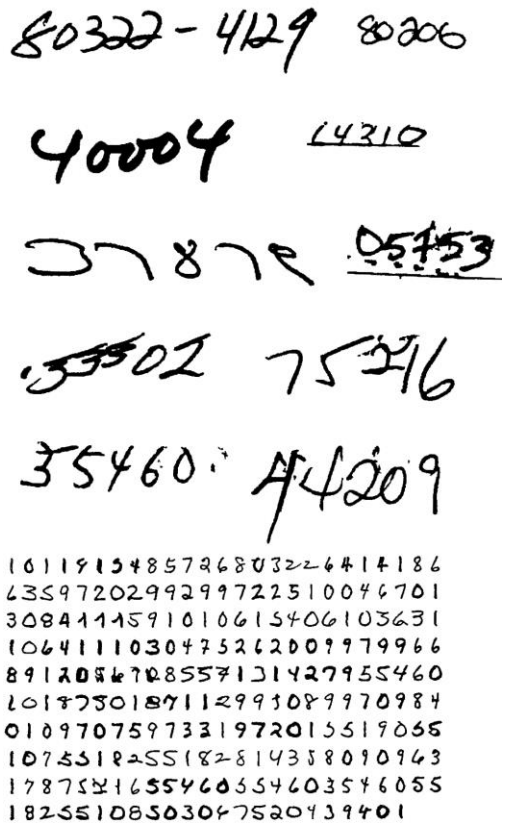


Figure 3

CNN for Handwritten Digits Recognition (LeCun 1989)

- The data base (MNIST) consisted of 9298 segmented numerals digitized from handwritten Zip-codes, as shown in Figure . The training set consists of 7291 examples and the remaining 2007 were used for testing the network.
- After completion of the training, only 10 digits from the entire training set had been misclassified.





ImageNet Challenge 2012

IMAGENET

[Deng et al. CVPR 2009]

- ~14 million labeled images, 20k classes
- Images gathered from Internet
- Human labels via Amazon Turk
- Challenge: 1.2 million training images, 1000 classes

ImageNet Large-Scale Visual Recognition Challenge (ILSVRC)
ILSVRC-10 is roughly 1.2M training data with 1000 classes

A. Krizhevsky, I. Sutskever, and G. Hinton, ImageNet Classification with Deep Convolutional Neural Networks, NIPS 2012

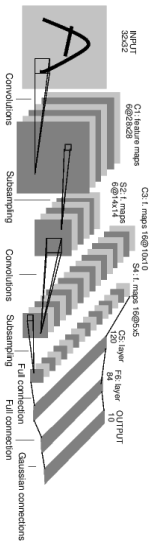
Golden Age of CNN Architectures

- Modern CNN Architectures
  - Krizhevsky, et al. 2012 (AlexNet)
    - Establishes new standard
  - Sermanet, et al. 2014 (overFeat)
    - Dense application of networks at multiple scales
  - Szegedy, et al. 2014 (GooGleNet)
    - Mixes depth with concatenated inceptions and new topologies
  - Zeiler & Fergus, 2013 (Zfnet)
    - Visualization and understanding convolutional networks, ECCV 2013
  - Howard, 2013,
    - Some Improvements on deep convolutional neural network-based image classification, ICLR 2013
  - Simonyan and Zisserman, 2015 (VGGNet)
    - Very deep convolutional networks for large-scale image recognition, ICLR 2015
  - K. He, et al., 2015 (ResNet)
    - Deep residual learning for image recognition, ICCV 2015

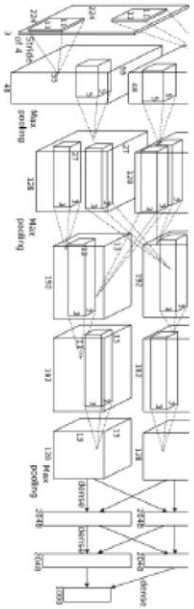
Revolution of Depth in CNN

There are several architectures in the field of Convolutional Networks

1998 LeNet
5

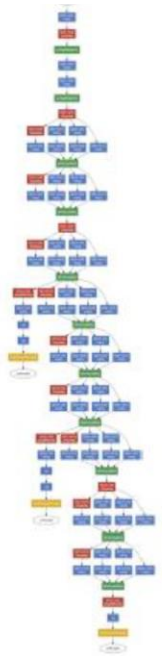


2012 AlexNet
8



2014 VGG
19

2014 GoogLeNet
22



2015 ResNET
152

- **There are several architectures in the field of Convolutional Networks that have a name. The most common are:**

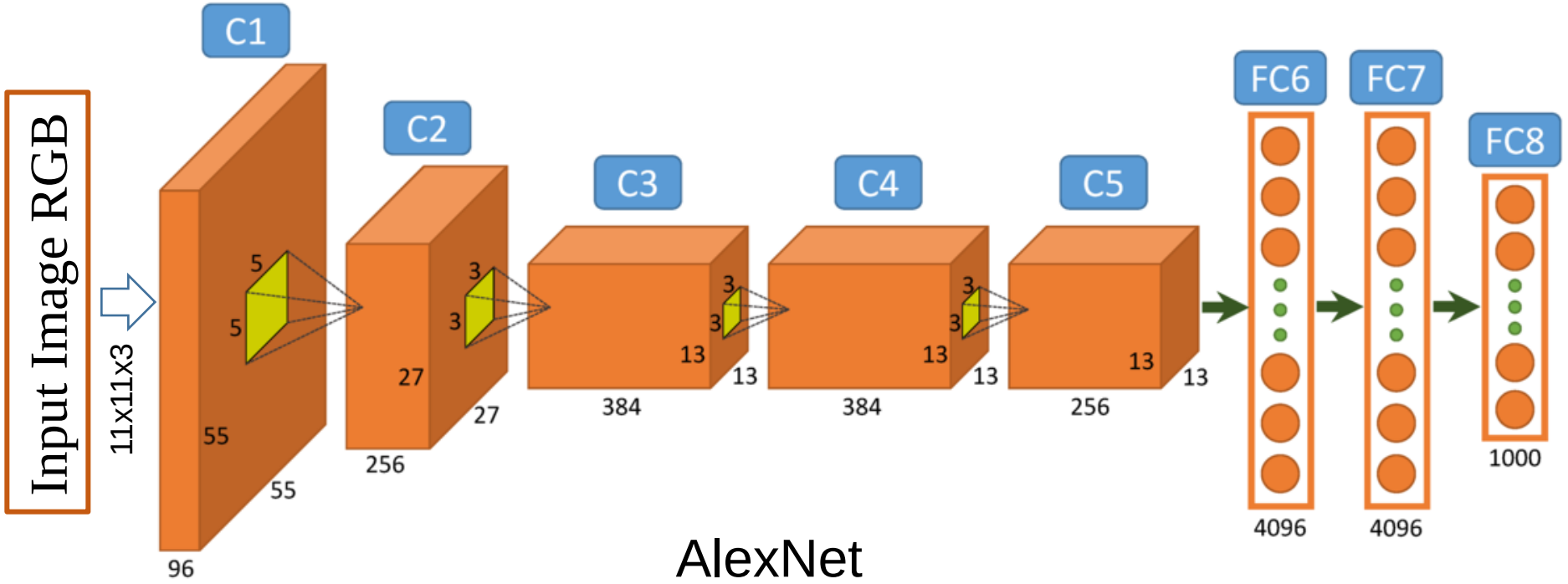
- **LeNet.** The first successful applications of Convolutional Networks were developed by Yann LeCun in 1990's. Of these, the best known is the LeNet architecture that was used to read zip codes, digits, etc.
- **AlexNet.** The first work that popularized Convolutional Networks in Computer Vision was the AlexNet, developed by Alex Krizhevsky, Ilya Sutskever and Geoff Hinton. The AlexNet was submitted to the ImageNet ILSVRC challenge in 2012 and significantly outperformed the second runner-up (top 5 error of 16% compared to runner-up with 26% error). The Network had a similar architecture basic as LeNet, but was deeper, bigger, and featured Convolutional Layers stacked on top of each other (previously it was common to only have a single CONV layer immediately followed by a POOL layer).
- **ZF Net**. The ILSVRC 2013 winner was a Convolutional Network from Matthew Zeiler and Rob Fergus. It became known as the ZFNet (short for Zeiler & Fergus Net). It was an improvement on AlexNet by tweaking the architecture hyperparameters, in particular by expanding the size of the middle convolutional layers.
- **GoogLeNet.** The ILSVRC 2014 winner was a Convolutional Network from Szegedy et al. from Google. Its main contribution was the development of an Inception Module that dramatically reduced the number of parameters in the network (4M, compared to AlexNet with 60M). Additionally, this paper uses Average Pooling instead of Fully Connected layers at the top of the ConvNet, eliminating a large amount of parameters that do not seem to matter much.
- **VGGNet.** The runner-up in ILSVRC 2014 was the network from Karen Simonyan and Andrew Zisserman that became known as the VGGNet. Its main contribution was in showing that the depth of the network is a critical component for good performance. Their final best network contains 16 CONV/FC layers and, appealingly, features an extremely homogeneous architecture that only performs 3x3 convolutions and 2x2 pooling from the beginning to the end. It was later found that despite its slightly weaker classification performance, the VGG ConvNet features outperform those of GoogLeNet in multiple transfer learning tasks. Hence, the VGG network is currently the most preferred choice in the community when extracting CNN features from images. In particular, their pretrained model is available for plug and play use in Caffe. A downside of the VGGNet is that it is more expensive to evaluate and uses a lot more memory and parameters (140M).
- **ResNet.** Residual Network developed by Kaiming He et al. was the winner of ILSVRC 2015. It features an interesting architecture with special skip connections and features heavy use of batch normalization. The architecture is also missing fully connected layers at the end of the network. The reader is also referred to Kaiming's presentation (video, slides), and some recent experiments that reproduce these networks in Torch.

AlexNet :
A Deep Convolutional Neural Networks Architecture

AlexNet (A Deep Convolutional Neural Networks Architecture)

Similar framework to LeCun'98 (LeNet) but:

- Bigger Model (7 Layers, 650,000 processing units, 60,000,000 parameters.
- More data (10^6 vs 10^3 images).

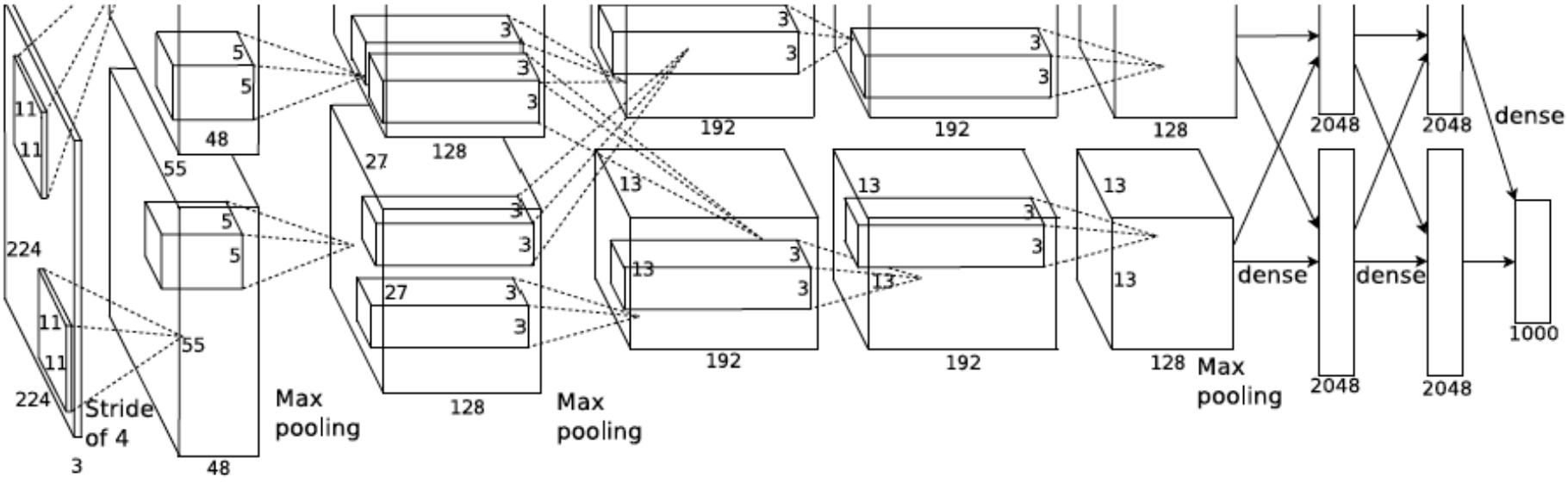


AlexNet

AlexNet (A Deep Convolutional Neural Networks Architecture)

Similar framework to LeCun'98 (LeNet) but:

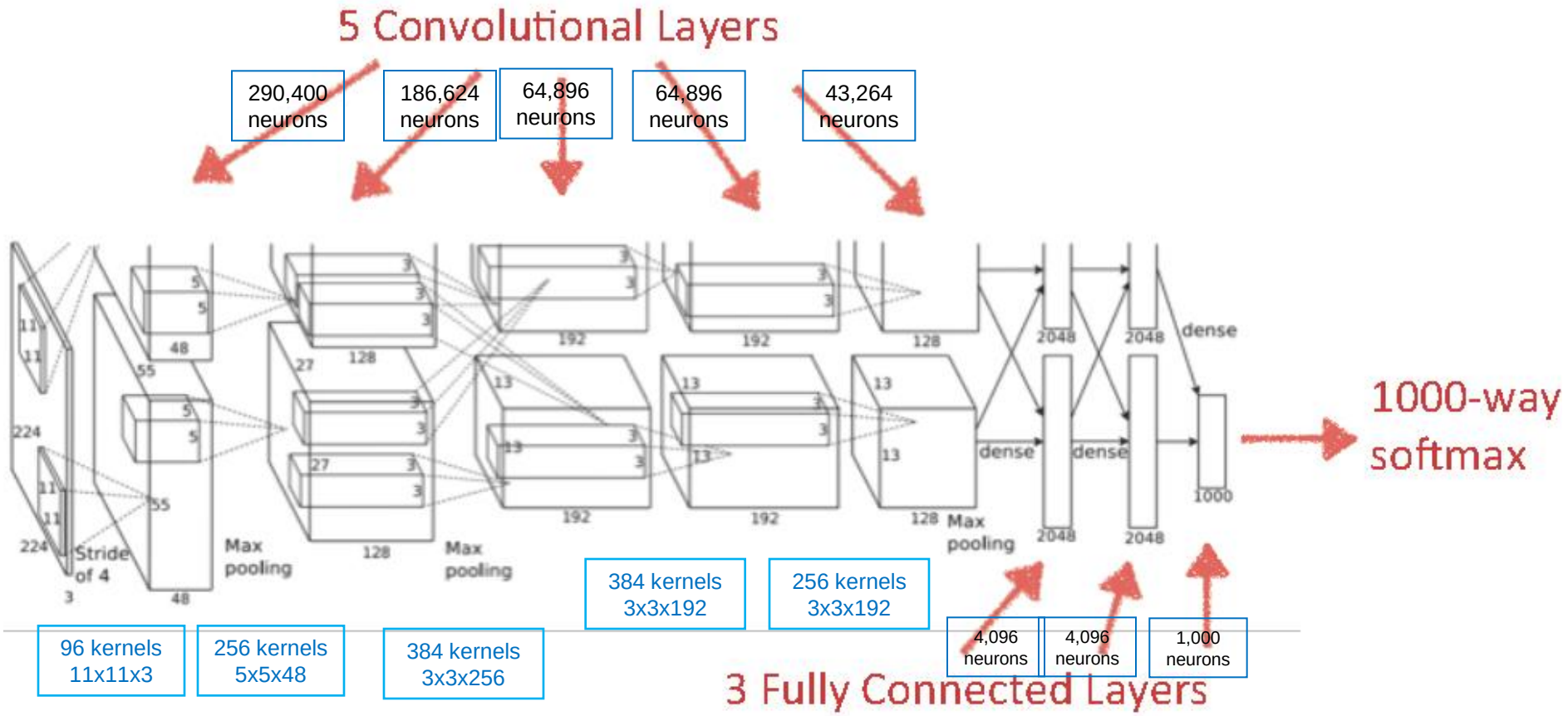
- Bigger Model (7 Layers, 650,000 processing units, 60,000,000 parameters.
- More data (10^6 vs 10^3 images).
- GPU implementation (50x speedup over CPU)
  - Trained on two GPUs for a week
- Better regularization for training (uses DropOut)



A. Krizhevsky, I. Sutskever, and G. Hinton, ImageNet Classification with Deep Convolutional Neural Networks, NIPS 2012

1.2 M training images, 1k classes, 50k validation images, 150k test images

Alexnet (A Deep Convolutional Neural Networks Architecture)

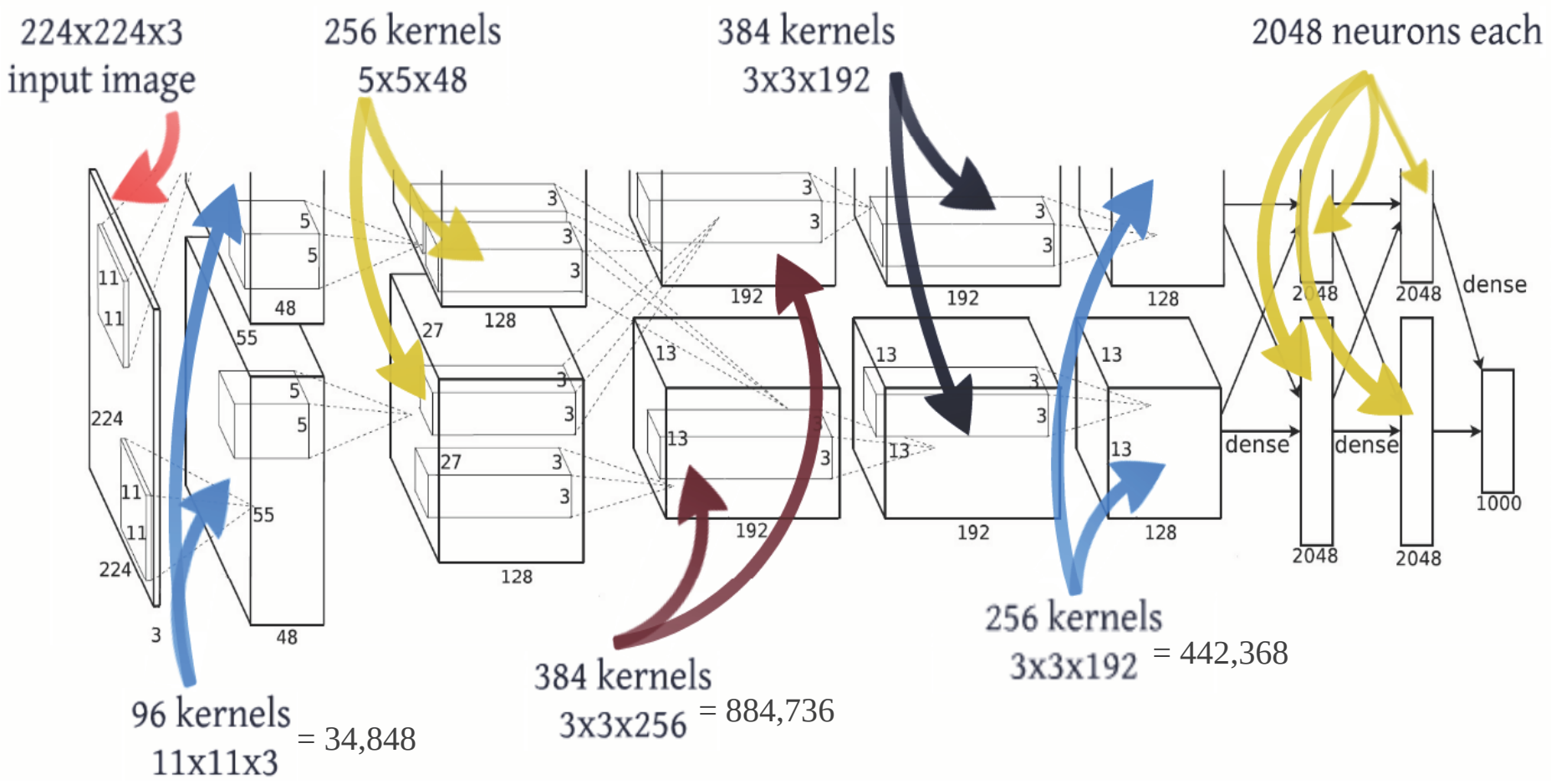


- Softmax classifier:

$f(\mathbf{x}_i; \mathbf{W}) = f(\mathbf{W}\mathbf{x}_i)$ where $\mathbf{W}_{1000\times4096}$ is a matrix of classifier parametrs for 1000 classes

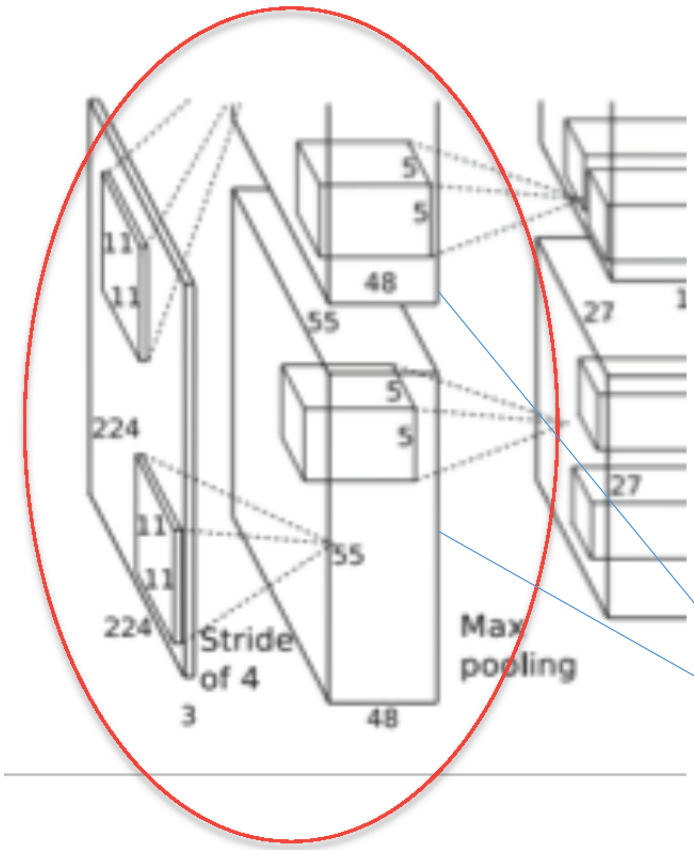
AlexNet (A Deep Convolutional Neural Networks Architecture)

- Number of distinct kernels at each layer



With stride of 4 pixels

AlexNet (Layer_1 Details)

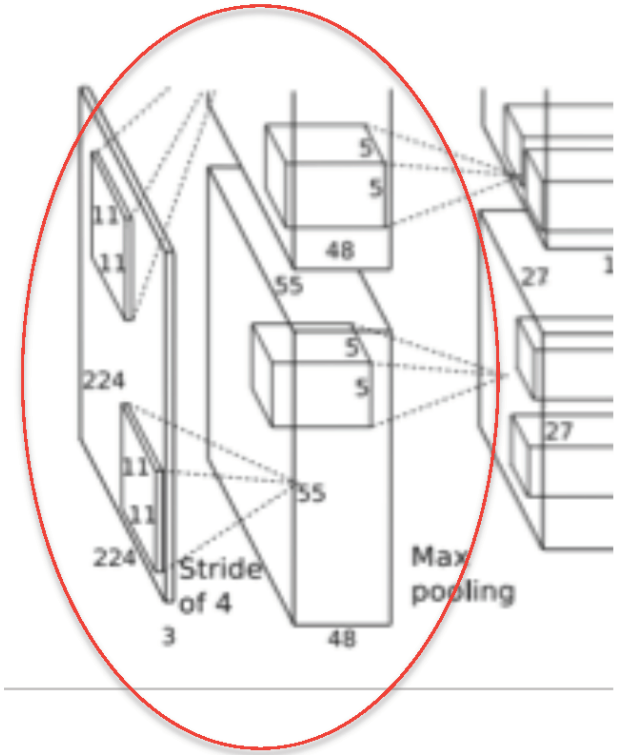


- Images: 224x224x3
- F (receptive field size): 11
- S (stride) = 4
- Conv layer output: 55x55x96

Kernel feature maps

- Stride is the distance between the receptive field centers of neighboring neurons in the kernel feature maps.
- 96 (48+48) kernels spread over two GPUs.

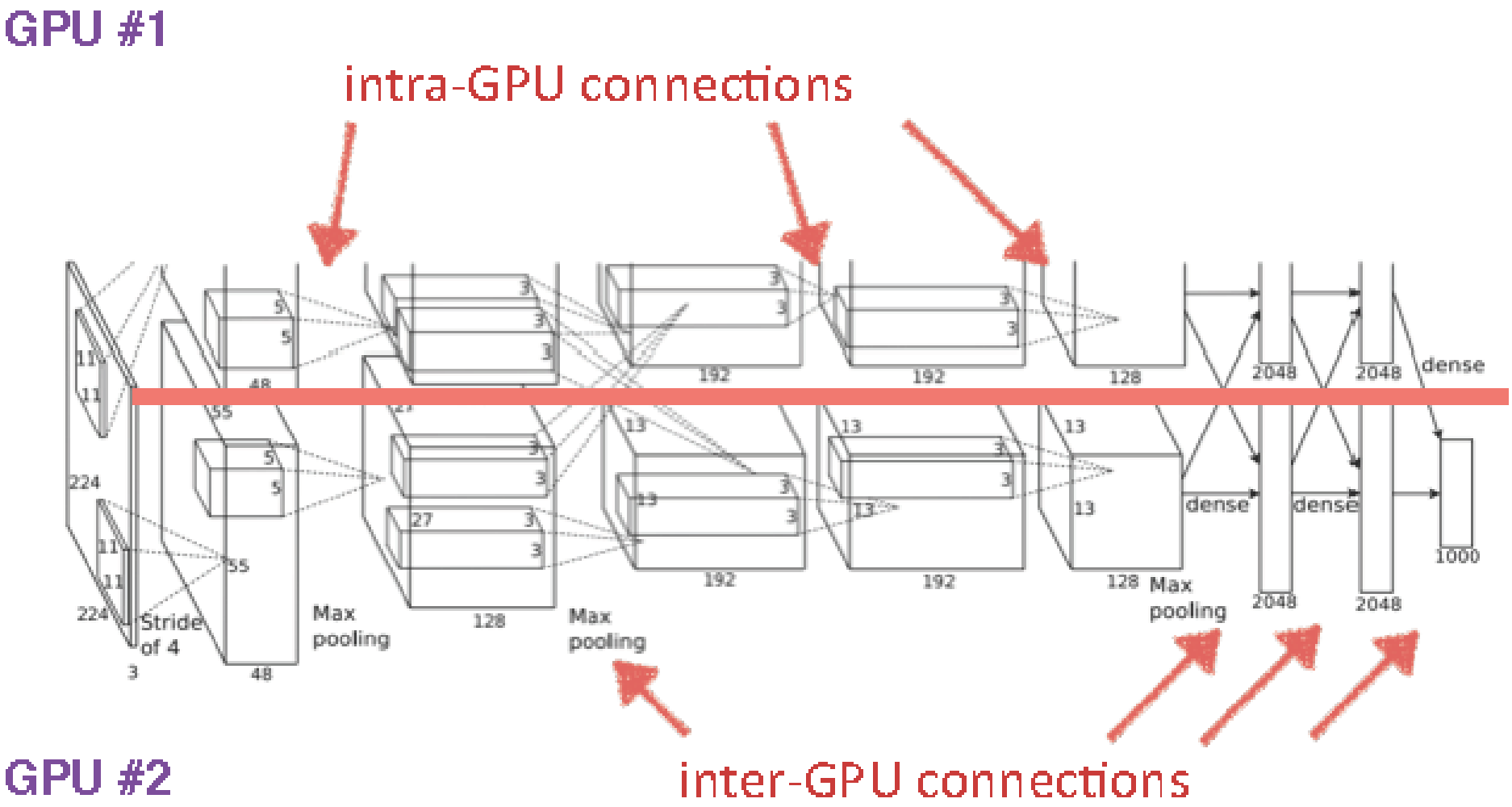
AlexNet (Layer_1 Details)



- 55*55*96 = 290,400 neurons
- each has 11*11*3 = 363 weights and 1 bias
- 290400 * 364 = 105,705,600 paramaters on the first layer of the AlexNet alone!

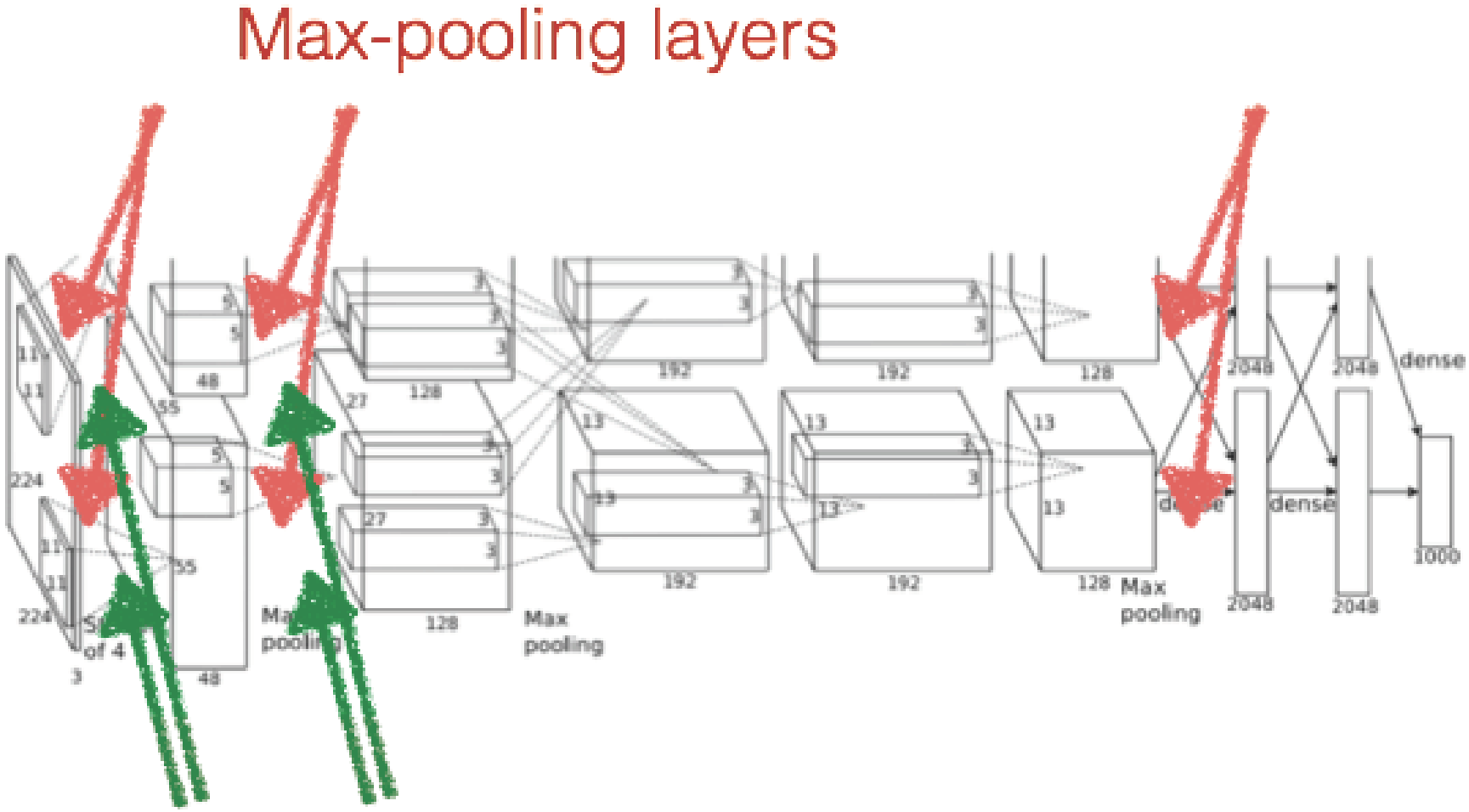
- But there are only 11*11*3*96 distinct parameters in the first layer of the AlexNet

AlexNet (Trained on Multiple GPUs)



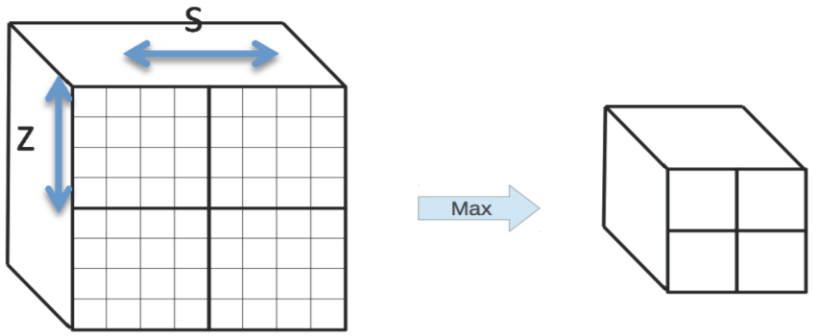
AlexNet (Overlapping Max-Pooling Function)

The third, fourth and fifth convolutional layers are connected together without any pooling or normalization layers



Overlapping Max-Pooling Function

- Traditional pooling (s = z)



- s < z ➔ overlapping pooling
- top-1 and top-5 error rates decrease by 0.4% and 0.3%, respectively, compared to the non-overlapping scheme s = 2, z = 2

AlexNet (Local Response Normalization)

- No need to input normalization with ReLUs.
- But still the following local normalization scheme helps generalization.

$$b^i_{x,y} = a^i_{x,y} \Big/ \left(k + \alpha \sum_{j=\max(0,i-n/2)}^{\min(N-1,i+n/2)} (a^j_{x,y})^2 \right)^{\beta}$$

Response-normalized activity

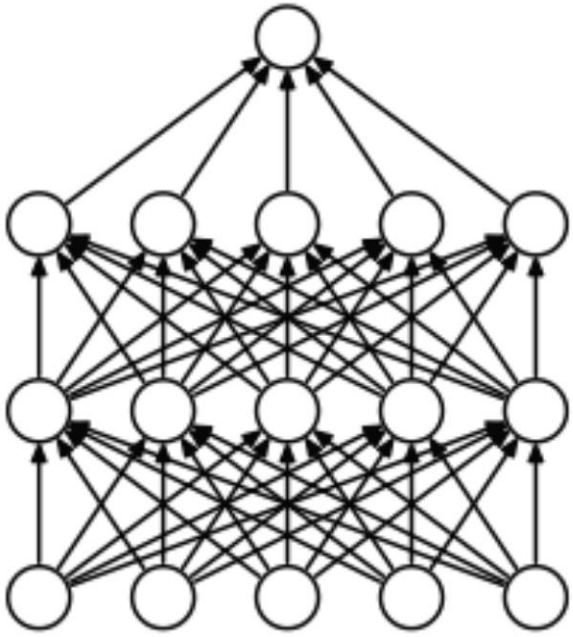
Activity of a neuron computed by applying kernel i at position (x,y) and then applying the ReLU nonlinearity

- Response normalization reduces top-1 and top-5 error rates by 1.4% and 1.2% , respectively.

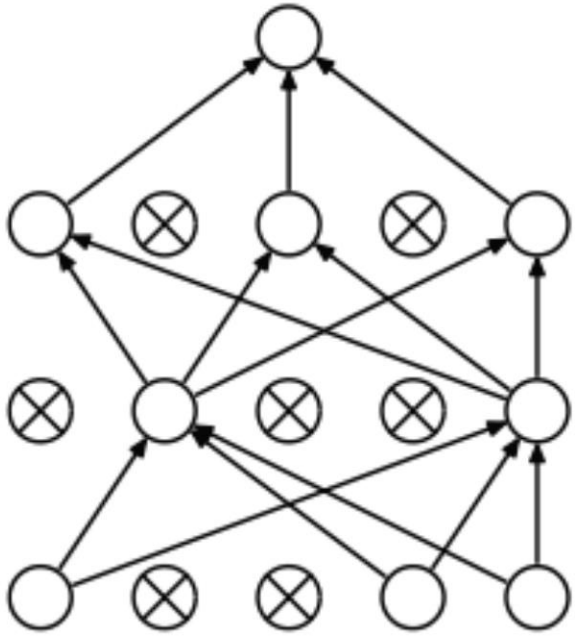
- The sum runs over n adjacent kernel maps at the same spatial location.
- Normalization implements a form of lateral inhabitation inspired by real neurons, creating competition for big activities amongst neuron outputs computed using different kernels. k and α are constants obtained by cross-validation.

Dropout to Help Generalization

- Dropout: setting to zero output of each hidden neuron with probability of 0.5 during presentation of each training sample.
- Neurons that are dropped this way do not participate in the training of the weights for this input.
- At testing time, all neurons are used but we multiply their outputs by 0.5 .



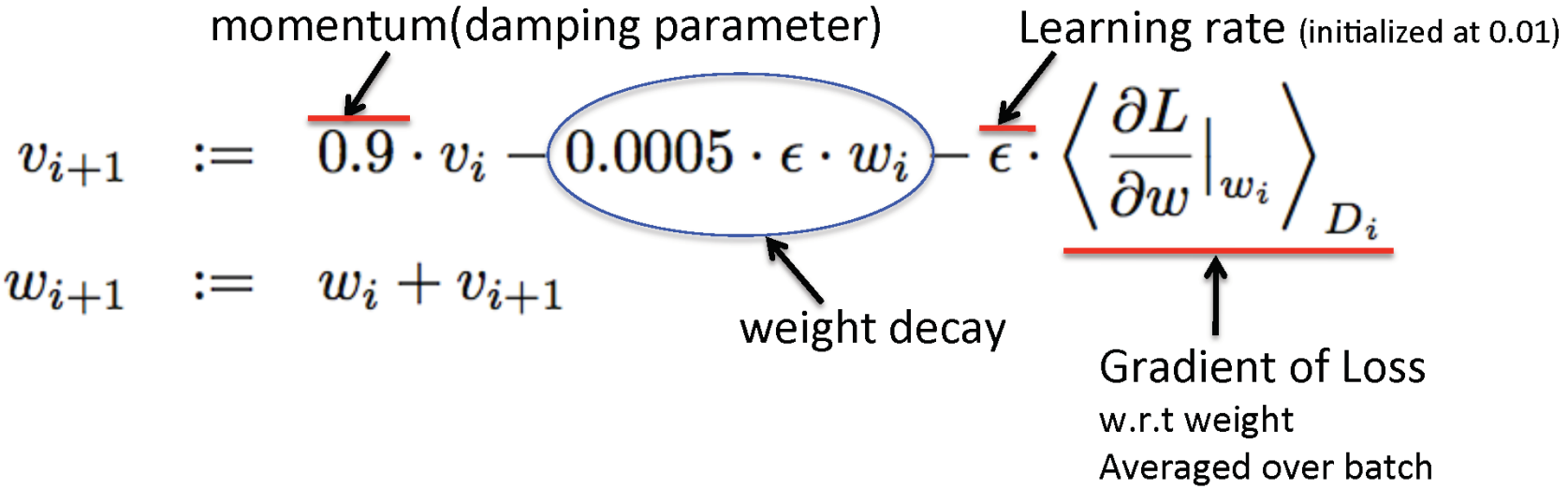
Standard Neural Net



After applying dropout.

Details of Stochastic Gradient Descent Learning

Momentum Update



Batch size: 128

- The training took 5 to 6 days on two NVIDIA GTX 580 3GB GPUs.
- Initial weights in each layer generated from a zero-mean Gaussian distribution with standard deviation 0.01 .
- Input to neuron biases are set to 1.

Reduce Overfitting

- The Sofmax classifier is regularized to reduce overfitting.

Softmax:

L_2 Regularization

$$L = -\frac{1}{N}\sum_{n}^{N}\sum_{k}^{C} y_{nk} \log\{P(class = k \mid \mathbf{x}_n; \mathbf{w})\} + \lambda \sum_{k}^{C}\sum_{l}^{L} w_{kl}^2$$

$$P(class = k \mid \mathbf{x}_n; \mathbf{w}) = \frac{e^{\mathbf{w}_k^T \mathbf{x}_n}}{\sum_{k}^{C} e^{\mathbf{w}_k^T \mathbf{x}_n}}$$

likelihood

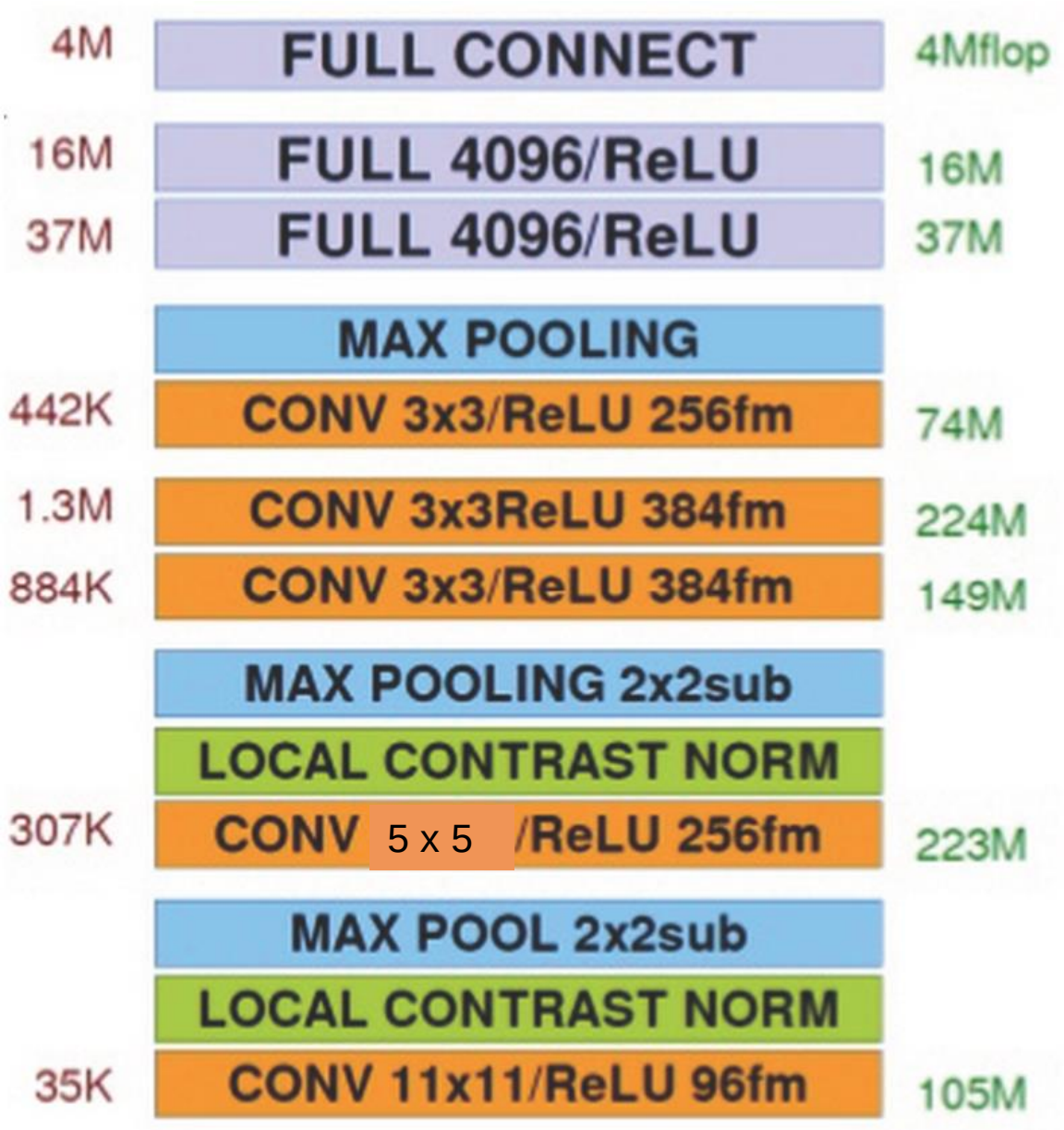
$$y_{nk} \in \{0,1\}$$

Data Augmentation

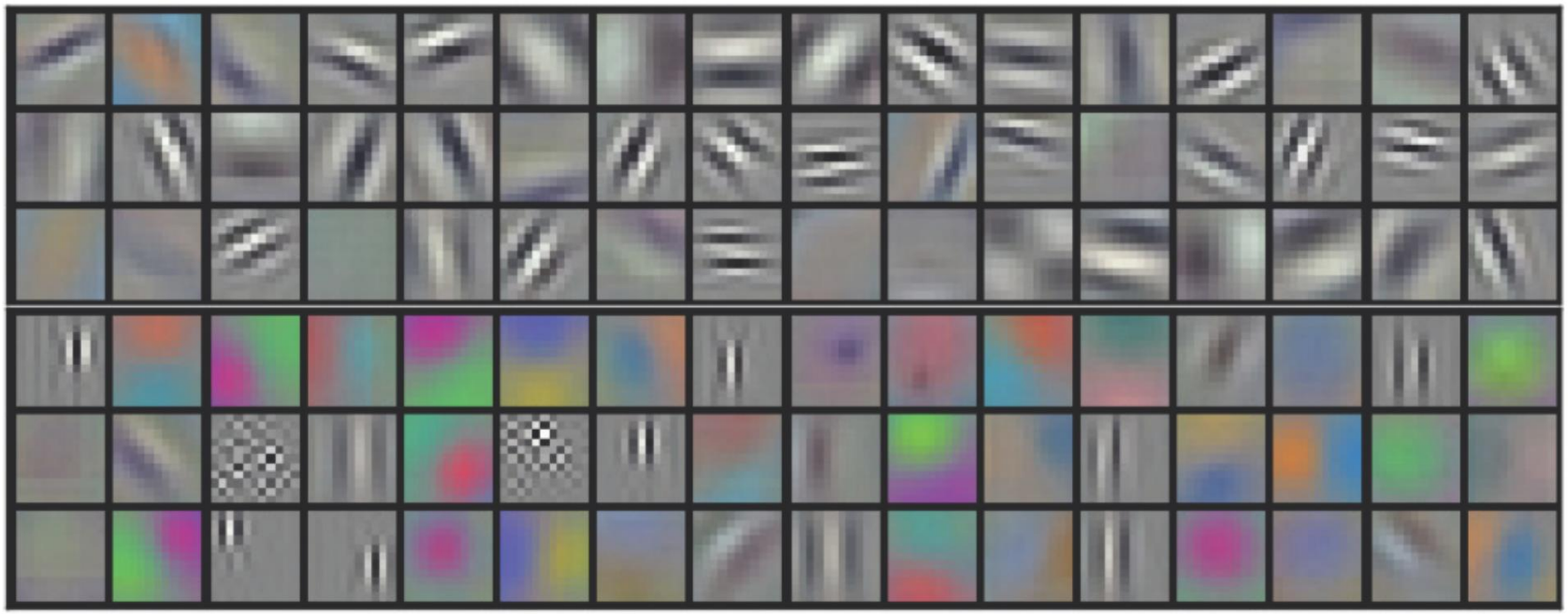
- Extract random 224x224 patches (and their horizontal reflections) from the 256x256 original images; this will increase the training set by a factor of 2048.
- At test time, network makes prediction by extracting **five** 224x224 patches (four corner patches and center patch) as well as their horizontal reflections (hence total of ten patches in all), and averaging the predictions made by Softmax layer on the ten patches.
- Another method of data augmentation is by adding the multiple of RGB PCA eigenvectors (in RGB domain) weighted with their eigenvalues times a random variable α_i (Gaussian with mean=0 and Standard Variation=0.1) to each RGB pixel:

$$\mathbf{I}_{xy} = [\mathbf{I}_{xy}^{R}, \mathbf{I}_{xy}^{G}, \mathbf{I}_{xy}^{B}] + [e_1, e_2, e_3][\alpha_1\lambda_1, \alpha_2\lambda_2, \alpha_3\lambda_3]^T$$

AlexNet (Overview Architecture)



96 Convolutional Kernels



- 11 x 11 x 3 size kernels.
- top 48 kernels on GPU 1 : color-agnostic
- bottom 48 kernels on GPU 2 : color-specific.

Why?

Visualizing and Understanding CNN

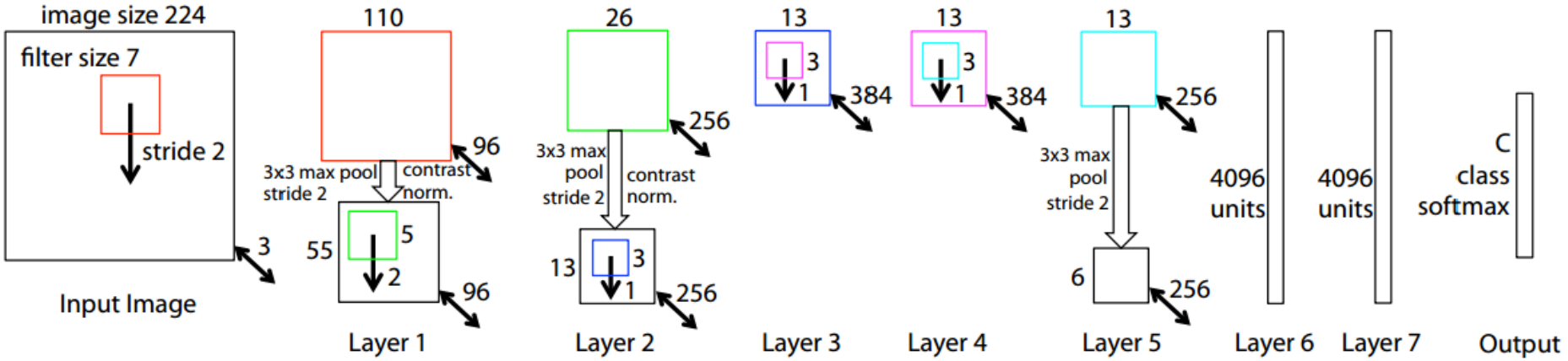
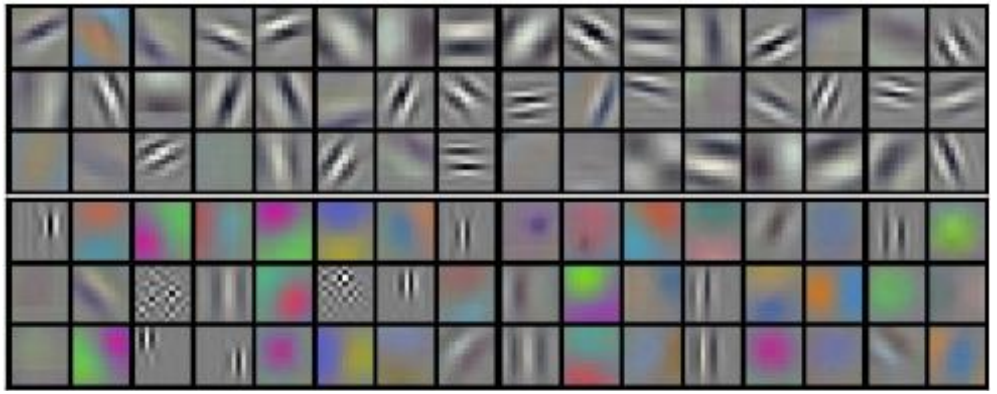
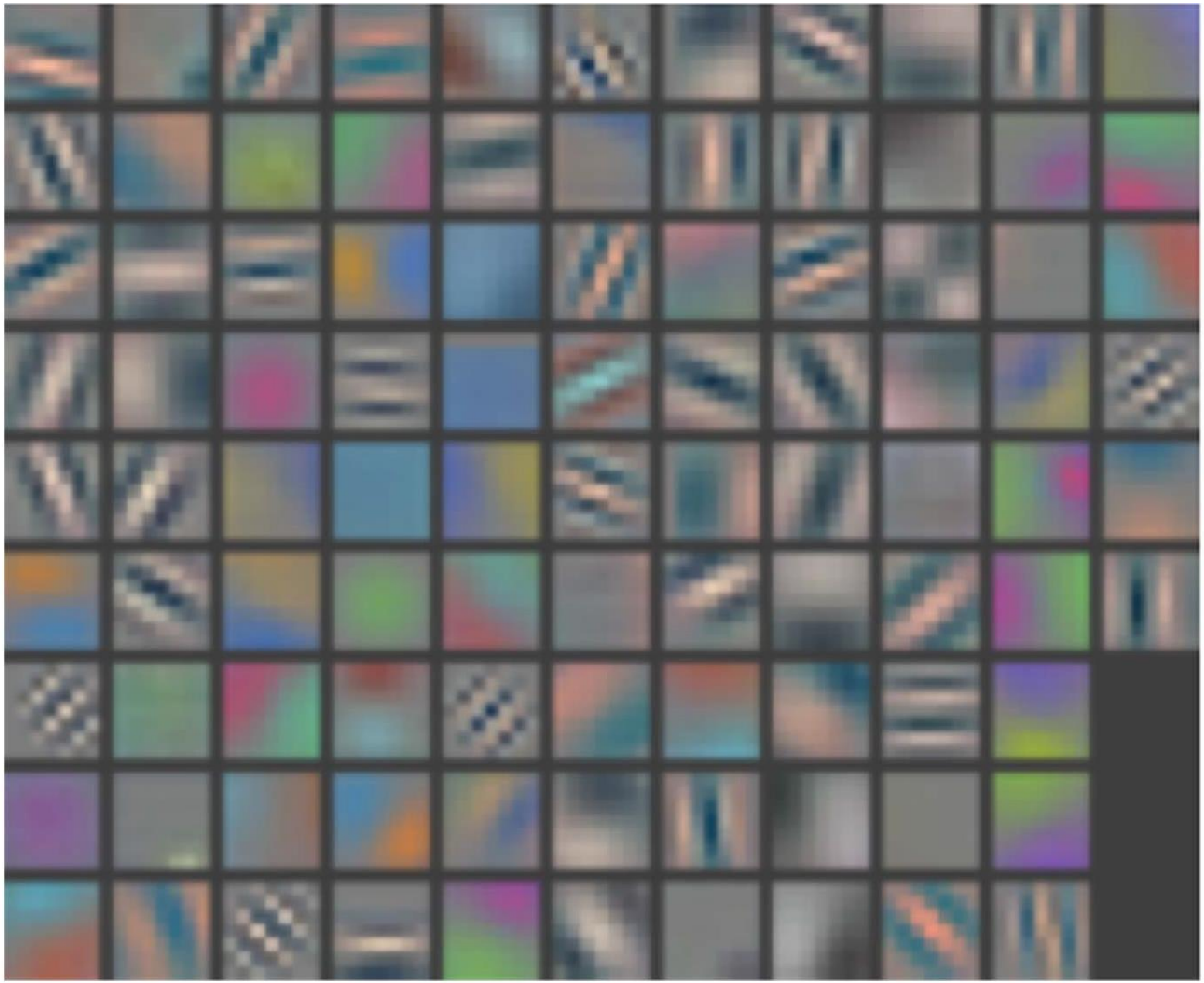


Figure 3. Architecture of our 8 layer convnet model. A 224 by 224 crop of an image (with 3 color planes) is presented as the input. This is convolved with 96 different 1st layer filters (red), each of size 7 by 7, using a stride of 2 in both x and y. The resulting feature maps are then: (i) passed through a rectified linear function (not shown), (ii) pooled (max within 3x3 regions, using stride 2) and (iii) contrast normalized across feature maps to give 96 different 55 by 55 element feature maps. Similar operations are repeated in layers 2,3,4,5. The last two layers are fully connected, taking features from the top convolutional layer as input in vector form ($6 \cdot 6 \cdot 256 = 9216$ dimensions). The final layer is a C-way softmax function, C being the number of classes. All filters and feature maps are square in shape.



M. Zeiler and R. Fergus, Visualizing and Understanding Convolutional Networks, arXiv preprint, 2013

96 Filters learned by Alex Krizhevsky, et al, AlexNet-CNN

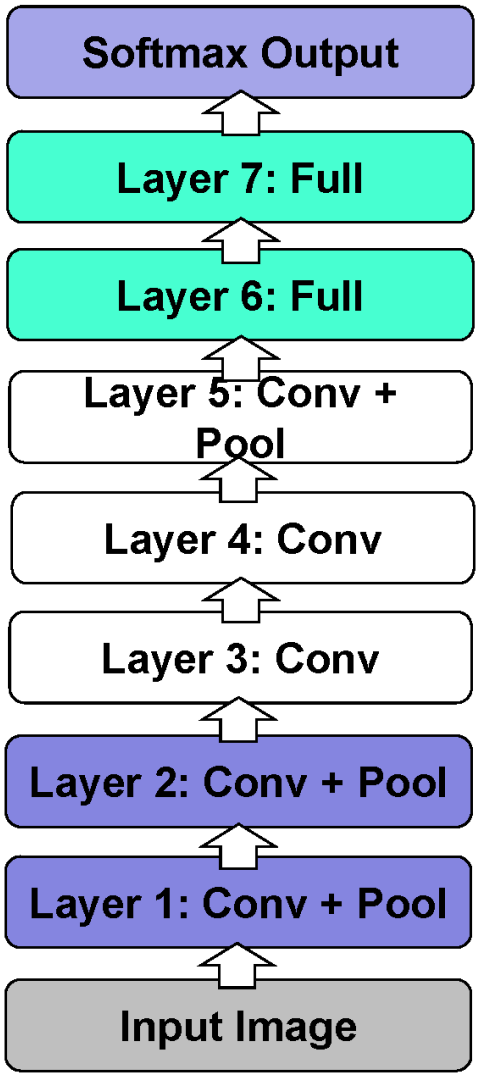


AlexNet CNN Architecture

How important is depth?

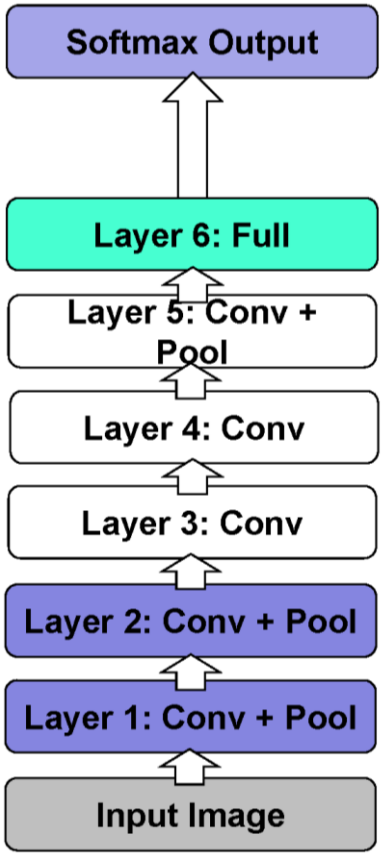
Architecture of Krizhevsky et al.

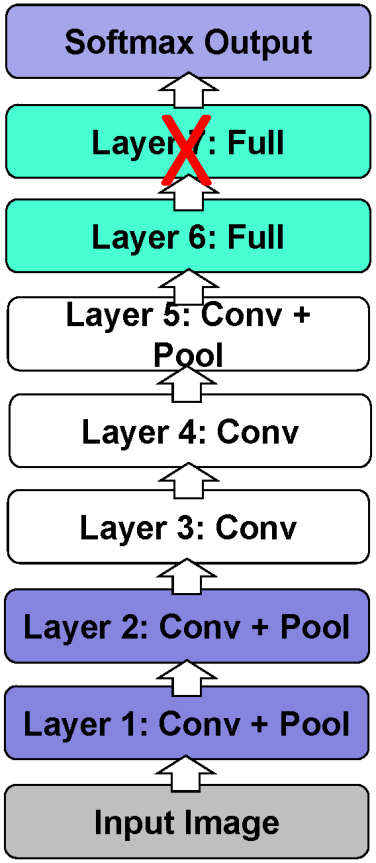
- 8 layers total
- Trained on ImageNet
- 18.1% top-5 error



AlexNet CNN Architecture (How important is depth?)

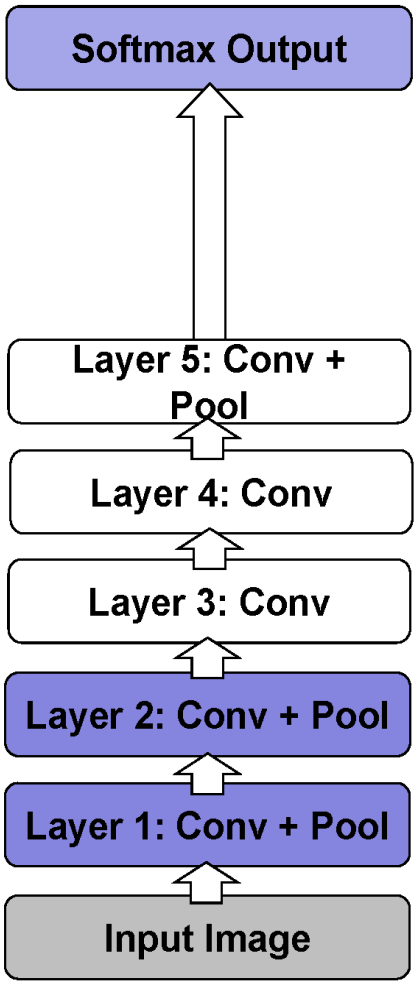
- Remove top fully connected layer (layer 7)
- Drop 16 million parameters
- Only 1.1% drop in performance!

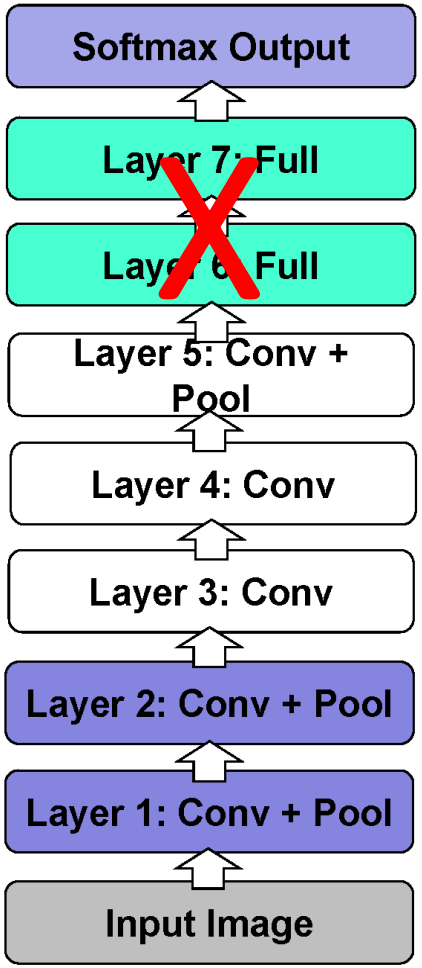




AlexNet CNN Architecture (How important is depth?)

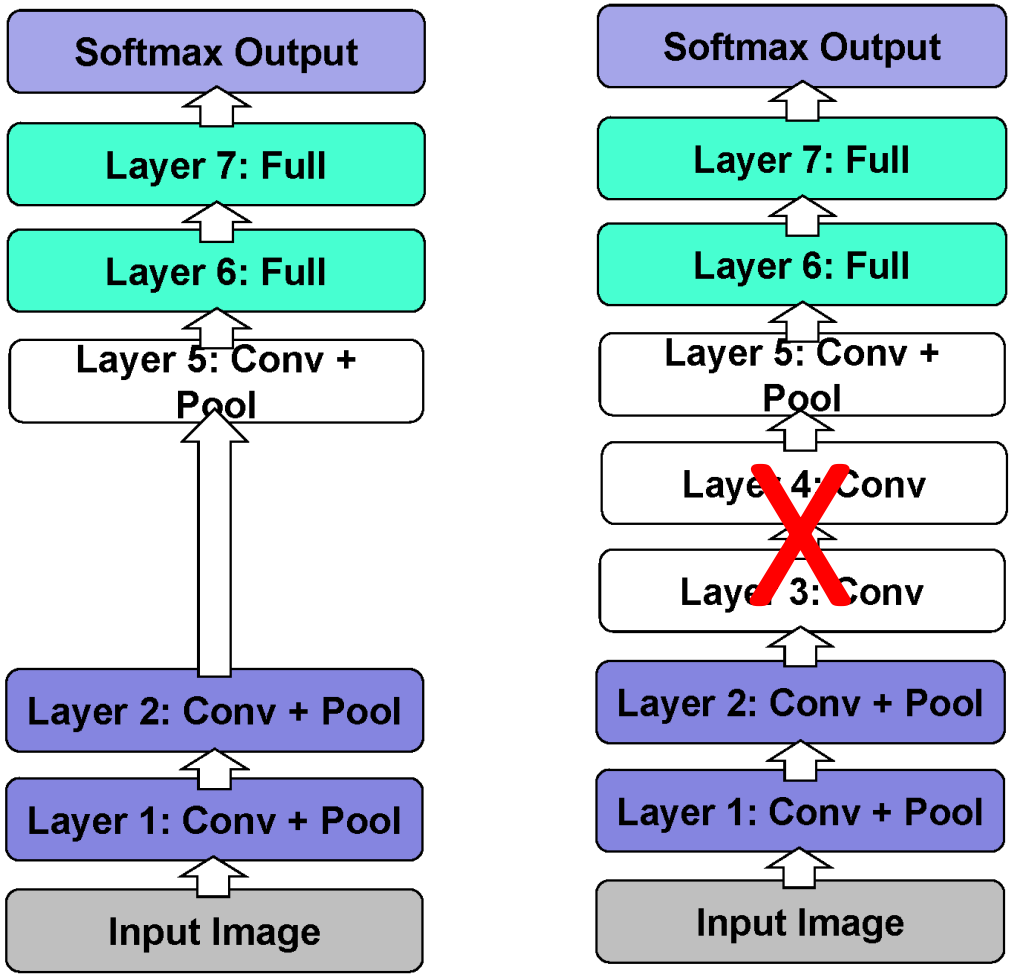
- Remove both fully connected layers (layers 6 &7)
- Drop ~ 60 million parameters
- Only 5.7% drop in performance!





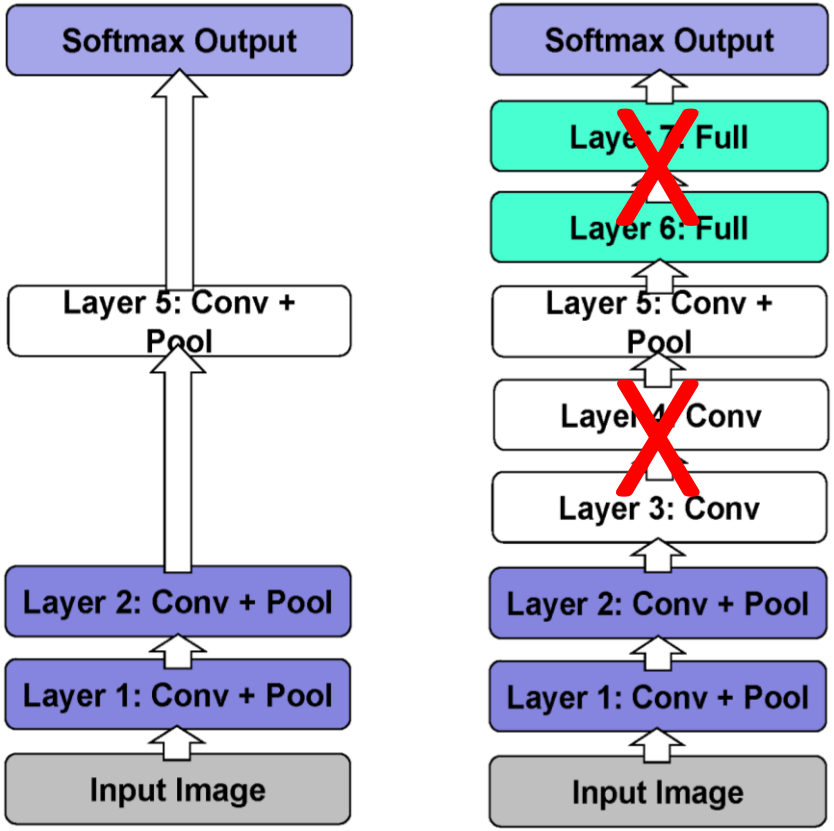
AlexNet CNN Architecture (How important is depth?)

- Remove upper layer feature extractor (layer3 3 &4)
- Drop ~ 1 million parameters
- 3.0% drop in performance!



AlexNet CNN Architecture (How important is depth?)

- Now try remove upper layer feature extractor layers + fully connected layers (layers 3, 4, 6,7)
- Now only 4 layers
- 33.5% drop in performance!
- Thus depth of network is key



Tappering off Features at Each Layer

Plug features from each layer into linear SVM or soft-max

	Cal-101 (30/class)	Cal-256 (60/class)
SVM (1)	44.8 ± 0.7	24.6 ± 0.4
SVM (2)	66.2 ± 0.5	39.6 ± 0.3
SVM (3)	72.3 ± 0.4	46.0 ± 0.3
SVM (4)	76.6 ± 0.4	51.3 ± 0.1
SVM (5)	$\mathbf{86.2 \pm 0.8}$	65.6 ± 0.3
SVM (7)	$\mathbf{85.5 \pm 0.4}$	$\mathbf{71.7 \pm 0.2}$
Softmax (5)	82.9 ± 0.4	65.7 ± 0.5
Softmax (7)	$\mathbf{85.4 \pm 0.4}$	$\mathbf{72.6 \pm 0.1}$

Face Recognition Milestone -DeepFace

1

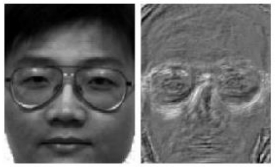
1966
W. W. Bledsoe
1st Tech. Report
on FR

2

1973
T. Kanade
1st Ph.D. Thesis
on FR

3

1991
Turk & Pentland
Concept of
Eigenfaces

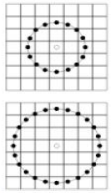


4

1997
Belhumeur et al.
Concept of
Fisherfaces

5

2001
Viola & Jones
Face Detection



6

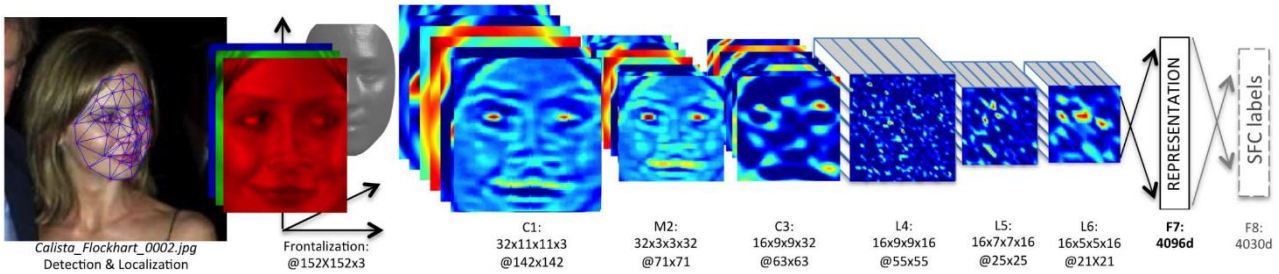
2006
Ahonen et al.
Concept of
Local Binary
Patterns (LBP)

7

2009
Wright et al.
Concept of
Sparse Rep/tions

[3] "Eigenfaces for recognition", JCN, 1991, [4] "Eigenfaces vs. Fisherfaces...", TPAMI, 1997, [5] "Robust real-time object detection", IJCV, 2001, [6] "Face Description with LBPs: Apps to FR", TPAMI 2006, [7] "Robust FR via SR", TPAMI, 2009.

8 DeepFace: Facebook (IEEE CVPR'14), FaceNet: Google (IEEE CVPR'15)



DeepFace Convolutional neural network

- In DeepFace 4M faces used as training to identify 4000 different identities
- In FaceNet 100M faces used as training to identify 8M different identities

DeepFace: Closing the Gap to Human-Level Performance in Face Verification-CVPR 2014

- A 3D-aligned 3-channels (RGB) face image of size 152 by 152 pixels is given to a convolutional layer (C1) with 32 filters of size 11x11x3 (we denote this by 32x11x11x3@142x142). The resulting 32 feature maps are then fed to a max-pooling layer (M2) which takes the max over 3x3 spatial neighborhoods with a stride of 2, separately for each channel. This is followed by another convolutional layer (C3) that has 16 filters of size 9x9x32. The purpose of these three layers is to extract low-level features, like simple edges and texture.
- The subsequent layers (L4, L5 and L6) are instead locally connected, like a convolutional layer they apply a filter bank, but every location in the feature map learns a different set of filters.
- Finally, the top two layers (F7 and F8) are fully connected: each output unit is connected to all inputs. These layers are able to capture correlations between features captured in distant parts of the face images, e.g., position and shape of eyes and position and shape of mouth.

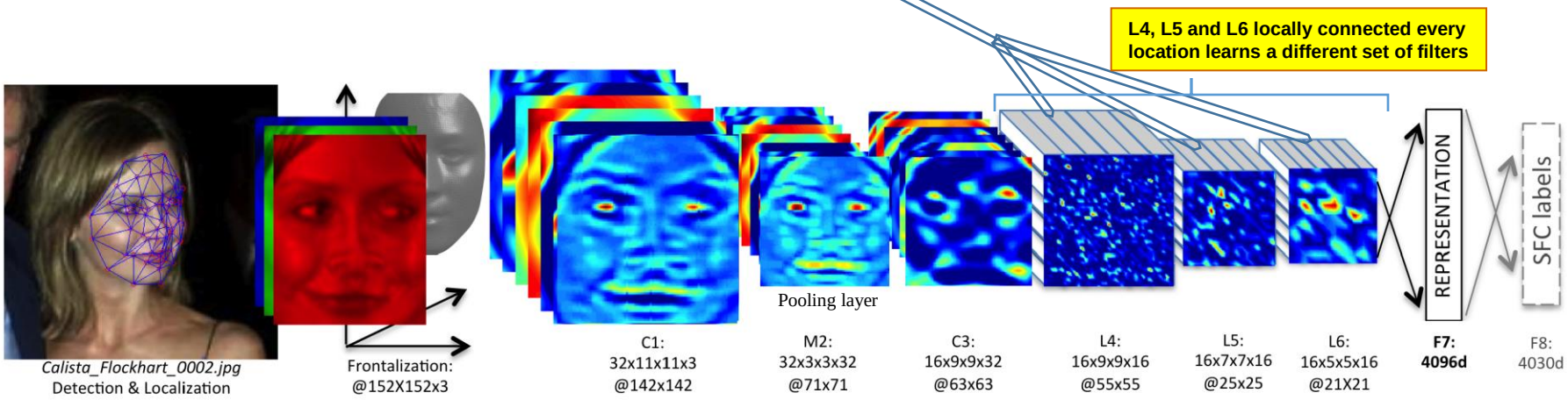
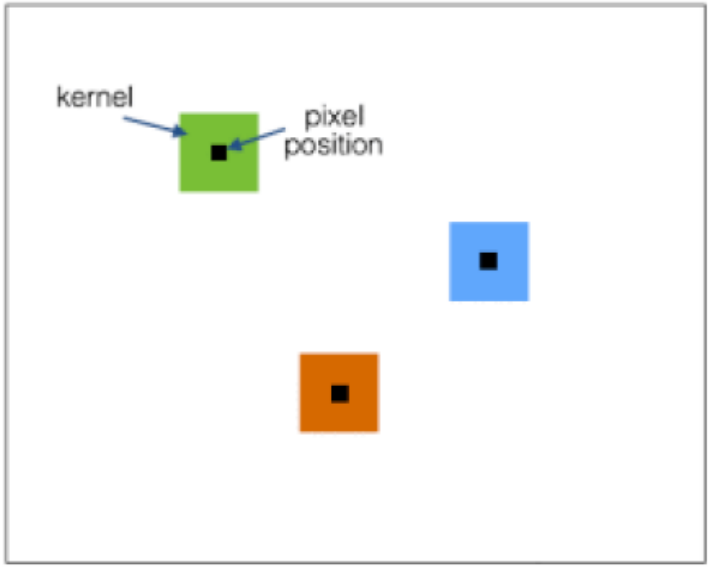


Figure 2. **Outline of the *DeepFace* architecture**. A front-end of a single convolution-pooling-convolution filtering on the rectified input, followed by three locally-connected layers and two fully-connected layers. Colors illustrate feature maps produced at each layer. The net includes more than 120 million parameters, where more than 95% come from the local and fully connected layers.

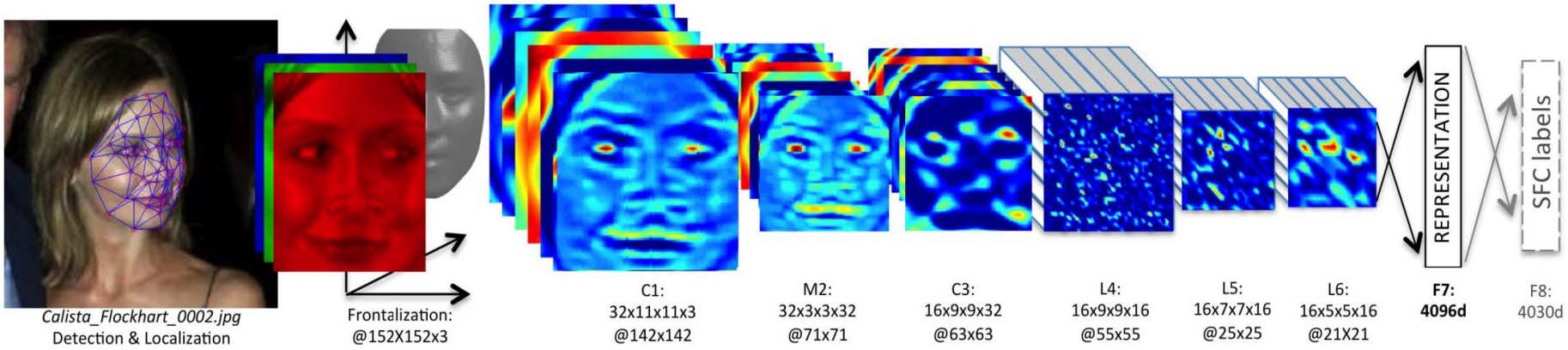
Understanding Locally Connected Layers in Convolutional Neural Networks

- In weight-shared convolutional NN on each 2D array of data, we train a whole bunch of N x N kernels. These kernels are nothing but filters that we run across the 2D array. For each pixel location (x,y), we compute the dot product summation between this kernel and the image values around that point.
- In the case of locally convolutional NN, we have a different filter for each (x,y) position. There's no convolution as such! It's just a dot product at each pixel position. So for an image of size 128x128 with a local kernel of size 5x5 there are 124x124 possible locations to fit 5x5 kernels. In weight-sharing convolutional layer we need only 5x5 number of parameters for each feature map. But in locally connected we need 5x5x124x124 number of parameters for each feature map.



- This results in an enormous number of hyperparameters, because we are building a separate filter for each pixel position and there are a lot of positions in the image.

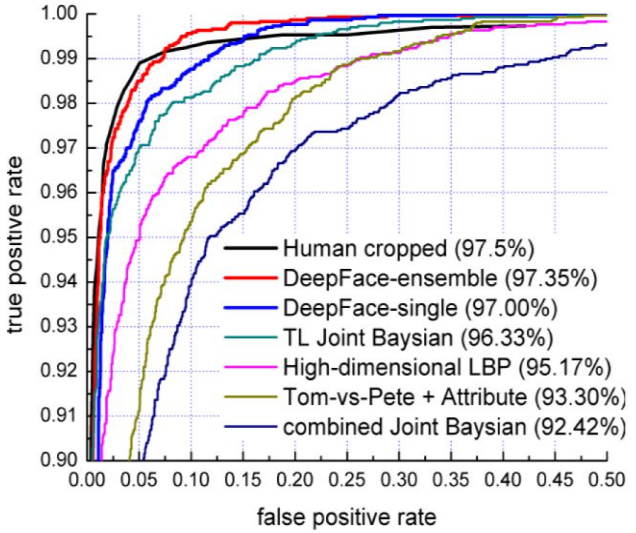
DeepFace Recognition

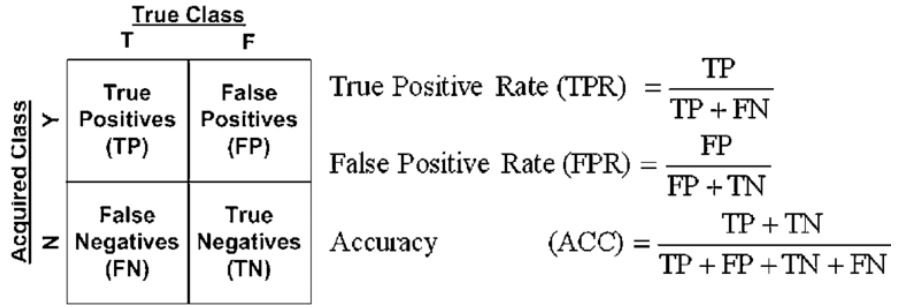


- The training data consists of FOUR million facial images of more than 4,030 subjects, with a network of 120 million parameters with accuracy of 97.35% on LFW dataset consisting of 13,233 images, 5749 people, and 1680 people with two or more image.

Method	Accuracy ± SE	Protocol
Joint Bayesian [6]	0.9242 ±0.0108	restricted
Tom-vs-Pete [4]	0.9330 ±0.0128	restricted
High-dim LBP [7]	0.9517 ±0.0113	restricted
TL Joint Bayesian [5]	0.9633 ±0.0108	restricted
DeepFace-single	**0.9592** ±0.0029	unsupervis
DeepFace-single	**0.9700** ±0.0028	restricte
DeepFace-ensemble	**0.9715** ±0.0027	restricte
DeepFace-ensemble	**0.9735** ±0.0025	unrestricte
Human, cropped	0.9753	

Table 3. Comparison with the state-of-the-art on the *LFW* d





CNN-based Face Classification

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} * 100$$

$$\text{Precision or } = \frac{TP}{TP + FP} * 100$$

$$\text{Recall or TPR } = \frac{TP}{TP + FN} * 100$$

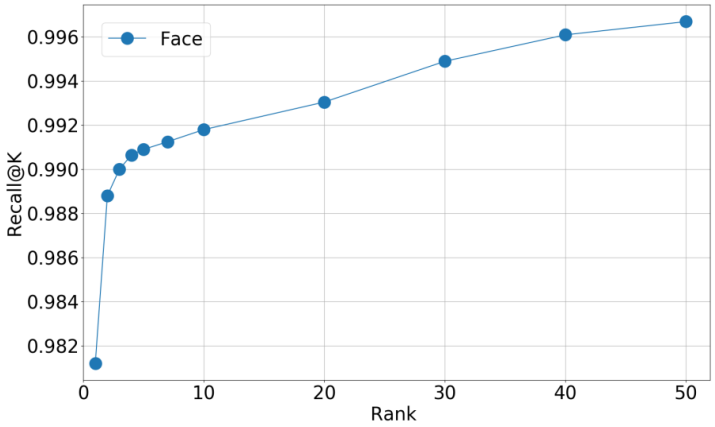
$$\text{Specificity or TNR } = \frac{TN}{TN + FP} * 100$$

$$\text{F1 Score } = 2 * \frac{\text{Recall } \times \text{Precision}}{\text{Recall} + \text{Precision}} * 100$$

$$\text{False Positive Ratio(FPR)} = \frac{FP}{FP+FN} * 100$$

- Train a 294-way Softmax for the CNN features

Number of Subjects	294
Training Set	6000
Validation Set	833
Test Set	6960

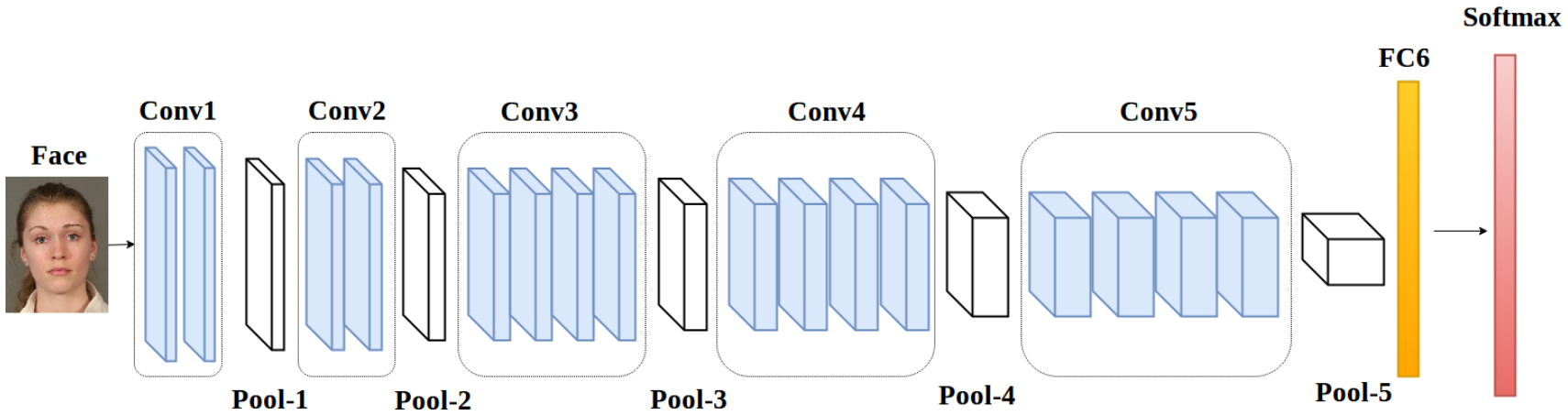


$$\textit{True Positive Rate} = \frac{TP}{\textit{Actual Positive}} = \frac{TP}{TP+FN}$$

$$\textit{False Negative Rate} = \frac{FN}{\textit{Actual Positive}} = \frac{FN}{TP+FN}$$

$$\textit{True Negative Rate} = \frac{TN}{\textit{Actual Negative}} = \frac{TN}{TN+FP}$$

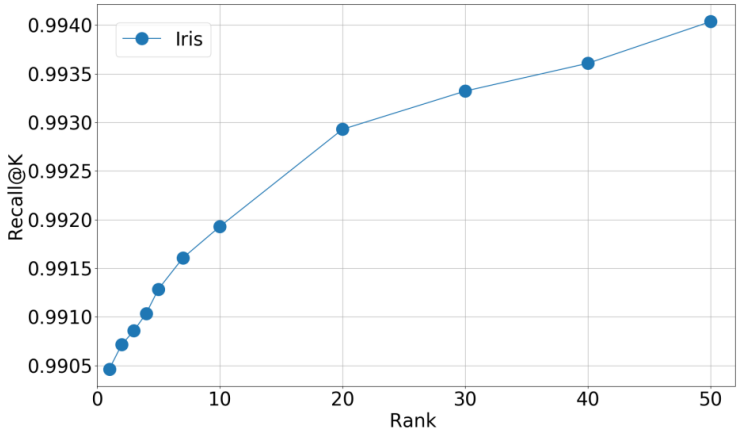
$$\textit{False Positive Rate} = \frac{FP}{\textit{Actual Negative}} = \frac{FP}{TN+FP}$$

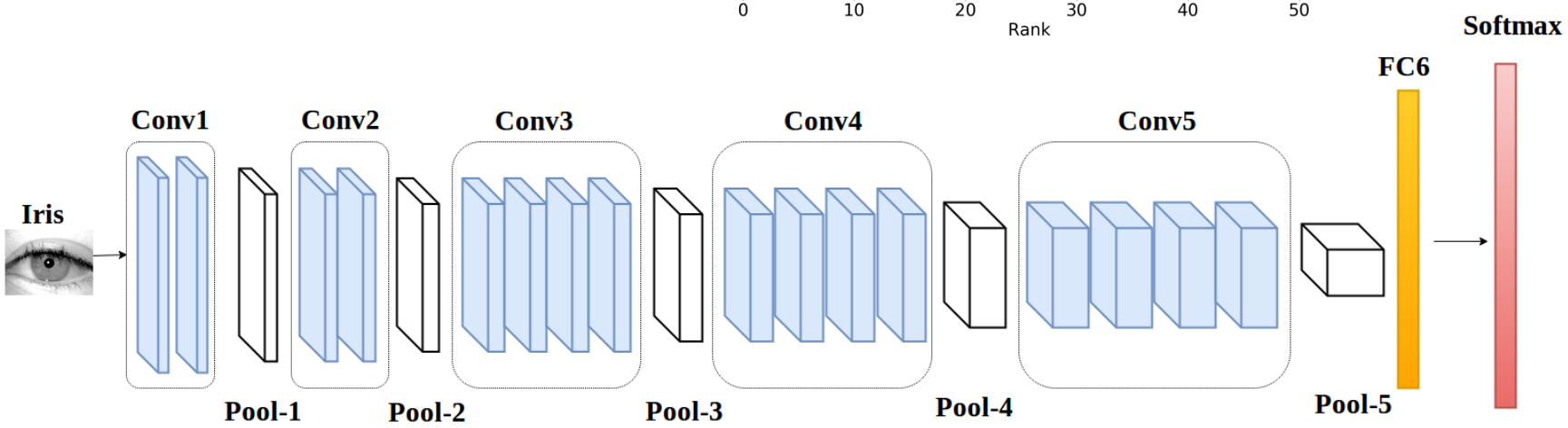


CNN-Based Iris Classification

- Train a 294-way Softmax for the CNN features

Number of Subjects	294
Training Set	33000
Validation Set	3636
Test Set	39725

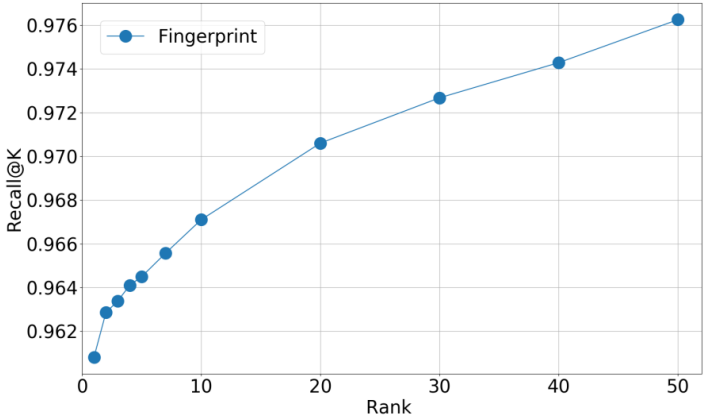


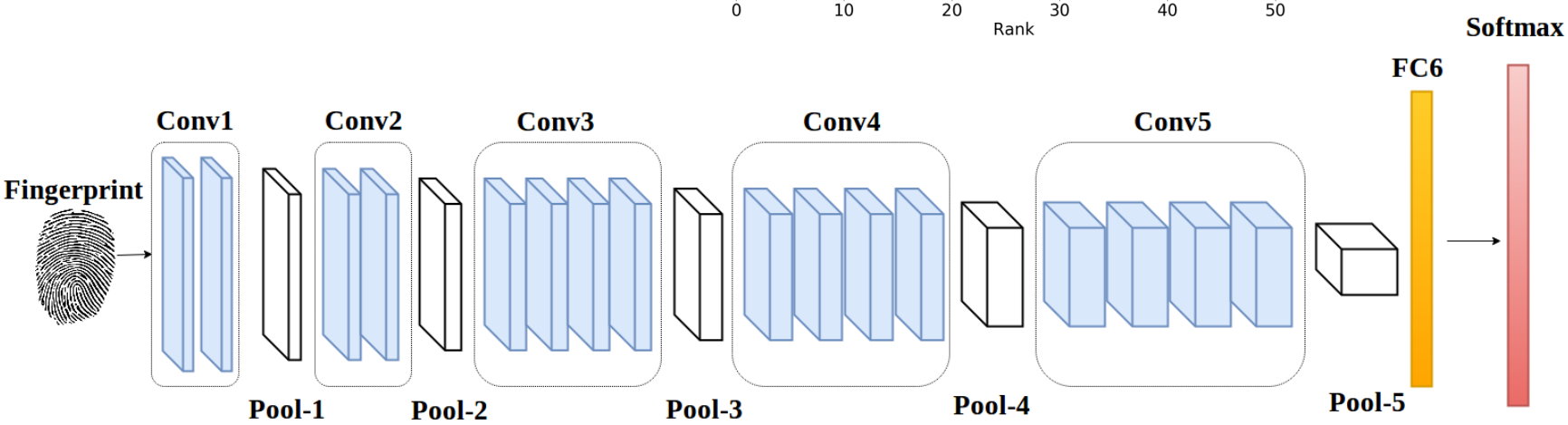


CNN-Based Fingerprint Classification

- Train a 294-way Softmax for the CNN features

Number of Subjects	294
Training Set	1600
Validation Set	222
Test Set	991

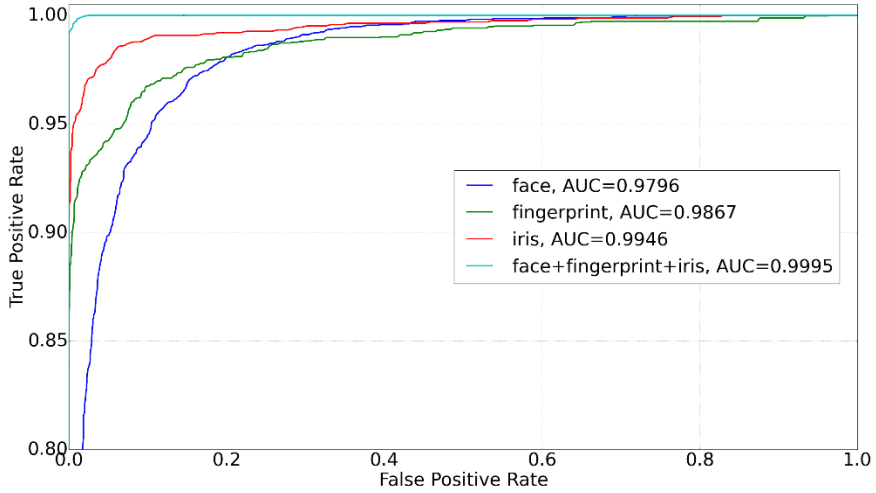


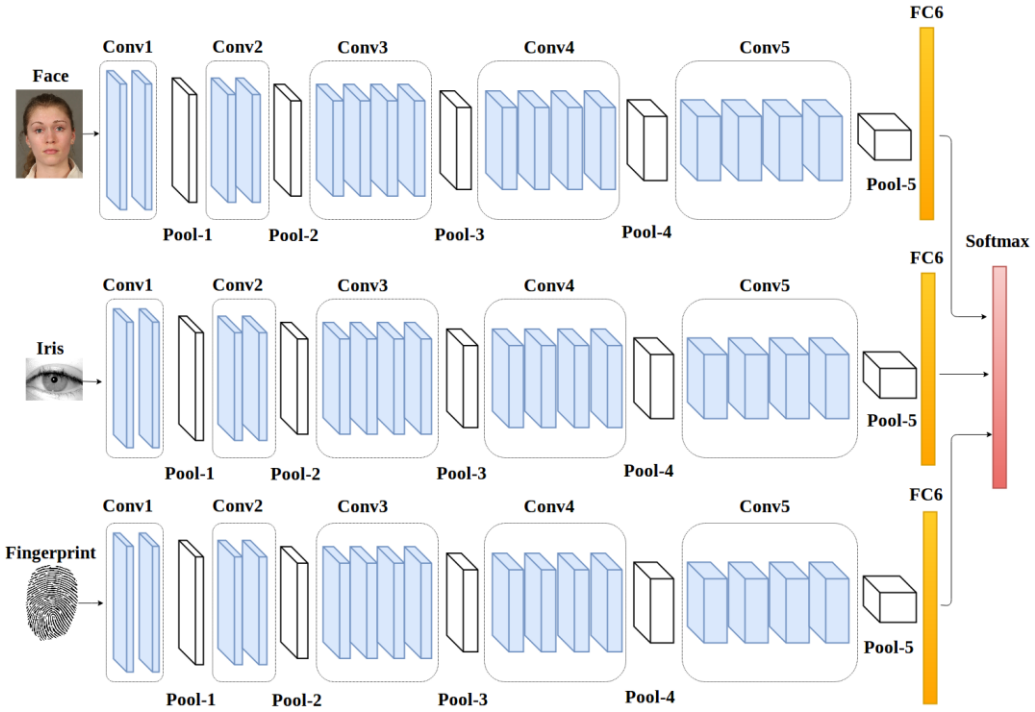


Deep Multi-modal CNN-Based Personnel Verification

Multi-modal Fusion:

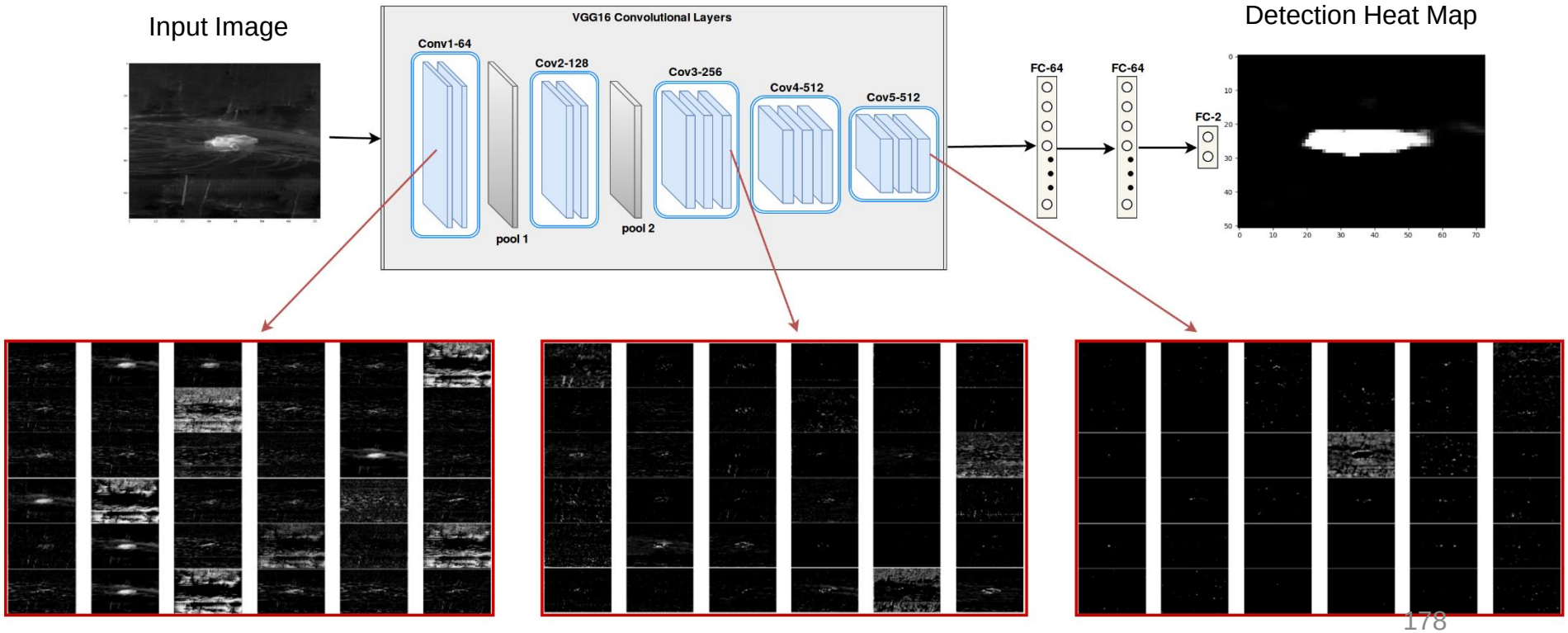
Number of Subjects	294
Training Set	58800
Validation Set	14700
Test Set	73500



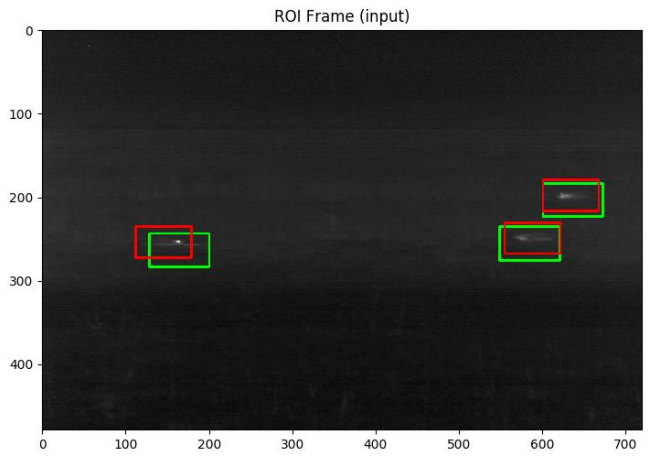


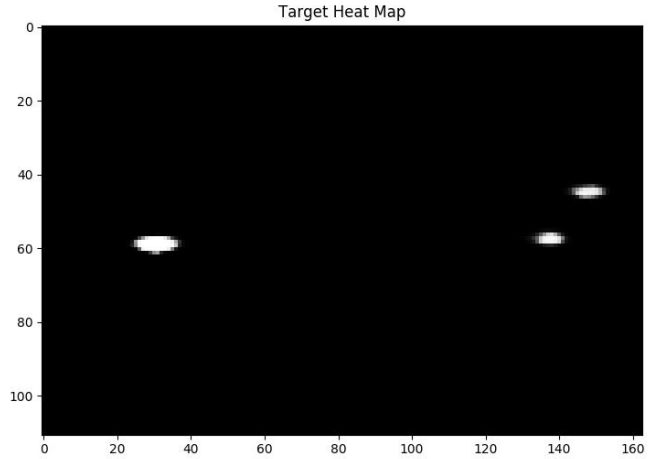
Example: Deep Target Detection Using CNN

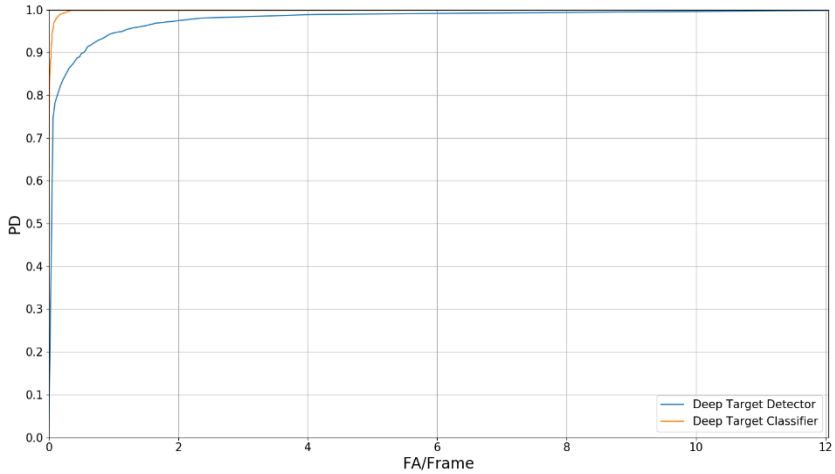
- Goal is to detect targets of interest using a CNN with a 2-class Softmax classifier.
- After detection a N-way CNN classifier is used on the detected region to perform target type identification.



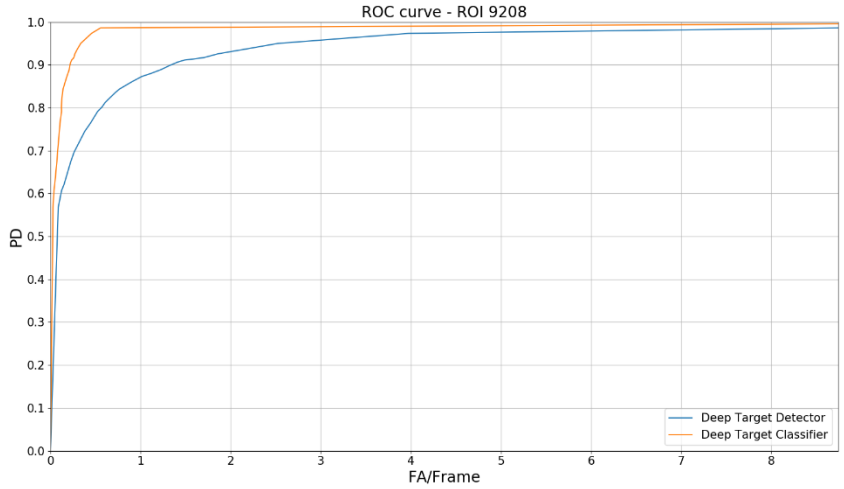
Deep Target Detector & Classification Using CNN





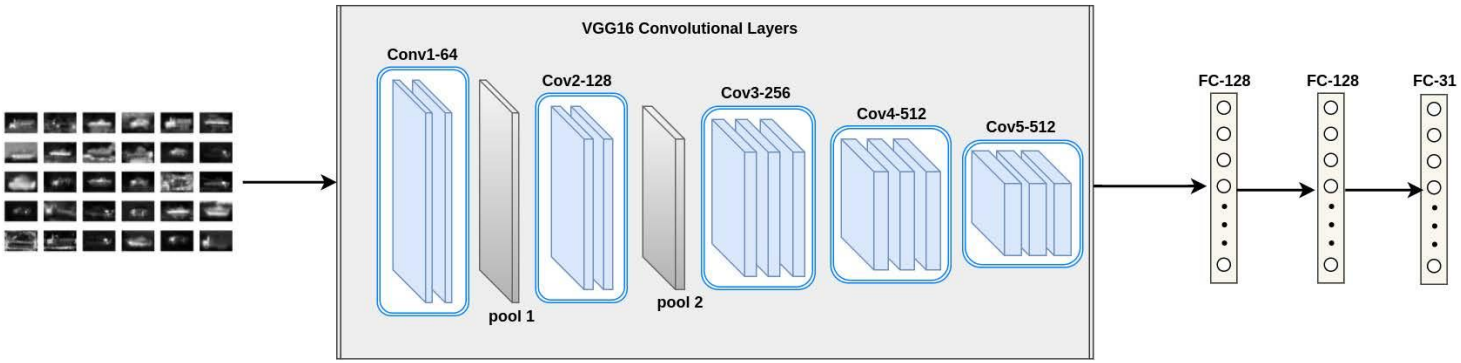


ROC for ROI-9201 test Dataset

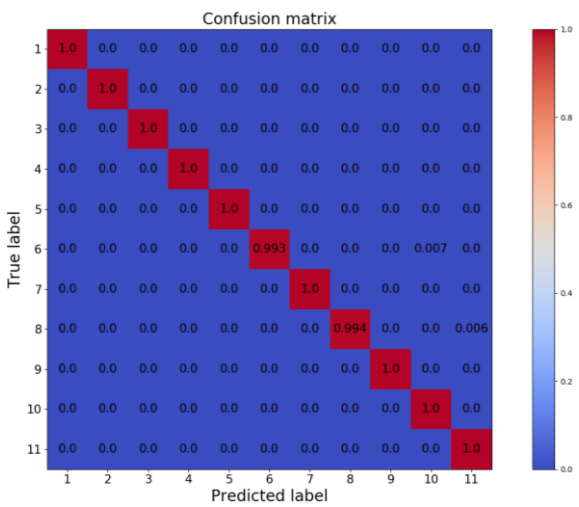


ROC for ROI-9208 Test Dataset

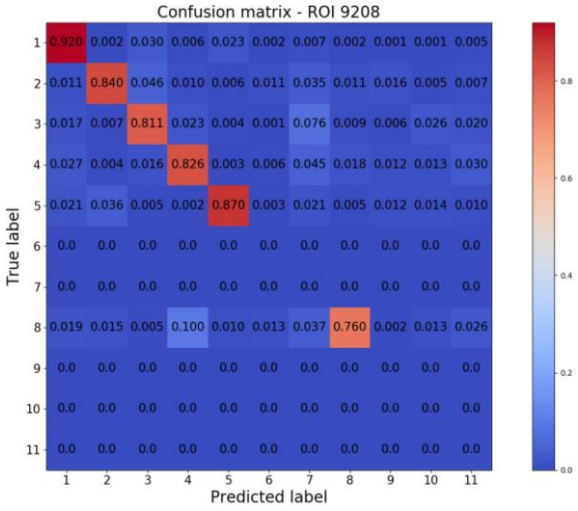
Deep Target classification Using CNN



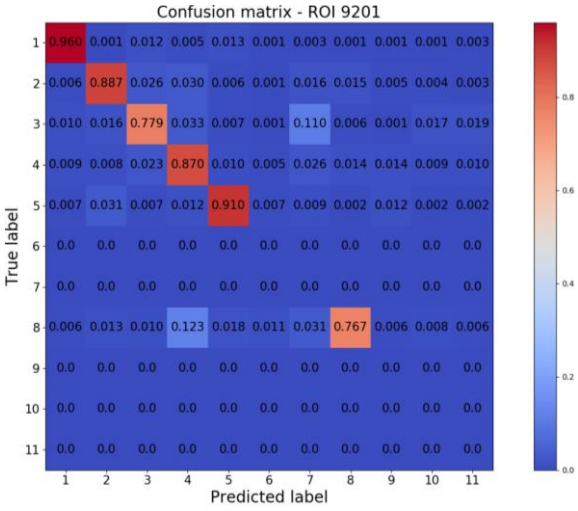
Deep Target Classifier



Training dataset



ROI-9208 dataset

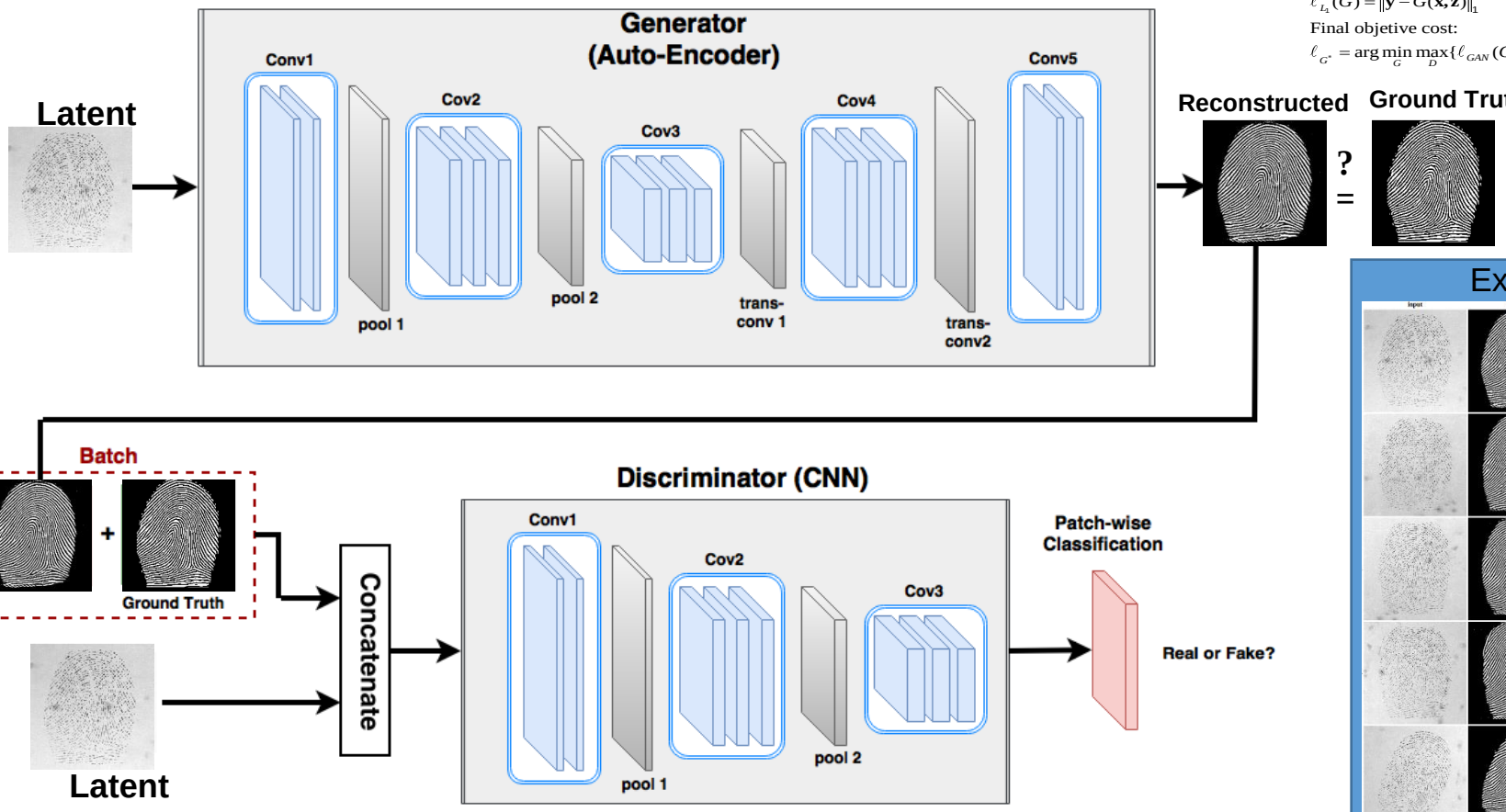


ROI-9201 dataset

Deep Latent Fingerprint Distortion Correction Using Generative Adversarial Network (GAN)

Deep Conditional Generative Adversarial Networks (DCGAN)

$$\ell_{GAN}(G,D) = [\log D(\mathbf{y})] + [\log(1 - D(G(\mathbf{x},\mathbf{z}))]$$
$$\ell_{L_1}(G) = \|\mathbf{y} - G(\mathbf{x},\mathbf{z})\|_1$$
Final objetive cost:
$$\ell_{G^*} = \arg\min_G \max_D \{\ell_{GAN}(G,D) + \lambda \ell_{L_1}(G)\}$$



Convolutional Neural Networks Packages

- Cuda-convnet (Alex Krizhevsky, Google)
- Caffe (Y. Jia, Berkeley)
  - C++ with Matlab and Python interfaces
    http://caffe.berkeleyvision.org
    Replacement of deprecated Decaf
- TensorFlow https://www.tensorflow.org/
- Keras: (https://keras.io/)
- Overfeat (NYU)
- Torch (http://torch.ch)
- PyTorch **– A Simple yet Powerful Deep Learning Library**
- Theano (Python, https://pypi.python.org/pypi/Theano)
- MatConvNet (Matlab, http://www.vlfeat.org/matconvnet/)
  https://www.youtube.com/watch?v=4nwpU7GFYe8
  https://www.youtube.com/watch?v=jw6nBWo21Zk

- **There are several architectures in the field of Convolutional Networks that have a name. The most common are:**

- **LeNet.** The first successful applications of Convolutional Networks were developed by Yann LeCun in 1990's. Of these, the best known is the LeNet architecture that was used to read zip codes, digits, etc.
- **AlexNet.** The first work that popularized Convolutional Networks in Computer Vision was the AlexNet, developed by Alex Krizhevsky, Ilya Sutskever and Geoff Hinton. The AlexNet was submitted to the ImageNet ILSVRC challenge in 2012 and significantly outperformed the second runner-up (top 5 error of 16% compared to runner-up with 26% error). The Network had a similar architecture basic as LeNet, but was deeper, bigger, and featured Convolutional Layers stacked on top of each other (previously it was common to only have a single CONV layer immediately followed by a POOL layer).
- **ZF Net**. The ILSVRC 2013 winner was a Convolutional Network from Matthew Zeiler and Rob Fergus. It became known as the ZFNet (short for Zeiler & Fergus Net). It was an improvement on AlexNet by tweaking the architecture hyperparameters, in particular by expanding the size of the middle convolutional layers.
- **GoogLeNet.** The ILSVRC 2014 winner was a Convolutional Network from Szegedy et al. from Google. Its main contribution was the development of an Inception Module that dramatically reduced the number of parameters in the network (4M, compared to AlexNet with 60M). Additionally, this paper uses Average Pooling instead of Fully Connected layers at the top of the ConvNet, eliminating a large amount of parameters that do not seem to matter much.
- **VGGNet.** The runner-up in ILSVRC 2014 was the network from Karen Simonyan and Andrew Zisserman that became known as the VGGNet. Its main contribution was in showing that the depth of the network is a critical component for good performance. Their final best network contains 16 CONV/FC layers and, appealingly, features an extremely homogeneous architecture that only performs 3x3 convolutions and 2x2 pooling from the beginning to the end. It was later found that despite its slightly weaker classification performance, the VGG ConvNet features outperform those of GoogLeNet in multiple transfer learning tasks. Hence, the VGG network is currently the most preferred choice in the community when extracting CNN features from images. In particular, their pretrained model is available for plug and play use in Caffe. A downside of the VGGNet is that it is more expensive to evaluate and uses a lot more memory and parameters (140M).
- **ResNet.** Residual Network developed by Kaiming He et al. was the winner of ILSVRC 2015. It features an interesting architecture with special skip connections and features heavy use of batch normalization. The architecture is also missing fully connected layers at the end of the network. The reader is also referred to Kaiming's presentation (video, slides), and some recent experiments that reproduce these networks in Torch.